

D B A L I B R A R Y

Vector Databases & AI

40 chapters · Free edition

© 2026 Rajesh Guntupalli

Chapter 1: What Are Embeddings and Why Databases Need Them

If you have spent years designing schemas, tuning indexes, and optimizing query plans, the idea of storing lists of floating-point numbers as a first-class database primitive probably sounds strange. This chapter explains why that instinct is worth revisiting. Embeddings are the data structure at the center of modern AI applications — from semantic search and recommendation engines to retrieval-augmented generation (RAG) pipelines — and understanding them at the infrastructure level is what separates engineers who bolt AI onto existing systems from those who build systems that actually perform at scale.

From Keywords to Coordinates: How Machines Represent Meaning

Traditional databases store meaning in structure. A product with `category = 'running shoes'` and `brand = 'Nike'` communicates meaning through schema design, foreign keys, and the queries you write against them. The database itself has no concept of what "running shoes" means — it only knows that two rows share the same string value in a column.

Natural language does not work this way. When a user searches for "comfortable sneakers for long runs," no keyword match will reliably surface products described as "cushioned athletic footwear for distance training." The words are different. The meaning is the same.

Embeddings solve this by converting text, images, audio, or any structured content into points in a high-dimensional vector space, where proximity encodes semantic similarity. Instead of asking "do these two strings share characters?", you ask "how close are these two points?"

A vector embedding is a fixed-length array of floating-point numbers — typically 384 to 3,072 dimensions depending on the model. Each dimension captures some latent feature learned during model training. No individual dimension has a human-readable meaning; the meaning emerges from the geometry of the entire space.

For a database professional, the most useful mental model is this: **an embedding is a computed column that captures semantic meaning, generated by a neural network rather than a SQL expression.** Just as a computed column like `LOWER(email)` transforms a raw value into a normalized form optimized for a specific access pattern, an embedding transforms raw content into a form optimized for similarity-based retrieval.

The critical difference is that you cannot reconstruct the original content from an embedding (it is a lossy, one-way transformation), and the "similarity" it enables is not byte-level equality — it is semantic relatedness, measured geometrically.

Anatomy of an Embedding: What the Numbers Actually Represent

To reason about embeddings as infrastructure, you need a concrete picture of what they look like and where they come from.

An embedding model takes an input — a sentence, a product description, a code snippet, an image — and produces a dense vector. "Dense" means nearly all dimensions have non-zero values, which distinguishes embeddings from the sparse TF-IDF or bag-of-words vectors that predated deep learning. Here is what generating an embedding looks like in practice using OpenAI's API:

```
# Python example – generating an embedding with OpenAI
from openai import OpenAI

client = OpenAI() # uses OPENAI_API_KEY from environment

text = "Cushioned athletic footwear for long-distance running"

response = client.embeddings.create(
    input=text,
    model="text-embedding-3-small" # 1536 dimensions, ~$0.02 per 1M tokens
)

vector = response.data[0].embedding
print(f"Dimensions: {len(vector)}")      # 1536
print(f"First 5 values: {vector[:5]}")   # [-0.012, 0.043, -0.071, 0.019, 0.055]
```

That list of 1,536 floats is the embedding. Store it alongside the original text, and you have the foundation of a semantic search system.

For local or self-hosted workloads — important when data cannot leave your environment — the `sentence-transformers` library gives you access to hundreds of open-source models:

```
# Python example – generating embeddings locally with sentence-transformers
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2") # 384 dimensions, fast, runs on CPU

sentences = [
    "Cushioned athletic footwear for long-distance running",
    "Comfortable sneakers for long runs",
    "Men's formal leather dress shoes",
]

embeddings = model.encode(sentences)

print(f"Shape: {embeddings.shape}") # (3, 384)

# Semantic similarity – closer to 1.0 means more similar
from sklearn.metrics.pairwise import cosine_similarity
sim_matrix = cosine_similarity(embeddings)

print(f"Sentence 0 vs 1: {sim_matrix[0][1]:.4f}") # high – very similar
print(f"Sentence 0 vs 2: {sim_matrix[0][2]:.4f}") # low – dissimilar
```

The output is telling. Sentences 0 and 1 use completely different words but score high on similarity. Sentence 2, which shares some nouns with sentence 0 ("footwear," "shoes"), scores much lower because the semantic context is entirely different. (Exact scores depend on model version and are illustrative here, not reproduced from a real run.) No traditional index — B-tree, full-text, or otherwise — can produce this result.

A few properties of embedding vectors that matter operationally:

Dimensionality determines storage and compute cost. A 1,536-dimension float32 vector occupies 6,144 bytes — about 6 KB per row. One million vectors requires roughly 6 GB of raw storage before indexes. A 3,072-dimension model (like `text-embedding-3-large`) doubles that cost for modest accuracy gains. Choosing the right model is an infrastructure decision as much as an accuracy decision.

The embedding space is model-specific. Vectors from `text-embedding-3-small` and `all-MiniLM-L6-v2` live in completely different coordinate spaces and cannot be compared. All vectors in a collection must come from the same model. This has significant operational implications: upgrading your embedding model means re-embedding your entire dataset.

Normalization affects which distance metric to use. Many models produce L2-normalized vectors (unit vectors), for which cosine similarity and dot product are equivalent. Some do not. Know your model's output characteristics before choosing a distance metric — the wrong choice degrades search quality silently.

Why Traditional Indexes Fail for Vector Search

At this point a reasonable question is: why not just store vectors in a standard `NVARCHAR(MAX)` column as JSON arrays and query them with computed similarity? Before SQL Server 2025, that workaround was exactly what teams did.

You can do this. For small datasets it even works. The problem is that similarity search over vectors does not have the algebraic properties that make B-tree and hash indexes effective.

A B-tree index works because values have a total ordering: for any two keys, one is definitively greater than, equal to, or less than the other. Queries like `WHERE price < 100` carve off a predictable subtree. Vector similarity has no total ordering. Asking "which of these 10 million vectors is closest to my query vector?" requires computing distance to every candidate — a full table scan — unless you use specialized data structures.

Consider what a naive implementation costs. At 10 million vectors of 1,536 dimensions each:

- → Full scan: $10M \times 1,536$ multiplications + additions per query
- → At ~10 billion FLOPs per second on a single CPU core, that is roughly 1.5 seconds per query
- → That is before any I/O overhead

This is why approximate nearest neighbor (ANN) algorithms exist, and why vector databases exist as a category. The index type you will encounter most in this book is **DiskANN** — the graph-based ANN algorithm behind SQL Server 2025's native vector index — alongside **HNSW**, which powers several dedicated vector databases:

DiskANN (SQL Server 2025's vector index): Builds a navigable graph where each vector connects to a bounded set of neighbors, designed for SSD-backed storage so billion-scale collections do not require terabytes of RAM. Query time is $O(\log n)$ with recall typically 95–99%. Created with `CREATE VECTOR INDEX ... WITH (TYPE = 'DiskANN')` and queried through `VECTOR_SEARCH`.

HNSW (Hierarchical Navigable Small World): Builds a layered graph structure where each node connects to its nearest neighbors, traversed from coarse upper layers to precise lower layers. Also $O(\log n)$ with excellent recall, but keeps the full graph in memory — fast at the cost of significant RAM. HNSW is the index inside Qdrant, Weaviate, and several other dedicated engines you will meet in Chapters 15–19.

The tradeoff table that matters in practice:

| | Exact scan (`VECTOR_DISTANCE`) | DiskANN index |

|---|---|---|

| Build time | None | Minutes–hours for 10M vectors |

| Memory | Buffer pool for scanned pages | Graph + vectors compete for buffer pool |

| Query speed | Slow at scale ($O(n)$) | Very fast ($O(\log n)$) |

| Recall | 100% | ~95–99% |

| Suitable for | Under ~50K vectors | 50K+ vectors, interactive latency SLAs |

Neither algorithm guarantees finding the exact nearest neighbor — that is the "approximate" in ANN. For most production use cases, 95% recall at 10ms latency is far more valuable than 100% recall at 1,500ms latency. This is a tradeoff database professionals sometimes resist initially because exact results are the norm in relational systems. In vector search, approximation is an engineering choice, not a deficiency.

Storing Vectors in SQL Server with the Native VECTOR Type

SQL Server 2025 is where database professionals running the Microsoft stack will encounter vectors first: the native `VECTOR(n)` data type is built into the engine — no extension to install, no separate process to run. Rather than immediately reaching for a specialized vector database, understanding what native vectors can do — and where they hit limits — gives you the foundation to make informed infrastructure decisions throughout this book.

```
graph LR
    A[Raw content  
text / image / audio] --> B[Embedding Model]
    B --> C[Dense Vector  
e.g. 1536 floats]
    C --> D[(VECTOR column  
SQL Server / Pinecone / etc.)]
    D --> E[ANN Index  
DiskANN]
    F[Query text] --> B
    E --> G[Ranked nearest  
neighbors]
```

The same embedding model encodes both stored content and incoming queries into one shared vector space — the ANN index only has to search that space, not the original content.

There is nothing to install — verify the engine version and create a table with a vector column:

```
-- SQL Server 2025: native VECTOR type, no extension needed
SELECT SERVERPROPERTY('ProductMajorVersion'); -- 17 = SQL Server 2025

-- Create a product table with an embedding column
CREATE TABLE dbo.Products (
    ProductId INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_Products PRIMARY KEY,
    Sku        NVARCHAR(50) NOT NULL UNIQUE,
    Name       NVARCHAR(200) NOT NULL,
    Description NVARCHAR(MAX) NULL,
    Category   NVARCHAR(50) NULL,
    Embedding  VECTOR(1536) NULL, -- OpenAI text-embedding-3-small dimensions
    CreatedAt  DATETIME2(0) NOT NULL CONSTRAINT DF_Products_CreatedAt DEFAULT
SYSUTCDATETIME()
);

-- Insert a row with a pre-computed embedding (abbreviated here)
-- In production, embeddings are populated by an application-layer pipeline
INSERT INTO dbo.Products (Sku, Name, Description, Category, Embedding)
VALUES (
    'NK-RUN-001',
    'Nike Air Zoom Pegasus 40',
    'Cushioned daily trainer for long-distance running',
    'running',
    CAST('[0.012, -0.043, 0.071, ...]' AS VECTOR(1536)) -- 1536 values in practice
);

-- DiskANN index for approximate nearest neighbor search (preview)
CREATE VECTOR INDEX IX_Products_Embedding
ON dbo.Products (Embedding)
WITH (METRIC = 'cosine', TYPE = 'DiskANN');

-- Semantic similarity search: 10 most similar products to a query vector
-- @q is the query embedding generated at the application layer
DECLARE @q VECTOR(1536) = '[0.012, -0.043, 0.071, ...]';

SELECT TOP (10)
    ProductId, Sku, Name, Category,
    VECTOR_DISTANCE('cosine', Embedding, @q) AS Distance
FROM dbo.Products
WHERE Category = 'running' -- relational pre-filter, same query
ORDER BY Distance;
```

The `VECTOR_DISTANCE` function takes the metric as its first argument: `'cosine'`, `'euclidean'`, or `'dot'` (negative dot product). The choice depends on your embedding model's normalization properties — cosine is the right default for normalized text embeddings.

A few things to notice in this schema. The pre-filter `WHERE Category = 'running'` combines with vector ranking in a single T-SQL statement planned by the regular query optimizer — relational predicates and similarity ranking compose naturally, which is the core advantage of keeping vectors inside the relational engine. The `VECTOR(1536)` type enforces dimensionality at write time: storing a 384-dimension vector in this column fails loudly instead of silently corrupting search quality.

When SQL Server native vectors are enough: Up to millions of vectors, single-server or Azure SQL deployments, workloads where vector search is one feature among many in a relational application, and teams that want to stay in the SQL Server operational model — transactions, row-level security, and backups included.

When to consider a dedicated vector database: Tens of millions of vectors with aggressive latency SLAs, high-QPS requirements (thousands of queries per second), complex metadata filtering at scale, SaaS-scale multi-tenant isolation, or first-class hybrid dense-plus-sparse search requirements.

The Ecosystem Map: SQL Server Vectors, Pinecone, Weaviate, and When to Use Each

The vector database market has grown rapidly, and the naming is not always consistent. Some are databases built specifically for vectors. Others are cloud services that abstract away the infrastructure entirely. Still others are the relational database you already run, with vectors added as a native type. A practical map of the landscape will ground the rest of this book.

SQL Server 2025 native vectors — built into the engine. Already discussed. Best for: teams already on SQL Server who want to add semantic search without new infrastructure. Also generally available on Azure SQL Database and Azure SQL Managed Instance.

Pinecone — Fully managed, cloud-native vector database. No infrastructure to operate. Supports hybrid search (dense + sparse), namespace-based multi-tenancy, and serverless tiers. Best for: teams that want production-grade vector search without operational overhead, and can accept SaaS pricing and data residency constraints.

Weaviate — Open-source vector database with GraphQL and REST APIs. Runs on-premises or in the cloud. Has built-in support for multiple modalities (text, image, audio) and an integrated inference API. Best for: teams needing a full object store with built-in vectorization, not just an index alongside another database.

Qdrant — Open-source, written in Rust. Excellent performance-per-dollar profile. Strong support for payload filtering with native scalar indexes that interact correctly with HNSW (solving the pre-filter problem mentioned above). Best for: self-hosted deployments where performance and operational simplicity matter.

Chroma — Lightweight, embedded-first vector store. Runs in-process with Python applications. Best for: prototyping, local development, and applications that do not need persistence beyond a single process.

Milvus — Open-source, cloud-native vector database designed for horizontal scaling. Kubernetes-native architecture with separate storage, query, and index layers. Best for: large-scale deployments requiring distributed operation and fine-grained resource management.

Here is a minimal example showing the same semantic search operation across two systems — SQL Server native vectors and Pinecone — so you can see how the abstractions differ:

```
# Python example – semantic search with Pinecone
from pinecone import Pinecone
from openai import OpenAI

openai_client = OpenAI()
pc = Pinecone(api_key="YOUR_PINECONE_API_KEY")

index = pc.Index("products-index") # must already exist

# Generate query embedding
query_text = "comfortable sneakers for long runs"
query_response = openai_client.embeddings.create(
    input=query_text,
    model="text-embedding-3-small"
)
query_vector = query_response.data[0].embedding

# Search with metadata filter – Pinecone handles pre-filtering correctly
results = index.query(
    vector=query_vector,
    top_k=10,
    filter={"category": {"$eq": "running"}}, # metadata pre-filter
    include_metadata=True
)

for match in results["matches"]:
    print(f"Score: {match['score']:.4f} | SKU: {match['metadata']['sku']} | {match['metadata']['name']}")
```

The Pinecone `filter` parameter is processed as a true pre-filter against the index, not a post-filter on ANN results. This is one of the concrete operational differences between a relational engine and a purpose-built vector system. At low vector counts the distinction is academic. At millions of vectors with selective filters it is the difference between 95% recall and 60% recall — and the reason SQL Server's single-query hybrid of relational predicates plus vector ranking matters.

The right tool for your workload depends on scale, operational constraints, data residency requirements, and where vectors fit in your overall data architecture. This book covers all of them — the goal is informed selection, not advocacy for any single system.

Key Takeaways

- → An embedding is a fixed-length array of floating-point numbers that represents the semantic content of text, images, or other data as a point in high-dimensional space. Two semantically similar inputs produce vectors that are geometrically close, regardless of surface-level similarity in the original content.
- → Traditional B-tree and hash indexes cannot support similarity search because vector similarity has no total ordering. Approximate nearest neighbor (ANN) algorithms — DiskANN in SQL Server 2025 — solve this problem by trading exact results for dramatically lower query latency.
- → SQL Server 2025 native vectors bring vector storage and ANN search into the relational engine, making them the natural starting point for SQL Server teams. They are well-suited for datasets into the millions of vectors, with hybrid relational-plus-vector queries as the defining strength — and the DiskANN index still in preview, worth tracking as it matures.
- → The embedding model you choose determines storage cost, query performance, and which distance metric is appropriate. All vectors in a collection must come from the same model, making embedding model upgrades a significant data migration event that must be planned for from the start.
- → The vector database ecosystem — SQL Server native vectors, Pinecone, Weaviate, Qdrant, Chroma, Milvus — spans a wide range of operational models from embedded libraries to fully managed cloud services. Selection criteria include dataset size, query-per-second requirements, metadata filtering complexity, and whether you need to own the infrastructure or prefer a managed service.

Chapter 2: How Vector Search Works: ANN vs Exact Search

Every vector query in SQL Server 2025 takes one of two paths: exact search, which measures the distance from your query vector to every stored vector and returns the true nearest neighbors, or approximate search, which navigates a DiskANN graph index to find nearly-the-nearest neighbors at a fraction of the cost. The two paths differ in guarantees, scaling behavior, operational overhead, and the T-SQL you write — and choosing the wrong one is the single most common performance mistake in production vector workloads. This chapter builds your intuition for both, with the numbers that determine where each one wins.

Exact Search: VECTOR_DISTANCE and the Full Scan

Exact search is the honest baseline: compute `VECTOR_DISTANCE` between the query vector and every row, sort, take the top k. No index to build, no parameters to tune, and — by construction — 100% recall. The true nearest neighbors are always returned because every candidate was measured.

```
-- SQL Server 2025: exact k-NN search, 100% recall, no index required
DECLARE @q VECTOR(1536) = '[0.12, 0.87, 0.34, ...]'; -- query embedding

-- Cosine distance ranking
SELECT TOP (10)
    DocumentId, Title,
    VECTOR_DISTANCE('cosine', Embedding, @q) AS Distance
FROM dbo.RagChunks
WHERE Embedding IS NOT NULL
ORDER BY Distance;

-- Euclidean variant: change only the metric argument
SELECT TOP (10) DocumentId,
    VECTOR_DISTANCE('euclidean', Embedding, @q) AS L2Distance
FROM dbo.RagChunks
WHERE Embedding IS NOT NULL
ORDER BY L2Distance;

-- Dot product variant (negative dot product: smaller = more similar)
SELECT TOP (10) DocumentId,
    VECTOR_DISTANCE('dot', Embedding, @q) AS NegDotProduct
FROM dbo.RagChunks
WHERE Embedding IS NOT NULL
ORDER BY NegDotProduct;
```

When exact search is the right choice:

- → **Small to medium collections.** Under roughly 50,000 vectors of 1536 dimensions, exact search on modern hardware typically completes in tens of milliseconds — fast enough for interactive use with zero index maintenance.
- → **Recall is non-negotiable.** Compliance search, legal discovery, and any workload where a missed neighbor has consequences beyond a slightly worse recommendation.
- → **Ground-truth measurement.** Exact search is how you compute recall for the approximate path — it is the ruler everything else is measured against (Chapter 9).
- → **Prototypes and new tables.** Start every new vector table on exact search. Add the DiskANN index when measurements — not anxiety — say you need it.

The Scaling Wall: Why Exact Search Stops Working

The performance ceiling of exact search is arithmetic, not engineering. Each query computes distance to every vector: for 1 million vectors of 1536 dimensions, that is about 1.5 billion floating-point multiply-add operations. With SIMD acceleration that runs in the low hundreds of milliseconds on a modern CPU — acceptable for a batch job, too slow for a user-facing search box. At 10 million vectors you are past a second per query before I/O, and the cost grows linearly with every row you add. No index trick short of approximation changes this; the math is the math.

This is also a CPU story, not just a latency story. Ten concurrent exact-search queries over a million-vector table each burn a core computing distances. Under load, exact search does not degrade gracefully — it saturates CPU and everything on the instance feels it. When `sys.dm_exec_query_stats` shows your retrieval procedure dominated by worker time rather than waits, and the corpus keeps growing, you have found the wall. The way through it is approximation.

Approximate Search: The DiskANN Graph Index

Approximate nearest neighbor (ANN) search trades a small, measurable amount of recall for dramatic speed. SQL Server 2025's approximate path is a DiskANN index — a navigable graph built over the vector collection where each vector is a node connected to its nearest neighbors. A query enters at a fixed entry point and greedily walks toward the query vector, touching only a tiny fraction of the collection. Query cost grows logarithmically with data size instead of linearly: going from 1 million to 10 million vectors barely moves latency.

```
-- SQL Server 2025: DiskANN approximate index (preview) + VECTOR_SEARCH
CREATE VECTOR INDEX IX_RagChunks_Embedding
ON dbo.RagChunks (Embedding)
WITH (METRIC = 'cosine', TYPE = 'DiskANN', MAXDOP = 4);

DECLARE @q VECTOR(1536) = '[0.12, 0.87, 0.34, ...]';

-- Approximate k-NN: graph walk instead of full scan
SELECT t.DocumentId, t.Title, s.distance
FROM VECTOR_SEARCH(
    TABLE = dbo.RagChunks AS t,
    COLUMN = Embedding,
    SIMILAR_TO = @q,
    METRIC = 'cosine',
    TOP_N = 10
) AS s
ORDER BY s.distance;
```

Recall is the metric that makes "approximate" respectable: the fraction of the true top-k neighbors the index actually returns. A DiskANN index delivering 0.97 recall@10 finds 97 of the 100 true nearest neighbors in milliseconds instead of finding all 100 in over a second. Since embeddings are themselves approximate representations of meaning, that last few percent of geometric precision almost never changes what the user experiences — but you must measure it (Chapter 9 gives the spot-check pattern, Chapter 14 makes it a scheduled SLI).

What DiskANN costs you: build time (minutes to hours for very large collections, scheduled in maintenance windows), the preview write-path constraint (drop and recreate the index to ingest new data — Chapter 10's staging-swap pattern), and memory competition between the graph and the buffer pool. What it buys you: flat latency into the millions of vectors.

```
graph TD
    A[Vector query] --> B{Corpus size?}
    B -->|Under ~50K| C[Exact: VECTOR_DISTANCE]
    B -->|Over ~50K| D[Approximate: DiskANN]
    C --> E[No index to build or maintain]
    D --> F[CREATE VECTOR INDEX]
    F --> G[measure recall@k]
```

HNSW in Dedicated Engines: The Same Idea, Different Home

You will encounter a second graph algorithm constantly in this book's later chapters: **HNSW** (Hierarchical Navigable Small World), the index inside Qdrant, Weaviate, Milvus, and Pinecone's serving layer. HNSW builds a layered graph — sparse long-range connections at the top, dense short-range connections at the bottom — and descends from coarse to fine. It delivers the same $O(\log n)$ search with 95–99% recall as DiskANN.

The practical difference is memory architecture. HNSW keeps its entire graph in RAM, which makes it extremely fast and extremely memory-hungry at billion-vector scale. DiskANN was designed for SSDs: most of the graph and the full-precision vectors live on disk, with only a working set in memory, issuing a small bounded number of random reads per query. That design maps naturally onto SQL Server's buffer-pool-and-storage architecture — which is why Microsoft chose it for the in-engine index. When you read vendor benchmarks comparing HNSW engines against SQL Server's vector index, remember you are comparing two legitimate answers to the same problem, optimized for different resource profiles.

Filtered Search: Combining Predicates with Similarity

Real queries are rarely pure similarity — they combine "similar to this" with "and in this category, published, visible to this tenant." SQL Server's answer is the ordinary relational engine: predicates and vector ranking compose in one query, planned by the regular optimizer.

```

-- Filtered vector search: relational predicate + similarity in one plan
DECLARE @q VECTOR(1536) = '[0.12, 0.87, 0.34, ...]';
DECLARE @tenantId INT = 42;

-- Exact path: the predicate narrows the scan before ranking
SELECT TOP (10) ChunkId, ChunkText,
       VECTOR_DISTANCE('cosine', Embedding, @q) AS Distance
FROM dbo.TenantChunks
WHERE TenantId = @tenantId
      AND Embedding IS NOT NULL
ORDER BY Distance;

-- Approximate path: over-fetch, because filters consume candidates
SELECT TOP (10) c.ChunkId, c.ChunkText, s.distance
FROM VECTOR_SEARCH(
    TABLE = dbo.TenantChunks AS t,
    COLUMN = Embedding,
    SIMILAR_TO = @q,
    METRIC = 'cosine',
    TOP_N = 50 -- over-fetch 5x, then filter down
) AS s
JOIN dbo.TenantChunks AS c ON c.ChunkId = s.ChunkId
WHERE c.TenantId = @tenantId
ORDER BY s.distance;

```

The over-fetch pattern in the approximate query deserves emphasis because it is the most common recall bug in production: if you ask the index for exactly 10 candidates and the tenant filter discards 6, you return 4 results — not because only 4 were relevant, but because the filter ate your candidate budget. Ask for 50, filter, then trim to 10. For highly selective filters over huge shared indexes, consider the tenancy patterns in Chapter 12 (schema-per-tenant gives each tenant a dedicated DiskANN index with no filtering penalty at all).

Putting It Together: The Decision You Revisit

Start every vector table on exact search. It is correct, simple, and fast enough until it isn't. Promote to the DiskANN index when production measurements show the corpus crossing ~50K vectors with latency pressure — or when `sys.dm_exec_query_stats` shows retrieval CPU dominating the instance. When you promote, measure recall@k against exact ground truth before and after, and keep measuring it on a schedule (Chapter 14). The decision is not permanent and not precious: the embedding model, the chunking strategy, and the retrieval procedure shape stay the same — only the access path changes, exactly like adding a B-tree index to a table that outgrew its scan.

Key Takeaways

- → Exact search (`VECTOR_DISTANCE` with `ORDER BY`) computes distance to every vector: 100% recall, no index, $O(n)$ cost per query — the right default under roughly 50,000 vectors and the ground truth everything else is measured against.
- → Approximate search (DiskANN index via `CREATE VECTOR INDEX` , queried with `VECTOR_SEARCH`) walks a navigable graph for $O(\log n)$ queries at 95–99% recall — the answer when exact search hits its arithmetic scaling wall.
- → DiskANN is designed for SSD-backed operation (small memory working set, bounded random reads); HNSW, used by Qdrant, Weaviate, and Milvus, keeps its graph in RAM. Same problem, different resource profiles — compare them on your workload, not on marketing.
- → Filtered vector search composes relational predicates with similarity ranking in one T-SQL statement; on the approximate path, over-fetch `TOP_N` well above your display count so filters do not eat your candidate budget.
- → Promote from exact to approximate based on production measurements (corpus size, retrieval CPU in `sys.dm_exec_query_stats`), measure recall@k before and after, and keep measuring it — the access path is allowed to change as the data grows.

Chapter 3: Distance Metrics: Cosine, Euclidean, Dot Product

Every vector search query asks the same fundamental question: how similar is this vector to that one? The answer depends entirely on how you measure similarity — and that choice is not arbitrary. The three dominant distance metrics in production vector systems — cosine similarity, Euclidean distance, and dot product — each make different mathematical assumptions about what "closeness" means, and each is optimized for different embedding types, storage architectures, and query patterns. Getting this wrong means your semantic search returns irrelevant results, your recommendation engine surfaces stale content, and your RAG pipeline retrieves the wrong context — all while your vector index silently operates on numbers that look reasonable but mean nothing useful. This chapter gives you a precise, production-grounded understanding of all three metrics: the math behind them, how they behave with real embeddings, and how to configure them correctly in SQL Server 2025, Pinecone, Weaviate, and Qdrant.

Cosine: Angle, Not Length

Cosine similarity measures the angle between two vectors, ignoring their lengths. Two documents that say the same thing in different amounts of detail point in the same direction — cosine treats them as similar. This is why cosine is the default metric for text embeddings: most text models produce vectors whose magnitude carries little semantic meaning, and discarding it removes noise.

In SQL Server, cosine distance is a first argument to `VECTOR_DISTANCE` and the `METRIC` of your vector index. Note the convention: `VECTOR_DISTANCE` returns **distance**, where 0 means identical and smaller is more similar. Cosine similarity, the thing humans talk about, is $1 - \text{distance}$, ranging from 1 (identical direction) through 0 (orthogonal) to -1 (opposite).

```
-- SQL Server 2025: cosine distance for text embeddings
CREATE TABLE dbo.Documents (
    DocId      INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_Documents PRIMARY KEY,
    Content    NVARCHAR(MAX) NOT NULL,
    SourceUrl  NVARCHAR(500) NULL,
    Embedding  VECTOR(1536) NULL      -- e.g. text-embedding-3-small
);

-- DiskANN index with cosine metric
CREATE VECTOR INDEX IX_Documents_Embedding
ON dbo.Documents (Embedding)
WITH (METRIC = 'cosine', TYPE = 'DiskANN', MAXDOP = 4);

-- Semantic search: TOP 5 by cosine distance
DECLARE @q VECTOR(1536) = '[0.12, 0.87, 0.34, ...]';

SELECT TOP (5)
    DocId, Content, SourceUrl,
    1 - VECTOR_DISTANCE('cosine', Embedding, @q) AS CosineSimilarity
FROM dbo.Documents
WHERE Embedding IS NOT NULL
ORDER BY VECTOR_DISTANCE('cosine', Embedding, @q);
```

Cosine is the correct default when: your embedding model outputs normalized or near-normalized vectors (most modern text models do), document length varies, or you are doing semantic text search. Practically speaking, if your embeddings come from an OpenAI, Cohere, or sentence-transformer text model, start with cosine.

Euclidean (L2): Straight-Line Distance

Euclidean distance measures the straight-line distance between two points in vector space — the ruler measurement. Unlike cosine, it cares about magnitude. Two vectors pointing the same direction but with very different lengths are far apart in L2. This makes L2 the right metric when magnitude carries meaning: image embeddings where brightness and contrast encode into vector length, audio embeddings where energy matters, or scientific sensor data where the actual values are the point.

```
-- SQL Server 2025: L2 distance for image embeddings
CREATE TABLE dbo.ProductImages (
    ImageId INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_ProductImages PRIMARY KEY,
    ProductId UNIQUEIDENTIFIER NOT NULL,
    ImageUrl NVARCHAR(500) NOT NULL,
    Embedding VECTOR(512) NULL          -- e.g. CLIP ViT-B/32 output
);

-- Same index DDL, different metric
CREATE VECTOR INDEX IX_ProductImages_Embedding
ON dbo.ProductImages (Embedding)
WITH (METRIC = 'euclidean', TYPE = 'DiskANN', MAXDOP = 4);

-- Visual similarity: smaller L2 distance = more similar
DECLARE @qi VECTOR(512) = '[0.05, 0.41, ...]';

SELECT TOP (10)
    ImageId, ProductId, ImageUrl,
    VECTOR_DISTANCE('euclidean', Embedding, @qi) AS L2Distance
FROM dbo.ProductImages
WHERE Embedding IS NOT NULL
ORDER BY VECTOR_DISTANCE('euclidean', Embedding, @qi);
```

A subtle but important relationship: for unit-normalized vectors, cosine distance and L2 distance produce identical rankings — the two metrics are monotonically related. This means if you normalize all embeddings at ingest time, you can use either metric and get the same top-k results. If you do not normalize, the choice genuinely changes results. Pick deliberately and document the decision next to the ingest code.

Dot Product: Magnitude-Aware Similarity

Dot product combines direction and magnitude: large dot products mean vectors point the same way *and* are long. Recommendation systems love it because magnitude often encodes a signal the model learned — popular or high-quality items get longer vectors in two-tower models, and dot product ranking surfaces them naturally. Dense Passage Retrieval (DPR) and many recommender architectures are trained with dot product as the scoring function, and switching metrics at serving time breaks the ranking the model learned.

```

-- SQL Server 2025: dot product for recommendation embeddings
-- Item embeddings are NOT normalized here – magnitude encodes the quality signal
CREATE TABLE dbo.ItemEmbeddings (
    ItemId      BIGINT NOT NULL CONSTRAINT PK_ItemEmbeddings PRIMARY KEY,
    ItemType    NVARCHAR(20) NOT NULL,    -- 'article', 'product', 'video'
    Embedding   VECTOR(256) NULL         -- two-tower model, 256-dim item tower
);

CREATE VECTOR INDEX IX_ItemEmbeddings_Embedding
ON dbo.ItemEmbeddings (Embedding)
WITH (METRIC = 'dot', TYPE = 'DiskANN', MAXDOP = 4);

-- VECTOR_DISTANCE('dot', ...) returns the NEGATIVE dot product:
-- ORDER BY ascending retrieves the highest dot product first
DECLARE @user VECTOR(256) = '[0.21, 0.03, ...]';

SELECT TOP (10)
    ItemId, ItemType,
    -VECTOR_DISTANCE('dot', Embedding, @user) AS DotProductScore
FROM dbo.ItemEmbeddings
WHERE ItemType = 'article'
    AND Embedding IS NOT NULL
ORDER BY VECTOR_DISTANCE('dot', Embedding, @user);    -- ascending = most similar

```

The sign convention is the classic source of dot-product bugs: `'dot'` returns the negative dot product so the "smaller distance = more similar" convention holds across all three metrics. Negate it in the SELECT list for a human-readable score, but always `ORDER BY` the raw distance ascending. Mixing the two — sorting by the negated score ascending — silently inverts your recommendations, and the results look plausible enough that nobody notices for weeks.

The Metric Must Match Everywhere: Index, Query, and Model

Here is the production rule that prevents the worst metric bugs: the metric declared in `CREATE VECTOR INDEX ... WITH (METRIC = ...)`, the metric argument in `VECTOR_DISTANCE` and `VECTOR_SEARCH`, and the metric your embedding model was trained with must all agree. If the index was built with `'cosine'` and your query asks for `'euclidean'`, the optimizer cannot use the index for ranking — it falls back to a full scan, and latency jumps from milliseconds to seconds with no error, no warning. The query is "correct" in the sense that it returns rows, and silently slow.

```
-- Index built for cosine ...
CREATE VECTOR INDEX IX_Documents_Embedding
ON dbo.Documents (Embedding)
WITH (METRIC = 'cosine', TYPE = 'DiskANN');

-- ... then someone queries with euclidean: index NOT used, full scan
DECLARE @q VECTOR(1536) = '[0.12, 0.87, ...]';
SELECT TOP (10) DocId,
       VECTOR_DISTANCE('euclidean', Embedding, @q) AS D    -- metric mismatch!
FROM dbo.Documents
ORDER BY D;

-- Confirm which path the plan takes (Chapter 14 covers vector plan analysis):
-- check sys.dm_exec_query_stats for CPU regression after any metric change
```

Encode the metric in infrastructure-as-code alongside the index definition, and treat any metric change as a schema migration: it means dropping and rebuilding the vector index (and possibly re-embedding with a different model). The same discipline applies across systems with different names for the same math — Pinecone calls it `dotproduct`, Qdrant calls it `DOT`, Weaviate uses `L2_SQUARED` for Euclidean. Build a reference table for your team so nobody has to guess across a system boundary.

Normalization: Decide It at Ingest Time

A decision that quietly interacts with everything above: whether vectors are L2-normalized before storage. SQL Server has no built-in normalize function for vectors, so normalization happens in your ingest pipeline (a one-liner in numpy or in the embedding client). Normalizing at ingest makes cosine and dot product rankings equivalent, makes Euclidean rankings equal to cosine rankings, and lets you store one representation that serves all three metrics. The cost is losing magnitude information — exactly the signal dot-product recommenders rely on. So the rule is simple: normalize at ingest for search and RAG workloads; keep raw magnitudes for recommender and retrieval models trained on dot product.

If you normalize at ingest, switching metrics later only requires rebuilding the index with the new `METRIC` — the stored data is already compatible. If you stored raw vectors, switching from dot product to cosine changes rankings and demands revalidation of your evaluation set. This is why the metric decision belongs in the same design review as chunking and the embedding model, not as a query-time afterthought.

```

graph TD
    A[Choose your metric] --> B{Embedding type?}
    B -->|Text / normalized| C[cosine]
    VECTOR_DISTANCE 'cosine'
    B -->|Image / audio / sensor| D[euclidean]
    VECTOR_DISTANCE 'euclidean'
    B -->|Recommender / DPR| E[dot]
    VECTOR_DISTANCE 'dot'
    keep raw magnitudes
    C --> F[METRIC = 'cosine']
    in CREATE VECTOR INDEX
    D --> G[METRIC = 'euclidean']
    in CREATE VECTOR INDEX
    E --> H[METRIC = 'dot']
    in CREATE VECTOR INDEX
    F --> I[Query with the same metric string]
    or the index silently goes unused

```

Evaluating Metrics on Your Data, Not on Intuition

When two metrics both seem plausible — cosine vs dot product for a new recommender, L2 vs cosine for image search — decide empirically. The evaluation harness pattern is the same regardless of system: a few hundred labeled query-document pairs, a recall@k or precision@k computation per metric, and the winner documented in the runbook.

```

# Python – comparing metrics offline before committing to an index config
import numpy as np

def recall_at_k(query_emb, doc_embs, relevant_ids, k=10, sim_fn=None):
    scores = sim_fn(query_emb, doc_embs)
    top_k = set(np.argsort(scores)[-k:]) # highest scores = most similar
    return len(top_k & relevant_ids) / len(relevant_ids)

def cosine_sim(q, docs):
    q = q / np.linalg.norm(q)
    docs = docs / np.linalg.norm(docs, axis=1, keepdims=True)
    return docs @ q

def dot_sim(q, docs):
    return docs @ q

def neg_l2(q, docs): # negate so higher = more similar
    return -np.linalg.norm(docs - q, axis=1)

# Run all three against your labeled set, pick the metric with the best
# recall@10, then build the DiskANN index with that METRIC value

```

Run this against a representative slice of your production query distribution — at least a few hundred labeled pairs — before committing to an index configuration. Rebuilding a DiskANN index with a different metric is non-trivial once millions of vectors are stored, and if the metric change also implies a different normalization policy, the re-ingest cost dwarfs the index rebuild.

Key Takeaways

- → Cosine ignores vector length and suits text embeddings; Euclidean measures straight-line distance for images, audio, and sensor data; dot product preserves magnitude signals for recommenders and DPR. Choose based on what the embedding model learned, not on habit.
- → In SQL Server, the metric is a string — `'cosine'`, `'euclidean'`, `'dot'` — used in `VECTOR_DISTANCE`, `VECTOR_SEARCH`, and `CREATE VECTOR INDEX ... WITH (METRIC = ...)`. `'dot'` returns the negative dot product: negate for display, sort ascending.
- → The index metric, the query metric, and the model's training metric must all match — a mismatch silently disables the DiskANN index and drops you to a full scan. Treat metric changes as schema migrations and encode them in infrastructure-as-code.
- → Decide normalization at ingest time: normalized vectors make all three metrics rank identically and keep your options open; raw magnitudes are required when the model uses magnitude as a signal. SQL Server has no built-in normalize function — do it in the ingest pipeline.
- → When two metrics both seem plausible, evaluate empirically with recall@k on labeled pairs before committing. Rebuilding a DiskANN index with a different metric after millions of vectors are stored is expensive — deciding up front is cheap.

Chapter 4: The Vector Database Landscape: SQL Server Vectors, Pinecone, Weaviate, Qdrant, Chroma

The vector database market has grown from a niche corner of machine-learning infrastructure into a crowded field of managed services, open-source engines, and — most importantly for this book — native support inside the relational databases you already operate. That last development changed the selection calculus more than any single product launch. For years the default advice was: keep your operational data in SQL Server, ship embeddings to a specialized vector store, and synchronize the two. SQL Server 2025 makes "keep everything in one engine" a legitimate architecture for a large share of workloads, and the rest of this chapter maps the landscape with that option as the center of gravity.

The naming in this market is not always consistent. Some products are databases built specifically for vectors. Others are cloud services that abstract away the infrastructure entirely. And one — SQL Server 2025 — is the database you already know, with vectors added as a first-class data type. A practical map of the landscape will ground every infrastructure decision in the rest of this book.

SQL Server 2025 Native Vectors: The Relational Anchor

SQL Server 2025 introduced a native `VECTOR(n)` data type — a first-class column type supporting 1 to 1998 dimensions, stored in an optimized binary format and exposed as JSON. There is no extension to install, no separate process to run, and no new operational model to learn. If you can create a table and an index, you can store and search vectors.

Similarity search comes in two flavors. For exact nearest-neighbor search, the built-in `VECTOR_DISTANCE` function computes cosine, Euclidean, or dot-product distance between two vectors directly in T-SQL:


```
-- SQL Server 2025: native VECTOR type and exact similarity search
CREATE TABLE dbo.Products (
    ProductId INT NOT NULL CONSTRAINT PK_Products PRIMARY KEY,
    Sku        NVARCHAR(50) NOT NULL UNIQUE,
    Name       NVARCHAR(200) NOT NULL,
    Category   NVARCHAR(50) NULL,
    Embedding  VECTOR(1536) NULL    -- OpenAI text-embedding-3-small dimensions
);

DECLARE @q VECTOR(1536) = '[0.012, -0.043, 0.071, ...]'; -- query embedding

-- Exact k-NN: computes distance to every row, 100% recall
SELECT TOP (10)
    ProductId, Sku, Name,
    VECTOR_DISTANCE('cosine', Embedding, @q) AS Distance
FROM dbo.Products
WHERE Category = 'running'           -- relational pre-filter, same query
ORDER BY Distance;
```

For larger collections, `CREATE VECTOR INDEX` builds an approximate nearest-neighbor index using the DiskANN algorithm — a graph-based index designed for SSD-backed, billion-scale vector search — and the `VECTOR_SEARCH` table-valued function queries it:

```
-- SQL Server 2025: DiskANN approximate index + VECTOR_SEARCH
CREATE VECTOR INDEX IX_Products_Embedding
ON dbo.Products (Embedding)
WITH (METRIC = 'cosine', TYPE = 'DiskANN');

DECLARE @q VECTOR(1536) = '[0.012, -0.043, 0.071, ...]';

-- Approximate k-NN through the vector index
SELECT t.ProductId, t.Name, s.distance
FROM VECTOR_SEARCH(
    TABLE      = dbo.Products AS t,
    COLUMN      = Embedding,
    SIMILAR_TO  = @q,
    METRIC      = 'cosine',
    TOP_N       = 10
) AS s
ORDER BY s.distance;
```

The killer feature is the hybrid query in the first example: a relational predicate (`WHERE Category = 'running'`) combined with vector ranking in a single T-SQL statement, planned by the same query optimizer that handles the rest of your workload. No synchronization pipeline between two systems, no second set of credentials, no separate backup story. Native vector support is also generally available in Azure SQL Database and Azure SQL Managed Instance, so the same code runs in the cloud with no changes.

Best for: teams already running SQL Server who want semantic search, recommendations, or RAG retrieval alongside their operational data; workloads up to millions of vectors; anyone who values transactions, joins, row-level security, and a single backup story over squeezing the last millisecond of ANN latency.

Pinecone: The Fully Managed Option

Pinecone is a fully managed, cloud-native vector database — no infrastructure to operate, no indexes to tune. You create an index, upsert vectors with metadata, and query. It supports hybrid dense-plus-sparse search, namespace-based multi-tenancy (each tenant gets a logical partition inside one index), and serverless tiers that scale to zero. Filtering on metadata is a true pre-filter against the index, which matters for selective predicates at scale.

The tradeoff is the classic SaaS bargain: you trade operational burden for pricing that scales with usage and data-residency constraints you do not control. There is also an architectural cost that teams underestimate — your embeddings now live in a second system. Every product row in SQL Server needs a corresponding vector in Pinecone, and the synchronization between the two (on insert, update, delete) is your code to write, monitor, and repair when it drifts.

```
# Python example – semantic search with Pinecone
from pinecone import Pinecone
from openai import OpenAI

openai_client = OpenAI()
pc = Pinecone(api_key="YOUR_PINECONE_API_KEY")
index = pc.Index("products-index") # must already exist

query_vector = openai_client.embeddings.create(
    input="comfortable sneakers for long runs",
    model="text-embedding-3-small"
).data[0].embedding

results = index.query(
    vector=query_vector,
    top_k=10,
    filter={"category": {"$eq": "running"}}, # true pre-filter on metadata
    include_metadata=True
)
for match in results["matches"]:
    print(f"Score: {match['score']:.4f} | SKU: {match['metadata']['sku']}")
```

Best for: teams that want production-grade vector search with zero operational overhead, can accept SaaS pricing and data-residency constraints, and have the engineering discipline to keep two systems synchronized.

Weaviate: The Open-Source Object Store

Weaviate is an open-source vector database with GraphQL and REST APIs that stores full objects — not just vectors — alongside their embeddings. It has built-in support for multiple modalities (text, image, audio) through its module system and can generate vectors itself via integrated inference modules, so your application can send raw text and let Weaviate handle the embedding call. Its hybrid search combines BM25 keyword scoring with vector similarity in a single query.

Weaviate runs on-premises or in the cloud, including a managed cloud offering. It is the right shape when you need a document store with vectorization baked in rather than an index bolted onto another database. The operational surface is real, though: you are running a JVM-based distributed system with its own backup, monitoring, and upgrade story — separate from your SQL Server estate.

Best for: teams needing a full object store with built-in vectorization and multi-modal support, who are comfortable operating (or paying for) a second database platform.

Qdrant: Performance-Per-Dollar, Self-Hosted

Qdrant is an open-source vector database written in Rust with an excellent performance-per-dollar profile. Its standout feature is payload filtering: scalar metadata indexes that interact correctly with the underlying HNSW graph, so filtered vector queries maintain high recall without the post-filter degradation that plagues naive implementations. It also supports quantization (scalar, product, and binary) to shrink memory footprint with modest recall cost, and offers both self-hosted and managed cloud deployments.

For a SQL Server shop, Qdrant is the strongest "dedicated engine, self-hosted" candidate: a single static binary, a straightforward HTTP API, and honest, well-documented performance characteristics. You still own the synchronization problem with your relational data, but the operational burden is lighter than JVM-based alternatives.

Best for: self-hosted deployments where filtered vector search performance and operational simplicity matter more than managed-service convenience.

Chroma: The Embedded Prototyping Store

Chroma is a lightweight, embedded-first vector store that runs in-process with Python applications. It is the fastest path from zero to a working semantic-search prototype: pip install, add documents, query. It supports persistent local storage and a client-server mode for small teams, but it is not designed for high-throughput production workloads or multi-tenant isolation at scale.

Chroma's role in a SQL Server architecture is usually temporary — prototype the retrieval logic locally, then promote the schema and queries to SQL Server 2025 native vectors or a dedicated engine for production. Treat it as a sketchpad, not a foundation.

Best for: prototyping, local development, and single-process applications that do not need production durability or scale.

Reading the Map: A Selection Framework

With the pieces on the table, the decision reduces to a handful of questions. Answer them in order and the right system usually selects itself.

1. Where does the vector query need to join relational data? If your retrieval query naturally combines semantic ranking with relational predicates — tenant filters, category constraints, permission checks, joins to orders or inventory — SQL Server 2025 native vectors win on simplicity. Every dedicated engine can store metadata and filter on it, but none of them joins to your `Orders` table. Keeping vectors next to the data they describe eliminates an entire class of consistency bugs.

2. What scale are you actually at? Under roughly 50,000 vectors, SQL Server's exact `VECTOR_DISTANCE` search is fast enough that an index is optional. Into the millions, the DiskANN vector index keeps latency flat. Past tens of millions of vectors with high query-per-second demands and aggressive recall targets, the dedicated engines' specialized storage layers start to justify their operational cost. Be honest about which bracket you are in today versus in three years — most teams overestimate.

3. Who operates it? SQL Server native vectors are operated by the DBA team you already have, with the backup, monitoring, and security tooling you already own. Pinecone is operated by Pinecone. Weaviate, Qdrant, and Milvus are operated by whoever you assign —

and "whoever" needs to learn a new system. Never underestimate the cost of a second operational model: on-call rotations, upgrade runbooks, and incident muscle memory all double.

4. What are the data-residency and compliance constraints? Embeddings derived from regulated data inherit its compliance requirements. Keeping vectors inside SQL Server means your existing TDE encryption, auditing, and row-level security policies apply unchanged. A SaaS vector database means a new data-processing agreement and a new entry in your compliance register. A self-hosted engine means you reimplement those controls yourself.

```
graph LR
    A[Vector workload] --> B{Joins relational data  
in the same query?}
    B -->|Yes| C[SQL Server 2025  
native VECTOR]
    B -->|No| D{Scale?}
    D -->|< millions| C
    D -->|10M+ vectors,  
high QPS| E{Who operates?}
    E -->|We do| F[Qdrant / Weaviate  
self-hosted]
    E -->|Vendor| G[Pinecone  
managed]
    G -->|Prototype| H[Chroma  
embedded]
```

One more consideration that cuts across all of these: **embedding model lock-in is independent of database choice**. Vectors from one model cannot be compared with vectors from another, so switching models means re-embedding your corpus regardless of where the vectors live. Choose the model carefully (Chapter 23), because it is the hardest thing in this stack to change.

Key Takeaways

- → SQL Server 2025 native vectors — the `VECTOR(n)` type, `VECTOR_DISTANCE`, DiskANN vector indexes, and `VECTOR_SEARCH` — make the relational engine itself a legitimate vector database for a large share of workloads, with hybrid relational-plus-vector queries as the killer feature.
- → Pinecone trades operational burden for SaaS pricing and a synchronization pipeline you must build between your relational data and the vector store. Choose it when you want zero infrastructure to operate and can accept the constraints.

- → Weaviate is an object store with vectorization baked in — the right shape when you need documents, multi-modal support, and built-in embedding generation rather than an index bolted onto another database.
- → Qdrant is the strongest self-hosted dedicated option, with correct pre-filtering against its HNSW graph and a light operational footprint. Chroma is a prototyping sketchpad, not a production foundation.
- → Select by answering four questions in order: does the query join relational data, what scale are you really at, who operates the system, and what do compliance constraints require. Most teams that answer honestly find SQL Server native vectors cover their needs.

Chapter 5: When to Use SQL Server Vectors vs a Dedicated Vector Database

Every team that adopts vector search eventually faces the same fork in the road: keep vectors inside SQL Server, or stand up a dedicated vector database alongside it. The wrong choice is expensive in both directions — a dedicated engine you did not need means a synchronization pipeline, a second operational model, and a second bill; vectors crammed into SQL Server past their sensible limits mean slow queries and frustrated users. This chapter gives you a decision framework grounded in how the two options actually behave: scale thresholds, filtering semantics, consistency guarantees, cost structure, and the organizational reality of who operates what.

The honest starting position is this: for most teams reading this book, SQL Server 2025 native vectors are enough. The dedicated engines exist for the workloads that genuinely outgrow the relational engine. Knowing exactly where that boundary lies — and how to recognize when you are approaching it — is the whole chapter.

What SQL Server Native Vectors Actually Give You

Before comparing, it helps to be precise about the SQL Server side of the comparison. SQL Server 2025 ships a native `VECTOR(n)` data type (1 to 1998 dimensions), the `VECTOR_DISTANCE` function for exact similarity search with cosine, Euclidean, and dot-product metrics, and — in preview — a DiskANN-based approximate vector index created with `CREATE VECTOR INDEX` and queried through the `VECTOR_SEARCH` table-valued function. Native vector support is generally available in Azure SQL Database and Azure SQL Managed Instance as well.

What this buys you beyond the raw capability is everything SQL Server already does: ACID transactions around vector writes, joins between vector results and relational tables in a single query, row-level security applied to embedding rows, Transparent Data Encryption covering vectors at rest, auditing, and a backup story that already exists. None of the dedicated engines give you all of that; they give you a faster ANN index and ask you to rebuild the rest.

The practical scale envelope, based on Microsoft's guidance and production experience: exact search with `VECTOR_DISTANCE` performs well up to roughly 50,000 vectors. Beyond that, the DiskANN vector index keeps approximate search latency flat into the millions of vectors.

That covers an enormous fraction of real deployments — internal knowledge bases, product catalogs, support-ticket search, and departmental RAG systems rarely exceed low millions of chunks.

Where Dedicated Engines Pull Ahead

Dedicated vector databases — Pinecone, Weaviate, Qdrant, Milvus, covered in Chapters 15 through 19 — are purpose-built for one job: approximate nearest-neighbor search at extreme scale. They pull ahead of SQL Server native vectors in four situations.

Extreme scale with high query throughput. Past tens of millions of vectors, especially with hundreds or thousands of queries per second and strict p99 latency budgets, the dedicated engines' specialized storage layers, quantization options, and distributed architectures justify their cost. SQL Server's DiskANN index is strong, but it runs inside a general-purpose engine sharing buffer pool, CPU, and I/O with everything else on the instance.

Complex filtered search at scale. When queries combine vector similarity with highly selective metadata filters across millions of vectors, engines like Qdrant — with scalar payload indexes that interact correctly with the ANN graph — maintain recall where simpler implementations degrade. SQL Server handles pre-filtering through ordinary relational predicates, which works well at moderate scale but can force expensive plans when a selective filter meets a large vector collection.

Hybrid dense-plus-sparse retrieval as a first-class feature. If your retrieval strategy depends on tightly integrated keyword-plus-vector scoring with built-in reranking (Pinecone's hybrid search, Weaviate's BM25 fusion), the dedicated engines package that as a product feature. In SQL Server you assemble hybrid search yourself from full-text predicates and `VECTOR_DISTANCE` — entirely doable (Chapter 11), but it is your code to write and tune.

Multi-tenant isolation at SaaS scale. Pinecone namespaces or separate Qdrant collections per tenant give hard isolation boundaries with per-tenant performance characteristics. SQL Server can do multi-tenant vectors well (Chapter 12 covers `tenant_id` filtering, separate schemas, and row-level security), but at thousands of tenants with noisy-neighbor concerns, physical separation in a dedicated engine is simpler to reason about.

The Hidden Cost of Two Systems: Synchronization

The most underestimated cost of a dedicated vector database is not the license or the SaaS bill — it is the synchronization pipeline between your system of record and the vector store. Every write path that creates, updates, or deletes a document must now update two systems, in the right order, with failure handling for the cases where one succeeds and the other does not.

Concretely, adopting a dedicated engine means building and operating:

- → **An embedding pipeline** that converts source rows into vectors and upserts them — with retry logic, dead-letter handling, and backfill tooling for the initial load and model changes.
- → **Change data capture** from SQL Server (triggers, CDC, or application-level events) so updates and deletes propagate; a vector store that never hears about deletes serves stale results indefinitely.
- → **Drift detection and reconciliation** — a periodic job that compares row counts, checksums, or timestamps between the two systems and repairs divergence, because divergence is a matter of when, not if.
- → **A second security model** — credentials, network rules, encryption settings, audit logging, and access reviews for the vector store, mapped to the same identities and policies as SQL Server.
- → **A second backup and DR story** — coordinated restore points across two systems, tested together, because restoring one without the other reintroduces the drift you built the pipeline to prevent.

None of this is exotic engineering, but it is permanent engineering — it needs owners, runbooks, and on-call coverage for as long as both systems exist. When you keep vectors in SQL Server, every item on this list collapses to zero: the embedding column is just a column, covered by the transactions, backups, and security you already have. This is the strongest argument for native vectors, and it has nothing to do with benchmarks.

```

graph TD
    A[New vector workload] --> B{Vectors join relational data in queries?}
    B -->|Yes, regularly| C[SQL Server native vectors]
    B -->|Rarely / never| D{Corpus size?}
    D -->|Under ~5M vectors| C
    D -->|5M - 50M| E{QPS and latency SLA?}
    E -->|Modest| C
    E -->|Aggressive| F{Team capacity?}
    F -->|Can operate new system| G[Dedicated engine Qdrant / Pinecone]
    F -->|Stretched thin| C
    D -->|50M+| G

```

Cost Structure: Licenses, SaaS, and People

Cost comparisons between "free because we already own SQL Server" and a dedicated engine's bill need to account for all three cost centers: infrastructure, software, and people.

Infrastructure. SQL Server native vectors consume the resources you already provisioned — buffer pool memory for hot vectors, CPU for distance computation, storage for the VECTOR columns and the DiskANN index. At moderate scale this is effectively free; at large scale you may need to size up the instance, and vector workloads are memory-hungry enough that the sizing deserves its own exercise (Chapter 10). A managed vector database bills per vector-dimension stored and per query, which is beautifully elastic for spiky workloads and punishing for steady high-volume ones. Self-hosted engines need their own VMs, disks, and memory — priced like any other stateful service you run.

Software. Native vectors ship in the SQL Server 2025 engine — no separate license line item beyond what you already pay. Managed vector DBs are SaaS subscriptions with pricing that rewards small pilots and taxes success. Open-source engines are free to run but not free to operate.

People. This is usually the deciding line item. SQL Server vectors are operated by your existing DBA team with existing tooling. A dedicated engine needs someone to learn it, tune it, upgrade it, and carry the pager for it. If your team is already stretched, the "free" open-source engine is the most expensive option on this list.

A useful rule of thumb: prototype on SQL Server native vectors first, measure, and only introduce a dedicated engine when you have production measurements — not projections — showing the relational engine cannot meet the requirement. The migration path is straightforward because the embedding model and chunking strategy (the hard parts) do not change; only the storage and query layer moves.

The Hybrid Answer: Both, Each for What It Does Best

The choice is not always exclusive. A common mature architecture keeps vectors in SQL Server for the workloads that benefit from relational proximity — RAG over internal documents with permission filtering, semantic search joined to live inventory — while a dedicated engine serves the one workload that genuinely needs it, such as a customer-facing recommendation API at thousands of QPS. The key discipline is making the split along workload boundaries, not along organizational habit: each vector collection should live in exactly one system, chosen deliberately, with the synchronization cost budgeted honestly for the collections that live outside SQL Server.

If you do split, keep SQL Server as the system of record for the source documents in all cases. The vector store holds derived data — embeddings computed from documents — and derived data can always be rebuilt from the source. That single principle makes every DR conversation, every model migration, and every vendor evaluation dramatically simpler.

Key Takeaways

- → SQL Server 2025 native vectors — `VECTOR(n)` , `VECTOR_DISTANCE` , DiskANN indexes, `VECTOR_SEARCH` — cover the majority of production vector workloads: up to millions of vectors, with transactions, joins, row-level security, and backups built in.
- → Dedicated engines earn their place at extreme scale (tens of millions of vectors with aggressive latency SLAs), complex filtered search, first-class hybrid retrieval, or SaaS-scale multi-tenancy — not at the prototype stage.
- → The dominant hidden cost of a dedicated engine is the synchronization pipeline: embedding generation, change propagation, drift reconciliation, a second security model, and coordinated DR. Budget it honestly before committing.
- → Prototype on native vectors first and introduce a dedicated engine only when production measurements — not projections — prove the need. The embedding model and chunking strategy transfer unchanged; only the storage layer moves.
- → In hybrid architectures, keep SQL Server as the system of record for source documents and treat every vector store as derived data that can be rebuilt. Each vector collection lives in exactly one system, chosen deliberately.

Chapter 6: Data Types in Vector Storage: Text, Images, Audio, Code

Vector databases are, at their core, indifferent to the original form of your data. Whether you hand them a paragraph from a legal document, a spectrogram of a customer service call, a JPEG thumbnail, or a Python function, the storage layer sees only a sequence of floating-point numbers. But the path from raw data to those numbers — the embedding pipeline — varies dramatically by modality, and the choices you make there determine search quality, index behavior, and infrastructure cost more than almost anything else. This chapter walks through the four modalities that drive the majority of production vector workloads: text, images, audio, and code. For each, you will see how embeddings are generated, what dimensionality and distance metrics apply, and how to wire it into real storage systems with runnable examples.

6.1 Text Embeddings: The Dominant Workload

Text is where most teams start, and for good reason. The tooling is mature, the models are well-benchmarked, and the use cases — semantic search, RAG pipelines, document classification, duplicate detection — map directly onto problems DBAs already understand. If you have maintained a full-text search index in SQL Server (`CONTAINS / FREETEXT`) or Elasticsearch, you already understand the tension text embeddings are meant to resolve: keyword search finds documents that share vocabulary, but not necessarily meaning. A query for "myocardial infarction" will miss documents that only use "heart attack." Embedding-based search collapses that gap.

How text embeddings work

A text embedding model takes a string and outputs a dense vector. The vector encodes semantic meaning in a high-dimensional space: documents about similar topics land near each other under cosine similarity or dot product distance. The dimensionality ranges widely — 384 dimensions for fast models like `all-MiniLM-L6-v2`, 1536 for OpenAI's `text-embedding-3-small`, 3072 for `text-embedding-3-large`. Higher dimension is not always better; it increases storage cost and slows index construction, and for many retrieval tasks a 768-dimensional model is indistinguishable in quality from a 3072-dimensional one.

Chunking is a database partitioning problem

Before you embed a document, you must decide how to cut it up. Embedding models have token limits: `text-embedding-3-small` supports 8,191 tokens, sentence-transformers models typically support 512 tokens. Feeding a 200-page PDF as a single string is not an option. Chunking strategies include fixed-size windows, sentence boundaries, paragraph boundaries, and semantic chunking. The right choice depends on your retrieval pattern — the same tradeoff you make when deciding between row-level and block-level access in a traditional DBMS.

A practical baseline: 512-token chunks with 50-token overlap. The overlap prevents semantic content from being split across chunk boundaries the way a page boundary in a B-tree splits a row.

```

# Python
# Text embedding pipeline using OpenAI + SQL Server
# Requirements: openai>=1.0, pyodbc, tiktoken

import openai
import pyodbc
import tiktoken
from typing import List, Tuple

client = openai.OpenAI() # reads OPENAI_API_KEY from environment

conn = pyodbc.connect(
    "DRIVER={ODBC Driver 18 for SQL Server};SERVER=localhost;"
    "DATABASE=vectordb;Trusted_Connection=yes;Encrypt=yes;"
    "TrustServerCertificate=yes"
)

def chunk_text(text: str, model: str = "text-embedding-3-small",
               chunk_tokens: int = 512, overlap_tokens: int = 50) -> List[str]:
    """Split text into overlapping token-bounded chunks."""
    enc = tiktoken.encoding_for_model(model)
    tokens = enc.encode(text)
    chunks = []
    start = 0
    while start < len(tokens):
        end = min(start + chunk_tokens, len(tokens))
        chunk_tokens_slice = tokens[start:end]
        chunks.append(enc.decode(chunk_tokens_slice))
        if end == len(tokens):
            break
        start += chunk_tokens - overlap_tokens
    return chunks

def embed_texts(texts: List[str], model: str = "text-embedding-3-small") ->
List[List[float]]:
    """Batch embed up to 2048 strings per API call."""
    response = client.embeddings.create(input=texts, model=model)
    return [item.embedding for item in response.data]

def to_vector_literal(embedding: List[float]) -> str:
    """Format an embedding as a VECTOR literal SQL Server can CAST."""
    return "[" + ",".join(str(v) for v in embedding) + "]"

def ingest_document(cursor, doc_id: str, title: str, body: str) -> int:
    """Chunk, embed, and store a document. Returns number of chunks inserted."""
    chunks = chunk_text(body)
    embeddings = embed_texts(chunks)
    for i, (chunk, emb) in enumerate(zip(chunks, embeddings)):
        cursor.execute(
            "INSERT INTO document_chunks (doc_id, title, chunk_index, chunk_text,
embedding) "
            "VALUES (?, ?, ?, ?, CAST(? AS VECTOR(1536)))",
            (doc_id, title, i, chunk, to_vector_literal(emb))

```

```

    )
    cursor.commit()
    return len(chunks)

# — Schema (run once) —————
SCHEMA_SQL = """
CREATE TABLE document_chunks (
    id          BIGINT IDENTITY(1,1) NOT NULL CONSTRAINT PK_document_chunks PRIMARY
KEY,
    doc_id      NVARCHAR(100) NOT NULL,
    title       NVARCHAR(200) NULL,
    chunk_index INT NOT NULL,
    chunk_text  NVARCHAR(MAX) NOT NULL,
    embedding   VECTOR(1536) NOT NULL,
    created_at  DATETIME2(0) NOT NULL CONSTRAINT DF_document_chunks_created
                DEFAULT SYSUTCDATETIME()
);

CREATE VECTOR INDEX document_chunks_embedding_idx
    ON document_chunks (embedding)
    WITH (METRIC = 'cosine', TYPE = 'DiskANN', MAXDOP = 4);
"""

# — Query —————
def semantic_search(cursor, query: str, top_k: int = 5) -> List[Tuple]:
    query_embedding = embed_texts([query])[0]
    cursor.execute(
        """
        SELECT TOP (?) doc_id, title, chunk_index, chunk_text,
            1 - VECTOR_DISTANCE('cosine', embedding, CAST(? AS VECTOR(1536)))
                AS cosine_similarity
        FROM    document_chunks
        WHERE   embedding IS NOT NULL
        ORDER  BY VECTOR_DISTANCE('cosine', embedding, CAST(? AS VECTOR(1536)))
        """,
        (top_k, to_vector_literal(query_embedding),
        to_vector_literal(query_embedding))
    )
    return cursor.fetchall()

```

`VECTOR_DISTANCE('cosine', embedding, @query)` is SQL Server's cosine distance. One minus that value gives you cosine similarity, which is more intuitive when you need to filter by a quality threshold (e.g., `WHERE 1 - VECTOR_DISTANCE('cosine', embedding, @query) > 0.75`).

Choosing a text embedding model

Model	Dimensions	Max Tokens	Cost / 1M tokens	Notes
-------	------------	------------	------------------	-------

---	---	---	---	---
-----	-----	-----	-----	-----

text-embedding-3-small	1536	8,191	\$0.020	Best price/quality for RAG
------------------------	------	-------	---------	----------------------------

<code>text-embedding-3-large</code>	3072	8,191	\$0.130	Marginal gains, 6.5× cost
<code>all-MiniLM-L6-v2</code>	384	512	Free (local)	Fast, low memory, good for high-volume
<code>BAAI/bge-large-en-v1.5</code>	1024	512	Free (local)	MTEB top-10, strong retrieval
<code>embed-english-v3.0</code> (Cohere)	1024	512	\$0.100	Strong multilingual variants

For offline or high-volume workloads, `BAAI/bge-large-en-v1.5` via sentence-transformers is the practical default. For OpenAI-native stacks, `text-embedding-3-small` covers most production cases without the cost penalty of the large variant.

6.2 Image Embeddings: Visual Similarity at Database Scale

Image search appears exotic, but the database framing is familiar: you are building an index over a feature space, and the features happen to come from a convolutional or transformer-based vision model rather than a B-tree key. The use cases span e-commerce (find visually similar products), content moderation (detect near-duplicate or policy-violating images), medical imaging (retrieve histology slides similar to a reference), and manufacturing (surface defect matching).

How image embeddings are generated

Vision models map an image to a dense vector by passing it through a neural network and extracting the activations at a specific layer — typically the penultimate layer before the classification head. CLIP (Contrastive Language-Image Pretraining) models are especially useful because they embed images and text into the same vector space, which allows cross-modal queries: you can retrieve images using a text string as the query.

Common models and their output dimensions:

- → **OpenAI CLIP (ViT-B/32):** 512 dimensions
- → **OpenAI CLIP (ViT-L/14):** 768 dimensions
- → **Google's ViT-G via Vertex AI:** 1408 dimensions
- → **ResNet-50 (ImageNet features):** 2048 dimensions (older, rarely used for search)

CLIP at ViT-L/14 is the standard starting point for production image search. It handles both similarity search within images and text-to-image retrieval, which reduces the number of specialized indexes you need to maintain.


```

# Python
# Image embedding pipeline using CLIP + Weaviate
# Requirements: weaviate-client>=4.0, open-clip-torch, Pillow, torch

import weaviate
import open_clip
import torch
from PIL import Image
from pathlib import Path
import base64, io, json

# — Load CLIP model (CPU or GPU depending on hardware) —————
model, _, preprocess = open_clip.create_model_and_transforms(
    "ViT-L-14", pretrained="openai"
)
model.eval()
tokenizer = open_clip.get_tokenizer("ViT-L-14")

def embed_image(image_path: str) -> list[float]:
    """Return a 768-dim CLIP vector for an image file."""
    img = preprocess(Image.open(image_path).convert("RGB")).unsqueeze(0)
    with torch.no_grad():
        features = model.encode_image(img)
        features /= features.norm(dim=-1, keepdim=True) # L2 normalize
    return features.squeeze().tolist()

def embed_text_for_image_search(text: str) -> list[float]:
    """Embed a text query into the same CLIP space as images."""
    tokens = tokenizer([text])
    with torch.no_grad():
        features = model.encode_text(tokens)
        features /= features.norm(dim=-1, keepdim=True)
    return features.squeeze().tolist()

# — Weaviate schema —————
client = weaviate.connect_to_local() # or weaviate.connect_to_weaviate_cloud(...)

from weaviate.classes.config import Configure, Property, DataType, VectorDistances

if not client.collections.exists("ProductImage"):
    client.collections.create(
        name="ProductImage",
        vectorizer_config=Configure.Vectorizer.none(), # we supply our own vectors
        vector_index_config=Configure.VectorIndex.hnsw(
            distance_metric=VectorDistances.COSINE,
            ef_construction=128,
            max_connections=32,
        ),
        properties=[
            Property(name="product_id", data_type=DataType.TEXT),
            Property(name="file_path", data_type=DataType.TEXT),
            Property(name="description", data_type=DataType.TEXT),
        ],
    )

```

```

)

# — Ingest images —————
def ingest_product_images(image_dir: str):
    collection = client.collections.get("ProductImage")
    with collection.batch.dynamic() as batch:
        for path in Path(image_dir).glob("*.jpg"):
            product_id = path.stem
            vector = embed_image(str(path))
            batch.add_object(
                properties={
                    "product_id": product_id,
                    "file_path": str(path),
                    "description": f"Product image {product_id}",
                },
                vector=vector,
            )

# — Text-to-image search —————
def search_images_by_text(query: str, top_k: int = 10):
    query_vector = embed_text_for_image_search(query)
    collection = client.collections.get("ProductImage")
    results = collection.query.near_vector(
        near_vector=query_vector,
        limit=top_k,
        return_metadata=["distance"],
    )
    for obj in results.objects:
        print(f"{obj.properties['product_id']} | "
              f"distance={obj.metadata.distance:.4f} | "
              f"path={obj.properties['file_path']}")

# Example: search_images_by_text("red running shoes with white sole")

```

Storage considerations for images

A 768-dimensional float32 vector consumes 3,072 bytes — about 3 KB per image. At one million images, that is 3 GB of vector data before indexing overhead. The HNSW index typically adds 1.5–2× the raw vector size, putting you at roughly 6–9 GB for the index alone. That fits comfortably in the memory of a cloud instance sized for this workload. The images themselves should not go into the vector database; store them in object storage (S3, GCS) and keep only the path or a content-addressed hash in the metadata fields.

Preprocessing matters

Unlike text, images require explicit preprocessing before embedding: resize to the model's expected input resolution (224×224 for ViT-B/32, 336×336 for ViT-L/14@336), normalize pixel values, and handle color mode. Feeding a grayscale medical image directly to a model

expecting three-channel RGB input without conversion will produce silently incorrect embeddings — the query will return results, they just will not be meaningful. Always validate your preprocessing pipeline with a small labeled set before bulk ingestion.

6.3 Audio Embeddings: Time-Series Signals as Vectors

Audio is the least common modality in general-purpose vector workloads, but it is heavily used in specific verticals: call center analytics, music recommendation, speaker verification, podcast search, and environmental sound classification. The embedding pipeline is more involved than text or images because audio must be converted from a raw waveform to a fixed-length vector in a way that captures the relevant signal while discarding noise.

Representations and models

There are two main approaches to audio embedding:

1. **Spectrogram-based:** Convert the waveform to a mel-spectrogram (a 2D time-frequency representation), then pass it through a CNN or transformer. Models: OpenL3, PANNs (Pretrained Audio Neural Networks), CLAP (Contrastive Language-Audio Pretraining).
2. **Waveform-based:** Feed raw audio samples directly to a transformer. Models: wav2vec 2.0, Whisper encoder features, EnCodec.

For semantic audio search — finding recordings similar in content or speaker — CLAP is the current standard because it supports cross-modal text-to-audio queries, the same pattern that makes CLIP useful for images.

For speaker identification, models like `speechbrain/spkrec-ecapa-voxceleb` produce 192-dimensional speaker embeddings optimized for cosine similarity between the same speaker across recordings.

```

# Python
# Audio embedding pipeline using CLAP + Qdrant
# Requirements: qdrant-client, msclap, librosa, numpy

import librosa
import numpy as np
from qdrant_client import QdrantClient
from qdrant_client.models import (
    Distance, VectorParams, PointStruct, SearchRequest
)
from msclap import CLAP # Microsoft CLAP implementation

# — Initialize CLAP and Qdrant —————
clap_model = CLAP(version="2023", use_cuda=False)
qdrant = QdrantClient(url="http://localhost:6333")

COLLECTION_NAME = "audio_clips"
VECTOR_DIM = 512 # CLAP 2023 audio embedding dimension

# — Create collection —————
qdrant.recreate_collection(
    collection_name=COLLECTION_NAME,
    vectors_config=VectorParams(size=VECTOR_DIM, distance=Distance.COSINE),
)

def embed_audio_file(file_path: str) -> np.ndarray:
    """
    Load an audio file and return a CLAP embedding.
    CLAP internally resamples to 44.1 kHz and expects mono or stereo.
    """
    embeddings = clap_model.get_audio_embeddings(
        [file_path], resample=True
    )
    vec = embeddings[0]
    # L2 normalize for cosine similarity
    return (vec / np.linalg.norm(vec)).tolist()

def embed_audio_text_query(text: str) -> list[float]:
    """Embed a text query into the CLAP audio space."""
    embeddings = clap_model.get_text_embeddings([text])
    vec = embeddings[0]
    return (vec / np.linalg.norm(vec)).tolist()

def ingest_audio_clips(clips: list[dict]):
    """
    clips: list of {"id": int, "file_path": str, "label": str, "duration_s": float}
    """
    points = []
    for clip in clips:
        vector = embed_audio_file(clip["file_path"])
        points.append(
            PointStruct(
                id=clip["id"],

```

```

        vector=vector,
        payload={
            "file_path": clip["file_path"],
            "label":      clip["label"],
            "duration_s": clip["duration_s"],
        },
    )
)
qdrant.upsert(collection_name=COLLECTION_NAME, points=points)

def search_audio_by_text(query: str, top_k: int = 5):
    """Find audio clips matching a natural language description."""
    query_vector = embed_audio_text_query(query)
    results = qdrant.search(
        collection_name=COLLECTION_NAME,
        query_vector=query_vector,
        limit=top_k,
        with_payload=True,
    )
    for hit in results:
        print(f"Score: {hit.score:.4f} | "
              f"Label: {hit.payload['label']} | "
              f"File: {hit.payload['file_path']}")

# Example: search_audio_by_text("dogs barking outdoors")
# Example: search_audio_by_text("angry customer complaining about billing")

```

Segmentation and temporal alignment

Long audio recordings — interviews, conference calls, podcasts — require segmentation before embedding, similar to text chunking. A common strategy is to use voice activity detection (VAD) to find speech segments, then embed each segment of 5–30 seconds independently. This gives finer retrieval granularity: instead of finding "the episode that discusses refund policies," you can surface the 20-second clip where refund policies are explicitly mentioned.

The metadata schema for audio should carry temporal anchors: `start_time_s` and `end_time_s` for each embedded segment. These allow the retrieval result to play back or link to the exact moment in a recording, which is the audio equivalent of returning a document chunk rather than a full document.

Sampling rate and codec normalization

Audio from different sources arrives at different sample rates (8 kHz telephony, 44.1 kHz music, 16 kHz podcast) and codecs (MP3, AAC, FLAC, OGG). Most embedding models expect a specific input: wav2vec 2.0 requires 16 kHz mono PCM. Feeding 8 kHz telephone audio without upsampling, or stereo audio without downmixing, degrades embedding quality.

Build a normalization step (librosa's `resample`, `ffmpeg` via subprocess) at the front of your ingestion pipeline and enforce it — treat it the same way you would enforce data type coercion in an ETL pipeline.

6.4 Code Embeddings: Semantic Search Across Source Files

Code is a modality that most vector database introductions skip, but it is one of the most commercially valuable. Developers search for how a function was used across a codebase, how a specific API was called, or what code handles a particular edge case. Traditional regex-based code search (`grep`, `ripgrep`) is syntactically exact; embedding-based search finds code that does the same thing expressed differently.

How code embeddings differ from text embeddings

Code has structure that plain text does not: function signatures, indentation scope, docstrings, variable names, and control flow. Models trained specifically on code — `microsoft/codebert-base`, `Salesforce/codet5-base`, OpenAI's `text-embedding-3-small` with code content, and `voyage-code-2` from Voyage AI — outperform generic text models on code retrieval tasks because they understand that `def add(a, b): return a + b` and `func sum(x int, y int) int { return x + y }` are semantically equivalent even across languages.

`voyage-code-2` from Voyage AI currently leads benchmarks for code retrieval, producing 1536-dimensional embeddings. For teams already on the OpenAI stack, `text-embedding-3-small` performs well enough on code and avoids introducing a second embedding provider.

Chunking strategies for code

File-level embedding loses too much precision; function-level or class-level chunking is the practical standard. Parse the AST of each source file and extract functions, methods, and classes as individual embedding units. Include the docstring, signature, and body as the text passed to the embedding model. For Python, the `ast` module handles this natively; for multi-language codebases, `tree-sitter` provides a consistent AST interface across languages.

```

# Python
# Code embedding pipeline using tree-sitter + Pinecone + Voyage AI
# Requirements: pinecone-client>=3.0, voyageai, tree-sitter, tree-sitter-python

import ast
import textwrap
from pathlib import Path
import voyageai
from pinecone import Pinecone, ServerlessSpec

voyage = voyageai.Client() # reads VOYAGE_API_KEY from environment
pc = Pinecone()            # reads PINECONE_API_KEY from environment

INDEX_NAME = "codebase-search"
DIMENSION = 1536           # voyage-code-2 output dimension

# — Create Pinecone index (serverless, cosine) —————
if INDEX_NAME not in [idx.name for idx in pc.list_indexes()]:
    pc.create_index(
        name=INDEX_NAME,
        dimension=DIMENSION,
        metric="cosine",
        spec=ServerlessSpec(cloud="aws", region="us-east-1"),
    )

index = pc.Index(INDEX_NAME)

# — Extract Python functions from source —————
def extract_python_functions(source_path: str) -> list[dict]:
    """
    Parse a Python file and return one dict per function/method,
    containing the name, source text, and file path.
    """
    source = Path(source_path).read_text(encoding="utf-8")
    tree = ast.parse(source)
    units = []

    for node in ast.walk(tree):
        if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):
            func_lines = source.splitlines()[
                node.lineno - 1 : node.end_lineno
            ]
            func_text = textwrap.dedent("\n".join(func_lines))
            units.append({
                "name": node.name,
                "file_path": source_path,
                "start_line": node.lineno,
                "end_line": node.end_lineno,
                "source": func_text,
            })

    return units

```

```

def embed_code_units(units: list[dict]) -> list[list[float]]:
    """Embed a list of code units using voyage-code-2."""
    texts = [u["source"] for u in units]
    response = voyage.embed(texts, model="voyage-code-2", input_type="document")
    return response.embeddings

def ingest_python_file(file_path: str, namespace: str = ""):
    """Parse, embed, and upsert all functions from a Python file."""
    units = extract_python_functions(file_path)
    if not units:
        return

    embeddings = embed_code_units(units)

    vectors = []
    for unit, embedding in zip(units, embeddings):
        # Pinecone IDs must be strings; use file+line as a stable key
        vec_id = f"{unit['file_path']}::{unit['name']}::{unit['start_line']}"
        vectors.append({
            "id": vec_id,
            "values": embedding,
            "metadata": {
                "name": unit["name"],
                "file_path": unit["file_path"],
                "start_line": unit["start_line"],
                "end_line": unit["end_line"],
                "source": unit["source"][:1000], # Pinecone metadata is size-
limited
            },
        })

    # Pinecone recommends upsert batches of 100 or fewer
    batch_size = 100
    for i in range(0, len(vectors), batch_size):
        index.upsert(vectors=vectors[i : i + batch_size], namespace=namespace)

def ingest_directory(root_dir: str, namespace: str = ""):
    """Walk a directory tree and ingest all .py files."""
    for path in Path(root_dir).rglob("*.py"):
        print(f"Ingesting {path}")
        ingest_python_file(str(path), namespace=namespace)

# — Search —
def search_code(query: str, top_k: int = 5, namespace: str = "") -> list[dict]:
    """
    Embed a natural language or code query and retrieve matching functions.
    """
    response = voyage.embed([query], model="voyage-code-2", input_type="query")
    query_vector = response.embeddings[0]

    results = index.query(
        vector=query_vector,
        top_k=top_k,

```



```

        include_metadata=True,
        namespace=namespace,
    )

    matches = []
    for match in results.matches:
        matches.append({
            "score": match.score,
            "name": match.metadata["name"],
            "file_path": match.metadata["file_path"],
            "start_line": match.metadata["start_line"],
            "source": match.metadata["source"],
        })
    return matches

# — Usage —————
# ingest_directory("/home/user/myproject/src")
# results = search_code("function that validates JWT tokens")
# for r in results:
#     print(f"{r['score']:.4f} | {r['file_path']}:{r['start_line']} | {r['name']}")

```

Multi-language codebases

When your codebase spans multiple languages — Go services, Python scripts, TypeScript frontends, SQL stored procedures — you have two practical options:

1. **Single shared index:** Embed all code regardless of language into the same Pinecone index or SQL Server vector table. This works well when queries are natural language ("find the function that handles payment retries"), because the model maps both the query and code across languages into a shared semantic space. Include `language` as a metadata field and filter when needed.
2. **Per-language indexes:** Maintain separate indexes or namespaces per language. This is useful when queries are themselves code ("find Go functions similar to this Python function") because it lets you tune chunking, model selection, and distance thresholds per language without cross-contamination.

For most teams, the single shared index is simpler to operate and surprisingly effective.

SQL and database code

SQL stored procedures, views, and migration scripts are a special case worth calling out. Developers routinely need to find queries that JOIN a particular table, update a specific column, or implement a pattern like a slowly changing dimension. Embedding SQL is straightforward — feed the SQL text to the embedding model — but chunking differs: a SQL file may contain dozens of independent statements. Split on statement boundaries (semicolons, `GO` delimiters in SQL Server) and embed each statement independently.

The metadata fields that pay off for SQL code search: `database_name`, `schema_name`, `object_type` (table, view, procedure, trigger), `tables_referenced` (extracted by parsing), and `last_modified`. This turns your vector database into a semantic catalog layer on top of your existing database assets.

6.5 Multi-Modal Pipelines: Combining Modalities in One Index

Real applications rarely deal with a single modality in isolation. A product catalog has a name (text), a description (text), images, and potentially a promotional audio clip. A customer support system has transcribed calls (text derived from audio), agent notes (text), and screenshots of error messages (images). Building a single search experience across all of these requires a decision: do you use a single unified embedding space, or do you maintain separate indexes and fuse results at query time?

Unified embedding space with CLIP/CLAP

CLIP and CLAP models are trained to project images and text (or audio and text) into a shared vector space using contrastive learning. A text query lands near matching images, and image queries land near matching captions. This makes it possible to maintain a single index and query it with any modality.

The limitation is that these shared spaces are trained for specific modality pairs. CLIP handles image↔text; CLAP handles audio↔text; there is no single off-the-shelf model that embeds text, images, audio, and code into one coherent space. Unified multi-modal models exist (Google's Gemini, Meta's ImageBind) but they are not yet standardized in the same way that CLIP is for image-text tasks.

Late fusion with per-modality indexes

```
graph TD
    T[Text] --> ET[Text embedding model]
    I[Image] --> EI[CLIP image encoder]
    A[Audio] --> EA[CLAP audio encoder]
    C[Code] --> EC[Code embedding model]
    ET --> IDX1[(Text index)]
    EI --> IDX2[(Image index)]
    EA --> IDX3[(Audio index)]
    EC --> IDX4[(Code index)]
    IDX1 --> RRF[Reciprocal Rank Fusion]
    IDX2 --> RRF
    IDX3 --> RRF
    IDX4 --> RRF
    RRF --> R[Fused results]
```

Each modality gets its own embedding model and index; results are merged at query time via Reciprocal Rank Fusion rather than forced into one shared vector space.

The alternative is to maintain separate indexes per modality and fuse results at query time. A search query is embedded by each relevant model, each index returns its top-K results, and a fusion layer combines the ranked lists. Reciprocal Rank Fusion (RRF) is the standard algorithm for this:

```
# Python
# Reciprocal Rank Fusion for multi-modal result merging
from collections import defaultdict
from typing import Any

def reciprocal_rank_fusion(
    ranked_lists: list[list[Any]],
    id_key: str = "id",
    k: int = 60,
) -> list[dict]:
    """
    Merge multiple ranked result lists using RRF.
    Each item in a ranked_list must be a dict with a unique id_key.
    k=60 is the standard constant that dampens the influence of high ranks.
    """
    scores: dict[Any, float] = defaultdict(float)
    items: dict[Any, dict] = {}

    for ranked_list in ranked_lists:
        for rank, item in enumerate(ranked_list, start=1):
            item_id = item[id_key]
            scores[item_id] += 1.0 / (k + rank)
            items[item_id] = item # keep the last-seen metadata

    fused = sorted(
        [{"rrf_score": score, **items[item_id]} for item_id, score in
        scores.items()],
        key=lambda x: x["rrf_score"],
        reverse=True,
    )
    return fused

# Example: fuse text search and image search results for a product catalog
# text_results = [{"id": "prod_001", "name": "Red Sneaker", ...}, ...]
# image_results = [{"id": "prod_003", "name": "Trail Runner", ...}, ...]
# fused = reciprocal_rank_fusion([text_results, image_results])
```

RRF does not require score normalization across modalities, which is a significant practical advantage: cosine similarity scores from a text index are not directly comparable to distance scores from an image index, but rank positions are comparable.

Metadata alignment across modalities

When operating a multi-modal pipeline, the metadata schema becomes the spine that connects records across indexes. A `product_id` field that exists in both the text chunk table and the image vector collection is what makes post-retrieval joining possible. Design your metadata schema before you build the ingestion pipelines, or you will find yourself with three indexes that cannot be joined without a full table scan.

A practical metadata pattern for a product catalog:

```
-- SQL Server: unified metadata table that all modality indexes reference
CREATE TABLE dbo.product_assets (
    asset_id          UNIQUEIDENTIFIER NOT NULL CONSTRAINT DF_product_assets_id DEFAULT
NEWID(),
    product_id        NVARCHAR(100) NOT NULL,
    modality           NVARCHAR(20) NOT NULL
                      CONSTRAINT CK_product_assets_modality
                      CHECK (modality IN ('text', 'image', 'audio', 'code')),
    source_ref         NVARCHAR(500) NOT NULL,    -- chunk text, S3 path, git SHA, etc.
    embedding_dim      INT NOT NULL,
    index_name         SYSNAME NOT NULL,          -- which index/table holds the vector
    external_id        NVARCHAR(100) NOT NULL,    -- the vector's ID in that index
    created_at         DATETIME2(0) NOT NULL CONSTRAINT DF_product_assets_created
                      DEFAULT SYSUTCDATETIME(),
    CONSTRAINT PK_product_assets PRIMARY KEY (asset_id)
);

CREATE INDEX IX_product_assets_product ON dbo.product_assets (product_id);
CREATE INDEX IX_product_assets_modality ON dbo.product_assets (modality,
index_name);
```

This table acts as a reverse-lookup registry: given a vector ID returned by Pinecone or Weaviate, you can join back to the canonical `product_id` and fetch any other modality's vectors or metadata.

6.6 Distance Metrics by Modality

Not all vector spaces are created equal, and the choice of distance metric affects both retrieval quality and index performance. The table below summarizes the practical defaults by modality:

Modality	Typical Model Family	Recommended Metric	Notes
----------	----------------------	--------------------	-------

---	---	---	---
-----	-----	-----	-----

| Text | Sentence-transformers, OpenAI | Cosine similarity | Vectors are L2-normalized by most models |

| Text (dot product tuned) | Matryoshka / OpenAI v3 | Dot product | Only if model is trained with dot product objective |

| Images | CLIP variants | Cosine similarity | L2-normalize outputs before storage |

| Audio (semantic) | CLAP | Cosine similarity | Same contrastive training as CLIP |

| Audio (speaker ID) | ECAPA-TDNN, x-vectors | Cosine similarity | Speaker embeddings respond well to cosine |

| Code | voyage-code-2, CodeBERT | Cosine similarity | Normalize outputs |

| Binary / hash | SimHash, MinHash | Hamming distance | Store hashes in `VARBINARY(MAX)` in SQL Server |

The near-universal answer is cosine similarity, which is why most production systems normalize vectors to unit length at ingestion time and then use dot product as a fast approximation (dot product on unit vectors equals cosine similarity). SQL Server's `VECTOR_DISTANCE` takes the metric as an argument (`'cosine'` vs `'dot'`), so you can switch metrics without changing the schema.

Euclidean (L2) distance is appropriate when the magnitude of the vector carries semantic information, which is rare in embedding models designed for retrieval — those models are typically trained with normalization objectives. Hamming distance is useful for binary fingerprinting and near-duplicate detection in image or document deduplication pipelines.

6.7 Storage Sizing and Cost Modeling

Before committing to a database architecture, teams need to estimate how much vector data they are actually dealing with. The arithmetic is simple but frequently underestimated.

Per-vector storage

A single float32 (4-byte) vector of dimension d consumes $4d$ bytes on disk. The HNSW index — the dominant index type in production vector stores — adds roughly 1.5–2× the raw vector size in graph structure overhead. A 1536-dimensional OpenAI embedding therefore consumes:

- → Raw vector: $4 \times 1536 = 6,144$ bytes (≈ 6 KB)
- → With HNSW overhead: ≈ 12 –15 KB per vector

At one million documents with five chunks each (five million vectors):

- → Raw data: $5M \times 6 \text{ KB} = 30 \text{ GB}$
- → Index: $5M \times 15 \text{ KB} = 75 \text{ GB}$ total on-disk

That is a meaningful infrastructure commitment. Dimensionality reduction strategies — Matryoshka representation learning (MRL), PCA post-processing, or switching to a smaller model — can cut that by 50–75% at a modest quality cost.

Quantization

Scalar quantization (SQ8) and binary quantization (BQ) reduce vector storage at the cost of some retrieval precision. SQL Server stores vectors as float32 in the native `VECTOR` type; application-side float16 arrays or binary hashes in `VARBINARY` trade retrieval precision for storage when you need it. Pinecone offers built-in int8 quantization. Weaviate supports both scalar and product quantization natively.

A common production pattern:

1. Store the full-precision vectors on disk for re-ranking.
2. Use a quantized version for the HNSW index to reduce memory footprint.
3. After ANN retrieval returns top-100 candidates, re-rank with exact dot product on the full-precision vectors.

This two-stage pattern maintains high recall while reducing the memory needed to serve the HNSW index, which must fit in RAM for low-latency queries.

Cost across platforms

Managed vector databases price on stored vectors, queries per second, and sometimes on the hosting tier (pod-based vs. serverless). Self-hosted SQL Server on a VM with adequate RAM is often the most cost-effective option at moderate scale — tens of millions of vectors — because you pay only for compute and storage, not a per-vector premium. Beyond a few hundred million vectors, the operational overhead of running a purpose-built system like Qdrant or Weaviate at scale often justifies the higher unit cost.

Key Takeaways

- → **Modality determines the embedding pipeline, not the storage layer.** SQL Server native vectors, Pinecone, Weaviate, and Qdrant are all modality-agnostic; what changes is the model you call before writing a vector to the index. Design your ingestion pipeline first, then select the database that fits your operational constraints.
- → **Chunking and segmentation are first-class engineering problems.** Whether you are splitting a legal document into 512-token windows, extracting Python functions from an AST, or segmenting a call recording by voice activity, how you divide data before embedding determines retrieval granularity more than model choice does.
- → **CLIP and CLAP enable cross-modal text-to-X queries without a separate query path.** By embedding images or audio into the same vector space as text, you avoid maintaining a specialized query interface for each modality — a single near-vector query handles both image similarity and text-to-image retrieval.
- → **Cosine similarity with L2-normalized vectors is the safe default across all modalities.** Euclidean distance is rarely appropriate for embedding-based retrieval; Hamming distance applies only to binary fingerprinting. When in doubt, normalize at ingestion and use cosine.
- → **Size your infrastructure before you build.** A 1536-dimensional embedding model applied to five million vectors produces roughly 75 GB of indexed data. Factor in HNSW overhead, quantization trade-offs, and the distinction between disk storage and RAM residency before selecting a hosting tier — vector indexes must fit in memory to serve low-latency queries.

Chapter 7: Enabling Native Vector Support in SQL Server

One of the pleasant surprises of SQL Server 2025's vector support is how little ceremony it requires. There is no extension to install, no separate service to deploy, no feature flag buried in a configuration file. Native vectors are part of the database engine itself: if your instance runs SQL Server 2025 (version 17.x), the `VECTOR` data type, the `VECTOR_DISTANCE` function, and the supporting infrastructure are already there. This chapter walks through verifying your environment, understanding what is generally available versus preview, planning the upgrade if you are on an older version, and creating your first vector-enabled table the right way.

Step 1: Verify the Engine Version

Native vector support requires SQL Server 2025, which reports itself as major version 17. Confirm what you are running before anything else:

```
-- Confirm the engine version: vector support needs 17.x (SQL Server 2025)
SELECT @@VERSION AS EngineVersion,
       SERVERPROPERTY('ProductMajorVersion') AS MajorVersion,
       SERVERPROPERTY('ProductLevel') AS ProductLevel,
       SERVERPROPERTY('Edition') AS Edition;
-- Expect: MajorVersion = 17, e.g. "Microsoft SQL Server 2025 (RTM) - 17.0.xxxx"
```

If you are on SQL Server 2022 or earlier, native vectors are not available in-engine. Your options are: upgrade to 2025 (the path this book assumes), use Azure SQL Database or Azure SQL Managed Instance where native vector support is generally available, or — as a stopgap only — store embeddings as serialized JSON in `NVARCHAR(MAX)` and compute similarity in the application tier. That stopgap is worth naming explicitly because many teams built it before 2025 shipped; Chapter 8 covers migrating those legacy columns to the native type.

Check that the type is actually usable in your database with a one-line smoke test:

```
-- Smoke test: if this runs, native vector support is live
DECLARE @v VECTOR(3) = '[1.0, 2.0, 3.0]';
SELECT VECTOR_DISTANCE('cosine', @v, '[1.0, 2.0, 3.0]') AS SelfDistance;
-- Returns 0: identical vectors have zero cosine distance
```

Step 2: Know What Is GA and What Is Preview

As of SQL Server 2025, the vector feature set is split across two maturity levels, and planning around that split saves unpleasant surprises later.

Generally available: the `VECTOR(n)` data type (1 to 1998 dimensions), `VECTOR_DISTANCE` with `'cosine'`, `'euclidean'`, and `'dot'` metrics, plus the helper functions `VECTOR_NORM` and `VECTOR_NORMALIZE`. These work on SQL Server 2025, Azure SQL Database, and Azure SQL Managed Instance, and you can build production workloads on them today. Exact nearest-neighbor search with `VECTOR_DISTANCE` performs well up to roughly 50,000 vectors — plenty for departmental knowledge bases, product catalogs, and pilot RAG systems.

In preview: the approximate vector index

(`CREATE VECTOR INDEX ... WITH (TYPE = 'DiskANN')`) and the `VECTOR_SEARCH` table-valued function, available in SQL Server 2025 (17.x), Azure SQL Database, and SQL database in Microsoft Fabric. Preview status matters for two reasons: the syntax and behavior can still change, and the current implementation carries real constraints — the table needs a single-column integer clustered primary key, the table becomes read-only while the vector index exists, the index must be dropped and recreated to incorporate new data, and partitioned tables, views, and temp tables are not supported.

The practical implication: design your schema so the GA features carry the production load and the preview index is an accelerator you can adopt, drop, or rebuild without redesigning tables. Keep the source documents and their embeddings in ordinary tables with ordinary keys; add the vector index as a performance layer, not a structural dependency. Chapters 9 and 10 go deep on DiskANN index design and tuning.

Step 3: Plan the Upgrade (If You Are Not on 2025 Yet)

Most SQL Server estates are not on 2025 yet, so the realistic path to native vectors runs through an upgrade project. Treat vector enablement as one workstream inside that project, not the reason for it — the upgrade justification should stand on its own (support lifecycle, performance, security), with vectors as a bonus capability.

A few upgrade specifics that matter for vector workloads:

- → **In-place vs. side-by-side.** For any workload you intend to add vectors to, prefer a side-by-side migration (new 2025 instance, restore and validate) over an in-place upgrade. It gives you a rollback path and a clean environment for the schema changes vector columns require.

- → **Compatibility level.** New engine features generally expect the database compatibility level to match the engine. After upgrading, test your workload at the new compatibility level before enabling anything vector-related, so vector issues and compat issues do not get conflated.
- → **Driver and client library versions.** The `VECTOR` type is new to the TDS type system. Older `SqlClient`, `ODBC`, or `JDBC` drivers may not understand it. Plan a driver upgrade alongside the engine upgrade, and verify that your ORM (Entity Framework Core 10+ has built-in vector support) handles the type.
- → **Cloud shortcut.** If the on-premises upgrade timeline is long, Azure SQL Database gives you native vectors immediately with no upgrade project at all — a legitimate interim home for vector workloads while the on-prem estate catches up. Chapter 13 covers the cloud deployment details.

Step 4: Create Your First Vector-Enabled Table

With the engine verified, creating vector storage is ordinary DDL. The pattern to internalize: keep the source content and the embedding in the same row, with the embedding nullable so documents can exist before they are embedded (backfill is a normal part of vector operations).

```

-- Canonical pattern: source content + embedding side by side
CREATE TABLE dbo.Documents (
    DocumentId INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_Documents PRIMARY KEY,
    Title      NVARCHAR(300) NOT NULL,
    Body       NVARCHAR(MAX) NOT NULL,
    SourceUri  NVARCHAR(1000) NULL,
    Embedding  VECTOR(1536) NULL,          -- NULL until embedded
    EmbeddedAt DATETIME2(0) NULL,         -- tracks backfill progress
    ModelName  NVARCHAR(100) NULL,        -- which model produced the vector
    CreatedAt  DATETIME2(0) NOT NULL CONSTRAINT DF_Documents_CreatedAt DEFAULT
    SYSUTCDATETIME()
);
GO

-- Insert a document; embedding arrives later from the embedding pipeline
INSERT INTO dbo.Documents (Title, Body, SourceUri)
VALUES ('Password reset procedure',
        'To reset your password, open Settings > Security > ...',
        'https://intranet/help/password-reset');

-- Store an embedding: implicit conversion from JSON array string
UPDATE dbo.Documents
SET Embedding  = CAST('[0.012, -0.043, 0.071, ...]' AS VECTOR(1536)),
    EmbeddedAt = SYSUTCDATETIME(),
    ModelName  = 'text-embedding-3-small'
WHERE DocumentId = 1;

```

Three design decisions in that script deserve emphasis. First, `VECTOR(1536)` fixes the dimensionality at the type level — attempting to store a 384-dimension vector in a `VECTOR(1536)` column fails loudly at write time, which is exactly what you want. Silent dimension mismatches are a notorious source of garbage search results. Second, `ModelName` records which embedding model produced each vector, because vectors from different models live in different coordinate spaces and cannot be mixed (Chapter 30 covers model versioning). Third, the single-column integer primary key is not just good practice — it is a prerequisite for the DiskANN vector index should you add one later.

```

graph LR
    A[Verify engine  
17.x] --> B[Smoke test  
VECTOR_DISTANCE]
    B --> C{On 2025?}
    C -->|No| D[Upgrade project  
or Azure SQL interim]
    C -->|Yes| E[Create VECTOR tables  
nullable embeddings]
    D --> E
    E --> F[Backfill embeddings  
Chapter 8]
    F --> G[Exact search  
GA today]
    G --> H[DiskANN index  
preview accelerator]

```

Common Gotchas on Day One

Dimension mismatches fail at write time — by design. If your embedding pipeline emits 1536-dimension vectors and someone points a 384-dimension model at the same table, the `CAST` fails. Treat that error as a guardrail, not a bug: it caught a model-configuration mistake before it poisoned your index.

NULL embeddings sort and filter normally. `VECTOR_DISTANCE` with a NULL input returns NULL, so un-embedded rows naturally fall out of `ORDER BY` ranking. Use `WHERE Embedding IS NOT NULL` in retrieval queries during backfill windows to keep plans clean.

Storage math. A 1536-dimension float32 vector occupies about 6 KB. One million vectors is roughly 6 GB of raw column data before indexes — trivial for modern storage, but worth knowing when sizing the buffer pool for hot vector workloads. Higher-dimension models (3072) double it; smaller models (384) cut it to about 1.5 KB per vector.

Do not skip the driver upgrade. The most common day-one failure is not the engine at all — it is an old `SqlClient` or `ODBC` driver that cannot parse the new TDS type metadata for `VECTOR` columns. If reads of vector columns fail while everything else works, the driver is the first suspect.

Key Takeaways

- → Native vector support needs SQL Server 2025 (version 17.x) — verify with `SERVERPROPERTY('ProductMajorVersion')` and a one-line `VECTOR_DISTANCE` smoke test. There is nothing to install: vectors are part of the engine.
- → The `VECTOR(n)` type and `VECTOR_DISTANCE` are generally available and production-ready; the DiskANN vector index and `VECTOR_SEARCH` are in preview with real constraints — design so the GA features carry production and the index is an accelerator.
- → If you are not on 2025 yet, run vector enablement as a workstream inside the upgrade project — side-by-side migration, compatibility-level validation, and driver upgrades included — or use Azure SQL Database as an interim home.
- → The canonical table pattern keeps source content and a nullable `VECTOR(n)` column in the same row, records the embedding model per row, and uses a single-column integer primary key to stay eligible for a future DiskANN index.
- → Day-one gotchas: dimension mismatches fail loudly at write time (a feature), NULL embeddings need explicit filtering during backfill, budget ~6 KB per 1536-dimension vector, and upgrade client drivers so they understand the new TDS type.

Chapter 8: Creating and Storing Embeddings in SQL Server

An embedding pipeline has two halves: generating vectors from content, and storing them where queries can reach them. Most teams overthink the first half and underthink the second. Generating an embedding is an API call; storing it well is a schema design, bulk-load, backfill, and lifecycle discipline that determines whether your vector search stays fast and correct for years. This chapter covers the full write path in SQL Server 2025: table patterns for documents and chunks, inserting vectors with proper type conversions, generating embeddings from inside the database with `sp_invoke_external_rest_endpoint`, bulk-loading millions of vectors efficiently, and backfilling the inevitable gaps.

Schema Patterns: Documents, Chunks, and Embeddings

Two schema shapes cover nearly every vector workload. The simple shape — one row per document with a `VECTOR` column — works when each document is embedded whole. The chunked shape — a parent document table plus a child chunk table carrying the embeddings — is what production RAG systems actually use, because retrieval quality depends on chunking (Chapter 22) and chunks are the unit of both embedding and ranking.

```
-- Production RAG schema: documents + chunks, embeddings on the chunks
CREATE TABLE dbo.RagDocuments (
    DocumentId INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_RagDocuments PRIMARY KEY,
    Title NVARCHAR(300) NOT NULL,
    SourceUri NVARCHAR(1000) NULL,
    DocHash BINARY(32) NOT NULL, -- SHA-256 of source: detects changes
    CreatedAt DATETIME2(0) NOT NULL CONSTRAINT DF_RagDocuments_CreatedAt DEFAULT
    SYSUTCDATETIME()
);

CREATE TABLE dbo.RagChunks (
    ChunkId INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_RagChunks PRIMARY KEY,
    DocumentId INT NOT NULL CONSTRAINT FK_RagChunks_Documents
        REFERENCES dbo.RagDocuments (DocumentId) ON DELETE CASCADE,
    ChunkSeq INT NOT NULL, -- order within the document
    ChunkText NVARCHAR(MAX) NOT NULL,
    TokenCount INT NOT NULL,
    Embedding VECTOR(1536) NULL, -- NULL until embedded
    ModelName NVARCHAR(100) NULL,
    EmbeddedAt DATETIME2(0) NULL,
    CONSTRAINT UQ_RagChunks_DocSeq UNIQUE (DocumentId, ChunkSeq)
);
CREATE INDEX IX_RagChunks_DocumentId ON dbo.RagChunks (DocumentId);
CREATE INDEX IX_RagChunks_NeedsEmbedding ON dbo.RagChunks (DocumentId)
    WHERE Embedding IS NULL; -- filtered index drives backfill
```

Several deliberate choices here. `DocHash` lets the indexing pipeline detect when source content changed and re-chunk only what moved (Chapter 28). The filtered index on `Embedding IS NULL` makes "find everything still needing embeddings" a trivial seek — the backfill query you will run most often. The single-column integer primary key on the chunk table keeps the door open for a DiskANN vector index later. And `ModelName` per chunk means a model migration never silently mixes coordinate spaces.

Inserting Vectors: Conversions That Keep You Honest

Application code almost always produces embeddings as JSON arrays of floats. SQL Server converts those strings to the native type with `CAST` — and the conversion is strict about dimensions, which is a guardrail worth keeping:

```

-- Strict conversion: wrong dimension count fails loudly (good)
UPDATE dbo.RagChunks
SET Embedding = CAST('[0.012, -0.043, 0.071, ...]' AS VECTOR(1536)),
    ModelName = 'text-embedding-3-small',
    EmbeddedAt = SYSUTCDATETIME()
WHERE ChunkId = 42;

-- Defensive pattern for pipeline code: TRY_CAST + explicit check
UPDATE dbo.RagChunks
SET Embedding = TRY_CAST(@embeddingJson AS VECTOR(1536)),
    ModelName = @modelName,
    EmbeddedAt = SYSUTCDATETIME()
WHERE ChunkId = @chunkId;

-- Catch rows where conversion failed instead of silently storing NULL
SELECT ChunkId, DocumentId
FROM dbo.RagChunks
WHERE EmbeddedAt IS NULL AND ChunkId IN (...); -- investigate these

```

From .NET, the cleanest path is `SqlClient` with the vector passed as a string parameter converted in T-SQL, or EF Core 10+, whose SQL Server provider maps `float[]` properties to `VECTOR(n)` columns with LINQ support for `VECTOR_DISTANCE`. Python pipelines typically use `pyodbc` with the JSON-string-plus-CAST pattern. Whichever client you use, parameterize — never concatenate embedding JSON into dynamic SQL.

Generating Embeddings Inside the Database

SQL Server can call out to embedding APIs directly through `sp_invoke_external_rest_endpoint`, which invokes an HTTPS endpoint — such as Azure OpenAI's embeddings API — and returns the response as JSON. This is genuinely useful for backfills, prototypes, and low-volume workloads: the database that stores the vectors also orchestrates their creation, with no middle-tier service to build.


```

-- Generate an embedding via Azure OpenAI without leaving T-SQL
-- (Requires a database-scoped credential for the endpoint; see docs)
DECLARE @payload NVARCHAR(MAX) = JSON_OBJECT(
    'input': (SELECT ChunkText FROM dbo.RagChunks WHERE ChunkId = 42),
    'model': 'text-embedding-3-small'
);
DECLARE @response NVARCHAR(MAX);

EXEC sp_invoke_external_rest_endpoint
    @url      = 'https://YOUR-RESOURCE.openai.azure.com/openai/deployments/YOUR-
DEPLOYMENT/embeddings?api-version=2024-10-21',
    @method   = 'POST',
    @payload  = @payload,
    @response = @response OUTPUT;

-- Extract the embedding array and store it as a native VECTOR
UPDATE dbo.RagChunks
SET Embedding = CAST(JSON_QUERY(@response, '$.data[0].embedding') AS VECTOR(1536)),
    ModelName  = 'text-embedding-3-small',
    EmbeddedAt = SYSUTCDATETIME()
WHERE ChunkId = 42;

```

Be clear-eyed about the limits of this pattern. It issues one HTTPS call per invocation, so embedding a million chunks row-by-row through the stored procedure is slow and expensive — and it couples your database to the latency and rate limits of an external API. The right mental model: `sp_invoke_external_rest_endpoint` is for backfills of modest size, administrative re-embedding, and prototypes. Production ingestion at scale belongs in an application pipeline (Azure Functions, a container job, ADF) that batches requests, honors rate limits, and writes vectors with bulk load — which is the next section.

Bulk Loading: Millions of Vectors Without the Pain

The initial load — and every model migration — is a bulk operation, and it should be treated like one. Row-by-row inserts of million-row embedding sets will take hours and generate transaction log pressure that affects the whole instance. The DBA playbook applies directly:

- → **Stage first, then merge.** Bulk-load raw embedding JSON into a staging table (heap, no indexes), then `INSERT ... SELECT` with `CAST(... AS VECTOR(n))` into the production table. Validation failures land in staging where they are cheap to inspect, not in production where they roll back a giant transaction.
- → **Batch the transaction.** Commit every 5,000–10,000 rows. This bounds transaction log growth, keeps lock escalation from surprising you, and gives restartable checkpoints if the load fails halfway.

- → **Drop the vector index first.** If a DiskANN vector index exists on the target table, drop it before the bulk load and rebuild after. Index maintenance during bulk insert is pure overhead, and the preview vector index requires drop-and-recreate to incorporate new data anyway.
- → **Use the fastest path available.** `bcp`, `BULK INSERT`, or `SqlBulkCopy` into staging; set-based `INSERT...SELECT` for the typed load. `TABLOCK` on the staging insert enables minimal logging where the recovery model allows it.
- → **Validate dimensions in staging.** A single malformed vector — wrong dimension count, NaN values, non-numeric tokens — aborts a batch. Check `TRY_CAST` results in staging and quarantine failures before they touch production.

```
-- Bulk pattern: stage raw JSON, validate, then typed insert in batches
CREATE TABLE dbo.StageEmbeddings (
    ChunkId          INT NOT NULL,
    EmbeddingJson    NVARCHAR(MAX) NOT NULL,
    ModelName        NVARCHAR(100) NOT NULL
); -- heap, no indexes: fastest possible load target

-- ... bcp / BULK INSERT the raw JSON here ...

-- Quarantine malformed rows before they poison the batch
SELECT ChunkId, LEFT(EmbeddingJson, 80) AS Sample
INTO dbo.QuarantineEmbeddings
FROM dbo.StageEmbeddings
WHERE TRY_CAST(EmbeddingJson AS VECTOR(1536)) IS NULL;

DELETE s FROM dbo.StageEmbeddings AS s
JOIN dbo.QuarantineEmbeddings AS q ON q.ChunkId = s.ChunkId;

-- Typed, batched insert into production
DECLARE @batch INT = 10000, @done INT = 0;
WHILE 1 = 1
BEGIN
    UPDATE TOP (@batch) c
    SET c.Embedding = CAST(s.EmbeddingJson AS VECTOR(1536)),
        c.ModelName = s.ModelName,
        c.EmbeddedAt = SYSUTCDATETIME()
    FROM dbo.RagChunks AS c
    JOIN dbo.StageEmbeddings AS s ON s.ChunkId = c.ChunkId
    WHERE c.Embedding IS NULL;

    IF @@ROWCOUNT = 0 BREAK;
    SET @done += @@ROWCOUNT;
    RAISERROR('Embedded %d chunks so far', 0, 1, @done) WITH NOWAIT;
END
```

```

graph LR
    A[Source documents] --> B[Chunking  
Chapter 22]
    B --> C[Embedding API  
batched]
    C --> D[Staging table  
raw JSON heap]
    D --> E[Validate  
TRY_CAST quarantine]
    E --> F[Typed batch insert  
VECTOR column]
    F --> G[Vector index  
rebuild]

```

Backfill and Lifecycle: Vectors Are Never "Done"

Embedding storage is a lifecycle, not a one-time load. Documents get added, edited, and deleted; models get upgraded; chunks get re-split. Design the write path for the steady state:

- → **New documents** arrive with NULL embeddings; a scheduled job (SQL Agent or an external scheduler) picks them up via the filtered index and embeds them in batches. The application never blocks on embedding latency.
- → **Edited documents** are detected by comparing `DocHash` ; only changed documents are re-chunked and re-embedded. Deleting the old chunks first (or versioning them) prevents stale vectors from outranking fresh ones.
- → **Deleted documents** cascade to chunks through the foreign key — no orphan vectors. If you soft-delete instead, add the delete flag to every retrieval query's filter.
- → **Model changes** mean re-embedding everything into a new column (or table) and cutting over atomically — never mixing coordinate spaces in one column. Chapter 30 covers the migration playbook.

A simple monitoring query keeps the lifecycle honest — run it on a schedule and alert when the un-embedded backlog grows:

```

-- Backlog health: how much content is waiting for embeddings?
SELECT
    COUNT(*) AS TotalChunks,
    SUM(CASE WHEN Embedding IS NULL THEN 1 ELSE 0 END) AS AwaitingEmbedding,
    MIN(CreatedAt) AS OldestUnembedded
FROM dbo.RagChunks AS c
JOIN dbo.RagDocuments AS d ON d.DocumentId = c.DocumentId;

```

Key Takeaways

- → Use the chunked schema — documents plus a child chunk table carrying the `VECTOR(n)` column — with a content hash for change detection, a filtered index on un-embedded rows, and the embedding model recorded per row.
- → Convert embedding JSON with `CAST(... AS VECTOR(n))` and let dimension mismatches fail loudly; use `TRY_CAST` plus quarantine tables in pipeline code so one malformed vector never aborts a batch.
- → `sp_invoke_external_rest_endpoint` lets T-SQL call embedding APIs directly — ideal for backfills and prototypes, but production ingestion at scale belongs in a batched application pipeline.
- → Bulk-load in three stages: raw JSON into an unindexed staging heap, validation with quarantine, then batched typed inserts — dropping any DiskANN vector index before the load and rebuilding after.
- → Treat embeddings as a lifecycle: NULL-then-backfill for new documents, hash-based change detection for edits, cascade deletes for removals, and a monitored backlog query so the pipeline never silently stalls.

Chapter 9: DiskANN Indexing in SQL Server 2025: Performance Tradeoffs

Vector search at scale lives or dies by its index structure. When you move beyond brute-force exact search — the vector equivalent of a full table scan — you enter the world of Approximate Nearest Neighbor (ANN) algorithms. SQL Server 2025 made its choice for you: the native vector index is built on **DiskANN**, a graph-based ANN algorithm designed at Microsoft Research for billion-scale search on SSD-backed storage. This chapter unpacks how DiskANN works, where it sits relative to the exact search you already know, what the index costs to build and maintain, and how to reason about the recall-versus-latency tradeoff that defines every ANN system. If B-tree fill-factor and fragmentation tradeoffs feel natural to you, this chapter will feel like home.

The ANN Problem: Why Exact Search Stops Scaling

Exact nearest-neighbor search with `VECTOR_DISTANCE` computes the distance from the query vector to every stored vector. At 1 million vectors of 1536 dimensions, that is roughly 1.5 billion floating-point operations per query — hundreds of milliseconds even with SIMD acceleration, before any I/O. It is perfectly accurate (100% recall by definition) and perfectly acceptable up to about 50,000 vectors. Past that, latency grows linearly with table size, and no amount of hardware generosity changes the math.

ANN algorithms trade a small, measurable amount of recall for dramatic speed improvements. **Recall** is the fraction of the true top-k nearest neighbors that the approximate search actually returns: 0.97 recall means 97 of the 100 true nearest neighbors were found. The core insight is identical to why you do not read every page of a book to find a topic — you use an index that organizes content so you can skip most of it. For most production AI workloads, 97% recall at 10 milliseconds is far more valuable than 100% recall at 1,500 milliseconds — and embeddings are already approximate representations of meaning, so the last few percent of geometric precision rarely changes what the user experiences.

How DiskANN Works: Graphs Instead of Trees

DiskANN builds a navigable graph over the vector collection: each vector becomes a node connected to a bounded set of its nearest neighbors. A query starts at a fixed entry point and greedily walks the graph — at each step moving to the neighbor closest to the query vector — until no unvisited neighbor is closer than the current best candidates. Because the graph is constructed so that greedy routing converges quickly, search touches only a tiny fraction of the collection: query time grows logarithmically with data size rather than linearly.

What distinguishes DiskANN from earlier graph indexes (notably HNSW, which powers several dedicated vector databases covered in Chapters 15–19) is its storage design. HNSW keeps the entire graph in RAM, which makes it fast but memory-hungry at billion-vector scale. DiskANN was designed for SSDs: it stores the full-precision vectors and most of the graph on disk, keeps only a compressed or sampled representation in memory, and issues a small, bounded number of random SSD reads per query. The result is an index that serves billion-scale collections from a single machine without requiring terabytes of RAM — a design that maps naturally onto SQL Server's buffer-pool-and-storage architecture.

For the DBA, the useful mental model is: **a DiskANN vector index is a specialized access path for ORDER BY VECTOR_DISTANCE queries, analogous to how a B-tree is a specialized access path for range predicates.** The query optimizer chooses between the exact scan path and the approximate index path the same way it chooses between a table scan and an index seek — based on cost, with the added dimension that the approximate path deliberately sacrifices a few percent of recall for orders of magnitude less work.

```
graph TD
    A[Query vector] --> B[Entry point node]
    B --> C{Greedy walk:
move to closest
unvisited neighbor}
    C -->|Closer neighbor exists| C
    C -->|No closer neighbor| D[Return top-k
candidates]
    D --> E[Rerank exact distances
optional]
```

Creating and Querying the Vector Index

The DDL is deliberately minimal — one statement, with the distance metric as the key decision:

```

-- DiskANN approximate vector index (preview in SQL Server 2025)
CREATE VECTOR INDEX IX_RagChunks_Embedding
ON dbo.RagChunks (Embedding)
WITH (
    METRIC = 'cosine',      -- must match the metric your queries use
    TYPE   = 'DiskANN',
    MAXDOP = 4              -- parallelism for the index build
);

-- Approximate k-NN through the index
DECLARE @q VECTOR(1536) = '[0.012, -0.043, 0.071, ...]';

SELECT t.ChunkId, t.ChunkText, s.distance
FROM VECTOR_SEARCH(
    TABLE      = dbo.RagChunks AS t,
    COLUMN      = Embedding,
    SIMILAR_TO  = @q,
    METRIC      = 'cosine',
    TOP_N       = 10
) AS s
ORDER BY s.distance;

```

The `METRIC` choice is the single most consequential parameter: it must match both your embedding model's normalization characteristics and the metric your queries specify. Cosine is the right default for normalized text embeddings (OpenAI, most sentence-transformer models). Euclidean suits unnormalized embeddings where magnitude carries meaning. Dot product matches models trained with inner-product objectives. An index built with the wrong metric does not error — it silently returns poorly ranked results, which is worse.

Note the current preview constraints, because they shape real designs: the table must have a single-column integer clustered primary key; the table becomes read-only while the vector index exists in SQL Server 2025; incorporating new data means dropping and recreating the index; and partitioned tables, views, and temporary tables are not supported. These constraints are why Chapter 8's schema patterns keep embeddings in ordinary tables with ordinary keys — the vector index stays a droppable performance layer, never a structural dependency.

The Tradeoff Table: Exact vs. DiskANN

Every ANN decision is a position on three axes — recall, latency, and build/maintenance cost. Here is where the two SQL Server search paths land:

	Exact (VECTOR_DISTANCE)	Approximate (DiskANN)

|---|---|---|

| Recall | 100% by definition | Typically 95–99%, tunable |

| Query cost | $O(n)$: full scan per query | $O(\log n)$: graph walk |

| Latency at 1M vectors | Hundreds of ms | Single-digit to tens of ms |

| Build cost | None | Minutes to hours for large collections |

| Write path | Normal DML | Drop index → load → rebuild (preview) |

| Memory pressure | Buffer pool for scanned pages | Graph + vectors compete for buffer pool |

| Suitable for | Under ~50K vectors; 100% recall required | 50K+ vectors; interactive latency SLAs |

The crossover point deserves emphasis because teams routinely misjudge it in both directions. Below roughly 50,000 vectors, exact search is so fast that the DiskANN index buys you almost nothing while adding build time and write-path friction — skip it. Above that, latency divergence becomes impossible to ignore, and the index pays for itself immediately. Between 50K and a few hundred thousand vectors, either path works; choose based on whether your latency SLA has headroom.

Tuning Recall: What to Measure

"Approximate" is only acceptable when you can measure how approximate. The standard methodology: sample a few hundred representative query vectors, run each through both the exact path (`VECTOR_DISTANCE` with `ORDER BY`) and the approximate path (`VECTOR_SEARCH`), and compute `recall@k` — the fraction of the exact top-k present in the approximate top-k. Automate this as a scheduled job; recall is the canary for index health, degrading when the data distribution drifts away from what the index was built on.


```

-- Recall spot-check pattern: compare exact vs approximate top-10
-- Run for a sample of query vectors; recall@10 = |intersection| / 10
DECLARE @q VECTOR(1536) = '[0.012, -0.043, 0.071, ...]';

-- Exact ground truth
SELECT TOP (10) ChunkId
INTO #exact
FROM dbo.RagChunks
WHERE Embedding IS NOT NULL
ORDER BY VECTOR_DISTANCE('cosine', Embedding, @q);

-- Approximate candidates
SELECT TOP (10) ChunkId
INTO #approx
FROM VECTOR_SEARCH(
    TABLE = dbo.RagChunks AS t, COLUMN = Embedding,
    SIMILAR_TO = @q, METRIC = 'cosine', TOP_N = 10
) AS s;

-- Overlap = recall@10 for this query
SELECT COUNT(*) * 1.0 / 10 AS recall_at_10
FROM #exact AS e JOIN #approx AS a ON a.ChunkId = e.ChunkId;

DROP TABLE #exact, #approx;

```

If recall slips below your target (0.95 is a common floor for RAG retrieval), the remedies in order of cost: increase `TOP_N` and rerank the surplus with exact `VECTOR_DISTANCE` (cheap, often sufficient); rebuild the index after significant data churn (moderate); reconsider chunking or the embedding model (expensive, but sometimes the real problem is upstream of the index).

Key Takeaways

- → SQL Server 2025's native vector index uses DiskANN — a graph-based ANN algorithm designed for SSD-backed, billion-scale search — created with `CREATE VECTOR INDEX ... WITH (TYPE = 'DiskANN')` and queried via the `VECTOR_SEARCH` table-valued function.
- → Exact search (`VECTOR_DISTANCE`) gives 100% recall with $O(n)$ cost per query and needs no index — it is the right choice under roughly 50,000 vectors. DiskANN gives 95–99% recall with $O(\log n)$ cost and is the right choice above that.
- → The `METRIC` parameter must match your embedding model and your queries; a mismatched metric silently degrades ranking quality instead of erroring.
- → Preview constraints shape the design: single-column integer clustered key required, table read-only while the index exists, drop-and-recreate to ingest new data — keep the vector index a droppable performance layer, not a structural dependency.

- → Measure recall@k by comparing `VECTOR_SEARCH` against exact `VECTOR_DISTANCE` on a query sample, on a schedule. When recall slips, over-fetch with larger `TOP_N` and rerank exactly before rebuilding anything.

Chapter 10: Tuning Vector Workloads in SQL Server

Vector workloads have a performance profile unlike anything else on a SQL Server instance: large sequential scans for exact search, random graph-walk I/O for DiskANN queries, and memory pressure from multi-kilobyte rows that dwarf typical OLTP rows. The good news is that the tuning toolkit is the one you already own — index design, query patterns, memory configuration, concurrency management, and Automatic Tuning. This chapter applies each of them to vector workloads specifically: when to choose exact versus approximate search, how to design the DiskANN index, how to write retrieval queries the optimizer loves, how to size memory, and how to keep hundreds of concurrent similarity searches from stepping on each other.

Choose the Search Path Deliberately

The first tuning decision is architectural, not knobs: exact search or approximate search. Exact search — `ORDER BY VECTOR_DISTANCE(...)` over the table — scans every vector and computes every distance. It needs no index, gives 100% recall, and performs well up to roughly 50,000 vectors. Approximate search through a DiskANN vector index and `VECTOR_SEARCH` trades a few percent of recall for logarithmic scaling into the millions.

The mistake to avoid is defaulting to one path for everything. A support-ticket search over 20,000 historical tickets does not need a vector index — the build time and the preview write-path constraints buy nothing. A product catalog with 2 million embeddings and a 100ms p99 SLA cannot live on exact search no matter how much hardware you add. Measure your corpus size, set the path explicitly per table, and revisit the decision as the corpus grows — the crossover is a function of data volume, not of ambition.

DiskANN Index Design

When the approximate path is warranted, the index design has exactly three decisions, and only one of them is subtle.

1. The metric must match the model and the queries. `METRIC = 'cosine'` for normalized text embeddings (the common case), `'euclidean'` where magnitude matters, `'dot'` for inner-product-trained models. The index does not validate this for you — a mismatched metric returns confidently wrong rankings. Standardize the metric per table in a data dictionary and assert it in deployment scripts.

2. Build with parallelism, then leave it alone. Index builds over millions of vectors are CPU-bound graph construction. `MAXDOP` in the `WITH` clause controls build parallelism — size it to what the instance can spare during the maintenance window, since a build competes with everything else for CPU and buffer pool. Schedule builds in low-traffic windows; there is no online build for vector indexes in the current preview.

3. Plan the write path around drop-and-recreate. In the current preview, the table is read-only while the vector index exists, and new data requires dropping and rebuilding the index. Do not fight this — design for it. The production pattern is a staging-table swap: bulk-load new embeddings into a staging table, build the vector index there, then switch it into production with a metadata operation (`ALTER SCHEMA ... TRANSFER` or partition switching where applicable). Retrieval queries see either the old or the new index, never a half-built one.

```
-- Production refresh pattern: build on staging, swap into production
-- 1. Load new embeddings into staging (ordinary table, same schema)
-- 2. Build the vector index on staging during the maintenance window
CREATE VECTOR INDEX IX_Stage_Embedding
ON dbo.RagChunks_Stage (Embedding)
WITH (METRIC = 'cosine', TYPE = 'DiskANN', MAXDOP = 8);

-- 3. Atomic swap: staging becomes production
BEGIN TRAN;
    EXEC sp_rename 'dbo.RagChunks', 'RagChunks_Old';
    EXEC sp_rename 'dbo.RagChunks_Stage', 'RagChunks';
COMMIT;
-- 4. Drop the old table after validation
```

Write Retrieval Queries the Optimizer Loves

Vector retrieval queries have a shape, and writing to that shape keeps plans stable. For exact search, the canonical pattern filters relationally first, then ranks:

```
-- Exact search: relational pre-filter, then rank by distance
DECLARE @q VECTOR(1536) = '...';

SELECT TOP (20)
    c.ChunkId, c.ChunkText,
    VECTOR_DISTANCE('cosine', c.Embedding, @q) AS Distance
FROM dbo.RagChunks AS c
JOIN dbo.RagDocuments AS d ON d.DocumentId = c.DocumentId
WHERE c.Embedding IS NOT NULL
    AND d.SourceUri LIKE 'https://intranet/%' -- relational pre-filter
ORDER BY Distance;
```

For approximate search, keep the `VECTOR_SEARCH` call clean and do relational work around it — join its results back to dimension tables for display columns and permission checks rather than stuffing predicates inside the TVF call:

```
-- Approximate search: ANN retrieval, then relational enrichment
DECLARE @q VECTOR(1536) = '...';

SELECT TOP (20)
    c.ChunkId, c.ChunkText, d.Title, s.distance
FROM VECTOR_SEARCH(
    TABLE = dbo.RagChunks AS t,
    COLUMN = Embedding,
    SIMILAR_TO = @q,
    METRIC = 'cosine',
    TOP_N = 50 -- over-fetch, then trim
) AS s
JOIN dbo.RagChunks AS c ON c.ChunkId = s.ChunkId
JOIN dbo.RagDocuments AS d ON d.DocumentId = c.DocumentId
WHERE d.IsPublished = 1 -- relational post-filter on the small result set
ORDER BY s.distance;
```

Two techniques in that query deserve names. **Over-fetch and trim:** request more candidates from the index (`TOP_N = 50`) than you display (20), because post-filters and reranking consume candidates — asking for exactly 20 and filtering down to 12 is the most common recall bug in production RAG. **Parameterize the query vector** like any other parameter: ad-hoc embedding literals in dynamic SQL pollute the plan cache with single-use plans. Pass `@q` as a parameter from `SqlClient` or a stored procedure argument.

Watch for plan-cache bloat from embedding literals the way you would watch for any non-parameterized workload: `sys.dm_exec_cached_plans` grouped by query hash will show it immediately. And enable Query Store on vector databases — vector queries are exactly the kind of workload where a plan regression (say, the optimizer abandoning the vector index path after a statistics update) is worth catching automatically rather than discovering from user complaints.

Memory: Vectors Are Hungry

A 1536-dimension float32 vector is about 6 KB — roughly a thousand times the size of a typical OLTP row. That changes the memory math. Ten thousand hot vectors fit comfortably in any buffer pool; ten million need ~60 GB before the DiskANN graph overhead. Size accordingly:

- → **Keep the working set in the buffer pool.** DiskANN is designed for SSD-backed operation and tolerates buffer-pool misses better than HNSW-style in-memory indexes tolerate RAM exhaustion — but every physical read adds latency to the p99. Monitor `sys.dm_os_buffer_descriptors` for the vector database to see how much of the corpus is actually resident, and size max server memory so the hot portion stays cached.
- → **Do not starve the rest of the instance.** On shared instances, cap the vector database's appetite with Resource Governor: a workload group limiting memory grants for the retrieval login prevents a burst of concurrent similarity searches from evicting everyone else's hot pages. Vector queries are the classic noisy neighbor because each one touches so much data.
- → **Mind the distance-computation CPU.** Exact search is a CPU burn — billions of multiply-adds per query. Track `sys.dm_exec_query_stats` total worker time for your retrieval procedures; if CPU, not I/O, dominates, the answer is the DiskANN index (fewer distance computations), not a bigger VM.

Concurrency: Pooling, Isolation, and Automatic Tuning

Vector search endpoints are typically chatty — a RAG application issues a retrieval query per user question, and user questions arrive concurrently. Three settings carry most of the weight.

Connection pooling is not optional. Establishing a new SQL Server connection costs milliseconds you do not have in a 100ms retrieval budget, and embedding JSON parameters are large enough that you want them flowing over warm connections. Use `SqlClient` connection pooling (enabled by default in .NET — do not disable it), size `Max Pool Size` for your actual concurrency rather than leaving the default 100 to be discovered under load, and keep the retrieval queries on a dedicated connection string so pool exhaustion in the OLTP workload cannot starve vector search (and vice versa).

Read committed snapshot for retrieval. Vector retrieval is overwhelmingly read-only, and it should never block on — or be blocked by — the embedding ingestion writes. Enable `READ_COMMITTED_SNAPSHOT` on vector databases so readers get row versions from tempdb instead of taking shared locks. Size tempdb accordingly; version-store pressure from a heavy backfill running alongside production retrieval is a real phenomenon.

Let Automatic Tuning watch the plans. SQL Server's Automatic Tuning (automatic plan correction) is well suited to vector workloads because retrieval queries are highly repetitive — the same parameterized shape executed thousands of times. When a plan regression appears (for example, after statistics update on a rapidly growing chunk table), automatic plan correction can force the last known good plan while you investigate. It is not a substitute for understanding the plans (Chapter 14), but it is a safety net that costs nothing to enable:

```
-- Safety net for repetitive retrieval queries: automatic plan correction
ALTER DATABASE VectorRag SET AUTOMATIC_TUNING (FORCE_LAST_GOOD_PLAN = ON);

-- Verify what it has done lately
SELECT reason, score, details
FROM sys.dm_db_tuning_recommendations;
```

```
graph TD
    A[Vector workload tuning] --> B{Corpus size?}
    B -->|Under ~50K| C[Exact search  
no vector index]
    B -->|Over ~50K| D[DiskANN index  
MAXDOP build]
    C --> E[Parameterized queries  
Query Store on]
    D --> E
    E --> F[Buffer pool sized  
for hot vectors]
    F --> G[SqlClient pooling  
RCSI reads]
    G --> H[Automatic Tuning  
plan correction]
```

Key Takeaways

- → Choose the search path by corpus size: exact `VECTOR_DISTANCE` under ~50K vectors with no index to maintain; DiskANN plus `VECTOR_SEARCH` above it. Revisit the decision as the corpus grows.

- → DiskANN design is three decisions: metric matching the model (cosine default), parallel builds in maintenance windows, and a staging-swap refresh pattern that respects the preview drop-and-recreate write path.
- → Write retrieval queries to a stable shape: relational pre-filter then rank for exact search; clean `VECTOR_SEARCH` with over-fetch (`TOP_N` above display count) and relational enrichment after. Parameterize the query vector and enable Query Store.
- → Vectors are memory-hungry (~6 KB per 1536-dim vector): size the buffer pool for the hot set, use Resource Governor on shared instances, and read `sys.dm_exec_query_stats` CPU to distinguish compute-bound exact search (needs the index) from I/O-bound retrieval (needs memory).
- → For concurrency: SqlConnection connection pooling on a dedicated connection string, `READ_COMMITTED_SNAPSHOT` so readers never block writers, and Automatic Tuning's plan correction as a safety net for the highly repetitive retrieval shapes.

Chapter 11: Hybrid Search: Combining Full-Text and Vector Search

Full-text search and vector search solve different problems, and in production AI applications you almost always need both. Full-text search excels at exact keyword matching, proper nouns, SKU codes, error numbers, and domain-specific terminology that embedding models may not have seen during training. Vector search excels at semantic similarity, handling paraphrase, typos, and intent that keywords miss entirely. This chapter covers how to combine these two retrieval mechanisms into a single pipeline on SQL Server 2025 — the architecture decisions, scoring fusion strategies, and the practical implementation details that determine whether hybrid search actually outperforms either approach alone. You will come away with working T-SQL for full-text plus vector fusion, the reciprocal-rank pattern used across the industry, and a mental model for tuning the balance between lexical and semantic signals.

Two Retrieval Models, Complementary Failures

Traditional full-text search — whether SQL Server Full-Text Search with `CONTAINS`, Elasticsearch BM25, or any TF-IDF-based engine — operates on term frequency and inverse document frequency with normalization. The retrieval model rewards documents that share exact terms with the query. This creates predictable failure modes:

- → **Vocabulary mismatch.** A query for "myocardial infarction" misses documents that only say "heart attack." An error search for "login timeout" misses "authentication expired."
- → **Paraphrase blindness.** "How do I make queries faster?" and "query performance tuning guide" share almost no tokens but mean the same thing.
- → **Typos and morphology.** "color" vs "colour," "optimise" vs "optimize" — stemming and word-breakers help, but only within the limits of the language rules installed.

Vector search inverts these strengths and weaknesses: it finds meaning without terms, but it is sloppy about exact identifiers. A query for the specific error number 18456 will match documents about login failures generally — but the one document that names 18456 in its title might rank below five loosely-related docs. Hybrid search exists to get both: the precision of lexical matching and the recall of semantic matching, fused into one ranked list.

Full-Text Search in SQL Server: The Lexical Side

SQL Server's full-text engine (iFTS) has been production-hardened for decades: full-text catalogs, word breakers per language, thesaurus files for synonyms, and the

`CONTAINSTABLE` / `FREETEXTTABLE` functions that return ranked results. If you already run full-text indexes, hybrid search does not replace them — it adds a second ranking signal on the same tables.

```
-- SQL Server: full-text catalog + index on the documents table
CREATE FULLTEXT CATALOG FTC_KnowledgeBase AS DEFAULT;

CREATE FULLTEXT INDEX ON dbo.KnowledgeChunks (ChunkText LANGUAGE 1033)
    KEY INDEX PK_KnowledgeChunks
    WITH STOPLIST = SYSTEM, CHANGE_TRACKING = AUTO;
-- CHANGE_TRACKING = AUTO keeps the full-text index current on every write

-- Ranked lexical search: CONTAINSTABLE returns [KEY] + [RANK]
DECLARE @keywords NVARCHAR(4000) = N'"login timeout" OR 18456';

SELECT TOP (50) c.ChunkId, c.Title, ft.[RANK] AS FtsRank
FROM CONTAINSTABLE(dbo.KnowledgeChunks, ChunkText, @keywords, 50) AS ft
JOIN dbo.KnowledgeChunks AS c ON c.ChunkId = ft.[KEY]
ORDER BY ft.[RANK] DESC;
```

`CONTAINSTABLE` gives you boolean control (phrases, `AND` / `OR` / `NEAR`, prefix terms) plus a rank; `FREETEXTTABLE` does the "find me documents like this text" expansion, which is itself a useful query-rewriting step before vector search. Both are maintained incrementally — with `CHANGE_TRACKING = AUTO`, freshness is not a concern, and neither is the full-text population schedule once you monitor `sys.fulltext_indexes` for crawl lag.

The Fusion Pattern: Reciprocal Rank Fusion in T-SQL

The industry-standard way to combine two ranked lists is **reciprocal rank fusion (RRF)**: for each document, sum $1 / (k + \text{rank})$ across the lexical and semantic rankings (k is typically 60). RRF needs no score normalization — it works on ranks alone, which is exactly what you want when one system returns BM25-ish ranks and the other returns cosine distances on incompatible scales. Documents appearing high in both lists get the strongest boost, which is precisely the behavior you want from a hybrid system.

```

-- SQL Server 2025: hybrid search with reciprocal rank fusion (RRF)
-- Lexical branch: CONTAINSTABLE rank. Semantic branch: cosine distance rank.
DECLARE @keywords NVARCHAR(4000) = N'"login timeout" OR 18456';
DECLARE @q VECTOR(1536) = '[0.12, 0.87, 0.34, ...]'; -- query embedding
DECLARE @k FLOAT = 60.0; -- RRF constant

WITH Lexical AS (
    SELECT ft.[KEY] AS ChunkId,
           ROW_NUMBER() OVER (ORDER BY ft.[RANK] DESC) AS LexRank
    FROM CONTAINSTABLE(dbo.KnowledgeChunks, ChunkText, @keywords, 50) AS ft
),
Semantic AS (
    SELECT TOP (50) ChunkId,
           ROW_NUMBER() OVER (
               ORDER BY VECTOR_DISTANCE('cosine', Embedding, @q) AS SemRank
            FROM dbo.KnowledgeChunks
            WHERE Embedding IS NOT NULL
        )
)
SELECT TOP (10)
    c.ChunkId, c.Title, c.ChunkText,
    COALESCE(1.0 / (@k + l.LexRank), 0)
    + COALESCE(1.0 / (@k + s.SemRank), 0) AS RrfScore
FROM dbo.KnowledgeChunks AS c
LEFT JOIN Lexical AS l ON l.ChunkId = c.ChunkId
LEFT JOIN Semantic AS s ON s.ChunkId = c.ChunkId
WHERE l.ChunkId IS NOT NULL OR s.ChunkId IS NOT NULL
ORDER BY RrfScore DESC;

```

A few design notes on this pattern. Each branch retrieves its own top-50 before fusion — the branches are independent, so both can use their indexes (full-text index for the lexical branch, DiskANN via `VECTOR_SEARCH` for the semantic branch at scale). The `COALESCE` handles documents that appear in only one list: they still get their single-branch score rather than being dropped. And the semantic branch shown uses exact `VECTOR_DISTANCE` ranking; swap it for the `VECTOR_SEARCH` approximate path from Chapter 2 once the corpus justifies the index.

Weighted Fusion: Tuning the Lexical/Semantic Balance

RRF treats both signals equally. In practice, the right balance depends on your query distribution. A support-knowledge-base workload heavy on error numbers and product codes wants lexical weight; a conceptual documentation search wants semantic weight. The weighted variant normalizes each branch's scores to [0,1] and blends with an alpha parameter:

```

-- Weighted hybrid: alpha = 1.0 is pure semantic, 0.0 is pure lexical
DECLARE @alpha FLOAT = 0.6;    -- start here, then tune against labeled queries

WITH Lexical AS (
    SELECT ft.[KEY] AS ChunkId,
           CAST(ft.[RANK] AS FLOAT) / NULLIF(MAX(CAST(ft.[RANK] AS FLOAT)) OVER (),
0) AS LexScore
    FROM CONTAINSTABLE(dbo.KnowledgeChunks, ChunkText, @keywords, 50) AS ft
),
Semantic AS (
    SELECT TOP (50) ChunkId,
           1.0 - VECTOR_DISTANCE('cosine', Embedding, @q) / 2.0 AS SemScore
           -- cosine distance in [0,2] maps to similarity in [0,1]
    FROM dbo.KnowledgeChunks
    WHERE Embedding IS NOT NULL
    ORDER BY VECTOR_DISTANCE('cosine', Embedding, @q)
)
SELECT TOP (10) c.ChunkId, c.Title,
               @alpha * COALESCE(s.SemScore, 0)
               + (1 - @alpha) * COALESCE(l.LexScore, 0) AS HybridScore
FROM dbo.KnowledgeChunks AS c
LEFT JOIN Lexical AS l ON l.ChunkId = c.ChunkId
LEFT JOIN Semantic AS s ON s.ChunkId = c.ChunkId
WHERE l.ChunkId IS NOT NULL OR s.ChunkId IS NOT NULL
ORDER BY HybridScore DESC;

```

Tune alpha empirically: run a few hundred labeled queries through alpha values of 0.3, 0.5, 0.7 and measure precision@10 or MRR against your relevance judgments. The curve usually has a broad plateau — hybrid beats either pure approach across most of the range — which means you do not need to over-tune. Log the alpha value in your deployment configuration next to the embedding model version; both are retrieval parameters that future you will need.

How the Dedicated Engines Do It

The same two-signal idea appears in every dedicated vector engine, with different mechanics. **Weaviate** implements hybrid search natively with an `alpha` parameter: 1.0 is pure vector, 0.0 is pure BM25, and the default fusion is a normalized weighted sum — conceptually the same blend as the query above, maintained inside the engine's inverted index plus HNSW graph. **Pinecone** takes the sparse-plus-dense route: you supply a sparse vector (term weights, often BM25-derived) alongside the dense embedding, and the index scores both with a configurable alpha. **Qdrant** supports query-time fusion across multiple named vectors per point, letting you run the same RRF idea across dense and sparse sub-indexes.

The SQL Server approach differs in one useful way: the fusion logic lives in T-SQL you own, not inside an engine's opaque implementation. That makes the blend auditable, testable with ordinary query tools, and adjustable without re-indexing — you can A/B test alpha values in a stored procedure parameter and watch the effect in Query Store the same afternoon.

```
sequenceDiagram
    participant Q as Query
    participant L as Lexical branch
    (CONTAINSTABLE)
    participant S as Semantic branch
    (VECTOR_DISTANCE)
    participant F as Fusion (RRF / weighted)
    Q->>L: keywords → ranked ChunkIds
    Q->>S: embedding → ranked ChunkIds
    L->>F: LexRank per ChunkId
    S->>F: SemRank per ChunkId
    F->>Q: TOP 10 by fused score
```

Pre-Filtering and the Approximate Path

One operational subtlety carries over from Chapter 2's filtered-search discussion: when the semantic branch runs through the approximate `VECTOR_SEARCH` path, over-fetch `TOP_N` generously before fusion. RRF only sees the candidates each branch returns — if the semantic branch returns 10 candidates and the tenant filter or fusion drops half, the lexical branch's documents dominate the final list not because they are better, but because the semantic branch was starved. Ask for 50–100 per branch, fuse, then trim to the display count. On the exact path this costs CPU; on the approximate path it costs almost nothing.

Also note the interplay between full-text and vector freshness. With `CHANGE_TRACKING = AUTO`, newly inserted chunks are searchable lexically within seconds, while the vector side is current as soon as the ingest pipeline writes the embedding. If you run the DiskANN index (preview write-path constraint: drop and recreate to ingest), the semantic branch sees new data only after a rebuild — a freshness asymmetry worth documenting in your runbook, since users will notice that keyword search finds a document before semantic search does.

Key Takeaways

- → Full-text search (SQL Server `CONTAINSTABLE` / `FREETEXTTABLE`) handles exact terms, identifiers, and error codes; vector search handles meaning, paraphrase, and intent. Production retrieval needs both, fused into one ranked list.
- → Reciprocal rank fusion — $1/(k + \text{rank})$ summed across branches — combines rankings without score normalization and is implementable in a single T-SQL statement with two CTEs.
- → Weighted fusion with an alpha parameter lets you tune the lexical/semantic balance for your query distribution; tune it empirically against labeled queries and version it with your embedding model.
- → Retrieve generously per branch (50–100 candidates) before fusing — RRF can only rank the candidates it sees, and a starved branch silently biases the final list toward the other signal.
- → Keeping fusion logic in T-SQL you own makes the blend auditable and A/B-testable without re-indexing; watch for freshness asymmetry between the incremental full-text index and the rebuild-cycle DiskANN index.

Chapter 12: Multi-Tenant Vector Storage in SQL Server

Multi-tenancy turns every vector decision up a notch. One tenant's support tickets must never surface in another tenant's RAG answers; a noisy tenant's indexing burst must not starve everyone else's retrieval latency; onboarding tenant number ten thousand should be a row insert, not a migration project. SQL Server's tenancy toolkit — shared tables with tenant keys, row-level security, separate schemas, separate databases, elastic pools — maps cleanly onto vector workloads, with one vector-specific wrinkle: the approximate vector index does not understand tenants, so filtered search quality needs deliberate design. This chapter walks through the four tenancy patterns, their vector implications, and how to keep retrieval both isolated and fast.

Pattern 1: Shared Tables with `tenant_id` (Pooled Tenancy)

The default pattern: every vector row carries a `TenantId`, every query filters on it, and tenants share tables, indexes, and the buffer pool. It is the cheapest to operate and the fastest to onboard — a new tenant is an `INSERT` into a tenants table, not a provisioning event.

```
-- Pooled tenancy: TenantId on every vector row, filtered at query time
CREATE TABLE dbo.TenantChunks (
    ChunkId      INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_TenantChunks PRIMARY KEY,
    TenantId     INT NOT NULL,
    DocumentId   INT NOT NULL,
    ChunkText    NVARCHAR(MAX) NOT NULL,
    Embedding    VECTOR(1536) NULL,
    CONSTRAINT FK_TenantChunks_Tenants FOREIGN KEY (TenantId)
        REFERENCES dbo.Tenants (TenantId)
);
-- Tenant-keyed access path: every retrieval seeks here first
CREATE INDEX IX_TenantChunks_Tenant ON dbo.TenantChunks (TenantId, ChunkId);

-- Tenant-scoped exact retrieval
DECLARE @tenantId INT = 101, @q VECTOR(1536) = '...';
SELECT TOP (10) ChunkId, ChunkText,
    VECTOR_DISTANCE('cosine', Embedding, @q) AS Distance
FROM dbo.TenantChunks
WHERE TenantId = @tenantId AND Embedding IS NOT NULL
ORDER BY Distance;
```

For exact search this pattern is flawless: the `TenantId` predicate seeks the tenant's slice and ranks only within it — perfect isolation, perfect recall. For DiskANN approximate search, the wrinkle appears: the vector index is built over all tenants' vectors together, and a tenant-scoped query is a filtered ANN search. With moderate tenant counts and reasonably balanced data volumes this works well — the relational filter narrows candidates and the graph walk stays efficient. It degrades when one tenant dominates the corpus (their vectors saturate the graph neighborhoods) or when a tenant's data is a tiny fraction of a huge shared index (the walk wastes effort on other tenants' vectors). Mitigations: over-fetch `TOP_N` aggressively for small tenants, and monitor per-tenant recall with the Chapter 9 spot-check pattern parameterized by tenant.

Enforcing Isolation: Row-Level Security

A `WHERE TenantId = @tenantId` clause is a convention; row-level security (RLS) makes it a guarantee. With RLS, the database itself filters every query — including ad-hoc queries, reporting tools, and that one stored procedure someone wrote at 2 AM without the tenant predicate:

```
-- RLS: the database enforces tenant isolation, not application discipline
CREATE SCHEMA Security;
GO
CREATE FUNCTION Security.fn_TenantAccessPredicate(@TenantId INT)
RETURNS TABLE
WITH SCHEMABINDING
AS RETURN SELECT 1 AS AccessResult
            WHERE @TenantId = CAST(SESSION_CONTEXT(N'TenantId') AS INT)
            OR IS_MEMBER('db_owner') = 1;
GO
CREATE SECURITY POLICY Security.TenantFilter
ADD FILTER PREDICATE Security.fn_TenantAccessPredicate(TenantId)
ON dbo.TenantChunks,
ADD BLOCK PREDICATE Security.fn_TenantAccessPredicate(TenantId)
ON dbo.TenantChunks AFTER INSERT;
GO
-- Application sets the tenant once per session/pooled connection reset
EXEC sp_set_session_context @key = N'TenantId', @value = 101;
```

Two operational notes. First, `SESSION_CONTEXT` must be set whenever a pooled connection is reused for a different tenant — reset it on connection checkout or set it at the start of every retrieval procedure; a stale tenant context is a data leak. Second, the security predicate composes with vector queries transparently: the optimizer applies it as part of the plan, so

`VECTOR_SEARCH` results joined back to the secured table are filtered before they reach the caller. Test this explicitly — write a test that sets a tenant context and asserts zero rows from other tenants leak through the full retrieval procedure, not just the base table.

Pattern 2: Schema per Tenant

When tenants need stronger isolation — per-tenant backup/restore, per-tenant index maintenance windows, or regulatory boundaries that shared tables cannot satisfy — schema-per-tenant gives each tenant their own set of tables (`tenant_101.RagChunks` , `tenant_102.RagChunks`) inside one database. Each tenant gets their own DiskANN vector index, which eliminates the filtered-ANN recall concern entirely: every index is single-tenant by construction.

The cost is operational multiplicity: schema migrations must roll across every tenant schema (script it — never hand-apply), and the preview vector index's drop-and-recreate refresh cycle now runs per tenant. This pattern fits tens to low hundreds of tenants with meaningful per-tenant data volumes. Past that, the migration fan-out becomes the dominant operational burden.

Pattern 3: Database per Tenant (with Elastic Pools)

Database-per-tenant is the strongest isolation SQL Server offers short of separate instances: per-tenant backups, per-tenant restores, per-tenant performance tuning, and a blast radius limited to one tenant when something goes wrong. Each database holds its own vector tables and its own DiskANN index — the cleanest possible vector story, since every index is single-tenant and sized to that tenant's corpus.

On Azure SQL Database, elastic pools make this economically sane: hundreds of tenant databases share a pool of DTUs or vCores, with per-database min/max limits preventing noisy-neighbor starvation. The tradeoff is management surface — provisioning, connection routing (a tenant-to-database catalog), and cross-tenant schema upgrades across hundreds of databases demand automation from day one. This is the pattern for SaaS products where tenants pay enough to justify per-tenant infrastructure and where compliance requires hard boundaries.

Pattern 4: Separate Instances (Rare, and Usually Wrong)

Instance-per-tenant — separate SQL Server instances or separate managed servers per tenant — appears in RFP responses more often than in sensible architectures. It gives absolute noisy-neighbor isolation at the cost of multiplying every operational burden by the tenant count: patching, monitoring, backups, and licensing. Reserve it for the handful of tenants whose contracts genuinely demand dedicated infrastructure, and run everyone else on one of the first three patterns.

```
graph TD
    A[Choose tenancy pattern] --> B{Tenant count?}
    B -->|Thousands+| C[Pooled: tenant_id]
    + RLS]
    B -->|Tens to hundreds| D{Isolation needs?}
    D -->|Standard| C
    D -->|Per-tenant backup/  
regulatory| E[Schema per tenant]
    D -->|Hard boundaries,  
SaaS at scale| F[Database per tenant  
elastic pool]
    C --> G[Watch filtered-ANN  
recall per tenant]
    E --> H[Per-tenant DiskANN  
index each]
    F --> H
```

Noisy Neighbors and Resource Governance

In pooled and schema-per-tenant patterns, tenants share compute, and vector workloads are disproportionately capable of hogging it — a single tenant's million-vector backfill or a burst of concurrent similarity searches can saturate CPU and evict other tenants' hot pages from the buffer pool. Two controls keep the peace. On-premises, Resource Governor with per-tenant workload groups (classified by login or session context) caps CPU and memory grants per tenant. On Azure SQL Database elastic pools, per-database max vCore/DTU settings provide the same isolation with less configuration. Either way, the principle is the same: decide the noisy-neighbor policy before the first incident, not during it.

Monitor per-tenant vector health with the same queries you use globally, parameterized by tenant: un-embedded backlog per tenant (Chapter 8's query with a `TenantId` group-by), recall spot-checks per tenant for filtered ANN search, and query-store wait statistics broken out by tenant context. Tenancy problems announce themselves in per-tenant metrics long before they show up in global ones.

Key Takeaways

- → Pooled tenancy (`TenantId` on every vector row) is the default: cheapest to operate, instant onboarding, perfect for exact search — but watch filtered-ANN recall for tiny tenants in huge shared indexes and over-fetch `TOP_N` to compensate.
- → Row-level security turns tenant filtering from application convention into database guarantee — set `SESSION_CONTEXT` on every pooled connection checkout and test the full retrieval procedure for cross-tenant leaks, not just the base table.
- → Schema-per-tenant gives each tenant their own DiskANN index (no filtered-search recall concerns) at the cost of migration fan-out; database-per-tenant on elastic pools gives hard boundaries and per-tenant backup/restore for SaaS scale.
- → Govern the noisy neighbor before the first incident: Resource Governor workload groups on-premises, per-database caps in elastic pools — vector workloads are disproportionately capable of hogging shared CPU and buffer pool.
- → Monitor tenancy in per-tenant metrics — embedding backlog, recall spot-checks, and Query Store waits broken out by tenant — because tenancy problems show up there long before they move global dashboards.

Chapter 13: Vector Support on Azure SQL and RDS for SQL Server

Native vectors are not just an on-premises story. The same `VECTOR` type and `VECTOR_DISTANCE` function run on Azure SQL Database and Azure SQL Managed Instance — generally available, in all regions — and on Amazon RDS for SQL Server wherever the 17.x engine is deployed. The T-SQL you write is identical across all of them, which is the real headline: develop once, deploy wherever your data lives. What differs is everything around the T-SQL — provisioning, scaling, backups, network integration with embedding APIs, and the managed-service constraints that shape production designs. This chapter maps vector support across the three major managed platforms and gives you a deployment checklist for each.

Azure SQL Database: Vectors Without an Upgrade Project

Azure SQL Database is the fastest path to production vectors for most teams: native vector type and functions are generally available, so there is no upgrade project, no patching window, and no capacity planning for the engine itself. Provision a database, create your tables, and the Chapter 7 through 12 patterns work unchanged.

Three Azure-specific considerations shape the design. First, **service tier sizing is memory sizing**. Vector workloads are memory-hungry (~6 KB per 1536-dimension vector plus DiskANN graph overhead), and in Azure SQL Database memory scales with the service tier. A General Purpose database that comfortably serves an OLTP workload can feel cramped once a few million vectors land in the buffer pool. Start on Hyperscale or a generously sized General Purpose tier for vector workloads, and watch buffer-pool residency before blaming query plans for slow retrieval.

Second, **elastic pools are the multi-tenant answer**. The database-per-tenant pattern from Chapter 12 maps directly onto elastic pools: hundreds of tenant databases share pooled vCores with per-database min/max caps for noisy-neighbor control. Each tenant database holds its own vector tables and DiskANN index — single-tenant indexes with no filtered-search recall concerns, at a cost structure that makes sense per tenant.

Third, **embedding generation integrates natively**. Azure SQL Database can call Azure OpenAI directly through `sp_invoke_external_rest_endpoint`, keeping the embedding pipeline inside the data platform for backfills and low-volume ingestion. For production scale, pair the

database with Azure Functions or Container Apps that batch embedding requests and bulk-load vectors — the same staged pattern from Chapter 8, with the staging table living in the same logical server.

The approximate vector index and `VECTOR_SEARCH` are in preview on Azure SQL Database, same as on-premises — design with the drop-and-recreate refresh cycle in mind, and keep exact `VECTOR_DISTANCE` search as the production path until the preview graduates.

Azure SQL Managed Instance: Near-Parity with On-Premises

Azure SQL Managed Instance gives you native vector support (GA for the type and functions) inside an instance-scoped managed service — the closest cloud analog to the on-premises SQL Server this book assumes elsewhere. If your vector architecture uses instance-level features — SQL Server Agent jobs for embedding backfills, cross-database queries, linked servers to data sources — Managed Instance preserves them, where single-database Azure SQL Database would force a redesign.

The practical guidance: choose Managed Instance when you are lifting an existing on-premises vector workload (or an on-premises estate that will grow vectors) with minimal refactoring. Choose single-database Azure SQL Database when you are building cloud-native from scratch and want the simplest operational surface. The vector T-SQL is identical; the decision is about everything around it.

Amazon RDS for SQL Server: Vectors Follow the Engine

Amazon RDS for SQL Server supports SQL Server 2025 (17.x), and native vector capabilities follow the engine version — where the 17.x engine runs, the `VECTOR` type and `VECTOR_DISTANCE` function are available. The T-SQL from earlier chapters ports directly. What changes are the RDS operational realities that every RDS DBA already knows:

- → **No sysadmin, no SQL Agent in the traditional sense.** Embedding backfill jobs run through RDS's managed job mechanisms or — more commonly — through external schedulers (EventBridge + Lambda, or an EC2-hosted scheduler) that connect over the RDS endpoint. Design the ingestion pipeline as an external process from the start rather than assuming in-engine scheduling.
- → **Backups are RDS snapshots.** Automated RDS backups capture the whole instance via storage snapshots — your vector columns ride along with everything else, which is exactly the single-system backup story that makes native vectors attractive. Point-in-time restore

works the same for vector tables as for OLTP tables. Test restore of a vector database specifically: restore time scales with total data size, and multi-GB vector columns make databases bigger than their row counts suggest.

- → **Instance sizing is fixed at provisioning.** Unlike Azure SQL Database's fluid scaling, RDS instance classes are chosen up front (with modification downtime for changes). Size for the vector working set — memory for hot vectors plus DiskANN overhead — at provisioning time, not after the first slow Monday morning.
- → **Multi-AZ for availability.** RDS Multi-AZ deployments (via Always On availability groups) replicate vector tables like any other data — no special handling needed, but failover testing should include a vector retrieval workload to confirm the secondary serves similarity queries at expected latency after failover.
- → **Embedding API calls go through your VPC.** Whether you generate embeddings from an external pipeline or via in-engine REST calls, the traffic to OpenAI or Bedrock-hosted models traverses your VPC networking — plan security groups, NAT, and PrivateLink the way you would for any external dependency, and keep API keys in Secrets Manager, never in stored procedures.

Deployment Checklist: Same T-SQL, Different Surroundings

Whichever managed platform you choose, run through this checklist before declaring the vector workload production-ready:

- → **Confirm engine version and feature maturity.** Verify 17.x (or the Azure SQL equivalent) and decide explicitly which features are GA-backed versus preview-backed in your architecture. Document the decision — preview features need a re-evaluation trigger, not just hope.
- → **Size memory for vectors, not just rows.** Estimate total vector bytes (dimensions × 4 bytes × row count × 1.3 for overhead), add the working-set fraction you need resident, and provision the tier/instance class accordingly. Revisit after the first month of production telemetry.
- → **Wire up the embedding pipeline's identity.** Managed identity (Azure) or IAM roles (AWS) for the pipeline's access to both the database and the embedding API — no connection strings with passwords in application config, no API keys in source control.
- → **Test backup and restore with vectors.** Measure restore time for the actual data volume, verify vector queries return correct results post-restore, and confirm the restore SLA still holds with vector columns included.

- → **Enable Query Store from day one.** Managed platforms give you less access to the OS-level telemetry you might lean on premises-side; Query Store plus the platform's monitoring (Azure Monitor, CloudWatch/RDS Performance Insights) is your performance record. Vector plan regressions are exactly what you want caught automatically.
- → **Load-test the refresh cycle.** If you use the DiskANN preview index, rehearse the drop-rebuild-swap cycle at production data volume in a non-production environment first — build duration, not just query latency, is part of the SLA.

```
graph TD
    A[Deploy vectors on managed SQL] --> B{Starting point?}
    B -->|Cloud-native| C[Azure SQL Database]
    C -->|Lift existing| D[Azure SQL Managed Instance]
    D -->|AWS standardized| E[RDS for SQL Server 17.x]
    E --> F[Checklist: memory sizing, identity, backup test, Query Store]
    F --> G[ ]
    G --> H[ ]
    G --> I[ ]
```

Key Takeaways

- → The vector T-SQL is identical across SQL Server 2025, Azure SQL Database, Azure SQL Managed Instance, and RDS for SQL Server 17.x — native `VECTOR` type and `VECTOR_DISTANCE` are GA on Azure SQL; the DiskANN index and `VECTOR_SEARCH` remain preview everywhere.
- → On Azure SQL Database, size the service tier for vector memory, use elastic pools for database-per-tenant architectures, and prefer external batched pipelines (Functions/Container Apps) over row-by-row in-engine embedding calls at scale.
- → On RDS for SQL Server, vectors follow the 17.x engine: plan for external job scheduling, snapshot-based backups (test restore time with vector data volumes), up-front instance sizing for the vector working set, and VPC-plumbed embedding API access with Secrets Manager.
- → Run the deployment checklist on every platform: confirm feature maturity explicitly, size memory for vectors, wire managed identities, test backup/restore with real vector volumes, enable Query Store day one, and rehearse the preview index refresh cycle at production scale.

Chapter 14: Monitoring Vector Query Performance in SQL Server

Vector queries fail differently from ordinary queries. An OLTP query that slows down usually shows it in duration and logical reads; a vector query can return in the same 40 milliseconds while silently dropping from 98% recall to 70% because the data distribution drifted under a stale DiskANN index. Monitoring vector performance therefore means tracking two things traditional monitoring ignores: **result quality** (recall) alongside **execution cost** (CPU, I/O, latency). This chapter builds the monitoring stack for both, using the tools SQL Server DBAs already own — DMVs, Query Store, execution plans, and Extended Events — applied to vector-specific questions.

Finding Vector Queries in the Wild: DMVs

The first monitoring task is inventory: which queries on the instance are vector queries, and what do they cost? Because retrieval procedures are parameterized and repetitive, `sys.dm_exec_query_stats` gives you the aggregate picture immediately. Filter the cached plans by the telltale function names:

```
-- Inventory: top vector queries by total CPU across the instance
SELECT TOP (20)
    qs.execution_count,
    qs.total_worker_time / 1000.0 / qs.execution_count AS avg_cpu_ms,
    qs.total_elapsed_time / 1000.0 / qs.execution_count AS avg_duration_ms,
    qs.total_logical_reads / qs.execution_count AS avg_logical_reads,
    SUBSTRING(st.text, 1, 120) AS query_snippet
FROM sys.dm_exec_query_stats AS qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) AS st
WHERE st.text LIKE '%VECTOR_DISTANCE%'
    OR st.text LIKE '%VECTOR_SEARCH%'
ORDER BY qs.total_worker_time DESC;
```

Read this output with vector-specific eyes. High `avg_cpu_ms` with low logical reads on a `VECTOR_DISTANCE` query means compute-bound exact search — the signature of a corpus that has outgrown the exact path and needs the DiskANN index. High logical reads on a `VECTOR_SEARCH` query means the graph walk is missing the buffer pool — a memory-sizing problem, not an index problem. And a sudden change in `execution_count` patterns usually traces to an application deployment changing retrieval behavior, which is worth correlating with release logs before tuning anything.

For the queries behind a stored procedure, join to `sys.dm_exec_procedure_stats` instead — retrieval almost always goes through procedures (Chapter 10's parameterization guidance), and procedure-level aggregates are cleaner than statement-level ones when one procedure issues both the ANN lookup and the relational enrichment joins.

Reading Vector Execution Plans

Pull the actual execution plan for your retrieval procedure and look for three things. First, **which access path the optimizer chose** — the vector index seek (approximate path) or a scan with a sort on the computed distance (exact path). After statistics updates on a fast-growing chunk table, the optimizer can flip between these paths; the flip is the most common cause of sudden vector latency changes, and Query Store's plan-forcing (or Automatic Tuning's plan correction from Chapter 10) is the safety net.

Second, **where the relational filters sit relative to the vector operation**. In a healthy plan, selective relational predicates (tenant, published flag, category) reduce the row set before or alongside the vector ranking. If you see the vector operation running over the full table with filters applied afterward on millions of rows, the query shape needs restructuring — pre-filter in a CTE or derived table so the optimizer can push the predicate down.

Third, **the sort**. Exact search plans show a Top-N Sort over the computed `VECTOR_DISTANCE` expression; its memory grant scales with the rows sorted. If the grant spills to tempdb (look for sort warnings), the query is sorting more rows than it should — usually because a pre-filter is missing or because the corpus has grown past the point where exact search is viable.

```
-- Which plan is the retrieval procedure actually using right now?
SELECT qs.plan_handle, qs.execution_count,
       qs.last_elapsed_time / 1000.0 AS last_duration_ms
FROM sys.dm_exec_procedure_stats AS qs
WHERE OBJECT_NAME(qs.object_id, qs.database_id) = 'usp_RetrieveChunks';
-- Then: SELECT * FROM sys.dm_exec_query_plan(plan_handle) for the actual plan
```

Query Store: The Vector Performance Record

Enable Query Store on every database holding vectors — it is the durable record that DMVs cannot be, since DMV counters reset on service restart and plan cache eviction. For vector workloads, Query Store earns its keep in two specific ways.

First, **regression detection on repetitive shapes**. Retrieval queries execute the same parameterized shape thousands of times a day, which makes regressions statistically obvious: a step change in average duration for `usp_RetrieveChunks` with no application change points at a plan flip or a data-distribution change. Query Store's "Top Resource Consuming Queries" and plan comparison views turn that investigation from archaeology into a five-minute exercise.

Second, **wait statistics per query**. Query Store's wait stats break down where vector query time goes: `SOS_SCHEDULER_YIELD` dominating means CPU-bound distance computation (exact search under load); `PAGEIOLATCH_SH` dominating means the vector data is not resident (memory sizing); `LCK_M_S` appearing on a retrieval query means your read-committed-snapshot setting from Chapter 10 is not actually enabled. Each wait type maps to a specific remediation — that is the whole point of collecting them.

```
-- Query Store: wait profile for the retrieval workload
SELECT qsq.query_id,
       ws.wait_category_desc,
       SUM(wrs.avg_query_wait_time_ms * wrs.count_executions) AS total_wait_ms
FROM sys.query_store_query AS qsq
JOIN sys.query_store_plan AS qsp ON qsp.query_id = qsq.query_id
JOIN sys.query_store_runtime_stats AS wrs ON wrs.plan_id = qsp.plan_id
JOIN sys.query_store_wait_stats AS ws ON ws.runtime_stats_id = wrs.runtime_stats_id
WHERE qsq.object_id = OBJECT_ID('dbo.usp_RetrieveChunks')
GROUP BY qsq.query_id, ws.wait_category_desc
ORDER BY total_wait_ms DESC;
```

Monitoring Recall: The Metric Other Databases Don't Have

Latency and CPU tell you the query ran well; recall tells you it ran *correctly*. No DMV measures recall — you have to instrument it. The Chapter 9 spot-check pattern (comparing `VECTOR_SEARCH` results against exact `VECTOR_DISTANCE` ground truth on a sample of queries) becomes a scheduled job that writes to a history table:

```
-- Recall history table: trend this like any other SLI
CREATE TABLE dbo.VectorRecallHistory (
    MeasuredAt DATETIME2(0) NOT NULL CONSTRAINT DF_Recall_MeasuredAt DEFAULT
    SYSUTCDATETIME(),
    SampleSize INT NOT NULL,
    AvgRecallAt10 DECIMAL(5,4) NOT NULL,
    MinRecallAt10 DECIMAL(5,4) NOT NULL,
    CONSTRAINT PK_VectorRecallHistory PRIMARY KEY (MeasuredAt)
);

-- Alert rule of thumb: page when avg recall@10 < 0.95,
-- investigate when it trends down three consecutive runs.
-- A falling recall with flat latency = stale index or drifted data distribution.
-- A falling recall with rising latency = resource pressure (memory/CPU).
```

The diagnostic split in that comment is worth memorizing because it determines the response. Recall falling while latency stays flat means the index no longer represents the data — rebuild it (or check whether a recent bulk load bypassed the refresh cycle). Recall and latency degrading together means resource pressure — the instance needs memory, CPU, or the noisy-neighbor governance from Chapter 12. One is an index maintenance problem; the other is a capacity problem; treating one as the other wastes a weekend.

Extended Events: Capturing the Slow Ones

For the queries that escape aggregate monitoring — the occasional 5-second retrieval in a sea of 40-millisecond ones — an Extended Events session capturing slow vector queries with their plans gives you the forensic record:

```
-- Capture vector queries running longer than 2 seconds, with plans
CREATE EVENT SESSION [SlowVectorQueries] ON SERVER
ADD EVENT sqlserver.rpc_completed(
    ACTION(sqlserver.sql_text, sqlserver.query_hash, sqlserver.client_app_name)
    WHERE ([duration] > 2000000
        AND ([sqlserver].[like_i_sql_unicode_string]([sqlserver].[sql_text],
N'%VECTOR%')))),
ADD EVENT sqlserver.sql_batch_completed(
    ACTION(sqlserver.sql_text, sqlserver.query_hash, sqlserver.client_app_name)
    WHERE ([duration] > 2000000
        AND ([sqlserver].[like_i_sql_unicode_string]([sqlserver].[sql_text],
N'%VECTOR%')))),
ADD TARGET package0.event_file(
    SET filename = N'C:\XEvents\SlowVectorQueries.xel',
    max_file_size = 100, max_rollover_files = 5);
GO
ALTER EVENT SESSION [SlowVectorQueries] ON SERVER STATE = START;
```

Correlate slow-query events with the recall history timestamps. A slow query at the same time recall dips tells a different story from a slow query with recall intact — the first is data/index health, the second is resource contention. Keep the session's threshold aligned with your retrieval SLA (the 2-second threshold above is illustrative; set it just above your p99 target so the session captures true outliers, not normal traffic).

```
graph TD
    A[Vector monitoring] --> B[Cost: DMVs + Query Store  
CPU, reads, waits, plans]
    A --> C[Quality: recall history  
scheduled spot-checks]
    A --> D[Forensics: Extended Events  
slow query capture]
    B --> E{Regression?}
    C --> F{Recall < 0.95?}
    E -->|Yes| G[Plan flip or  
resource pressure]
    F -->|Yes, latency flat| H[Stale index:  
rebuild]
    F -->|Yes, latency up| I[Capacity:  
memory/CPU]
```

Key Takeaways

- → Inventory vector queries with `sys.dm_exec_query_stats` filtered on `VECTOR_DISTANCE / VECTOR_SEARCH`: CPU-bound exact search signals a corpus ready for the DiskANN index; I/O-bound approximate search signals a memory-sizing problem.

- → Read vector execution plans for three things: which access path the optimizer chose (index seek vs. scan-plus-sort), where relational filters sit relative to the vector operation, and whether the Top-N sort is spilling to tempdb.
- → Enable Query Store on every vector database: its regression detection catches plan flips on the highly repetitive retrieval shapes, and per-query wait stats (`SOS_SCHEDULER_YIELD` vs `PAGEIOLATCH_SH` vs `LCK_M_S`) map directly to remediations.
- → Instrument recall as a scheduled SLI: compare `VECTOR_SEARCH` against exact ground truth, trend recall@10, and alert below 0.95. Falling recall with flat latency means rebuild the index; falling recall with rising latency means add capacity.
- → Capture slow vector queries with Extended Events (threshold just above your p99 SLA) and correlate the timestamps with recall history — the combination distinguishes data/index health problems from resource contention.

Chapter 15: Pinecone: Architecture, Indexing, and Namespaces

Pinecone is the managed vector database that most teams encounter first when building production AI applications. Unlike SQL Server native vectors, which add vector search to the relational engine you already operate, Pinecone is purpose-built from the ground up for one job: storing high-dimensional vectors and returning approximate nearest neighbors at low latency and high throughput. Understanding how Pinecone works internally—its pod architecture, indexing strategies, namespace model, and operational boundaries—gives you the mental model to make informed decisions about when to use it, how to configure it correctly, and where its constraints will push back against your application requirements. This chapter maps Pinecone's architecture to concepts you already know from relational database operations, then walks through real implementation patterns for the most common production scenarios.

15.1 Pinecone's Architecture Through a Database Lens

Before writing a single line of code, it helps to understand what kind of system Pinecone actually is. Database professionals often expect a familiar storage hierarchy: pages, extents, tablespaces, and transaction logs. Pinecone's architecture looks different, but the underlying concerns—storage, indexing, query routing, and replication—are recognizable once you know where to look.

The Index as the Primary Object

In Pinecone, the fundamental unit of organization is the **index**, not a table or schema. An index holds vectors of a fixed dimension, uses a single distance metric, and has a defined capacity configuration. You cannot change the dimension or metric after creation—this is analogous to choosing a data type for a column, except you cannot alter it later. Plan accordingly.

Each index runs on infrastructure that Pinecone calls **Pods**. Pods are the physical compute and storage units allocated to your index. Think of them loosely like tablespaces with defined IOPS and memory characteristics. The pod type you choose controls query latency, storage capacity, and cost.

Pod Types: The Capacity Model

Pinecone exposes three pod families:

| Pod Type | Optimized For | Approximate Capacity (p1) | Approximate QPS |

| --- | --- | --- | --- |

| `s1` | Storage efficiency | 5M vectors/pod | ~200 QPS |

| `p1` | Balanced workloads | 1M vectors/pod | ~200 QPS |

| `p2` | Low latency | 1M vectors/pod | ~200 QPS with lower p99 |

Pod types can be suffixed with a size multiplier: `p1.x1`, `p1.x2`, `p1.x4`, `p1.x8`. Each step up doubles memory and storage but also doubles cost. The `s1` pod type uses roughly five times less pod capacity per vector, making it the right choice when you're storing tens of millions of vectors that are not queried at high frequency—essentially a cold tier.

This maps directly to what a DBA knows as capacity planning. You're choosing between storage-optimized vs. compute-optimized instance types, a decision familiar from RDS instance selection. The key difference is that Pinecone makes the tradeoff explicit in the pod type label rather than hiding it behind instance size tiers.

Replicas and High Availability

Replicas in Pinecone behave like read replicas in a traditional RDBMS. Adding replicas increases query throughput and provides redundancy, but does not increase storage capacity. If you need more capacity, you add pods. If you need more QPS, you add replicas. These are orthogonal levers.

Index Capacity Model:

Total storage = pods × capacity_per_pod

Total QPS = replicas × QPS_per_pod

A common mistake is adding replicas when you've hit a storage limit. More replicas will not help—you've exhausted pod storage, not query throughput. Pinecone will surface this as `ResourceExhausted` errors on upserts, not on queries.

Serverless Indexes: The Architecture Shift

In 2024, Pinecone introduced **serverless indexes**, which abandon the pod model entirely. Serverless indexes decouple storage from compute: vectors live in object storage (backed by cloud blob stores), and compute resources are provisioned on demand per query. You no longer provision pods, replicas, or capacity multipliers.

The tradeoff is query latency predictability. Pod-based indexes keep vectors in memory or fast NVMe storage, delivering single-digit millisecond p50 latency. Serverless indexes have higher and more variable p99 latency because cold data must be fetched from object storage. For applications that can tolerate 50–200ms query latency and have bursty or unpredictable workloads, serverless significantly reduces operational overhead and cost. For real-time recommendation systems or latency-sensitive retrieval, pod-based indexes remain the right choice.

Treat serverless as you would a provisioned IOPS vs. burst-credit EBS volume decision: great for workloads that don't require sustained low latency, but not a substitute for dedicated resources when latency SLAs are tight.

15.2 Indexing Algorithms: What HNSW Means at the Pinecone Layer

Pinecone uses **HNSW** (Hierarchical Navigable Small World) as its primary indexing algorithm for pod-based indexes, and a proprietary algorithm for serverless indexes. DBAs familiar with B-tree indexes have a strong intuition about how indexes trade write overhead for read performance—HNSW follows the same logic but operates in high-dimensional vector space rather than on sorted keys.

HNSW Recap from a Database Perspective

HNSW builds a multi-layer graph where each node represents a vector. The top layers are sparse long-range connections; the bottom layers are dense short-range connections. A query starts at the top layer, navigates toward the approximate nearest region, then descends through layers for refinement. The analogy is a skip list combined with a proximity graph.

The relevant parameters are:

- → `M` — The number of bidirectional connections per node in the graph. Higher `M` means better recall but more memory. Pinecone sets this internally; you don't tune it directly.
- → `ef_construction` — The size of the candidate list during index build. Higher values produce a more accurate graph but slower upsert throughput. Again, managed internally.
- → `top_k` — You control this at query time. It's the number of approximate nearest neighbors to return.

What you *do* control in Pinecone is the `top_k` value and metadata filtering, which together govern how aggressively the index traversal is scoped. This is covered in detail in the query patterns section below.

Dimension and Distance Metric Selection

The distance metric is set at index creation and cannot be changed. Pinecone supports:

- → **Cosine similarity** — Default for most embedding models. Measures angle between vectors; normalizes for magnitude. Use this unless you have a specific reason not to.
- → **Euclidean distance** (`euclidean`) — Measures absolute distance in vector space. Appropriate for spatial data or embeddings where magnitude carries meaning.
- → **Dot product** (`dotproduct`) — Measures the inner product. Appropriate for models that output non-normalized embeddings where magnitude is intentional (e.g., some fine-tuned retrieval models).

A common mistake is using dot product with normalized embeddings from OpenAI or Cohere, where cosine similarity is the correct metric. The mathematical result is the same only when vectors are unit-normalized—but relying on this coincidence without understanding it creates subtle bugs when you swap embedding models.

Creating an Index: Pod-Based vs. Serverless

```

# Python
import pinecone
from pinecone import Pinecone, ServerlessSpec, PodSpec

pc = Pinecone(api_key="YOUR_API_KEY")

# Serverless index – recommended for development and moderate production workloads
pc.create_index(
    name="document-embeddings-serverless",
    dimension=1536,          # Must match your embedding model output dimension
    metric="cosine",
    spec=ServerlessSpec(
        cloud="aws",
        region="us-east-1"
    )
)

# Pod-based index – for production workloads with strict latency requirements
pc.create_index(
    name="document-embeddings-pods",
    dimension=1536,
    metric="cosine",
    spec=PodSpec(
        environment="us-east1-gcp",
        pod_type="p1.x2",    # x2 = 2GB memory per pod, doubles capacity
        pods=2,              # Total storage = 2 × ~1M vectors per p1.x1 × 2 = ~4M
        replicas=2,          # Read replicas for query throughput
        shards=1             # Horizontal partitioning; increase for very large
    )
)

# Verify index status before connecting
import time

while not pc.describe_index("document-embeddings-pods").status["ready"]:
    print("Waiting for index to initialize...")
    time.sleep(5)

print("Index ready.")

```

The `pods`, `replicas`, and `shards` parameters map directly to a familiar infrastructure model:

- → ``pods`` = storage capacity units
- → ``replicas`` = read scaling factor
- → ``shards`` = horizontal partitions for indexes exceeding single-pod capacity

15.3 Upserting Vectors and Working with Metadata

In Pinecone's data model, each record is a **vector** paired with a unique string ID and an optional metadata dictionary. There are no schemas enforced at the database layer—metadata is schemaless JSON, similar to an NVARCHAR(MAX) column holding JSON in SQL Server with no CHECK constraints. The flexibility is useful but requires application-level discipline.

The Upsert Pattern

Pinecone uses upsert semantics: if a record with the given ID already exists, it is replaced entirely. There is no partial update—if you change one metadata field, you must resubmit the entire record including the full vector. This is analogous to a full row replacement via `MERGE` in T-SQL. Plan your data pipelines accordingly, because re-embedding is expensive if done unnecessarily.

```

# Python
import openai
from pinecone import Pinecone

pc = Pinecone(api_key="YOUR_API_KEY")
index = pc.Index("document-embeddings-serverless")

openai_client = openai.OpenAI(api_key="YOUR_OPENAI_KEY")

def embed_texts(texts: list[str], model: str = "text-embedding-3-small") ->
list[list[float]]:
    """
    Batch embed texts using OpenAI.
    text-embedding-3-small outputs 1536 dimensions.
    """
    response = openai_client.embeddings.create(
        input=texts,
        model=model
    )
    return [item.embedding for item in response.data]

# Sample document corpus
documents = [
    {
        "id": "doc-001",
        "text": "SQL Server index rebuild reclaims space from fragmented indexes
caused by updates and deletes.",
        "metadata": {
            "source": "sql-server-internals",
            "category": "maintenance",
            "doc_type": "technical",
            "created_at": "2024-01-15"
        }
    },
    {
        "id": "doc-002",
        "text": "Index fragmentation in SQL Server accumulates when maintenance
windows cannot keep pace with write load.",
        "metadata": {
            "source": "sql-server-internals",
            "category": "performance",
            "doc_type": "technical",
            "created_at": "2024-01-16"
        }
    },
    {
        "id": "doc-003",
        "text": "Pinecone serverless indexes use object storage for vectors with on-
demand compute scaling.",
        "metadata": {
            "source": "vector-db-guide",
            "category": "architecture",

```

```

        "doc_type": "technical",
        "created_at": "2024-02-01"
    }
}
]

# Generate embeddings in batch (reduces API calls and cost)
texts = [doc["text"] for doc in documents]
embeddings = embed_texts(texts)

# Build upsert batch – Pinecone accepts dicts with id, values, metadata
vectors_to_upsert = [
    {
        "id": doc["id"],
        "values": embedding,
        "metadata": doc["metadata"]
    }
    for doc, embedding in zip(documents, embeddings)
]

# Batch upsert – recommended batch size is 100 vectors per request
# Larger batches risk hitting the 2MB request size limit
upsert_response = index.upsert(
    vectors=vectors_to_upsert,
    namespace="production"    # Namespaces are covered in section 15.4
)

print(f"Upserted {upsert_response['upserted_count']} vectors")

# Verify a vector was stored correctly
fetch_result = index.fetch(ids=["doc-001"], namespace="production")
print(f"Fetched vector ID: {list(fetch_result['vectors'].keys())}")
print(f"Metadata: {fetch_result['vectors']['doc-001']['metadata']}")

```

Metadata Filtering: The Most Important Operational Detail

Metadata filtering in Pinecone behaves differently from what you'd expect coming from SQL WHERE clauses, and misunderstanding this distinction causes serious performance problems in production.

In a SQL database, a filtered query touches only the rows that match the WHERE clause. Pinecone's metadata filter is applied **after** the approximate nearest neighbor search narrows the candidate set. The filter reduces the result set from the ANN results, it does not direct the ANN traversal (except in the specialized case of Pinecone's indexed metadata for serverless, which is handled differently).

This means that if your filter is highly selective—say, filtering to a `tenant_id` that matches only 0.1% of your vectors—the ANN search still traverses the full index, and the post-filter step discards most candidates. You end up retrieving far fewer results than `top_k` requested, and in extreme cases, you get zero results even when matching vectors exist.

The operational rule of thumb: **metadata filters work well when they match at least 10–20% of the index.** Below that threshold, increase `top_k` to compensate, or consider using namespaces for true data isolation (see Section 15.4).

```

# Python
# Querying with metadata filters

query_text = "How does SQL Server reclaim space from fragmented indexes?"
query_embedding = embed_texts([query_text])[0]

# Basic query without filter
results = index.query(
    vector=query_embedding,
    top_k=5,
    namespace="production",
    include_metadata=True
)

print("=== Unfiltered Results ===")
for match in results["matches"]:
    print(f" ID: {match['id']}, Score: {match['score']:.4f}, "
          f"Category: {match['metadata'].get('category', 'N/A')}")

# Filtered query – metadata filter uses MongoDB-style operators
results_filtered = index.query(
    vector=query_embedding,
    top_k=5,
    namespace="production",
    include_metadata=True,
    filter={
        "category": {"$in": ["maintenance", "performance"]},
        "doc_type": {"$eq": "technical"}
    }
)

print("\n=== Filtered Results (maintenance or performance, technical docs) ===")
for match in results_filtered["matches"]:
    print(f" ID: {match['id']}, Score: {match['score']:.4f}, "
          f"Category: {match['metadata'].get('category', 'N/A')}")

# Supported filter operators:
# $eq, $ne          – equality, inequality
# $gt, $gte         – greater than comparisons (numeric fields)
# $lt, $lte         – less than comparisons (numeric fields)
# $in, $nin         – membership in list
# $and, $or         – logical combinators

```

Metadata Storage Limits

Pinecone enforces a 40KB metadata limit per vector. This is rarely a problem for standard metadata fields (strings, numbers, small arrays), but it matters if you're tempted to store the full document text in metadata for retrieval. The practice of storing full text in Pinecone metadata is common but wrong at scale—you're using a vector index as a key-value store for

blob data it was not designed to hold. Store document text in your primary database (SQL Server, a document store, or object storage), and use Pinecone's IDs as foreign keys to retrieve it. This pattern is covered in depth in Chapter 22 on RAG pipelines.

15.4 Namespaces: Multi-Tenancy Without Schema Separation

Namespaces are one of Pinecone's most useful features for production applications, and one of the most frequently misused. A namespace is a logical partition within an index. Vectors in different namespaces are completely isolated from each other—a query targeting `namespace="tenant-a"` will never return vectors from `namespace="tenant-b"`, regardless of similarity scores.

The Database Analogy

Namespaces map most closely to **schemas** within a SQL Server database—they provide logical separation within a shared storage engine. Unlike schemas, you don't need to define namespaces in advance. They are created implicitly when you first upsert a vector into them and destroyed when they become empty.

The critical distinction from schemas: namespaces share the index's pod capacity. You cannot assign different pod types or capacity budgets to different namespaces. If `tenant-a` stores 900,000 vectors and the index capacity is 1,000,000, there are only 100,000 slots remaining across all other namespaces. This is capacity pooling, not isolation.

Multi-Tenant Application Pattern

The canonical use case for namespaces is multi-tenant applications where each tenant's data must be queryable only in the context of that tenant. This is more reliable than metadata filtering for tenant isolation because it is enforced at the storage layer, not the application query layer.


```

# Python
# Multi-tenant vector upsert and query using namespaces

from pinecone import Pinecone
import openai

pc = Pinecone(api_key="YOUR_API_KEY")
index = pc.Index("document-embeddings-serverless")
openai_client = openai.OpenAI(api_key="YOUR_OPENAI_KEY")

def get_embedding(text: str) -> list[float]:
    response = openai_client.embeddings.create(
        input=[text],
        model="text-embedding-3-small"
    )
    return response.data[0].embedding

# ---- Tenant A: Legal firm document corpus ----
tenant_a_docs = [
    {"id": "legal-001", "text": "Force majeure clauses excuse contract non-
performance during extraordinary events."},
    {"id": "legal-002", "text": "Indemnification provisions transfer liability risk
between contracting parties."},
]

tenant_a_vectors = [
    {"id": doc["id"], "values": get_embedding(doc["text"]), "metadata": {"type":
"contract"}}
    for doc in tenant_a_docs
]

index.upsert(vectors=tenant_a_vectors, namespace="tenant-legal-firm-a")

# ---- Tenant B: Medical records system ----
tenant_b_docs = [
    {"id": "med-001", "text": "Systolic blood pressure above 140 mmHg indicates
stage 2 hypertension."},
    {"id": "med-002", "text": "HbA1c levels above 6.5% are diagnostic criteria for
type 2 diabetes."},
]

tenant_b_vectors = [
    {"id": doc["id"], "values": get_embedding(doc["text"]), "metadata": {"type":
"clinical"}}
    for doc in tenant_b_docs
]

index.upsert(vectors=tenant_b_vectors, namespace="tenant-medical-system-b")

# ---- Namespace-isolated queries ----
def search_tenant(tenant_namespace: str, query: str, top_k: int = 3):

```

```

"""
Query is strictly scoped to the tenant's namespace.
No cross-tenant data leakage regardless of query content.
"""

query_vector = get_embedding(query)
results = index.query(
    vector=query_vector,
    top_k=top_k,
    namespace=tenant_namespace,
    include_metadata=True
)
return results["matches"]

# This query runs only against tenant A's data
legal_results = search_tenant(
    "tenant-legal-firm-a",
    "What contract clauses address unexpected events beyond control?"
)
print("Legal firm results:", [r["id"] for r in legal_results])

# Same query against tenant B – will return medical data, not legal
medical_results = search_tenant(
    "tenant-medical-system-b",
    "What contract clauses address unexpected events beyond control?"
)
print("Medical system results:", [r["id"] for r in medical_results])
# Output: medical vectors closest to the query in B's namespace

# Namespace statistics – how many vectors per namespace
index_stats = index.describe_index_stats()
print("\nVectors per namespace:")
for ns, stats in index_stats.get("namespaces", {}).items():
    print(f"  {ns}: {stats['vector_count']} vectors")

```

Namespace Deletion and Maintenance

Deleting all vectors from a namespace effectively destroys it. There is no explicit DROP NAMESPACE command—the namespace simply ceases to exist when empty. For tenant offboarding, you can delete by namespace directly:

```
# Python
# Delete all vectors within a namespace – equivalent to DROP SCHEMA CASCADE
index.delete(delete_all=True, namespace="tenant-legal-firm-a")

# Delete specific vectors within a namespace
index.delete(
    ids=["legal-001", "legal-002"],
    namespace="tenant-legal-firm-a"
)

# List all namespaces currently in use
stats = index.describe_index_stats()
active_namespaces = list(stats.get("namespaces", {}).keys())
print(f"Active namespaces: {active_namespaces}")
```

When Namespaces Are Not Enough

Namespaces provide logical isolation but share physical resources. For strict compliance requirements (GDPR right-to-erasure with audit proof, HIPAA physical data isolation), namespaces within a shared index are generally insufficient. In those cases, use **separate indexes per tenant**, accepting the higher cost. A single-tenant index on a `p1.x1` pod has a cost per tenant that varies by Pinecone's current pricing model—viable for enterprise SaaS but may be prohibitive for consumer-scale applications. Verify current costs with Pinecone's pricing documentation.

The namespace vs. separate-index decision follows the same logic as shared vs. dedicated database servers for tenant isolation in traditional SaaS architectures.

15.5 Production Operations: Monitoring, Scaling, and Failure Patterns

Running Pinecone in production requires operational discipline that mirrors what you'd apply to any managed database service: understand your capacity headroom, monitor query latency percentiles, and have runbooks for failure modes before they occur.

Index Statistics and Health Monitoring

```

# Python
# Comprehensive index monitoring – suitable for integration into Datadog,
# CloudWatch custom metrics, or a Prometheus exporter

from pinecone import Pinecone
import time

pc = Pinecone(api_key="YOUR_API_KEY")
index = pc.Index("document-embeddings-pods")

def collect_index_metrics(index_name: str) -> dict:
    """
    Collect key operational metrics from a Pinecone index.
    Call this on a schedule (e.g., every 60 seconds) and push to your monitoring
    system.
    """
    pc_client = Pinecone(api_key="YOUR_API_KEY")
    idx = pc_client.Index(index_name)

    # describe_index_stats returns vector count and namespace breakdown
    stats = idx.describe_index_stats()

    # describe_index returns pod config, status, and replicas
    index_config = pc_client.describe_index(index_name)

    total_vectors = stats.get("total_vector_count", 0)
    dimension = stats.get("dimension", 0)

    # Capacity utilization – critical for pod-based indexes
    # p1.x1 holds ~1M vectors; x2 = ~2M, x4 = ~4M, x8 = ~8M
    pod_type = index_config.spec.pod.pod_type if hasattr(index_config.spec, 'pod')
    else "serverless"
    pods = index_config.spec.pod.pods if hasattr(index_config.spec, 'pod') else None
    replicas = index_config.spec.pod.replicas if hasattr(index_config.spec, 'pod')
    else None

    namespace_stats = {
        ns: data["vector_count"]
        for ns, data in stats.get("namespaces", {}).items()
    }

    return {
        "index_name": index_name,
        "total_vectors": total_vectors,
        "dimension": dimension,
        "pod_type": pod_type,
        "pods": pods,
        "replicas": replicas,
        "namespace_vector_counts": namespace_stats,
        "collected_at": time.time()
    }

```

```

def estimate_capacity_utilization(metrics: dict) -> float:
    """
    Estimate percentage of pod capacity consumed.
    Useful for alerting before you hit ResourceExhausted errors.
    Capacity estimates are approximate – refer to Pinecone documentation
    for current per-pod limits.
    """
    pod_type = metrics.get("pod_type", "")
    pods = metrics.get("pods", 1) or 1
    total_vectors = metrics.get("total_vectors", 0)

    # Approximate per-pod capacities by pod family
    # These change as Pinecone updates infrastructure – treat as guidelines
    per_pod_capacity = {
        "p1": 1_000_000,
        "p2": 1_000_000,
        "s1": 5_000_000,
    }

    pod_family = pod_type.split(".")[0] if pod_type else "p1"
    base_capacity = per_pod_capacity.get(pod_family, 1_000_000)

    # x2, x4, x8 multipliers double/quadruple capacity
    multiplier = 1
    if "." in pod_type:
        suffix = pod_type.split(".")[1] # e.g., "x2"
        if suffix.startswith("x"):
            try:
                multiplier = int(suffix[1:])
            except ValueError:
                multiplier = 1

    total_capacity = base_capacity * multiplier * pods
    return (total_vectors / total_capacity) * 100 if total_capacity > 0 else 0.0

# Example usage
metrics = collect_index_metrics("document-embeddings-pods")
utilization = estimate_capacity_utilization(metrics)

print(f"Index: {metrics['index_name']}")
print(f"Vectors: {metrics['total_vectors']:,}")
print(f"Pod type: {metrics['pod_type']}, Pods: {metrics['pods']}, Replicas: {metrics['replicas']}")
print(f"Estimated capacity utilization: {utilization:.1f}%")

if utilization > 80:
    print("WARNING: Index approaching capacity. Plan pod upgrade or horizontal expansion.")
elif utilization > 95:
    print("CRITICAL: Index near capacity. Upserts will fail with ResourceExhausted.")

```

Latency Measurement Patterns

Pinecone does not expose p99 query latency directly through its Python client—that telemetry lives in your own instrumentation. Wrap every query call with a timing decorator and feed the results into your observability stack.

```

# Python
import time
import statistics
from functools import wraps
from collections import deque

# Simple sliding-window latency tracker
class LatencyTracker:
    def __init__(self, window_size: int = 1000):
        self.latencies = deque(maxlen=window_size)

    def record(self, latency_ms: float):
        self.latencies.append(latency_ms)

    def percentile(self, p: float) -> float:
        if not self.latencies:
            return 0.0
        sorted_latencies = sorted(self.latencies)
        idx = int(len(sorted_latencies) * p / 100)
        return sorted_latencies[min(idx, len(sorted_latencies) - 1)]

    def report(self) -> dict:
        if not self.latencies:
            return {}
        return {
            "p50_ms": self.percentile(50),
            "p95_ms": self.percentile(95),
            "p99_ms": self.percentile(99),
            "mean_ms": statistics.mean(self.latencies),
            "sample_count": len(self.latencies),
        }

query_latency = LatencyTracker(window_size=500)

def timed_query(index, vector, top_k, namespace, **kwargs):
    """
    Execute a Pinecone query and record wall-clock latency.
    Use this wrapper everywhere instead of calling index.query() directly.
    """
    start = time.perf_counter()
    result = index.query(
        vector=vector,
        top_k=top_k,
        namespace=namespace,
        **kwargs
    )
    elapsed_ms = (time.perf_counter() - start) * 1000
    query_latency.record(elapsed_ms)
    return result

```

```
# After a load test or production traffic window:  
print("Query latency report:", query_latency.report())
```

Common Failure Modes and Mitigations

| Failure Mode | Cause | Mitigation |

| --- | --- | --- |

| `ResourceExhausted` on upsert | Index at pod capacity | Upgrade pod size or add pods before hitting 90% |

| Fewer results than `top_k` | Metadata filter too selective | Increase `top_k`; consider namespace isolation |

| Stale vectors after upsert | Pinecone indexes asynchronously | Add read-your-writes wait loop; see below |

| High p99 latency spikes | Replica saturation | Add replicas; or switch to pod type with better cache behavior |

| 400 on dimension mismatch | Embedding model changed | Validate vector dimensions before every upsert batch |

| Timeout on large batch upsert | Request body exceeds 2MB | Limit batch to 100 vectors; reduce metadata payload size |

Read-Your-Writes Consistency

Pinecone is eventually consistent on write paths. An upsert call that returns success does not guarantee the vector is immediately queryable. For workflows that require confirming vector availability after upsert—such as in integration tests or post-ingest verification jobs—poll `describe_index_stats` until the vector count increments, or use a small polling loop on `fetch`.


```
# Python
# Wait for a vector to become queryable after upsert
def wait_for_vector(index, vector_id: str, namespace: str,
                    timeout_seconds: int = 30, poll_interval: float = 0.5):
    """
    Poll until a vector with the given ID is fetchable.
    Useful in tests and post-ingest verification pipelines.
    Not recommended for hot paths – adds latency.
    """
    deadline = time.time() + timeout_seconds
    while time.time() < deadline:
        result = index.fetch(ids=[vector_id], namespace=namespace)
        if vector_id in result.get("vectors", {}):
            return True
        time.sleep(poll_interval)
    raise TimeoutError(
        f"Vector '{vector_id}' not visible in namespace '{namespace}' "
        f"after {timeout_seconds}s"
    )

# Usage: after upserting a critical document, verify before proceeding
index.upsert(
    vectors=[{"id": "critical-doc-001", "values": get_embedding("important text"),
"metadata": {}}],
    namespace="production"
)
wait_for_vector(index, "critical-doc-001", "production")
print("Vector confirmed queryable.")
```

Scaling Playbook: When to Take Each Action

Database professionals expect a capacity scaling decision matrix. Here is the equivalent for Pinecone pod-based indexes:

Observed Symptom	Diagnosis	Action
ResourceExhausted on upsert	Pod storage full	Scale pod size (x1 → x2 → x4) or add pods
Rising p99 query latency	Replica throughput saturated	Add replicas
top_k results consistently short	Filter selectivity < 10%	Increase top_k or move to namespaces
Upsert throughput degrading	Saturating pod write path	Increase batch concurrency or pod count

| Cold start latency spikes (serverless) | Object storage fetch on cold data | Migrate hot data to pod-based index |

| High cost with low query rate | Over-provisioned pods | Migrate to serverless for cost reduction |

15.6 Advanced Patterns: Hybrid Search and Re-Ranking

Pinecone's core operation is approximate nearest neighbor search over dense vectors. However, most production retrieval systems need more than raw ANN scores—they need the ability to blend keyword relevance with semantic similarity, and to re-rank initial retrieval results with a more expensive model before returning the final set.

Sparse-Dense Hybrid Search

Pinecone supports **sparse-dense hybrid search** on pod-based indexes using the `dotproduct` metric. Sparse vectors represent keyword or BM25-style term-frequency signals; dense vectors represent semantic embeddings. Combining them lets you retrieve results that are both keyword-relevant and semantically similar—the same goal that Elasticsearch's Reciprocal Rank Fusion (RRF) achieves at the query rewrite layer.

The architecture requires generating both a dense embedding and a sparse vector for each document at index time, then combining them at query time with an `alpha` weighting parameter.

```

# Python
# Sparse-dense hybrid search
# Requires: pod-based index with dotproduct metric and sparse vector support

from pinecone import Pinecone
from pinecone_text.sparse import BM25Encoder
import openai

pc = Pinecone(api_key="YOUR_API_KEY")

# Create an index that supports sparse vectors
# Note: hybrid search requires dotproduct metric and pod-based index
pc.create_index(
    name="hybrid-search-index",
    dimension=1536,
    metric="dotproduct",
    spec=PodSpec(
        environment="us-east1-gcp",
        pod_type="p1.x1",
        pods=1
    )
)

index = pc.Index("hybrid-search-index")
openai_client = openai.OpenAI(api_key="YOUR_OPENAI_KEY")

# BM25 encoder – fit on your document corpus for accurate IDF weights
# In production, fit once on the full corpus, serialize, and reuse
bm25 = BM25Encoder()

documents = [
    "SQL Server index rebuild reclaims space from fragmented indexes caused by updates and deletes.",
    "Index fragmentation in SQL Server accumulates when maintenance windows cannot keep pace with write load.",
    "BM25 is a term frequency ranking function used in full-text search systems.",
]

# Fit BM25 on corpus (computes IDF weights)
bm25.fit(documents)

# Encode sparse vectors for each document
sparse_doc_vectors = bm25.encode_documents(documents)

# Encode dense vectors for each document
dense_response = openai_client.embeddings.create(
    input=documents,
    model="text-embedding-3-small"
)
dense_doc_vectors = [item.embedding for item in dense_response.data]

# Upsert hybrid vectors
hybrid_vectors = []

```

```

for i, (dense, sparse) in enumerate(zip(dense_doc_vectors, sparse_doc_vectors)):
    hybrid_vectors.append({
        "id": f"hybrid-doc-{i:03d}",
        "values": dense,
        "sparse_values": {
            "indices": sparse["indices"],
            "values": sparse["values"]
        },
        "metadata": {"text": documents[i]}
    })

index.upsert(vectors=hybrid_vectors, namespace="hybrid")

# Hybrid query with alpha blending
# alpha = 1.0 → pure dense search
# alpha = 0.0 → pure sparse (keyword) search
# alpha = 0.5 → equal blend

def hybrid_query(query_text: str, alpha: float = 0.7, top_k: int = 5):
    """
    Execute a hybrid sparse-dense query.
    alpha controls the balance: higher = more semantic, lower = more keyword.
    """
    # Encode query as both dense and sparse
    dense_query = openai_client.embeddings.create(
        input=[query_text],
        model="text-embedding-3-small"
    ).data[0].embedding

    sparse_query = bm25.encode_queries(query_text)

    # Scale sparse and dense components by alpha
    def scale_sparse(sparse_vec, scale: float):
        return {
            "indices": sparse_vec["indices"],
            "values": [v * scale for v in sparse_vec["values"]]
        }

    def scale_dense(dense_vec, scale: float):
        return [v * scale for v in dense_vec]

    results = index.query(
        vector=scale_dense(dense_query, alpha),
        sparse_vector=scale_sparse(sparse_query, 1 - alpha),
        top_k=top_k,
        namespace="hybrid",
        include_metadata=True
    )
    return results["matches"]

# Compare pure semantic vs. hybrid results
print("=== Pure Semantic Search (alpha=1.0) ===")

```

```

semantic_results = hybrid_query("dead rows storage reclamation", alpha=1.0)
for r in semantic_results:
    print(f"  {r['id']}: {r['score']:.4f} - {r['metadata']['text'][:60]}...")

print("\n=== Hybrid Search (alpha=0.7, 30% BM25) ===")
hybrid_results = hybrid_query("dead rows storage reclamation", alpha=0.7)
for r in hybrid_results:
    print(f"  {r['id']}: {r['score']:.4f} - {r['metadata']['text'][:60]}...")

```

Re-Ranking with a Cross-Encoder

Approximate nearest neighbor retrieval returns a candidate set quickly, but ANN scores are coarse signals. Cross-encoder re-ranking runs a more expensive model over the (query, candidate) pairs to produce precise relevance scores. The pattern is always two-stage:

1. **Retrieval** (fast, approximate): Pinecone returns top-100 candidates.
2. **Re-ranking** (slower, precise): Cross-encoder scores each candidate; return top-10.

```

# Python
# Two-stage retrieval and cross-encoder re-ranking

from sentence_transformers import CrossEncoder
from pinecone import Pinecone
import openai

pc = Pinecone(api_key="YOUR_API_KEY")
index = pc.Index("document-embeddings-serverless")
openai_client = openai.OpenAI(api_key="YOUR_OPENAI_KEY")

# Cross-encoder from sentence-transformers – runs locally
# ms-marco-MiniLM-L-6-v2 is a well-validated choice for passage reranking
cross_encoder = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

def two_stage_retrieval(
    query: str,
    namespace: str,
    retrieve_k: int = 50,
    rerank_top_k: int = 5
) -> list[dict]:
    """
    Stage 1: Retrieve retrieve_k candidates from Pinecone using ANN.
    Stage 2: Re-rank with cross-encoder and return rerank_top_k results.

    retrieve_k should be at least 5–10× rerank_top_k to give the
    cross-encoder enough candidates to work with.
    """
    # Stage 1: ANN retrieval
    query_embedding = openai_client.embeddings.create(
        input=[query],
        model="text-embedding-3-small"
    ).data[0].embedding

    ann_results = index.query(
        vector=query_embedding,
        top_k=retrieve_k,
        namespace=namespace,
        include_metadata=True
    )

    candidates = ann_results["matches"]
    if not candidates:
        return []

    # Stage 2: Cross-encoder re-ranking
    # Cross-encoder expects (query, passage) pairs
    # Requires document text to be stored in metadata or fetched from primary DB
    pairs = [
        (query, candidate["metadata"].get("text", ""))
        for candidate in candidates
    ]

```

```

# Predict returns a score per pair – higher is more relevant
rerank_scores = cross_encoder.predict(pairs)

# Attach rerank scores and sort descending
for candidate, score in zip(candidates, rerank_scores):
    candidate["rerank_score"] = float(score)

reranked = sorted(candidates, key=lambda x: x["rerank_score"], reverse=True)
return reranked[:rerank_top_k]

# Example usage
query = "How does SQL Server reclaim space from deleted rows?"
final_results = two_stage_retrieval(
    query=query,
    namespace="production",
    retrieve_k=20,
    rerank_top_k=3
)

print(f"Query: {query}\n")
print("Re-ranked results:")
for i, result in enumerate(final_results, 1):
    print(f"  {i}. [{result['rerank_score']:.3f}] {result['id']}: "
          f"{result['metadata'].get('text', '')[:80]}...")

```

The cross-encoder step runs in the application tier—it has no dependency on Pinecone and can be parallelized. For latency-sensitive applications, run the cross-encoder inference on GPU or use a hosted re-ranking API (Cohere's Rerank API provides this as a managed service) rather than the CPU-bound local model.

15.7 Integrating Pinecone with Your Data Stack

Pinecone does not exist in isolation. In production architectures, it sits alongside a primary relational database, an event streaming layer, and an embedding generation service. Getting the integration topology right determines whether your vector search layer stays consistent with the rest of your data.

The Canonical Integration Topology

```

graph TD
    App[Application]
    SQL[(SQL Server / Primary DB  
source of truth for text + metadata)]
    CDC[(CDC / Event Stream)]
    Embed[Embedding Service  
OpenAI / Cohere / self-hosted]
    Pine[(Pinecone Index  
vector storage, ANN search)]
    Query[Pinecone ANN Query]
    Fetch[Fetch full docs from SQL Server by returned IDs]
    Result[Return to application / LLM context]

    App --> SQL
    SQL --> CDC
    CDC --> Embed
    Embed --> Pine
    App --> Query
    Query --> Fetch
    Fetch --> Result
    Pine -. Query .-> Query
    PG -. Fetch .-> Fetch

```

Write path (left): SQL Server changes flow through CDC into the embedding service and land in Pinecone. Read path (right): the application queries Pinecone for nearest-neighbor IDs, then fetches authoritative text from SQL Server.

The pattern is clear: Pinecone stores vectors and returns IDs. SQL Server stores authoritative text and structured data. Your application joins the two at query time using the IDs Pinecone returns. Never treat Pinecone's metadata as the sole copy of document content.

Keeping Pinecone Synchronized with SQL Server

The most common production headache with Pinecone is index drift: records updated or deleted in SQL Server that still exist (stale) in Pinecone. Address this with one of three patterns, in order of reliability:

Pattern 1: Transactional outbox with CDC

Write changes to an outbox table in SQL Server in the same transaction as the business record change. A downstream consumer (SQL Server CDC, Debezium, or a Service Broker queue) reads the outbox and triggers re-embedding + Pinecone upsert. Deletes in SQL Server produce a tombstone event that triggers `index.delete()`.


```

-- SQL Server
-- Outbox table for Pinecone sync events
CREATE TABLE dbo.pinecone_sync_queue (
    id                BIGINT IDENTITY(1,1) NOT NULL CONSTRAINT PK_pinecone_sync_queue
PRIMARY KEY,
    operation         NVARCHAR(20) NOT NULL,          -- 'upsert' or 'delete'
    document_id       NVARCHAR(100) NOT NULL,
    namespace         NVARCHAR(100) NOT NULL CONSTRAINT DF_pinecone_sync_ns DEFAULT
('default'),
    payload           NVARCHAR(MAX) NULL,             -- document text + metadata as JSON
    created_at        DATETIME2(0) NOT NULL CONSTRAINT DF_pinecone_sync_created
                    DEFAULT SYSUTCDATETIME(),
    processed_at      DATETIME2(0) NULL
);

CREATE INDEX IX_pinecone_sync_unprocessed
    ON dbo.pinecone_sync_queue (created_at)
    WHERE processed_at IS NULL;

-- Trigger: queue a Pinecone sync event on document change
CREATE OR ALTER TRIGGER trg_queue_pinecone_sync
ON dbo.documents
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;

    -- Deletes: tombstone event
    INSERT INTO dbo.pinecone_sync_queue (operation, document_id, namespace)
    SELECT 'delete', CAST(d.id AS NVARCHAR(100)), 'production'
    FROM deleted AS d
    WHERE NOT EXISTS (SELECT 1 FROM inserted AS i WHERE i.id = d.id);

    -- Inserts and updates: upsert event with JSON payload
    INSERT INTO dbo.pinecone_sync_queue (operation, document_id, namespace, payload)
    SELECT 'upsert',
        CAST(i.id AS NVARCHAR(100)),
        'production',
        (SELECT i.content AS [text],
            i.category AS [metadata.category],
            i.created_at AS [metadata.created_at],
            i.doc_type AS [metadata.doc_type]
        FOR JSON PATH, WITHOUT_ARRAY_WRAPPER)
    FROM inserted AS i;
END;

```

```

# Python
# Outbox consumer – runs as a background worker or scheduled job

import json
import time
import pyodbc
from pinecone import Pinecone
import openai

pc = Pinecone(api_key="YOUR_API_KEY")
index = pc.Index("document-embeddings-serverless")
openai_client = openai.OpenAI(api_key="YOUR_OPENAI_KEY")

conn = pyodbc.connect(
    "DRIVER={ODBC Driver 18 for SQL Server};SERVER=localhost;"
    "DATABASE=vectordb;Trusted_Connection=yes;Encrypt=yes;"
    "TrustServerCertificate=yes"
)

def process_sync_queue(sql_conn, batch_size: int = 50):
    """
    Fetch unprocessed events from the outbox, execute Pinecone operations,
    then mark events as processed. Runs in a polling loop.
    """
    cur = sql_conn.cursor()
    # Fetch a batch of unprocessed events. READPAST + UPDLOCK skip rows
    # locked by another consumer – the T-SQL equivalent of SKIP LOCKED.
    cur.execute("""
        SELECT TOP (?) id, operation, document_id, namespace, payload
        FROM dbo.pinecone_sync_queue WITH (READPAST, UPDLOCK, ROWLOCK)
        WHERE processed_at IS NULL
        ORDER BY created_at
    """, (batch_size,))

    events = cur.fetchall()
    if not events:
        return 0

    upsert_batch = []
    delete_ids_by_namespace = {}

    for event_id, operation, document_id, namespace, payload in events:
        if operation == 'delete':
            delete_ids_by_namespace.setdefault(namespace, []).append(document_id)

        elif operation == 'upsert' and payload:
            data = json.loads(payload)
            text = data.get("text", "")
            metadata = data.get("metadata", {})

            # Generate embedding for the document
            embedding = openai_client.embeddings.create(
                input=[text],

```

```

        model="text-embedding-3-small"
    ).data[0].embedding

    upsert_batch.append({
        "id": document_id,
        "values": embedding,
        "metadata": {**metadata, "text": text},
        "_namespace": namespace # Store namespace for grouping
    })

# Execute upserts grouped by namespace
upserts_by_ns = {}
for vec in upsert_batch:
    ns = vec.pop("_namespace")
    upserts_by_ns.setdefault(ns, []).append(vec)

for ns, vectors in upserts_by_ns.items():
    index.upsert(vectors=vectors, namespace=ns)

# Execute deletes grouped by namespace
for ns, ids in delete_ids_by_namespace.items():
    index.delete(ids=ids, namespace=ns)

# Mark events as processed
cur.executemany(
    "UPDATE dbo.pinecone_sync_queue SET processed_at = SYSUTCDATETIME() WHERE id"
    = "?",
    [(e[0],) for e in events]
)
sql_conn.commit()
return len(events)

# Polling loop – in production, deploy as a dedicated service with retry logic
while True:
    processed = process_sync_queue(conn, batch_size=50)
    if processed == 0:
        time.sleep(2) # Back off when queue is empty
    else:
        print(f"Processed {processed} sync events")

```

Pattern 2: Application-layer dual write

On every create/update/delete operation in your application, write to SQL Server first, then synchronously upsert/delete from Pinecone. Simpler than the outbox pattern but introduces a failure mode: if the Pinecone call fails after the SQL Server commit, your systems diverge. Acceptable for low-stakes applications; not recommended for any dataset where consistency matters.

Pattern 3: Periodic full re-index

For datasets that change infrequently—large document corpora where new versions are published in batches—a nightly full re-index is often the most operationally straightforward approach. Export all documents from SQL Server, batch-embed them, and upsert into Pinecone. Stale data exists for at most one index cycle, which is acceptable for many knowledge base applications.

The three patterns represent the classic reliability vs. complexity tradeoff: outbox CDC is the most reliable but operationally heaviest; full re-index is the simplest but least timely.

Key Takeaways

- → **Pinecone's capacity model maps to familiar infrastructure concepts**: pods correspond to storage units (analogous to tablespace capacity), replicas correspond to read scaling (analogous to read replicas), and shards provide horizontal partitioning. Understanding which lever to pull—pods for storage, replicas for throughput—prevents the common mistake of adding replicas when you've hit a storage limit.
- → **Namespaces provide logical tenant isolation enforced at the query layer**, not at the storage hardware layer. They are the right choice for multi-tenant applications where data privacy between tenants is required at the application level, but not a substitute for separate indexes when regulatory compliance demands physical data isolation (HIPAA, GDPR right-to-erasure audit trails).
- → **Metadata filtering operates post-ANN, not pre-ANN**, which means highly selective filters on large indexes return fewer results than `top_k` requests. The practical threshold is that filters should match at least 10–20% of the index to produce reliable result sets. For cardinality-based isolation below that threshold, namespace separation is more reliable than metadata filtering.
- → **Serverless and pod-based indexes represent fundamentally different tradeoff profiles**: serverless eliminates capacity provisioning overhead and is cost-efficient for bursty or moderate workloads, but trades away the predictable low-latency (single-digit millisecond p50) that pod-based indexes deliver. Choose based on your application's latency SLA, not on index size alone.
- → **Keeping Pinecone synchronized with your primary database requires an explicit consistency strategy**. The transactional outbox pattern—queuing sync events in SQL Server within the same transaction as business record changes—is the most robust approach. Application-layer dual writes are simpler but introduce a divergence window on partial failures, and periodic full re-indexing is appropriate only when data freshness requirements allow for batch-cycle latency.

Chapter 16: Weaviate: Schema Design and Hybrid Search

Weaviate occupies a distinct position in the vector database landscape: it ships with built-in vectorization modules, a graph-inspired object model, and first-class hybrid search that combines dense vector retrieval with BM25 keyword scoring — all within a single system that requires no external embedding pipeline at insert time. For database engineers, the learning curve is less about grasping vector concepts and more about accepting a schema model that sits somewhere between a document store and a relational system, with a query language (GraphQL) that rewards understanding before it rewards productivity. This chapter walks through Weaviate's schema design philosophy, its class-and-property model, practical hybrid search configuration, multi-tenancy, and the operational patterns that make Weaviate viable at production scale.

16.1 Weaviate's Data Model: Classes, Properties, and the Object Graph

Weaviate organizes data into **classes** — the rough equivalent of tables in a relational database, or collections in MongoDB. Each class holds **objects** that carry scalar properties plus an optional vector. Unlike most vector databases where you bring your own vector, Weaviate can generate vectors automatically using a configured vectorizer module (OpenAI, Cohere, sentence-transformers via its `text2vec-transformers` module, and others). The vector becomes a first-class citizen of the object, not an add-on.

The class definition is the schema contract. Every class declaration specifies:

- → The vectorizer module (or ``none`` if you supply vectors yourself)
- → Per-property data types (``text``, ``int``, ``number``, ``boolean``, ``date``, ``uuid``, cross-references)
- → Indexing behavior per property (BM25 inverted index, filterable, sortable)
- → Vector index configuration (HNSW parameters, quantization)
- → Module-specific settings (which model, tokenization strategy)

The **cross-reference** system is Weaviate's most database-distinct feature. A property can reference objects in another class, creating a directed relationship that can be traversed in GraphQL queries. Think of it as a foreign key with graph traversal semantics — closer to a JOIN than a document embed, but resolved lazily at query time.

Tokenization matters more than it looks. For BM25 indexing, each `text` property has a tokenization setting: `word`, `lowercase`, `whitespace`, or `field`. The choice affects how terms are indexed and matched. Product codes, URLs, and structured identifiers should use `field` (exact match) or `whitespace`. Natural language content should use `word` (default). Getting this wrong produces silent retrieval failures — your keyword search simply misses matches because the indexed tokens don't align with the query tokens.

Here is a realistic schema definition for a document retrieval system using the Weaviate Python client v4:

```

# Python
import weaviate
import weaviate.classes as wvc
from weaviate.classes.config import (
    Configure,
    Property,
    DataType,
    Tokenization,
    VectorDistances,
)

client = weaviate.connect_to_local() # or connect_to_weaviate_cloud(...)

# Define a DocumentChunk class with OpenAI vectorizer
client.collections.create(
    name="DocumentChunk",
    description="Chunked document passages for RAG retrieval",
    vectorizer_config=Configure.Vectorizer.text2vec_openai(
        model="text-embedding-3-small",
        dimensions=1536,
    ),
    vector_index_config=Configure.VectorIndex.hnsw(
        distance_metric=VectorDistances.COSINE,
        ef_construction=128,
        max_connections=16,
        ef=64, # query-time ef, higher = more accurate, slower
        dynamic_ef_factor=8, # auto-scales ef based on result limit
        dynamic_ef_min=100,
        dynamic_ef_max=500,
    ),
    properties=[
        Property(
            name="content",
            data_type=DataType.TEXT,
            description="The chunk text content",
            tokenization=Tokenization.WORD, # BM25 indexed on words
            vectorize_property_name=False, # don't include prop name in
vector            skip_vectorization=False, # this prop drives the vector
        ),
        Property(
            name="source_document",
            data_type=DataType.TEXT,
            tokenization=Tokenization.FIELD, # exact match on document path/
ID            skip_vectorization=True,
        ),
        Property(
            name="chunk_index",
            data_type=DataType.INT,
            skip_vectorization=True,
        ),
        Property(

```

```

        name="created_at",
        data_type=DataType.DATE,
        skip_vectorization=True,
    ),
    Property(
        name="metadata_tags",
        data_type=DataType.TEXT_ARRAY,
        tokenization=Tokenization.WORD,
        skip_vectorization=True,
    ),
],
# Enable inverted index for BM25 hybrid search
inverted_index_config=Configure.inverted_index(
    bm25_b=0.75,
    bm25_k1=1.2,
    index_null_values=False,
    index_property_length=False,
),
)

print("DocumentChunk class created successfully")
client.close()

```

The BM25 parameters `k1` and `b` are tunable. `k1=1.2` (term frequency saturation) and `b=0.75` (document length normalization) are the classic defaults from Elasticsearch and Lucene — reasonable starting points for general English text. For short chunks (128–256 tokens), reduce `b` toward `0.5` because length normalization matters less when documents are uniformly short.

16.2 Schema Design Patterns for Production RAG Systems

Schema design mistakes in Weaviate are more expensive to fix than in a traditional RDBMS because the vector index is built during ingestion. Rebuilding it from scratch on a dataset of tens of millions of objects is an hours-long operation. The following patterns reflect hard-won production decisions.

Pattern 1: Separate classes for separate embedding models. Do not mix objects vectorized with `text-embedding-3-small` and objects vectorized with `text-embedding-3-large` in the same class. The vector space is different even though both are OpenAI models. Weaviate v1.24+ supports named vectors (multiple vectors per object), which solves this cleanly for cross-modal retrieval, but for standard single-model pipelines, one model per class is the rule.

Pattern 2: Push filtering metadata into scalar properties, not cross-references. Cross-references are powerful but expensive to filter on at scale. If you need to filter chunks by `tenant_id`, `document_type`, or `language`, store those as scalar properties with `index_filterable=True`. Filterable scalar indexes in Weaviate use a roaring bitmap structure — sub-millisecond for high-cardinality filters. Cross-reference traversal requires a separate resolution step and doesn't compose well with pre-filtering.

Pattern 3: Use `skip_vectorization=True` aggressively. By default, Weaviate concatenates all non-skipped text properties before sending them to the vectorizer. A `source_document` path like `s3://my-bucket/reports/Q4-2023-financials.pdf` pollutes the semantic vector. Mark every property that isn't semantic content as `skip_vectorization=True`.

Pattern 4: Named vectors for multi-representation objects. Weaviate v1.24 introduced named vectors, enabling one object to carry multiple vector representations. This matters for hybrid retrieval strategies where you want to search by question embedding AND summary embedding simultaneously:

```

# Python
import weaviate
import weaviate.classes as wvc
from weaviate.classes.config import Configure, Property, DataType, Tokenization

client = weaviate.connect_to_local()

# Multi-vector schema: one object, two named vector spaces
client.collections.create(
    name="ResearchPaper",
    properties=[
        Property(name="title", data_type=DataType.TEXT,
tokenization=Tokenization.WORD),
        Property(name="abstract", data_type=DataType.TEXT,
tokenization=Tokenization.WORD),
        Property(name="full_text", data_type=DataType.TEXT,
tokenization=Tokenization.WORD),
        Property(name="authors", data_type=DataType.TEXT_ARRAY,
skip_vectorization=True),
        Property(name="year", data_type=DataType.INT, skip_vectorization=True),
        Property(name="doi", data_type=DataType.TEXT,
tokenization=Tokenization.FIELD, skip_vectorization=True),
    ],
    vectorizer_config=[
        # Named vector 1: abstract-level semantic search
        Configure.NamedVectors.text2vec_openai(
            name="abstract_vector",
            source_properties=["title", "abstract"],
            model="text-embedding-3-small",
        ),
        # Named vector 2: full document retrieval
        Configure.NamedVectors.text2vec_openai(
            name="fulltext_vector",
            source_properties=["full_text"],
            model="text-embedding-3-large",
        ),
    ],
)

client.close()

```

At query time, you specify which named vector to search against. The overhead is roughly 2× storage and index memory, but the retrieval flexibility is worth it for heterogeneous content where title/abstract search and full-text search have materially different precision characteristics.

Property index flags reference:

Flag	Type	Purpose	Default
------	------	---------	---------

|-----|-----|-----|-----|

| `index_filterable` | bool | Roaring bitmap for pre-filtering | true |

| `index_searchable` | bool | BM25 inverted index | true for text |

| `index_range_filters` | bool | Numeric range index | false |

| `skip_vectorization` | bool | Exclude from embedding input | false |

Turn off `index_searchable` for properties you never BM25-search. It saves disk and speeds up writes. For an INT property like `chunk_index` you'll never keyword-search, both `index_searchable` and `skip_vectorization` should be `true` and `true` respectively.

16.3 Hybrid Search: BM25 + Vector Fusion

Hybrid search is the single most operationally significant feature Weaviate offers over simpler vector stores. Pure vector search fails on proper nouns, product codes, rare terminology, and out-of-vocabulary tokens that the embedding model hasn't seen at training time. Pure BM25 fails on semantic paraphrase and synonym matching. Hybrid search fuses both signals.

Weaviate implements hybrid search via **Reciprocal Rank Fusion (RRF)** or a **relative score fusion** strategy. Both take the ranked lists from BM25 and vector ANN independently, then merge them. The `alpha` parameter controls the blend: `alpha=0.0` is pure BM25, `alpha=1.0` is pure vector, `alpha=0.5` is an equal blend. The default RRF formula is:

$$\text{RRF_score}(d) = \text{sum_over_rankings}(1 / (k + \text{rank}(d)))$$

where `k=60` is a smoothing constant. This is the same formula used in Elasticsearch's hybrid search and in academic IR literature. It is robust to outliers and scale differences between BM25 and cosine similarity scores.

```

# Python
import weaviate
import weaviate.classes as wvc
from weaviate.classes.query import HybridFusion, MetadataQuery, Filter

client = weaviate.connect_to_local()

collection = client.collections.get("DocumentChunk")

# Basic hybrid search
response = collection.query.hybrid(
    query="transformer architecture self-attention mechanism",
    alpha=0.7,          # 70% weight toward vector, 30% BM25
    limit=10,
    return_metadata=MetadataQuery(score=True, explain_score=True),
)

for obj in response.objects:
    print(f"Score: {obj.metadata.score:.4f}")
    print(f"Explain: {obj.metadata.explain_score}")
    print(f"Content: {obj.properties['content'][:120]}")
    print("---")

# Hybrid search with pre-filtering
response_filtered = collection.query.hybrid(
    query="quarterly revenue forecast",
    alpha=0.5,
    limit=10,
    filters=Filter.by_property("source_document").contains_any(
        ["finance/Q4-2023.pdf", "finance/Q3-2023.pdf"]
    ),
    return_metadata=MetadataQuery(score=True),
)

# Hybrid search with named vectors (multi-vector schema)
research_collection = client.collections.get("ResearchPaper")

response_named = research_collection.query.hybrid(
    query="attention is all you need transformer",
    alpha=0.6,
    target_vector="abstract_vector",  # search in abstract embedding space
    limit=5,
    return_metadata=MetadataQuery(score=True, explain_score=True),
)

client.close()

```

Alpha tuning is empirical, not theoretical. Run your benchmark query set against your actual data, vary `alpha` from 0.3 to 0.9 in 0.1 increments, and measure recall@10 or NDCG@10 against human-labeled relevance judgments. Published benchmarks across IR datasets suggest `alpha` between 0.6 and 0.75 is optimal for most English-language

knowledge retrieval tasks. For technical queries with precise terminology (SQL error codes, API method names, CVE identifiers), shift toward `0.3 – 0.4` to give BM25 more weight. For conversational or natural language questions, `0.7 – 0.8` is typical.

Score fusion strategy choice:

- → ``HybridFusion.RANKED`` (RRF): rank-based, insensitive to raw score magnitudes. Recommended when BM25 and vector scores have very different distributions (they almost always do).
- → ``HybridFusion.RELATIVE_SCORE``: normalizes both score lists to `[0,1]` then linearly combines. More sensitive to score distribution; can produce better results when both signals are calibrated.

RRF is the safer default for production deployments where score distribution is unknown or shifts with data updates.

16.4 Multi-Tenancy: One Schema, Many Isolated Datasets

Multi-tenancy in Weaviate is a first-class schema feature, not an application-layer workaround. This matters for SaaS platforms and enterprise deployments where you need per-customer data isolation without operating separate Weaviate instances.

Weaviate's multi-tenancy model assigns each tenant a dedicated shard. The implications are significant:

- → **Vector index isolation**: each tenant's HNSW graph is independent. A query for Tenant A never scans Tenant B's index.
- → **Lifecycle control**: tenants can be activated (hot), deactivated (warm, on disk), or offloaded (cold, off disk) independently. This is the key mechanism for cost-optimized multi-tenant deployments where not all customers are active simultaneously.
- → **No cross-tenant contamination**: a malformed insert or a slow query on one tenant cannot degrade another tenant's shard.

The tradeoff is memory. Each active tenant's HNSW graph must be loaded into memory. At 10,000 objects per tenant with `ef_construction=128` and 1536-dimension float32 vectors, expect roughly 70–90 MB per tenant in memory. At 1,000 active tenants that's 70–90 GB — plan your node sizing accordingly, or use tenant activation/deactivation to manage the active set.

```

# Python
import weaviate
import weaviate.classes as wvc
from weaviate.classes.config import Configure, Property, DataType, Tokenization
from weaviate.classes.tenants import Tenant, TenantActivityStatus

client = weaviate.connect_to_local()

# Create a multi-tenant collection
client.collections.create(
    name="CustomerDocuments",
    multi_tenancy_config=Configure.multi_tenancy(
        enabled=True,
        auto_tenant_creation=False, # explicit tenant management
    ),
    vectorizer_config=Configure.Vectorizer.text2vec_openai(
        model="text-embedding-3-small",
    ),
    properties=[
        Property(name="content", data_type=DataType.TEXT,
tokenization=Tokenization.WORD),
        Property(name="doc_type", data_type=DataType.TEXT,
tokenization=Tokenization.FIELD),
        Property(name="uploaded_by", data_type=DataType.TEXT,
skip_vectorization=True),
    ],
)

collection = client.collections.get("CustomerDocuments")

# Create tenants (one per customer)
collection.tenants.create([
    Tenant(name="customer_acme", activity_status=TenantActivityStatus.ACTIVE),
    Tenant(name="customer_globex", activity_status=TenantActivityStatus.ACTIVE),
    Tenant(name="customer_initech", activity_status=TenantActivityStatus.INACTIVE),
# warm
])

# Insert objects into a specific tenant
tenant_collection = collection.with_tenant("customer_acme")

with tenant_collection.batch.dynamic() as batch:
    batch.add_object(properties={
        "content": "Q4 revenue exceeded targets by 12% driven by APAC
expansion.",
        "doc_type": "financial_report",
        "uploaded_by": "finance_team",
    })
    batch.add_object(properties={
        "content": "New product roadmap approved by executive committee.",
        "doc_type": "strategy_doc",
        "uploaded_by": "product_team",
    })

```

```

# Query within a tenant
response = tenant_collection.query.hybrid(
    query="revenue targets quarterly results",
    alpha=0.6,
    limit=5,
)

for obj in response.objects:
    print(obj.properties["content"])

# Deactivate an inactive tenant to free memory (tenant data stays on disk)
collection.tenants.update([
    Tenant(
        name="customer_globex",
        activity_status=TenantActivityStatus.INACTIVE,
    )
])

# Reactivate when needed
collection.tenants.update([
    Tenant(
        name="customer_globex",
        activity_status=TenantActivityStatus.ACTIVE,
    )
])

client.close()

```

Tenant activation patterns in practice:

| Scenario | Strategy |

|-----|-----|

| Enterprise SaaS, most tenants active | All ACTIVE, scale horizontally |

| Long-tail SaaS, 20% tenants use 80% of queries | Active set management via usage metrics |

| Archive/compliance, tenants rarely queried | OFFLOADED by default, activate on-demand |

| Testing/staging environments | INACTIVE until test run starts |

Auto-tenant creation (`auto_tenant_creation=True`) creates a tenant shard the first time an object is inserted for an unknown tenant key. It's convenient for development but dangerous in production — a client bug with an unexpected tenant ID can create thousands of orphaned shards.

16.5 Batch Ingestion, HNSW Tuning, and Query Performance

Weaviate's HNSW implementation is configurable at class definition time. The parameters are identical in semantics to those in FAISS and hnswlib because Weaviate's implementation derives from the same academic specification, but the defaults are conservative — intentionally so, because Weaviate serves latency-sensitive HTTP workloads, not offline batch retrieval.

The three HNSW parameters you will tune:

- → ``ef_construction`` (default: 128): graph quality during build. Higher = better recall, slower ingestion. Range 64–256 covers most use cases. Don't go below 64 — recall degrades sharply. Don't exceed 256 unless you have a recall requirement above 99.5%.
- → ``max_connections`` (M, default: 64): neighbors per node. Higher = better recall on large datasets, higher memory. For datasets under 10M objects, M=16 is enough. For 100M+ objects, M=32–64.
- → ``ef`` (default: 64 or dynamic): query-time beam width. The primary recall vs. latency knob. Set via ``dynamic_ef_factor`` to auto-scale based on ``limit``.

Memory estimation for HNSW:

```
Memory (bytes) ≈ num_objects × (dimensions × 4 + max_connections × 8 × 2)
```

For 1M objects, 1536 dimensions, M=16:

```
1,000,000 × (1536 × 4 + 16 × 8 × 2) = 1,000,000 × (6144 + 256) = ~6.4 GB
```

This is the graph memory floor. Add scalar property storage and inverted index overhead on top.

Batch ingestion with error handling:


```

# Python
import weaviate
import weaviate.classes as wvc
from weaviate.classes.config import ConsistencyLevel
import json
from pathlib import Path
from tqdm import tqdm

client = weaviate.connect_to_local()
collection = client.collections.get("DocumentChunk")

def load_chunks(filepath: str) -> list[dict]:
    """Load pre-chunked documents from JSONL."""
    with open(filepath) as f:
        return [json.loads(line) for line in f]

chunks = load_chunks("chunks.jsonl")

# Dynamic batching: Weaviate auto-tunes batch size based on server capacity
failed_objects = []

with collection.batch.dynamic() as batch:
    for chunk in tqdm(chunks, desc="Ingesting chunks"):
        batch.add_object(
            properties={
                "content": chunk["text"],
                "source_document": chunk["source"],
                "chunk_index": chunk["index"],
                "metadata_tags": chunk.get("tags", []),
            },
            # Optionally supply a deterministic UUID for idempotency
            uuid=wvc.util.generate_uuid5(
                f"{chunk['source']}_{chunk['index']}"
            ),
        )

# Check for failed objects after batch completes
if batch.number_errors > 0:
    print(f"Batch completed with {batch.number_errors} errors")
    for failed in collection.batch.failed_objects:
        print(f"Failed: {failed.message} | Object: {failed.object_.properties}")

print(f"Total objects in collection: {collection.aggregate.over_all().total_count}")
client.close()

```

Key batch performance observations from production deployments:

- → Dynamic batching (`batch.dynamic()`) outperforms fixed-size batching in most cases because Weaviate's server-side admission control adapts to memory pressure during HNSW graph construction.

- → With external vectorization (OpenAI, Cohere), the embedding API call is the bottleneck — not Weaviate's insert throughput. Use async batching or parallel chunk pipelines with a semaphore to control concurrent embedding calls.
- → With built-in vectorizers (`text2vec-transformers` running locally), GPU availability dominates throughput. A single T4 GPU handles roughly 2,000–4,000 tokens per second for `sentence-transformers/all-MiniLM-L6-v2`.
- → Set `WEAVIATE\_GRPC\_PORT` and use gRPC transport for batch inserts in v1.23+. gRPC batch ingestion is meaningfully faster than the REST API for large payloads because it avoids JSON serialization overhead — benchmark against your own payload shapes rather than assuming a fixed improvement.

Query latency benchmarks (approximate, single-node Weaviate, 1M objects):

| Search Type | ef=64 | ef=128 | ef=256 |

|-----|-----|-----|-----|

| Pure vector | ~8ms | ~14ms | ~25ms |

| Pure BM25 | ~5ms | ~5ms | ~5ms |

| Hybrid (RRF) | ~12ms | ~18ms | ~30ms |

| Hybrid + filter | ~15ms | ~22ms | ~38ms |

BM25 latency is largely independent of `ef` because it uses a separate inverted index path. Hybrid latency is dominated by the vector search component, so `ef` tuning applies to hybrid queries the same way it applies to pure vector queries.

16.6 Filtering Strategies: Pre-Filter vs. Post-Filter

Weaviate's approach to filtered vector search has evolved significantly. Early versions used post-filtering — execute ANN search, then discard results that don't match the filter predicate. Post-filtering is simple to implement but statistically problematic: if your filter eliminates 90% of the dataset, you may retrieve only 1–2 results even when you asked for 10, because the ANN graph was built over the full dataset and doesn't know which nodes will survive the filter.

Weaviate v1.17+ implements **flat search fallback** and **ACORN-style filtered ANN**, where the execution strategy adapts based on filter selectivity:

- → **High selectivity filter** (small result set, e.g., 0.1% of objects match): Weaviate falls back to flat (brute-force) search over the filtered subset. This is faster than navigating the HNSW graph when the candidate pool is small enough.
- → **Low selectivity filter** (large result set, e.g., 80% of objects match): standard HNSW graph traversal with filter evaluation at candidate inspection time.

The threshold for switching strategies is configurable via `flat_search_cutoff` in the collection's vector index configuration. The default threshold is typically around 40,000 objects — if fewer objects match the filter, flat search is used.

```
# Python
from weaviate.classes.config import Configure, VectorDistances

client.collections.create(
    name="ProductCatalog",
    vector_index_config=Configure.VectorIndex.hnsw(
        distance_metric=VectorDistances.COSINE,
        ef_construction=128,
        max_connections=16,
        ef=96,
        flat_search_cutoff=40_000, # tune based on your filter distribution
    ),
    # ... properties omitted for brevity
)
```

Filter composition rules that matter:

1. Use `Filter.by_property("field").equal()` for equality filters — these hit the roaring bitmap index directly.
2. Avoid filtering on cross-references when a scalar property would suffice. Cross-reference filters require graph traversal before the vector search even begins.
3. Combine multiple filters with `Filter.all_of([...])` (AND) or `Filter.any_of([...])` (OR). The query planner intersects roaring bitmaps, which is fast.
4. Range filters on numeric properties require `index_range_filters=True` at schema definition time. Without this flag, numeric range queries fall back to a full-scan of the property store — expensive at scale.

```
# Python
from weaviate.classes.query import Filter, MetadataQuery

collection = client.collections.get("ProductCatalog")

# Compound filter: category AND price range AND in-stock
response = collection.query.hybrid(
    query="wireless noise-cancelling headphones",
    alpha=0.65,
    limit=20,
    filters=Filter.all_of([
        Filter.by_property("category").equal("electronics"),
        Filter.by_property("price_usd").greater_than(50.0),
        Filter.by_property("price_usd").less_than(500.0),
        Filter.by_property("in_stock").equal(True),
    ]),
    return_metadata=MetadataQuery(score=True),
)

for obj in response.objects:
    print(f"{obj.properties['name']:40s}  score={obj.metadata.score:.4f}")
```

16.7 Generative Search and RAG Modules

Weaviate's generative search module (`generative-openai`, `generative-cohere`, `generative-anthropic`, and others) integrates LLM generation directly into the query pipeline. Instead of retrieving objects and then calling an LLM in application code, you declare the generation step in the query itself.

This is architecturally convenient but not always the right choice. The tradeoff is flexibility vs. simplicity: Weaviate's built-in generative module handles the simplest RAG patterns well, but complex prompt engineering, multi-step retrieval, and retrieval with re-ranking often require application-layer orchestration anyway.

When to use the generative module:

- → Prototyping RAG systems quickly
- → Simple question-answering over a well-bounded corpus
- → Single-turn queries with straightforward summarization requirements

When to move generation to the application layer:

- → Multi-turn conversations requiring context management
- → Re-ranking retrieved results before generation

- → Prompt templates that depend on user-specific data
- → Cost optimization across multiple providers based on query type

```
# Python
from weaviate.classes.generate import GenerativeConfig
from weaviate.classes.query import MetadataQuery

client = weaviate.connect_to_local() # Weaviate configured with generative-openai
module

collection = client.collections.get("DocumentChunk")

# Single-prompt generation: apply one prompt to all retrieved objects
response = collection.generate.hybrid(
    query="climate change mitigation strategies",
    alpha=0.65,
    limit=5,
    single_prompt="Summarize the key point of this passage in one sentence:
{content}",
    return_metadata=MetadataQuery(score=True),
)

for obj in response.objects:
    print(f"Source: {obj.properties['source_document']}")
    print(f"Generated: {obj.generated}")
    print()

# Grouped task: synthesize a response across all retrieved objects
response_grouped = collection.generate.hybrid(
    query="what are the main approaches to carbon capture?",
    alpha=0.65,
    limit=8,
    grouped_task=(
        "Based on the provided passages, write a concise technical summary "
        "of the main carbon capture approaches discussed, citing specific methods."
    ),
    grouped_properties=["content", "source_document"],
)

print("Synthesized answer:")
print(response_grouped.generated)
```

The `single_prompt` mode calls the LLM once per retrieved object — useful for per-passage annotation or extraction. The `grouped_task` mode sends all retrieved passages to the LLM in a single call — the standard RAG synthesis pattern.

Module configuration in the Weaviate instance settings (set at startup, not per-query):

```
# docker-compose.yml excerpt
environment:
  ENABLE_MODULES: "text2vec-openai,generative-openai"
  OPENAI_APIKEY: "${OPENAI_API_KEY}"
  # For Azure OpenAI:
  # AZURE_APIKEY: "${AZURE_OPENAI_API_KEY}"
```

If you operate Weaviate in a zero-trust network where outbound API calls are restricted, the `text2vec-transformers` and `generative-ollama` modules run fully locally using self-hosted models — no external API calls.

16.8 Backup, Restore, and Operational Concerns

Weaviate's backup system operates at the collection level and supports pluggable backends: filesystem, S3, GCS, and Azure Blob Storage. Backups are **live** — they do not require stopping the database or taking it offline.

```
# Python
import weaviate

client = weaviate.connect_to_local()

# Trigger a backup to S3 (requires backup-s3 module enabled)
result = client.backup.create(
    backup_id="backup-2024-01-15-prod",
    backend="s3",
    include_collections=["DocumentChunk", "ResearchPaper"],
    wait_for_completion=True,
)
print(f"Backup status: {result.status}")

# Restore from backup
restore_result = client.backup.restore(
    backup_id="backup-2024-01-15-prod",
    backend="s3",
    include_collections=["DocumentChunk"],
    wait_for_completion=True,
)
print(f"Restore status: {restore_result.status}")

client.close()
```

Operational checklist for production Weaviate deployments:

Concern	Recommendation

|-----|-----|

| Memory sizing | 2× the HNSW memory estimate; leave headroom for batch ingestion spikes |

| Disk sizing | 3–4× raw vector data size (inverted index, WAL, metadata) |

| Replication | `replication_factor=3` for production; `consistency_level=QUORUM` for reads |

| Backup cadence | Daily backup to object storage; verify restore quarterly |

| Module isolation | Run `text2vec-transformers` as a separate container; GPU contention with Weaviate's own Go runtime causes latency spikes |

| Schema versioning | Treat class definitions as code; store in version control alongside migration scripts |

| Monitoring | Export Prometheus metrics via `/v1/metrics`; alert on `batch_durations_ms_bucket`, `query_dimensions_ms_bucket`, and heap memory |

Replication configuration:

```
# Python
from weaviate.classes.config import Configure, Reconfigure

# At class creation time
client.collections.create(
    name="DocumentChunk",
    replication_config=Configure.replication(factor=3),
    # ... other config
)

# Consistency level at query time
from weaviate.classes.query import MetadataQuery
from weaviate.classes.config import ConsistencyLevel

collection = client.collections.get("DocumentChunk")

# Read with quorum consistency
response = collection.with_consistency_level(
    ConsistencyLevel.QUORUM
).query.hybrid(
    query="distributed systems consensus",
    alpha=0.6,
    limit=10,
)
```

Weaviate uses a leaderless replication model similar to Cassandra's — all replica nodes are equal, and consistency is controlled per-operation via `ONE`, `QUORUM`, or `ALL`. For RAG workloads, `QUORUM` reads with `QUORUM` writes provides the right balance between consistency and availability.

16.9 Weaviate vs. SQL Server Native Vectors: Decision Framework

After the preceding chapters on SQL Server native vectors and this chapter on Weaviate, the natural question is when to choose one over the other. This is not a "best database wins" comparison — it is a context-dependent architectural decision.

Choose SQL Server native vectors when:

- → Your application data already lives in SQL Server and you want to avoid a second operational system
- → Your team's operational expertise is SQL and relational databases
- → Your vector dataset is under ~10M objects and hybrid search requirements are modest
- → Strong ACID transactions matter (financial records, inventory, anything with coordinated writes)
- → You need arbitrary JOIN semantics between vector results and relational data

Choose Weaviate when:

- → You need production-grade hybrid search (BM25 + vector) with minimal configuration
- → Multi-tenancy with per-tenant shard isolation is a hard requirement
- → Built-in vectorization modules remove embedding pipeline complexity from your architecture
- → Your dataset grows to tens or hundreds of millions of objects with demanding recall/latency SLAs
- → Named vectors (multi-representation objects) fit your retrieval model
- → You want generative search integrated into the query layer for prototyping

The operational cost of adding Weaviate to a SQL Server shop is real. You now operate two stateful systems, two backup procedures, two monitoring stacks. Many teams start with SQL Server native vectors, hit a concrete limitation (multi-tenancy isolation, hybrid search quality, recall at scale), and then migrate to Weaviate for the specific workload that needs it while keeping SQL Server for transactional data.

The reverse migration — from Weaviate back to SQL Server native vectors — is rare but happens when teams underestimate SQL Server's capacity or overestimate the complexity of their vector workload.

Key Takeaways

- → **Weaviate's class-and-property schema is a design contract**: tokenization choices, ``skip_vectorization`` flags, and index configuration flags (``index_filterable``, ``index_searchable``, ``index_range_filters``) have permanent performance consequences that cannot be changed without rebuilding the index, so schema design deserves the same rigor as DDL in a relational database.
- → **Hybrid search with Reciprocal Rank Fusion combines BM25 and vector ANN into a single ranked result**: the ``alpha`` parameter blends the two signals, and empirical tuning against labeled relevance judgments — rather than theoretical defaults — consistently produces the best retrieval quality for a specific domain and query type.
- → **Multi-tenancy is a first-class Weaviate feature backed by shard isolation**: each tenant gets an independent HNSW graph, enabling per-tenant lifecycle management (ACTIVE, INACTIVE, OFFLOADED) that controls memory consumption in large-scale SaaS deployments, but active-tenant memory budgeting must be part of node sizing from day one.
- → **Named vectors (Weaviate v1.24+) allow one object to carry multiple independent vector representations**: this supports cross-modal and multi-granularity retrieval strategies — for example, searching by abstract embedding and full-text embedding independently — at the cost of roughly proportional additional storage and index memory.
- → **Filter selectivity drives Weaviate's execution strategy**: high-selectivity filters trigger a flat brute-force search over the matching subset, which is faster than HNSW graph traversal when the candidate pool is small; schema-time flags like ``index_range_filters=True`` on numeric properties are prerequisites for efficient range filtering, and omitting them causes silent full-scan fallbacks at query time.

Chapter 17: Qdrant: Payload Filtering and Precision Tuning

Qdrant occupies a distinctive position in the vector database landscape: it was built from the ground up in Rust with a clear architectural opinion that vector search and structured metadata filtering should be first-class equals, not an afterthought. Where other systems treat payload filters as a post-processing step applied after ANN retrieval, Qdrant's filtered HNSW implementation weaves payload conditions directly into the graph traversal itself. The result is a system where adding filters often costs less than you expect, and where the gap between "give me the nearest vectors" and "give me the nearest vectors matching these exact business conditions" nearly disappears. This chapter walks through Qdrant's data model, its payload indexing system, the mechanics of filtered HNSW, and the production configuration knobs that separate a tuned deployment from one that simply runs.

17.1 Qdrant's Data Model: Collections, Points, and Payloads

Before touching query syntax, you need to understand how Qdrant organizes data. The mental model maps cleanly to relational concepts, which makes it approachable for DBAs.

A **collection** is Qdrant's equivalent of a table. It holds a set of **points**, where each point contains:

- → A **unique ID** — either a 64-bit unsigned integer or a UUID string
- → One or more **vectors** — dense, sparse, or multivector depending on configuration
- → A **payload** — an arbitrary JSON document, roughly analogous to an NVARCHAR(MAX) column holding JSON in SQL Server

Unlike traditional databases where schema is enforced at write time, Qdrant's payload schema is flexible by default. You can insert points with completely different payload shapes into the same collection. However, the analogy to SQL Server's JSON support goes further than you might expect: just as SQL Server benefits from indexed computed columns over JSON paths before those paths become filter-efficient, Qdrant requires you to create **payload indexes** before payload fields become efficient filter predicates.

This distinction matters. Without a payload index, Qdrant performs a full scan of all payload documents to evaluate a filter condition — effectively a sequential scan. With a payload index, filter evaluation happens in a purpose-built data structure before or during vector traversal.

Supported payload types and their indexed forms:

| Payload Type | Qdrant Index Type | Analogy |

| --- | --- | --- |

| String/keyword | Keyword index | B-tree on VARCHAR |

| Integer | Integer index | B-tree on BIGINT |

| Float | Float index | B-tree on DOUBLE |

| Boolean | — (inline, fast) | BIT column |

| Geo coordinates | Geo index | PostGIS spatial index |

| Full text | Text index (tokenized) | GIN index on tsvector |

| UUID | UUID index | B-tree on UUID |

The collection-level configuration also controls vector parameters, distance metrics, and the HNSW graph parameters that affect both search quality and resource consumption. Let's stand up a collection and populate it with something realistic.

```

# Python
from qdrant_client import QdrantClient
from qdrant_client.models import (
    Distance,
    VectorParams,
    PointStruct,
    CreateCollection,
)
from sentence_transformers import SentenceTransformer
import uuid

# Connect to a local Qdrant instance (default port 6333)
client = QdrantClient(host="localhost", port=6333)

COLLECTION_NAME = "product_catalog"
VECTOR_DIM = 384 # all-MiniLM-L6-v2 output dimension

# Create collection with explicit HNSW configuration
client.create_collection(
    collection_name=COLLECTION_NAME,
    vectors_config=VectorParams(
        size=VECTOR_DIM,
        distance=Distance.COSINE,
        # HNSW index params – tunable per collection
        hnsw_config={
            "m": 16, # edges per node; higher = better recall, more
RAM
            "ef_construct": 100, # build-time beam width
            "full_scan_threshold": 10_000, # switch to brute force below this count
        },
    ),
)

# Create payload indexes BEFORE bulk insert (more efficient)
client.create_payload_index(
    collection_name=COLLECTION_NAME,
    field_name="category",
    field_schema="keyword",
)
client.create_payload_index(
    collection_name=COLLECTION_NAME,
    field_name="price",
    field_schema="float",
)
client.create_payload_index(
    collection_name=COLLECTION_NAME,
    field_name="in_stock",
    field_schema="bool",
)
client.create_payload_index(
    collection_name=COLLECTION_NAME,
    field_name="description",
    field_schema={

```

```

        "type": "text",
        "tokenizer": "word",
        "min_token_len": 2,
        "max_token_len": 20,
        "lowercase": True,
    },
)

# Encode some product descriptions
encoder = SentenceTransformer("all-MiniLM-L6-v2")

products = [
    {"id": str(uuid.uuid4()), "text": "Wireless noise-canceling over-ear headphones",
     "category": "electronics", "price": 149.99, "in_stock": True, "brand": "SoundPro"},
    {"id": str(uuid.uuid4()), "text": "Ergonomic mechanical keyboard with RGB backlight",
     "category": "electronics", "price": 89.99, "in_stock": True, "brand": "TypeMaster"},
    {"id": str(uuid.uuid4()), "text": "Standing desk converter, 36-inch wide surface",
     "category": "furniture", "price": 229.00, "in_stock": False, "brand": "DeskRise"},
    {"id": str(uuid.uuid4()), "text": "Portable USB-C hub with 4K HDMI output",
     "category": "electronics", "price": 59.99, "in_stock": True, "brand": "HubTech"},
    {"id": str(uuid.uuid4()), "text": "Adjustable lumbar support office chair",
     "category": "furniture", "price": 399.00, "in_stock": True, "brand": "ErgoSeat"},
]

vectors = encoder.encode([p["text"] for p in products]).tolist()

points = [
    PointStruct(
        id=p["id"],
        vector=v,
        payload={
            "category": p["category"],
            "price": p["price"],
            "in_stock": p["in_stock"],
            "brand": p["brand"],
            "description": p["text"],
        },
    )
    for p, v in zip(products, vectors)
]

# Upsert in a single batch
operation_info = client.upsert(
    collection_name=COLLECTION_NAME,
    wait=True, # synchronous – waits for indexing to complete

```

```

        points=points,
    )
    print(f"Upsert status: {operation_info.status}")

    # Verify collection info
    info = client.get_collection(COLLECTION_NAME)
    print(f"Points count: {info.points_count}")
    print(f"Indexed vectors: {info.indexed_vectors_count}")

```

The `wait=True` parameter deserves mention. Qdrant is eventually consistent by default — vectors are accepted into a write-ahead buffer before being incorporated into the HNSW graph. Setting `wait=True` blocks until the operation is confirmed durable, which is the correct choice for transactional writes. For bulk loading, omitting it (or setting `wait=False`) allows pipelining and can improve throughput by an order of magnitude, provided you validate completeness afterward.

17.2 Payload Filtering: From Simple Conditions to Complex Predicates

Qdrant's filter language uses a nested boolean structure that maps directly to SQL WHERE clauses. Once you see the correspondence, the API becomes predictable.

The top-level filter is always one of: `must` (AND), `should` (OR), `must_not` (NOT), or their nested combinations. Each condition targets a specific payload field using a typed matcher.

```

# Python
from qdrant_client.models import (
    Filter,
    FieldCondition,
    MatchValue,
    MatchAny,
    Range,
    SearchRequest,
)

encoder = SentenceTransformer("all-MiniLM-L6-v2")
query_text = "comfortable headphones for long work sessions"
query_vector = encoder.encode(query_text).tolist()

# -----
# Example 1: Simple AND filter
# SQL equivalent: WHERE category = 'electronics' AND in_stock = true
# -----
simple_filter = Filter(
    must=[
        FieldCondition(key="category", match=MatchValue(value="electronics")),
        FieldCondition(key="in_stock", match=MatchValue(value=True)),
    ]
)

results = client.search(
    collection_name=COLLECTION_NAME,
    query_vector=query_vector,
    query_filter=simple_filter,
    limit=5,
    with_payload=True,
    with_vectors=False,
)

for r in results:
    print(f"Score: {r.score:.4f} | {r.payload['description']} | $
    {r.payload['price']}")

# -----
# Example 2: Range filter on numeric field
# SQL equivalent: WHERE price BETWEEN 50.00 AND 200.00
# -----
price_filter = Filter(
    must=[
        FieldCondition(key="price", range=Range(gte=50.0, lte=200.0)),
        FieldCondition(key="in_stock", match=MatchValue(value=True)),
    ]
)

results_price = client.search(
    collection_name=COLLECTION_NAME,
    query_vector=query_vector,
    query_filter=price_filter,

```

```

        limit=10,
        with_payload=["description", "price", "brand"],
    )

# -----
# Example 3: OR condition with MatchAny
# SQL equivalent: WHERE category IN ('electronics', 'accessories')
# -----
category_filter = Filter(
    must=[
        FieldCondition(
            key="category",
            match=MatchAny(any=["electronics", "accessories"]),
        ),
        FieldCondition(key="price", range=Range(lt=100.0)),
    ]
)

# -----
# Example 4: Nested boolean – (must AND) combined with (should OR)
# SQL equivalent:
#   WHERE in_stock = true
#         AND (category = 'electronics' OR price < 100.0)
# -----
nested_filter = Filter(
    must=[
        FieldCondition(key="in_stock", match=MatchValue(value=True)),
        Filter(
            should=[
                FieldCondition(key="category",
match=MatchValue(value="electronics")),
                FieldCondition(key="price", range=Range(lt=100.0)),
            ]
        ),
    ]
)

results_nested = client.search(
    collection_name=COLLECTION_NAME,
    query_vector=query_vector,
    query_filter=nested_filter,
    limit=5,
    with_payload=True,
)

# -----
# Example 5: must_not exclusion
# SQL equivalent: WHERE brand != 'SoundPro' AND in_stock = true
# -----
exclusion_filter = Filter(
    must=[
        FieldCondition(key="in_stock", match=MatchValue(value=True)),
    ],

```



```
must_not=[
    FieldCondition(key="brand", match=MatchValue(value="SoundPro")),
],
)
```

One thing that trips up developers coming from SQL: Qdrant filters operate on the **payload document as a whole**, not on join results. There is no equivalent of a multi-table join at query time. If your data model requires join semantics — say, filtering products by a property that lives in a separate entity — you must denormalize that property into the point's payload before inserting. This is standard practice in vector databases and mirrors the document database pattern: write-time denormalization in exchange for read-time simplicity and speed.

Nested payload fields are accessed with dot notation. If a payload contains `{"metadata": {"region": "us-east-1"}}`, the filter key is `"metadata.region"`. Arrays are also supported: if a payload field contains `["tag1", "tag2", "tag3"]`, a `MatchValue` condition checks whether any element matches.

17.3 Filtered HNSW: How Qdrant Avoids the Post-Filter Penalty

To understand why Qdrant's filtered search is competitive with unfiltered search, you need to understand what most other systems do and why it's costly.

The naive approach — post-filtering:

1. Retrieve the top-K neighbors from the HNSW graph
2. Apply payload filters to those K results
3. Return however many survive the filter

This works when the filter selectivity is low (most documents pass). When selectivity is high — say, only 0.1% of documents match the filter — you need to retrieve $K/0.001 = 1000K$ candidates to expect K results. At that point, you have effectively done a full graph scan, and the ANN index provides no benefit.

Qdrant's approach — filtered HNSW traversal:

Qdrant maintains an additional data structure: a **payload filter index** for each indexed field, which maps field values to point IDs. During HNSW traversal, Qdrant uses this index to evaluate filter conditions per candidate node **before** computing distances or following graph edges. Nodes that fail the filter condition are skipped during graph traversal, not just discarded afterward.

The practical effect is that the search explores the graph only through nodes that satisfy the filter. This is closer to how a SQL Server filtered index works: the index was built over a specific subset of data, and queries that match that predicate can use the index directly rather than filtering after retrieval.

There is a tradeoff: HNSW graphs can become less connected when many nodes are excluded. Qdrant addresses this with an automatic fallback mechanism. When the expected number of matching points falls below `full_scan_threshold` (configured per collection), Qdrant switches to brute-force search over only the matching points. For small result sets, brute force over a tiny filtered subset is faster than graph traversal anyway.

```

# Python
# Demonstrating the performance difference between filtered and unfiltered search,
# and verifying that Qdrant is using its index vs. brute force path.

import time
from qdrant_client.models import SearchParams

# For a larger dataset, create and populate a collection programmatically
import random
import numpy as np

LARGE_COLLECTION = "benchmark_collection"
N_POINTS = 100_000
DIM = 128

# Create collection
try:
    client.delete_collection(LARGE_COLLECTION)
except Exception:
    pass

client.create_collection(
    collection_name=LARGE_COLLECTION,
    vectors_config=VectorParams(
        size=DIM,
        distance=Distance.COSINE,
        hnsw_config={
            "m": 16,
            "ef_construct": 128,
            "full_scan_threshold": 10_000,
        },
    ),
)

# Create payload index before bulk insert
client.create_payload_index(
    collection_name=LARGE_COLLECTION,
    field_name="tenant_id",
    field_schema="keyword",
)
client.create_payload_index(
    collection_name=LARGE_COLLECTION,
    field_name="score",
    field_schema="float",
)

# Generate random points with tenant labels (100 tenants, ~1000 points each)
BATCH_SIZE = 1000
tenants = [f"tenant_{i:03d}" for i in range(100)]

for batch_start in range(0, N_POINTS, BATCH_SIZE):
    batch_ids = list(range(batch_start, min(batch_start + BATCH_SIZE, N_POINTS)))
    batch_vectors = np.random.rand(len(batch_ids), DIM).tolist()

```

```

batch_points = [
    PointStruct(
        id=i,
        vector=v,
        payload={
            "tenant_id": random.choice(tenants),
            "score": random.uniform(0.0, 1.0),
        },
    )
    for i, v in zip(batch_ids, batch_vectors)
]
client.upsert(
    collection_name=LARGE_COLLECTION,
    wait=False, # async for bulk load throughput
    points=batch_points,
)

# Wait for indexing to complete
import time
while True:
    info = client.get_collection(LARGE_COLLECTION)
    if info.status.value == "green":
        break
    time.sleep(1)

query_vector = np.random.rand(DIM).tolist()

# Benchmark 1: Unfiltered search
start = time.perf_counter()
for _ in range(50):
    client.search(
        collection_name=LARGE_COLLECTION,
        query_vector=query_vector,
        limit=10,
    )
unfiltered_ms = (time.perf_counter() - start) / 50 * 1000
print(f"Unfiltered search avg latency: {unfiltered_ms:.2f}ms")

# Benchmark 2: Filtered search – 1% selectivity (1 of 100 tenants)
tenant_filter = Filter(
    must=[FieldCondition(key="tenant_id", match=MatchValue(value="tenant_042"))]
)
start = time.perf_counter()
for _ in range(50):
    client.search(
        collection_name=LARGE_COLLECTION,
        query_vector=query_vector,
        query_filter=tenant_filter,
        limit=10,
        search_params=SearchParams(hnsw_ef=128), # explicit ef at query time
    )
filtered_ms = (time.perf_counter() - start) / 50 * 1000
print(f"Filtered search (1% selectivity) avg latency: {filtered_ms:.2f}ms")

```

```
# Benchmark 3: High-selectivity filter – 0.1% with range condition
narrow_filter = Filter(
    must=[
        FieldCondition(key="tenant_id", match=MatchValue(value="tenant_042")),
        FieldCondition(key="score", range=Range(gte=0.95)),
    ]
)
start = time.perf_counter()
for _ in range(50):
    client.search(
        collection_name=LARGE_COLLECTION,
        query_vector=query_vector,
        query_filter=narrow_filter,
        limit=10,
    )
narrow_ms = (time.perf_counter() - start) / 50 * 1000
print(f"High-selectivity filtered search avg latency: {narrow_ms:.2f}ms")
# Note: at very high selectivity, Qdrant switches to brute-force over
# the matching subset, which is often faster than graph traversal
```

Exact numbers depend heavily on hardware, dataset size, and dimensionality, so run this benchmark against your own deployment rather than trusting a printed figure — but the general shape holds: unfiltered and 1%-selectivity filtered search land in a similar low-millisecond range, and high-selectivity search (where brute-force kicks in) is often just as fast because the matching set is small. The filtered HNSW penalty is far smaller than the equivalent post-filter architecture.

17.4 Precision Tuning: ef, Quantization, and Segment Configuration

Qdrant exposes more precision knobs than most vector databases. Understanding them requires understanding the search accuracy vs. speed tradeoff in concrete terms.

The ef parameter:

In HNSW, `ef` (exploration factor) controls how many candidates the graph explorer keeps in its priority queue during traversal. Higher `ef` means more candidates explored, which increases recall at the cost of latency. Two separate `ef` values exist:

- → ``ef_construct``: Used during index build. Set this once at collection creation. Typical values: 100–400. Higher values produce better graph connectivity but slow down writes.
- → ``hnsw_ef`` (or ``ef`` at search time): Used during query. Can be set per-query via ``SearchParams``. Typical range: 64–512.

The `m` parameter controls the number of bidirectional edges each node maintains in the graph. Higher `m` increases RAM usage proportionally but improves recall, especially at lower `ef` values. For 100k–1M vectors, `m=16` is a reasonable default. For datasets requiring very high recall (>99%) at large scale, `m=32` or `m=64` is warranted.

Quantization:

Qdrant supports three quantization strategies that trade accuracy for speed and memory reduction:

Strategy	Memory Reduction	Recall Impact	Typical Use
Scalar (int8)	~4x	<1% at ef rescore	General purpose
Product Quantization (PQ)	8–32x	2–5% without rescore	Memory-constrained
Binary	~32x	Significant without rescore	Pre-filtering, speed priority

All three support **rescoring**: after the initial quantized search narrows the candidate set, Qdrant fetches the original full-precision vectors for the top candidates and re-ranks them. This recovers most of the accuracy penalty. The tradeoff is I/O for the rescore step.

```

# Python
from qdrant_client.models import (
    ScalarQuantizationConfig,
    ProductQuantizationConfig,
    BinaryQuantizationConfig,
    QuantizationConfig,
    ScalarType,
    CompressionRatio,
)

# -----
# Creating a collection with scalar quantization (recommended default)
# -----
client.create_collection(
    collection_name="quantized_products",
    vectors_config=VectorParams(
        size=1536,          # OpenAI text-embedding-3-small dimension
        distance=Distance.COSINE,
        hnsw_config={
            "m": 16,
            "ef_construct": 128,
            "full_scan_threshold": 10_000,
        },
    ),
    quantization_config=ScalarQuantizationConfig(
        type=ScalarType.INT8,
        quantile=0.99,      # clip outliers; 0.99 is conservative
        always_ram=True,    # keep quantized vectors in RAM, originals on disk
    ),
)

# -----
# Creating a collection with product quantization for memory-constrained
# environments
# 1536-dim OpenAI vectors compressed to ~192 bytes (8x reduction at m=256)
# -----
client.create_collection(
    collection_name="compressed_products",
    vectors_config=VectorParams(
        size=1536,
        distance=Distance.COSINE,
    ),
    quantization_config=ProductQuantizationConfig(
        compression=CompressionRatio.X16, # 16x compression
        always_ram=True,
    ),
)

# -----
# Per-query control over quantization behavior
# -----
from qdrant_client.models import QuantizationSearchParams

```

```

results = client.search(
    collection_name="quantized_products",
    query_vector=query_vector,
    limit=10,
    search_params=SearchParams(
        hnsw_ef=256,          # higher ef compensates for quantization noise
        quantization=QuantizationSearchParams(
            ignore=False,      # use quantization
            rescore=True,      # rescore top candidates with original vectors
            oversampling=2.5,  # fetch 2.5x candidates for rescore pool
        ),
    ),
)

# -----
# Optimizer and segment configuration
# These control how Qdrant manages internal storage segments
# -----
from qdrant_client.models import OptimizersConfigDiff

client.update_collection(
    collection_name="quantized_products",
    optimizer_config=OptimizersConfigDiff(
        # Flush segments to disk when they exceed this size (in KB)
        max_segment_size=200_000,      # 200MB per segment
        # How many points trigger a segment merge
        memmap_threshold=50_000,
        # Minimum number of vectors before HNSW index is built
        # (below this, brute force is used instead)
        indexing_threshold=20_000,
        # Number of concurrent optimizer threads
        max_optimization_threads=2,
        # Flush interval in seconds
        flush_interval_sec=5,
        # Vacuum deleted vectors (tombstone cleanup)
        deleted_threshold=0.2,
        vacuum_min_vector_number=1000,
    ),
)

```

Understanding segments:

Qdrant partitions each collection into **segments** — immutable or appendable units of storage that each contain a subset of points. New points land in a mutable "appendable" segment. When that segment grows beyond `memmap_threshold`, the optimizer merges it into a larger immutable segment backed by a memory-mapped file. The immutable segment then gets an HNSW index built on it (once it exceeds `indexing_threshold`).

This architecture means that very fresh data that hasn't been incorporated into a segment HNSW index yet is searched via brute force, while indexed segments use HNSW. If your collection receives a heavy write burst followed immediately by queries, you may see temporarily elevated latency or reduced recall until the optimizer catches up. Monitoring `indexed_vectors_count` vs. `points_count` from `get_collection()` gives you visibility into this lag.

17.5 Named Vectors and Multivector Collections

A practical scenario that arises frequently: you want to search the same items with multiple embeddings — perhaps a text embedding for semantic search and a separate image embedding for visual similarity. Qdrant handles this natively with **named vectors**.

```

# Python
from qdrant_client.models import NamedVector, NamedSparseVector, SparseVector

# Create a collection with multiple named vector spaces
client.create_collection(
    collection_name="multimodal_products",
    vectors_config={
        "text": VectorParams(
            size=384,
            distance=Distance.COSINE,
        ),
        "image": VectorParams(
            size=512,
            distance=Distance.COSINE,
        ),
        "price_embedding": VectorParams(
            size=32,
            distance=Distance.EUCLID,
        ),
    },
)

# Insert a point with all three vector representations
client.upsert(
    collection_name="multimodal_products",
    points=[
        PointStruct(
            id=1,
            vector={
                "text": [0.1, 0.2, 0.3] * 128, # 384-dim placeholder
                "image": [0.4, 0.5, 0.6] * (512 // 3) + [0.4] * (512 % 3),
                "price_embedding": [0.7] * 32,
            },
            payload={"name": "Wireless Headphones", "price": 149.99},
        )
    ],
    wait=True,
)

# Search using only one named vector
text_query = [0.15, 0.25, 0.35] * 128 # placeholder 384-dim query

results = client.search(
    collection_name="multimodal_products",
    query_vector=NamedVector(name="text", vector=text_query),
    limit=5,
    with_payload=True,
)

# Hybrid search: combine results from text and image vectors
# using Qdrant's query API (v1.7+)
from qdrant_client.models import (
    Prefetch,

```

```

        FusionQuery,
        Fusion,
    )

    image_query = [0.45, 0.55, 0.65] * (512 // 3) + [0.45] * (512 % 3)

    # Reciprocal Rank Fusion across two vector spaces
    hybrid_results = client.query_points(
        collection_name="multimodal_products",
        prefetch=[
            Prefetch(
                query=NamedVector(name="text", vector=text_query),
                limit=20,
            ),
            Prefetch(
                query=NamedVector(name="image", vector=image_query),
                limit=20,
            ),
        ],
        query=FusionQuery(fusion=Fusion.RRF),
        limit=10,
        with_payload=True,
    )

```

Named vectors let you avoid maintaining separate collections per modality, which simplifies payload management considerably — you only need one set of payload indexes rather than synchronized copies across multiple collections.

Sparse vectors for hybrid dense+sparse search:

Qdrant also supports sparse vectors natively, enabling BM25-style keyword search alongside dense semantic search within the same system.

```

# Python
from qdrant_client.models import SparseVectorParams

# Collection with both dense and sparse vector slots
client.create_collection(
    collection_name="hybrid_search_products",
    vectors_config={
        "dense": VectorParams(size=384, distance=Distance.COSINE),
    },
    sparse_vectors_config={
        "sparse": SparseVectorParams(),
    },
)

# Simulate BM25 sparse weights (in production, use a sparse encoder like SPLADE)
def mock_sparse_vector(text: str) -> SparseVector:
    """Toy TF-weighted sparse representation."""
    tokens = text.lower().split()
    token_ids = [hash(t) % 30_000 for t in tokens]
    weights = [1.0 / (tokens.count(t) ** 0.5) for t in tokens]
    # deduplicate by index
    combined = {}
    for idx, w in zip(token_ids, weights):
        combined[idx] = combined.get(idx, 0.0) + w
    indices = list(combined.keys())
    values = list(combined.values())
    return SparseVector(indices=indices, values=values)

# Upsert a point with both dense and sparse vectors
import numpy as np

client.upsert(
    collection_name="hybrid_search_products",
    points=[
        PointStruct(
            id=1,
            vector={
                "dense": np.random.rand(384).tolist(),
                "sparse": mock_sparse_vector("wireless noise canceling headphones"),
            },
            payload={"name": "Wireless Headphones"},
        )
    ],
    wait=True,
)

# Hybrid query: fuse dense semantic search with sparse keyword search
dense_query = np.random.rand(384).tolist()
sparse_query = mock_sparse_vector("wireless headphones long battery")

hybrid_results = client.query_points(
    collection_name="hybrid_search_products",
    prefetch=[

```

```

    Prefetch(
        query=NamedVector(name="dense", vector=dense_query),
        limit=20,
    ),
    Prefetch(
        query=NamedSparseVector(name="sparse", vector=sparse_query),
        limit=20,
    ),
],
query=FusionQuery(fusion=Fusion.RRF),
limit=10,
with_payload=True,
)

```

17.6 Scroll, Recommendation, and Discovery APIs

Qdrant exposes several query modes beyond the standard `search()` endpoint. Each addresses a different retrieval pattern.

Scroll — paginated full collection scan with filters:

When you need to iterate over all matching points (not just the nearest neighbors), `scroll()` provides cursor-based pagination. This is useful for bulk exports, backfill operations, and building recommendation candidate sets from structured criteria alone.

```

# Python
from qdrant_client.models import ScrollRequest

# Retrieve all in-stock electronics, paginated
offset = None # start from the beginning
all_electronics = []

while True:
    results, next_offset = client.scroll(
        collection_name=COLLECTION_NAME,
        scroll_filter=Filter(
            must=[
                FieldCondition(key="category",
match=MatchValue(value="electronics")),
                FieldCondition(key="in_stock", match=MatchValue(value=True)),
            ]
        ),
        limit=100,
        offset=offset,
        with_payload=True,
        with_vectors=False,
    )
    all_electronics.extend(results)
    if next_offset is None:
        break
    offset = next_offset

print(f"Total matching points: {len(all_electronics)}")

```

Recommendation API — finding similar items without a query vector:

The recommendation endpoint accepts one or more positive example point IDs and optional negative example IDs. Qdrant internally fetches those points' vectors, computes an average positive direction, and performs the search. This is the correct abstraction for "more like this" UI patterns without requiring the client to store or re-encode vectors.

```
# Python
from qdrant_client.models import RecommendRequest

# "More like this product, but not like that one"
recommendations = client.recommend(
    collection_name=COLLECTION_NAME,
    positive=["product-id-1", "product-id-2"], # IDs of liked items
    negative=["product-id-3"],                # IDs of disliked items
    query_filter=Filter(
        must=[FieldCondition(key="in_stock", match=MatchValue(value=True))]
    ),
    limit=5,
    with_payload=True,
)

for rec in recommendations:
    print(f"Score: {rec.score:.4f} | {rec.payload['description']}")
```

Discovery API — constrained exploration:

The discovery endpoint extends the recommendation concept with **context pairs** — pairs of (positive, negative) examples that define a preference direction without requiring query vectors. It is useful when you want to explore a region of the vector space defined by relative preferences rather than absolute query proximity.

```
# Python
from qdrant_client.models import DiscoverRequest, ContextExamplePair

discovery_results = client.discover(
    collection_name=COLLECTION_NAME,
    target="product-id-1", # item to find neighbors of
    context=[
        ContextExamplePair(
            positive="product-id-4",
            negative="product-id-3",
        )
    ],
    query_filter=Filter(
        must=[FieldCondition(key="category", match=MatchValue(value="electronics"))]
    ),
    limit=5,
    with_payload=True,
)
```

17.7 Production Deployment Patterns

Running Qdrant in production involves decisions about persistence, sharding, replication, and access control. This section covers the patterns that matter most for teams moving from prototype to production.

Persistence and snapshots:

Qdrant stores its data in a configurable `storage_path` directory. The default on-disk format uses memory-mapped files for large segments, which means the OS page cache provides an implicit read buffer. For cloud deployments, map this path to a persistent volume — losing the storage directory means losing all data.

Qdrant provides a snapshot API for point-in-time backups:

```
# Python
# Create a snapshot of a collection
snapshot_info = client.create_snapshot(collection_name=COLLECTION_NAME)
print(f"Snapshot created: {snapshot_info.name}")

# List available snapshots
snapshots = client.list_snapshots(collection_name=COLLECTION_NAME)
for s in snapshots:
    print(f"  {s.name} - {s.size} bytes - created at {s.creation_time}")

# Restore from snapshot (via REST, since Python SDK uses the same endpoint)
# POST /collections/{collection_name}/snapshots/recover
# { "location": "file:///qdrant/snapshots/collection-name/snapshot-name.snapshot" }
```

Distributed mode — shards and replication:

Qdrant's distributed mode partitions a collection into **shards**, each of which is an independent HNSW index. Shards can be replicated across nodes for fault tolerance. The configuration happens at collection creation:

```
# Python
# Create a distributed collection – requires Qdrant cluster mode
client.create_collection(
    collection_name="distributed_catalog",
    vectors_config=VectorParams(size=384, distance=Distance.COSINE),
    shard_number=4,           # 4 shards distributed across nodes
    replication_factor=2,     # each shard replicated to 2 nodes
    write_consistency_factor=1, # 1 replica must confirm writes
)
```


Queries in distributed mode are automatically broadcast to all shards, and results are merged. The write consistency factor controls the durability vs. latency tradeoff: requiring all replicas to confirm (`write_consistency_factor=replication_factor`) maximizes durability but adds write latency proportional to replica count.

API key authentication and TLS:

Qdrant supports simple API key authentication and TLS termination, configured via the `config.yaml` file rather than the client:

```
# qdrant config.yaml (server-side)
service:
  api_key: "your-secret-api-key"
  enable_tls: true
  tls:
    cert: /path/to/server.crt
    key: /path/to/server.key

storage:
  storage_path: /qdrant/storage
  snapshots_path: /qdrant/snapshots

# Distributed mode
cluster:
  enabled: true
  p2p:
    port: 6335
  consensus:
    tick_period_ms: 100
```

On the client side, pass the API key at construction:

```
# Python
client = QdrantClient(
    host="your-qdrant-host",
    port=6333,
    api_key="your-secret-api-key",
    https=True,          # enables TLS verification
    prefix="optional/prefix", # useful when behind a path-routing reverse proxy
)
```

Monitoring collection health:

```
# Python
def collection_health_report(client: QdrantClient, collection_name: str) -> dict:
    """
    Returns a structured health summary useful for alerting dashboards.
    """
    info = client.get_collection(collection_name)
    config = info.config

    total_points = info.points_count
    indexed_vectors = info.indexed_vectors_count
    index_lag = total_points - indexed_vectors

    segments = client.get_collection(collection_name).segments_count

    return {
        "collection": collection_name,
        "status": info.status.value,
        "total_points": total_points,
        "indexed_vectors": indexed_vectors,
        "index_lag": index_lag,
        "index_lag_pct": round(index_lag / max(total_points, 1) * 100, 2),
        "segments": segments,
        "vector_size": config.params.vectors.size,
        "distance": config.params.vectors.distance.value,
        "m": config.hnsw_config.m,
        "ef_construct": config.hnsw_config.ef_construct,
    }

report = collection_health_report(client, COLLECTION_NAME)
for k, v in report.items():
    print(f" {k:<22} {v}")
```

Write-ahead log and crash recovery:

Qdrant uses a write-ahead log (WAL) to ensure that accepted writes survive crashes even before segments are flushed. The WAL is replayed on startup. This means your data is safe after `wait=True` upserts, but a large WAL that hasn't been checkpointed will slow down restart time. The `flush_interval_sec` optimizer setting controls how frequently WAL entries are checkpointed to segments.

17.8 Practical Integration Pattern: Tenant-Isolated Search

A common production pattern when building multi-tenant SaaS applications on Qdrant is to use a single collection with a `tenant_id` payload field and a keyword index, rather than a separate collection per tenant. This avoids the overhead of managing thousands of collections while still delivering tenant-isolated results.

```

# Python
import hashlib
from qdrant_client.models import (
    Filter, FieldCondition, MatchValue,
    PointStruct, VectorParams, Distance,
)
from sentence_transformers import SentenceTransformer
from qdrant_client import QdrantClient

client = QdrantClient(host="localhost", port=6333)
encoder = SentenceTransformer("all-MiniLM-L6-v2")

COLLECTION = "saas_documents"

# One-time setup
client.recreate_collection(
    collection_name=COLLECTION,
    vectors_config=VectorParams(size=384, distance=Distance.COSINE),
)
client.create_payload_index(
    collection_name=COLLECTION,
    field_name="tenant_id",
    field_schema="keyword",
)
client.create_payload_index(
    collection_name=COLLECTION,
    field_name="doc_type",
    field_schema="keyword",
)
client.create_payload_index(
    collection_name=COLLECTION,
    field_name="created_at",
    field_schema="integer", # Unix timestamp – integer index supports range queries
)

def deterministic_point_id(tenant_id: str, doc_id: str) -> int:
    """
    Generate a stable integer point ID from tenant + document ID.
    Qdrant integer IDs must be uint64, so we truncate the hash.
    """
    h = hashlib.sha256(f"{tenant_id}:{doc_id}".encode()).digest()
    return int.from_bytes(h[:8], "big") >> 1 # shift to keep within signed range

def upsert_document(
    tenant_id: str,
    doc_id: str,
    text: str,
    doc_type: str,
    created_at: int,
) -> None:
    vector = encoder.encode(text).tolist()
    point_id = deterministic_point_id(tenant_id, doc_id)
    client.upsert(

```

```

        collection_name=COLLECTION,
        points=[
            PointStruct(
                id=point_id,
                vector=vector,
                payload={
                    "tenant_id": tenant_id,
                    "doc_id": doc_id,
                    "doc_type": doc_type,
                    "created_at": created_at,
                    "text": text,
                },
            )
        ],
        wait=True,
    )

def tenant_search(
    tenant_id: str,
    query: str,
    doc_type: str | None = None,
    min_created_at: int | None = None,
    limit: int = 10,
) -> list[dict]:
    """
    Vector search scoped strictly to a single tenant,
    with optional document type and recency filters.
    """
    must_conditions = [
        FieldCondition(key="tenant_id", match=MatchValue(value=tenant_id))
    ]
    if doc_type is not None:
        must_conditions.append(
            FieldCondition(key="doc_type", match=MatchValue(value=doc_type))
        )
    if min_created_at is not None:
        must_conditions.append(
            FieldCondition(key="created_at", range=Range(gte=min_created_at))
        )

    query_vector = encoder.encode(query).tolist()

    results = client.search(
        collection_name=COLLECTION,
        query_vector=query_vector,
        query_filter=Filter(must=must_conditions),
        limit=limit,
        with_payload=["doc_id", "doc_type", "text", "created_at"],
    )

    return [
        {
            "score": r.score,

```

```

        "doc_id":      r.payload["doc_id"],
        "doc_type":   r.payload["doc_type"],
        "text":       r.payload["text"],
        "created_at": r.payload["created_at"],
    }
    for r in results
]

# Example usage
import time

upsert_document(
    tenant_id="acme_corp",
    doc_id="kb-001",
    text="How to reset your password using the account settings page",
    doc_type="knowledge_base",
    created_at=int(time.time()),
)

upsert_document(
    tenant_id="acme_corp",
    doc_id="ticket-0042",
    text="User reports login failure after password reset attempt",
    doc_type="support_ticket",
    created_at=int(time.time()),
)

results = tenant_search(
    tenant_id="acme_corp",
    query="login problems after changing password",
    limit=5,
)

for r in results:
    print(f"[{r['score']:.4f}] ({r['doc_type']}) {r['text'][:80]}")

```

This pattern has one significant consideration: if a single tenant accounts for a very large fraction of all points (say, one tenant holds 80% of the data), that tenant's searches will traverse more nodes during HNSW graph construction and querying than a balanced distribution would imply. For severely skewed tenants, the dedicated-collection approach for that specific tenant is worth the operational complexity.

Key Takeaways

- → **Qdrant's filtered HNSW** integrates payload conditions directly into graph traversal rather than applying them as a post-filter step. This keeps filtered search latency close to unfiltered latency even at selectivities of 1% or lower, and automatically falls back to brute-force over the matching subset when the result set is very small.
- → **Payload indexes must be created explicitly** before those fields become efficient filter predicates. The field type you choose — keyword, integer, float, geo, or text — determines which comparison operators are available and how the index is physically structured. Creating indexes before bulk insert is faster than creating them afterward.
- → **Quantization (scalar, product, or binary) combined with rescoring** is the primary lever for reducing memory footprint without sacrificing meaningful recall. The ``always_ram=True`` option for quantized vectors keeps the compressed representation in memory for fast candidate retrieval while the full-precision originals can live on disk and are only read for the rescore step.
- → **Named vectors and sparse vector support** allow a single collection to serve multiple retrieval modalities — dense semantic search, sparse keyword search, and cross-modal fusion — without synchronizing parallel collections. The ``prefetch`` + ``FusionQuery(RRF)`` pattern is the idiomatic way to implement hybrid search in Qdrant v1.7+.
- → **Multi-tenant isolation via payload filtering** (single collection, ``tenant_id`` keyword index) is operationally simpler than per-tenant collections at moderate tenant counts. The pattern breaks down when tenant data is severely skewed, at which point dedicated collections for the largest tenants become preferable.

Chapter 18: Chroma: Local-First Vector DB for Development

Chroma occupies a unique position in the vector database landscape: it is the tool developers reach for first, the one that runs in a Python process with zero infrastructure, and the one that disappears from the conversation the moment a team decides to move to production. That reputation is both earned and slightly unfair. Chroma is a genuinely capable vector store that handles millions of embeddings, supports persistent disk-backed storage, and can run as a standalone server with a client/server split that mirrors how teams actually work. What it lacks is the operational maturity of Pinecone or Weaviate — but for local development, rapid prototyping, and small-to-medium deployments where you control the machine, it is often the most productive choice. This chapter covers Chroma's architecture, its embedding and collection model, persistent versus in-memory operation, querying patterns that transfer directly to production systems, and the specific scenarios where Chroma remains the right answer even after your application ships.

18.1 Chroma's Architecture: An Embedded Database Perspective

Database professionals will recognize Chroma's design immediately once you map it to familiar concepts. Chroma is, at its core, an embedded vector database in the same lineage as SQLite — it runs inside your process, writes to a local directory, and requires no separate daemon. This is its defining characteristic and the source of both its appeal and its limitations.

The storage model

Chroma uses a layered storage architecture. The vector index is backed by HNSW (Hierarchical Navigable Small World), the same algorithm used internally by Weaviate, Qdrant, and Milvus. Metadata and document text are stored in DuckDB in versions prior to 0.4, and in Chroma's own embedded SQLite-based system in the 0.4+ architecture. This separation — vectors in an HNSW structure, metadata in a columnar or relational store — mirrors what you see in every production vector database. Chroma just makes the layers visible in a single local directory.

The `chroma` directory after initialization looks like this:

```
# shell
ls -la ./chroma_data/
# chroma.sqlite3      <- metadata, collections, embeddings catalog
# <uuid>/            <- one directory per collection
# data_level0.bin     <- HNSW graph (bottom layer, all nodes)
# header.bin          <- index parameters
# length.bin          <- node count
# link_lists.bin      <- HNSW upper-layer links
```

If you have managed SQL Server databases, think of `chroma.sqlite3` as the system catalog and the HNSW binary files as a specialized index tablespace. The collection UUID directory is analogous to a table's data files — the collection is the primary organizing unit in Chroma, equivalent to a table with a mandatory vector column and optional metadata columns.

Collections as tables

A Chroma collection holds:

- → **IDs** — string primary keys (you supply these, no auto-increment)
- → **embeddings** — float arrays of fixed dimension (dimensionality is inferred from the first insert)
- → **documents** — optional raw text (the source that was embedded)
- → **metadatas** — optional key/value dictionaries (analogous to non-indexed columns)

The dimension constraint works like a typed column: once you insert a 1536-dimensional embedding, every subsequent insert must match. Mixing models is a schema violation Chroma enforces at insert time.

Client modes

Chroma supports three operational modes that map to a deployment progression:

Mode	Use Case	Analogy
Ephemeral (in-memory)	Unit tests, notebooks	:memory: SQLite
Persistent (embedded)	Local dev, single-process apps	File-based SQLite
Client/Server (HTTP)	Multi-process, shared dev environment	Separate DB server

Most tutorials show only the ephemeral mode. Production-adjacent usage requires understanding the persistent and client/server modes, which this chapter covers in detail.

18.2 Getting Started: Collections, Embeddings, and Basic Queries

Install Chroma with its default dependencies, which include the HNSW library and sentence-transformers for local embedding generation:

```
# shell
pip install chromadb sentence-transformers openai
# For the server mode:
pip install "chromadb[server]"
```

Ephemeral client (testing and notebooks)

```

# Python
import chromadb

# In-memory client – resets on process exit
client = chromadb.EphemeralClient()

# Create a collection with cosine similarity (default is L2)
collection = client.create_collection(
    name="product_docs",
    metadata={"hnsw:space": "cosine"}
)

# Add documents – Chroma uses its default embedding function if you don't specify
one
# Default is all-MiniLM-L6-v2 via sentence-transformers (384 dimensions)
collection.add(
    documents=[
        "SQL Server supports JSON in NVARCHAR columns for semi-structured data",
        "SQL Server 2025 adds native vector similarity search",
        "Chroma is an open-source embedding database",
        "HNSW indexing provides approximate nearest neighbor search",
        "RAG pipelines combine retrieval with language model generation",
    ],
    ids=["doc_001", "doc_002", "doc_003", "doc_004", "doc_005"],
    metadatas=[
        {"source": "db_docs", "category": "database"},
        {"source": "db_docs", "category": "vector"},
        {"source": "chroma_docs", "category": "vector"},
        {"source": "research", "category": "algorithms"},
        {"source": "ml_docs", "category": "architecture"},
    ]
)

# Query – Chroma embeds the query text automatically using the same function
results = collection.query(
    query_texts=["how does vector similarity search work?"],
    n_results=3,
    include=["documents", "metadatas", "distances"]
)

for i, (doc, meta, dist) in enumerate(zip(
    results["documents"][0],
    results["metadatas"][0],
    results["distances"][0]
)):
    print(f"Rank {i+1} | Distance: {dist:.4f} | Source: {meta['source']}")
    print(f"  {doc}\n")

```

The `include` parameter controls what Chroma returns — omit `"embeddings"` unless you need the raw vectors, since returning full embedding arrays for large result sets is expensive.

Persistent client with metadata filtering

```

# Python
import chromadb
from chromadb.utils import embedding_functions

# Persistent client – survives process restart
client = chromadb.PersistentClient(path="./chroma_data")

# Use OpenAI embeddings instead of the default local model
openai_ef = embedding_functions.OpenAIEmbeddingFunction(
    api_key="sk-...",          # or read from env: os.environ["OPENAI_API_KEY"]
    model_name="text-embedding-3-small" # 1536 dimensions
)

# get_or_create is idempotent – safe to call on restart
collection = client.get_or_create_collection(
    name="engineering_kb",
    embedding_function=openai_ef,
    metadata={"hnsw:space": "cosine"}
)

# Batch insert – Chroma handles the embedding calls internally
documents = [
    "Connection pooling reduces database overhead by reusing connections",
    "Read replicas distribute query load across multiple database instances",
    "Partitioning splits large tables into smaller, manageable segments",
    "Index rebuild reclaims space from fragmented indexes in SQL Server",
    "The transaction log ensures durability and enables replication",
]
ids = [f"kb_{i:04d}" for i in range(len(documents))]
metadatas = [
    {"topic": "performance", "db": "general"},
    {"topic": "scaling", "db": "general"},
    {"topic": "storage", "db": "general"},
    {"topic": "maintenance", "db": "sqlserver"},
    {"topic": "reliability", "db": "sqlserver"},
]

collection.add(documents=documents, ids=ids, metadatas=metadatas)

# Query with metadata filter – equivalent to WHERE clause
results = collection.query(
    query_texts=["how to improve query performance?"],
    n_results=3,
    where={"db": "sqlserver"},          # metadata filter
    include=["documents", "metadatas", "distances"]
)

print(f"Found {len(results['ids'][0])} results filtered to SQL Server topics:")
for doc, meta, dist in zip(
    results["documents"][0],
    results["metadatas"][0],
    results["distances"][0]

```

```
):  
    print(f" [{meta['topic']}] dist={dist:.4f}: {doc[:60]}...")
```

The `where` clause supports operators that map directly to SQL predicates:

```
# Python  
# Single condition  
where={"category": "vector"}  
  
# AND condition  
where={"$and": [{"category": "vector"}, {"source": "db_docs"}]}  
  
# OR condition  
where={"$or": [{"db": "sqlserver"}, {"db": "general"}]}  
  
# Comparison operators  
where={"score": {"$gte": 0.8}}          # score >= 0.8  
where={"year": {"$in": [2023, 2024]}}   # year IN (2023, 2024)  
where={"status": {"$ne": "archived"}}   # status != 'archived'
```

For DBA readers: these operators compile down to SQLite predicates against the metadata store before the HNSW search executes. Pre-filtering on metadata is more efficient than post-filtering large result sets, particularly when the metadata column has high cardinality.

18.3 Embedding Functions: Wiring Chroma to Your Model Layer

Chroma's embedding function abstraction is one of its more thoughtful design decisions. Rather than requiring you to compute embeddings externally and pass float arrays, Chroma lets you attach an embedding function to a collection and then work exclusively with text. The function is called transparently during `add()` and `query()`.

Built-in embedding functions

```
# Python
from chromadb.utils import embedding_functions

# 1. Default: all-MiniLM-L6-v2 (384-dim, runs locally, no API key needed)
default_ef = embedding_functions.DefaultEmbeddingFunction()

# 2. OpenAI – text-embedding-3-small (1536-dim) or text-embedding-3-large (3072-dim)
openai_ef = embedding_functions.OpenAIEmbeddingFunction(
    api_key="sk-...",
    model_name="text-embedding-3-small"
)

# 3. Cohere – good multilingual support
cohere_ef = embedding_functions.CohereEmbeddingFunction(
    api_key="...",
    model_name="embed-english-v3.0"
)

# 4. Sentence Transformers – local, no API costs, many model choices
st_ef = embedding_functions.SentenceTransformerEmbeddingFunction(
    model_name="BAAI/bge-large-en-v1.5" # 1024-dim, strong retrieval performance
)

# 5. HuggingFace Inference API – cloud-hosted open models
hf_ef = embedding_functions.HuggingFaceEmbeddingFunction(
    api_key="hf_...",
    model_name="sentence-transformers/all-mpnet-base-v2"
)
```

Custom embedding function for Azure OpenAI or AWS Bedrock

When your organization's models are hosted on enterprise AI platforms, you write a custom embedding function by subclassing `EmbeddingFunction`:

```

# Python
import numpy as np
from chromadb import EmbeddingFunction, Documents, Embeddings
import boto3
import json

class BedrockEmbeddingFunction(EmbeddingFunction):
    """Chroma embedding function backed by Amazon Bedrock Titan Embeddings."""

    def __init__(self, region_name: str = "us-east-1"):
        self.client = boto3.client("bedrock-runtime", region_name=region_name)
        self.model_id = "amazon.titan-embed-text-v1"

    def __call__(self, input: Documents) -> Embeddings:
        embeddings = []
        for text in input:
            body = json.dumps({"inputText": text})
            response = self.client.invoke_model(
                modelId=self.model_id,
                body=body,
                contentType="application/json",
                accept="application/json"
            )
            result = json.loads(response["body"].read())
            embeddings.append(result["embedding"])
        return embeddings

class AzureOpenAIEmbeddingFunction(EmbeddingFunction):
    """Chroma embedding function backed by Azure OpenAI."""

    def __init__(
        self,
        azure_endpoint: str,
        api_key: str,
        deployment_name: str,
        api_version: str = "2024-02-01"
    ):
        from openai import AzureOpenAI
        self.client = AzureOpenAI(
            azure_endpoint=azure_endpoint,
            api_key=api_key,
            api_version=api_version
        )
        self.deployment_name = deployment_name

    def __call__(self, input: Documents) -> Embeddings:
        # Azure OpenAI supports batch embedding – send all texts at once
        response = self.client.embeddings.create(
            input=input,
            model=self.deployment_name
        )
        # Sort by index to preserve order

```

```

        sorted_data = sorted(response.data, key=lambda x: x.index)
        return [item.embedding for item in sorted_data]

# Usage
bedrock_ef = BedrockEmbeddingFunction(region_name="us-west-2")
azure_ef = AzureOpenAIEmbeddingFunction(
    azure_endpoint="https://your-resource.openai.azure.com",
    api_key="...",
    deployment_name="text-embedding-3-small"
)

client = chromadb.PersistentClient(path="./chroma_data")
collection = client.get_or_create_collection(
    name="enterprise_docs",
    embedding_function=azure_ef,
    metadata={"hnsw:space": "cosine"}
)

```

A dimension mismatch is a hard schema error. If you create a collection with `text-embedding-3-small` (1536 dimensions) and later try to query it with a custom function that returns 768-dimensional vectors, Chroma raises a `ValueError` at query time. There is no silent dimension reduction. Treat the embedding function attached to a collection as a schema constraint — document it alongside your collection metadata.

18.4 Client/Server Mode and LangChain Integration

Running Chroma as a server unlocks multi-process access, which is the minimum viable architecture for any application with a web server, background workers, and a database — they cannot all share an embedded client safely. The server exposes a FastAPI HTTP interface, and the Python client speaks to it over HTTP.

Starting the Chroma server


```
# shell
# Install with server dependencies
pip install "chromadb[server]"

# Start server on default port 8000
chroma run --path ./chroma_data --host 0.0.0.0 --port 8000

# Or with Docker – useful for CI and team shared environments
docker run -d \
  --name chroma \
  -p 8000:8000 \
  -v $(pwd)/chroma_data:/chroma/chroma \
  -e IS_PERSISTENT=TRUE \
  chromadb/chroma:latest

# Verify the server is running
curl http://localhost:8000/api/v1/heartbeat
# {"nanosecond heartbeat": 1234567890}

curl http://localhost:8000/api/v1/collections
# Returns list of existing collections as JSON
```

Connecting from Python (client/server mode)

```

# Python
import chromadb

# HTTP client – connects to running Chroma server
client = chromadb.HttpClient(
    host="localhost",
    port=8000,
    # Optional: add auth headers if you've configured Chroma with token auth
    # settings=chromadb.Settings(chroma_client_auth_provider="token",
    #                             chroma_client_auth_credentials="secret-token")
)

# Everything else is identical to the embedded client API
collection = client.get_or_create_collection(
    name="shared_collection",
    metadata={"hnsw:space": "cosine"}
)

# Test round-trip
collection.add(
    documents=["Server mode enables multi-process access to Chroma"],
    ids=["server_test_001"]
)

count = collection.count()
print(f"Collection contains {count} documents")

results = collection.query(
    query_texts=["multi-process database access"],
    n_results=1
)
print(results["documents"][0])

```

LangChain integration

LangChain treats Chroma as a `VectorStore`, which means you can swap it for Pinecone or Weaviate by changing two lines. This is exactly the abstraction you want during development — build against Chroma, promote to a production vector database without rewriting retrieval logic.

```

# Python
from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings
from langchain_core.documents import Document
import chromadb

# Option 1: Let LangChain manage the Chroma client (simple case)
embedding_function = OpenAIEmbeddings(model="text-embedding-3-small")

vectorstore = Chroma(
    collection_name="langchain_demo",
    embedding_function=embedding_function,
    persist_directory="./chroma_langchain_data"
)

# Add LangChain Document objects (they carry page_content and metadata)
docs = [
    Document(
        page_content="Indexes in SQL Server speed up query execution by reducing full table scans",
        metadata={"source": "mssql_manual", "chapter": "11", "topic": "indexing"}
    ),
    Document(
        page_content="B-tree indexes are the default index type in SQL Server",
        metadata={"source": "mssql_manual", "chapter": "11", "topic": "indexing"}
    ),
    Document(
        page_content="Full-text indexes are optimal for CONTAINS and FREETEXT queries on text columns",
        metadata={"source": "mssql_manual", "chapter": "11", "topic": "indexing"}
    ),
    Document(
        page_content="Filtered indexes include only rows matching a WHERE condition",
        metadata={"source": "mssql_manual", "chapter": "11", "topic": "indexing"}
    ),
]

vectorstore.add_documents(docs)

# Similarity search – returns Documents ranked by relevance
query = "what index type should I use for text search?"
similar_docs = vectorstore.similarity_search(query, k=2)
for doc in similar_docs:
    print(f"[{doc.metadata['topic']}] {doc.page_content}")

# Similarity search with scores
docs_with_scores = vectorstore.similarity_search_with_score(query, k=2)
for doc, score in docs_with_scores:
    print(f"Score: {score:.4f} | {doc.page_content[:60]}...")

# Option 2: Connect LangChain to a running Chroma server
http_client = chromadb.HttpClient(host="localhost", port=8000)

```

```

server_vectorstore = Chroma(
    client=http_client,
    collection_name="langchain_server_collection",
    embedding_function=embedding_function
)

# Use as a retriever in a RAG chain
retriever = server_vectorstore.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 4, "filter": {"topic": "indexing"}}
)

# The retriever works with any LangChain chain
retrieved = retriever.invoke("how do partial indexes work?")
for doc in retrieved:
    print(doc.page_content)

```

The `as_retriever()` call is the bridge to the full LangChain ecosystem — you can pass this retriever directly to `RetrievalQA`, `ConversationalRetrievalChain`, or `LangGraph` nodes without modification.

18.5 HNSW Tuning, Scaling Limits, and When to Graduate

Chroma exposes HNSW parameters through collection metadata. Understanding these parameters gives you control over the precision/performance tradeoff that every vector index presents — the same tradeoff exists in SQL Server's DiskANN configuration and Weaviate's HNSW configuration.

HNSW parameters exposed by Chroma

```
# Python
import chromadb

client = chromadb.PersistentClient(path="./chroma_data")

# Create collection with explicit HNSW tuning
collection = client.get_or_create_collection(
    name="tuned_collection",
    metadata={
        "hsw:space": "cosine",          # Distance metric: cosine, l2, ip (inner
product)
        "hsw:construction_ef": 200,    # Candidates examined during index build
(default: 100)
        "hsw:search_ef": 100,          # Candidates examined during search (default:
10)
        "hsw:M": 32,                   # Connections per node in graph (default: 16)
        "hsw:num_threads": 4,          # Parallel threads for index operations
        "hsw:resize_factor": 1.2,      # Growth factor when index needs to expand
        "hsw:batch_size": 100,         # Vectors per batch during construction
        "hsw:sync_threshold": 1000,    # Flush to disk after N mutations
    }
)
```

Parameter guidance from database operations experience:

Parameter	Default	When to Increase	Memory Impact
<code>hsw:M</code>	16	Recall < 0.90, dataset > 100K	~2x at M=32
<code>hsw:construction_ef</code>	100	Recall < 0.95 at query time	Index build time only
<code>hsw:search_ef</code>	10	Recall < 0.90 in production	Per-query CPU
<code>hsw:num_threads</code>	1	Batch ingestion too slow	None (CPU only)

The key insight: `hsw:construction_ef` affects index quality permanently. `hsw:search_ef` can be tuned per-query without rebuilding the index. If you index with `construction_ef=100` and find recall is poor, you must rebuild — there is no online reindex in Chroma.

Measuring recall and query performance

```

# Python
import chromadb
import time
import numpy as np
from chromadb.utils import embedding_functions

def benchmark_chroma_collection(
    collection,
    query_texts: list[str],
    n_results: int = 10
) -> dict:
    """
    Basic benchmark for Chroma query latency and result consistency.
    In production you'd compare against brute-force results for recall measurement.
    """
    latencies = []

    for query in query_texts:
        start = time.perf_counter()
        results = collection.query(
            query_texts=[query],
            n_results=n_results,
            include=["distances"]
        )
        elapsed_ms = (time.perf_counter() - start) * 1000
        latencies.append(elapsed_ms)

    latency_array = np.array(latencies)
    return {
        "n_queries": len(query_texts),
        "p50_ms": np.percentile(latency_array, 50),
        "p95_ms": np.percentile(latency_array, 95),
        "p99_ms": np.percentile(latency_array, 99),
        "mean_ms": latency_array.mean(),
        "collection_size": collection.count(),
    }

# Example usage
client = chromadb.PersistentClient(path="./chroma_data")
col = client.get_collection("tuned_collection")

test_queries = [
    "vector similarity search algorithms",
    "SQL Server index optimization",
    "RAG pipeline architecture",
    "distributed database scaling",
    "embedding model selection",
]

stats = benchmark_chroma_collection(col, test_queries)
print(f"Collection size: {stats['collection_size']:,}")
print(f"P50 latency: {stats['p50_ms']:.1f} ms")

```

```
print(f"P95 latency: {stats['p95_ms']:.1f} ms")
print(f"P99 latency: {stats['p99_ms']:.1f} ms")
```

Scaling limits and the graduation decision

Chroma does not publish hard row limits, but practical experience with HNSW implementations establishes soft boundaries that matter for planning:

- → ****Under 500K vectors:**** Chroma is indistinguishable from production vector databases in most workloads. Latency stays in single-digit milliseconds on commodity hardware with tuned `hnsw:search_ef``.
- → ****500K to 2M vectors:**** Performance degrades predictably — HNSW memory footprint grows, and single-node Chroma cannot distribute the index across machines. Query latency at P99 becomes notable.
- → ****Above 2M vectors:**** Chroma is a wrong-size tool. The HNSW index lives entirely in RAM; a 2M vector collection at 1536 dimensions requires roughly 12 GB of RAM for the index alone ($\text{float32} \times 1536 \times 2\text{M} \approx 12\text{ GB}$), before the graph structure overhead.

The other signals that indicate it is time to graduate from Chroma:

1. **Multiple services need concurrent read/write access** — Chroma's server mode works, but it has no connection pooling, no query planner, and no replication.
2. **You need filtered search at scale** — Chroma's pre-filtering approach works well when the metadata filter is highly selective; it degrades when filters are loose and the HNSW graph still needs to scan large subgraphs.
3. **SLA requirements exist** — Chroma has no built-in monitoring, no replication lag metrics, and no managed backup. Operationalizing it requires wrapping it with infrastructure you build yourself.
4. **You need multi-tenancy** — Chroma's collection-per-tenant model works at small tenant counts; at hundreds of tenants, the collection overhead becomes unwieldy.
5. **Your team needs audit logs or access control** — Chroma has no row-level security, no audit trail, and only token-level authentication in server mode.

None of these limitations are criticisms of Chroma's design philosophy. It is an embedded database, and embedded databases are not wrong tools — they are differently scoped tools.

18.6 Real RAG Pipeline with Chroma: End-to-End Example

This section assembles the concepts from earlier in the chapter into a complete, runnable RAG pipeline. The pattern here — document loading, chunking, embedding, storage, retrieval, generation — is identical regardless of which vector database backs the retrieval step.

```
graph LR
  subgraph "Ingestion"
    A[Load documents] --> B[Chunk text]
    B --> C[Embed chunks]
    C --> D[(Chroma collection)]
  end
  end
  subgraph "Query time"
    E[User question] --> F[Embed question]
    F --> G[Query Chroma]
    D -. -> G
    G --> H[Retrieved chunks]
    H --> I[Assemble prompt]
    I --> J[LLM generation]
    J --> K[Answer + sources]
  end
  end
```

Ingestion (top) populates the Chroma collection once per document set; query time (bottom) embeds the question, retrieves the nearest chunks, and passes them to the LLM as grounding context.


```

# Python
import chromadb
from chromadb.utils import embedding_functions
from openai import OpenAI
import hashlib
import textwrap
from typing import Optional

# — Configuration —————
CHROMA_PATH = "./rag_chroma_data"
COLLECTION_NAME = "technical_kb"
EMBEDDING_MODEL = "text-embedding-3-small"
GENERATION_MODEL = "gpt-4o-mini"
CHUNK_SIZE = 400          # characters per chunk
CHUNK_OVERLAP = 80        # overlap between consecutive chunks
TOP_K = 4                  # number of chunks to retrieve

# — Chunking —————
def chunk_text(text: str, chunk_size: int, overlap: int) -> list[str]:
    """
    Naive fixed-size chunking with overlap.
    In production, prefer semantic chunking or recursive character splitting.
    """
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunks.append(text[start:end].strip())
        start += chunk_size - overlap
    return [c for c in chunks if len(c) > 50] # drop tiny tail chunks

def doc_id(source: str, chunk_index: int) -> str:
    """Deterministic ID so re-ingestion is idempotent via upsert."""
    raw = f"{source}::{chunk_index}"
    return hashlib.md5(raw.encode()).hexdigest()

# — Ingestion —————
def ingest_documents(
    collection: chromadb.Collection,
    documents: list[dict], # each: {"text": str, "source": str, "category": str}
    chunk_size: int = CHUNK_SIZE,
    overlap: int = CHUNK_OVERLAP
) -> int:
    """
    Chunk and upsert documents into a Chroma collection.
    Returns total chunks ingested.
    """
    all_ids, all_docs, all metas = [], [], []

    for doc in documents:

```

```

        chunks = chunk_text(doc["text"], chunk_size, overlap)
        for i, chunk in enumerate(chunks):
            all_ids.append(doc_id(doc["source"], i))
            all_docs.append(chunk)
            all metas.append({
                "source": doc["source"],
                "category": doc["category"],
                "chunk_index": i,
                "total_chunks": len(chunks),
            })

# Upsert is idempotent – safe to call on re-ingestion
collection.upsert(
    documents=all_docs,
    ids=all_ids,
    metadatas=all_metas
)
return len(all_ids)

# — Retrieval —————
def retrieve(
    collection: chromadb.Collection,
    query: str,
    k: int = TOP_K,
    category_filter: Optional[str] = None
) -> list[dict]:
    """
    Retrieve top-k chunks relevant to the query, with optional category filter.
    Returns list of dicts with keys: text, source, distance.
    """
    where = {"category": category_filter} if category_filter else None

    results = collection.query(
        query_texts=[query],
        n_results=k,
        where=where,
        include=["documents", "metadatas", "distances"]
    )

    chunks = []
    for doc, meta, dist in zip(
        results["documents"][0],
        results["metadatas"][0],
        results["distances"][0]
    ):
        chunks.append({
            "text": doc,
            "source": meta["source"],
            "category": meta["category"],
            "distance": dist,
        })
    return chunks

```

```

# — Generation —————
def generate_answer(
    openai_client: OpenAI,
    query: str,
    context_chunks: list[dict],
    model: str = GENERATION_MODEL
) -> str:
    """
    Generate a grounded answer using retrieved context.
    """
    context_text = "\n\n--\n\n".join(
        f"[Source: {c['source']}] \n{c['text']}"
        for c in context_chunks
    )

    system_prompt = textwrap.dedent("""
        You are a technical assistant answering questions about databases and AI
        systems.
        Answer using only the information in the provided context.
        If the context does not contain enough information, say so explicitly.
        Cite the source document for each claim you make.
    """).strip()

    user_message = f"""Context:
{context_text}

Question: {query}

Answer: """

    response = openai_client.chat.completions.create(
        model=model,
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_message},
        ],
        temperature=0.1,  # low temperature for factual retrieval tasks
        max_tokens=512,
    )
    return response.choices[0].message.content

# — Pipeline assembly —————
def build_rag_pipeline():
    """Wire up the Chroma collection and OpenAI clients."""
    openai_client = OpenAI()  # reads OPENAI_API_KEY from environment

    openai_ef = embedding_functions.OpenAIEmbeddingFunction(
        model_name=EMBEDDING_MODEL
    )

```

```

chroma_client = chromadb.PersistentClient(path=CHROMA_PATH)
collection = chroma_client.get_or_create_collection(
    name=COLLECTION_NAME,
    embedding_function=openai_ef,
    metadata={"hnsw:space": "cosine"}
)

return openai_client, collection

# — Demo run —————
if __name__ == "__main__":
    openai_client, collection = build_rag_pipeline()

    # Sample corpus – in production, load from files, databases, or APIs
    corpus = [
        {
            "source": "db_indexing",
            "category": "database",
            "text": (
                "SQL Server offers several index types. B-tree (clustered and
nonclustered) "
                "indexes support equality and range queries and are the default. "
                "Columnstore indexes accelerate analytical scans. Full-text indexes
"
                "support CONTAINS and FREETEXT queries. Filtered indexes apply a "
                "WHERE clause during construction so only qualifying rows are "
                "indexed, reducing both index size and maintenance cost. Computed "
                "column indexes store the result of an expression, useful when "
                "queries filter on transformed columns such as persisted JSON "
                "paths extracted with JSON_VALUE."
            ),
        },
        {
            "source": "sqlserver_vectors_guide",
            "category": "vector",
            "text": (
                "SQL Server 2025 adds native vector similarity search. It introduces
"
                "the VECTOR data type, which stores fixed-dimension float arrays, "
                "and the VECTOR_DISTANCE function with cosine, euclidean, and dot "
                "metrics. Approximate search uses DiskANN indexes created with "
                "CREATE VECTOR INDEX. DiskANN is a navigable graph index designed "
                "for SSD-backed operation: most of the graph lives on disk with a "
                "small memory working set, delivering high recall at large scale "
                "without the RAM requirements of in-memory graph indexes."
            ),
        },
        {
            "source": "rag_architecture",
            "category": "ai",
            "text": (
                "Retrieval-Augmented Generation (RAG) combines a retrieval system "

```

```

        "with a language model. At query time, the pipeline embeds the user
    "
        "question, retrieves semantically similar document chunks from a "
        "vector store, concatenates those chunks as context, and passes the
    "
        "combined prompt to the language model for generation. The quality "
        "of the final answer depends on retrieval precision – irrelevant "
        "chunks dilute the context window and degrade generation. Reranking
    "
        "models, such as cross-encoders, can improve retrieval precision by
    "
        "re-scoring the top-k retrieved chunks before passing them to the "
        "generator. Chunk size, overlap strategy, and metadata filtering all
    "
        "affect end-to-end RAG quality significantly."
    ),
},
]

print("Ingesting corpus...")
n_chunks = ingest_documents(collection, corpus)
print(f"Ingested {n_chunks} chunks. Collection size: {collection.count()}\n")

# Run example queries
queries = [
    ("What index should I use for full-text search in SQL Server?", None),
    ("How does DiskANN compare to HNSW in dedicated vector engines?", "vector"),
    ("How does chunk size affect RAG quality?", "ai"),
]

for query, category in queries:
    print(f"Query: {query}")
    if category:
        print(f"Filter: category={category}")

    chunks = retrieve(collection, query, k=TOP_K, category_filter=category)
    print(f"Retrieved {len(chunks)} chunks:")
    for c in chunks:
        print(f"    [{c['source']}] dist={c['distance']:.4f}: {c['text'][:
70]}...")

    answer = generate_answer(openai_client, query, chunks)
    print(f"\nAnswer:\n{answer}\n")
    print("-" * 70 + "\n")

```

This pipeline deliberately avoids framework magic so every component is visible. The `ingest_documents` function uses `upsert` rather than `add` so that re-running the script does not create duplicate entries — a mistake that is easy to make in development and expensive to diagnose when a RAG system returns duplicate context windows.

18.7 Migrating from Chroma to a Production Vector Database

When the graduation signals from Section 18.5 appear, the migration path is straightforward because Chroma's API surface maps cleanly onto Pinecone, Weaviate, and SQL Server native vectors. The core operation is: read vectors and metadata from Chroma, write them to the target database.

Exporting from Chroma

```

# Python
import chromadb
import json

def export_chroma_collection(
    chroma_path: str,
    collection_name: str,
    output_path: str,
    batch_size: int = 500
) -> int:
    """
    Export all vectors, documents, and metadata from a Chroma collection
    to a newline-delimited JSON file for migration.
    """
    client = chromadb.PersistentClient(path=chroma_path)
    collection = client.get_collection(collection_name)

    total = collection.count()
    print(f"Exporting {total:}, documents from '{collection_name}'...")

    exported = 0
    with open(output_path, "w") as f:
        # Chroma does not support cursor-based pagination natively;
        # use offset/limit via get() for batch export
        offset = 0
        while offset < total:
            batch = collection.get(
                limit=batch_size,
                offset=offset,
                include=["embeddings", "documents", "metadatas"]
            )

            for record_id, embedding, document, metadata in zip(
                batch["ids"],
                batch["embeddings"],
                batch["documents"],
                batch["metadatas"]
            ):
                record = {
                    "id": record_id,
                    "embedding": embedding,
                    "document": document,
                    "metadata": metadata,
                }
                f.write(json.dumps(record) + "\n")
                exported += 1

            offset += batch_size
            print(f"  Exported {min(offset, total):} / {total:}, end="\r")

    print(f"\nExport complete: {exported:} records written to {output_path}")
    return exported

```

```
# Export
export_chroma_collection(
    chroma_path="./chroma_data",
    collection_name="engineering_kb",
    output_path="./migration_export.jsonl"
)
```

Importing to SQL Server


```

# Python
import json
import pyodbc

def to_vector_literal(embedding) -> str:
    return "[" + ",".join(str(v) for v in embedding) + "]"

def import_to_sqlserver(
    jsonl_path: str,
    connection_string: str,
    table_name: str,
    batch_size: int = 500
) -> int:
    """
    Load Chroma export into a SQL Server table with a native VECTOR column.
    Creates the table and a DiskANN index if they do not exist.
    """
    conn = pyodbc.connect(connection_string)
    cur = conn.cursor()

    # Inspect the first record to get the dimension
    with open(jsonl_path) as f:
        first = json.loads(f.readline())
    dim = len(first["embedding"])

    # Create table if it does not exist
    cur.execute(f"""
        IF OBJECT_ID('{table_name}', 'U') IS NULL
        CREATE TABLE {table_name} (
            id          NVARCHAR(100) NOT NULL CONSTRAINT PK_{table_name} PRIMARY KEY,
            embedding   VECTOR({dim}) NOT NULL,
            document    NVARCHAR(MAX) NULL,
            metadata    NVARCHAR(MAX) NULL    -- JSON payload
        )
    """)
    cur.execute(f"""
        IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = '{table_name}
        _diskann_idx')
        CREATE VECTOR INDEX {table_name}_diskann_idx
            ON {table_name} (embedding)
            WITH (METRIC = 'cosine', TYPE = 'DiskANN', MAXDOP = 4)
    """)
    conn.commit()

    total_inserted = 0
    batch = []

    def flush(rows):
        # MERGE gives idempotent upserts: re-running the import updates rows
        cur.executemany(f"""
            MERGE {table_name} AS t
            USING (SELECT ? AS id, CAST(? AS VECTOR({dim})) AS embedding,
                ? AS document, ? AS metadata) AS s
        """, rows)
        total_inserted += len(rows)
        batch = []

    return total_inserted

```

```

        ON t.id = s.id
        WHEN MATCHED THEN
            UPDATE SET embedding = s.embedding, document = s.document,
                      metadata = s.metadata
        WHEN NOT MATCHED THEN
            INSERT (id, embedding, document, metadata)
            VALUES (s.id, s.embedding, s.document, s.metadata);
    """
    rows)
    conn.commit()

with open(jsonl_path) as f:
    for line in f:
        record = json.loads(line)
        batch.append((
            record["id"],
            to_vector_literal(record["embedding"]),
            record["document"],
            json.dumps(record["metadata"])
        ))
        if len(batch) >= batch_size:
            flush(batch)
            total_inserted += len(batch)
            batch = []

# Final batch
if batch:
    flush(batch)
    total_inserted += len(batch)

cur.close()
conn.close()
print(f"Imported {total_inserted:,} records into {table_name}")
return total_inserted

# Import
import_to_sqlserver(
    jsonl_path="./migration_export.jsonl",
    connection_string=(
        "DRIVER={ODBC Driver 18 for SQL Server};SERVER=localhost;"
        "DATABASE=vectordb;Trusted_Connection=yes;Encrypt=yes;"
        "TrustServerCertificate=yes"
    ),
    table_name="engineering_kb"
)

```

Importing to Pinecone

```

# Python
import json
from pinecone import Pinecone, ServerlessSpec

def import_to_pinecone(
    jsonl_path: str,
    api_key: str,
    index_name: str,
    batch_size: int = 100
) -> int:
    """
    Load Chroma export into a Pinecone serverless index.
    """
    pc = Pinecone(api_key=api_key)

    # Infer dimension from first record
    with open(jsonl_path) as f:
        first = json.loads(f.readline())
    dim = len(first["embedding"])

    # Create index if it does not exist
    if index_name not in [idx.name for idx in pc.list_indexes()]:
        pc.create_index(
            name=index_name,
            dimension=dim,
            metric="cosine",
            spec=ServerlessSpec(cloud="aws", region="us-east-1")
        )

    index = pc.Index(index_name)
    total_upserted = 0
    batch = []

    with open(jsonl_path) as f:
        for line in f:
            record = json.loads(line)
            # Pinecone metadata values must be strings, numbers, booleans, or lists
            flat_metadata = {
                k: v for k, v in record["metadata"].items()
                if isinstance(v, (str, int, float, bool, list))
            }
            flat_metadata["document"] = record["document"] # store text in metadata

            batch.append({
                "id": record["id"],
                "values": record["embedding"],
                "metadata": flat_metadata
            })

            if len(batch) >= batch_size:
                index.upsert(vectors=batch)
                total_upserted += len(batch)
                batch = []

```

```

        print(f"  Upserted {total_upserted:,} vectors", end="\r")

    if batch:
        index.upsert(vectors=batch)
        total_upserted += len(batch)

    print(f"\nImported {total_upserted:,} vectors into Pinecone index
'{index_name}''")
    return total_upserted

# Import
import_to_pinecone(
    jsonl_path="./migration_export.jsonl",
    api_key="pcsk...",
    index_name="engineering-kb"
)

```

The migration scripts above re-use the embeddings stored in Chroma rather than re-embedding from scratch — a significant cost saving when the collection contains hundreds of thousands of documents. Re-embedding is only necessary when you switch to a different embedding model as part of the migration, which is a natural time to do so if you are upgrading from a local model to a higher-quality API model.

18.8 Chroma in CI/CD: Testing Vector Features Without a Server

One of Chroma's most underrated capabilities is its role in automated testing. Every team that builds RAG features eventually needs to test retrieval quality, embedding consistency, and metadata filtering logic without spinning up a full vector database service in CI. Chroma's ephemeral client makes this possible in a single `pytest` fixture.

```

# Python – conftest.py
import pytest
import chromadb
from chromadb.utils import embedding_functions
import numpy as np

@pytest.fixture(scope="session")
def chroma_client():
    """Session-scoped in-memory Chroma client – no disk I/O, resets between test runs."""
    return chromadb.EphemeralClient()

@pytest.fixture(scope="function")
def empty_collection(chroma_client):
    """Function-scoped collection – isolated between individual tests."""
    col = chroma_client.get_or_create_collection(
        name=f"test_collection_{id(object())}",
        metadata={"hnsw:space": "cosine"}
    )
    yield col
    chroma_client.delete_collection(col.name)

@pytest.fixture(scope="session")
def deterministic_ef():
    """
    Deterministic embedding function for testing.
    Produces consistent embeddings so test results are reproducible.
    Uses a simple hash-based approach rather than a real model.
    """
    class HashEmbeddingFunction:
        def __call__(self, input):
            embeddings = []
            for text in input:
                # Seed numpy RNG with hash of text for reproducibility
                rng = np.random.default_rng(abs(hash(text)) % (2**32))
                emb = rng.standard_normal(384).astype(np.float32)
                emb = emb / np.linalg.norm(emb) # normalize to unit sphere
                embeddings.append(emb.tolist())
            return embeddings

    return HashEmbeddingFunction()

@pytest.fixture(scope="session")
def populated_collection(chroma_client, deterministic_ef):
    """Session-scoped collection with known content for retrieval tests."""
    col = chroma_client.get_or_create_collection(
        name="populated_test",
        embedding_function=deterministic_ef,
        metadata={"hnsw:space": "cosine"}
    )

```

```

)
col.add(
    documents=[
        "SQL Server B-tree index for range queries",
        "SQL Server DiskANN index for similarity search",
        "Redis sorted sets for leaderboard ranking",
        "Elasticsearch inverted index for full-text search",
        "MongoDB atlas search for document retrieval",
    ],
    ids=["btree_idx", "hnsf_idx", "redis_sorted", "es_inverted", "mongo_atlas"],
    metadatas=[
        {"db": "sqlserver", "type": "index"},
        {"db": "sqlserver", "type": "vector"},
        {"db": "redis", "type": "data_structure"},
        {"db": "elasticsearch", "type": "index"},
        {"db": "mongodb", "type": "search"},
    ]
)
yield col
chroma_client.delete_collection(col.name)

```

— Test cases —————

```

def test_collection_add_and_count(empty_collection):
    empty_collection.add(
        documents=["test document one", "test document two"],
        ids=["t1", "t2"]
    )
    assert empty_collection.count() == 2

def test_metadata_filter_returns_correct_subset(populated_collection):
    results = populated_collection.query(
        query_texts=["database index"],
        n_results=5,
        where={"db": "sqlserver"}
    )
    returned_ids = results["ids"][0]
    # All returned results must be SQL Server documents
    assert all(rid in ("btree_idx", "hnsf_idx") for rid in returned_ids)
    # Non-SQL Server documents must not appear
    assert "redis_sorted" not in returned_ids
    assert "es_inverted" not in returned_ids

def test_upsert_updates_existing_document(empty_collection):
    empty_collection.add(
        documents=["original content"],
        ids=["doc_1"],
        metadatas=[{"version": 1}]
    )
    empty_collection.upsert(

```

```

        documents=["updated content"],
        ids=["doc_1"],
        metadatas=[{"version": 2}]
    )
    result = empty_collection.get(ids=["doc_1"], include=["documents", "metadatas"])
    assert result["documents"][0] == "updated content"
    assert result["metadatas"][0]["version"] == 2
    assert empty_collection.count() == 1    # no duplicate created

def test_delete_removes_document(empty_collection):
    empty_collection.add(
        documents=["doc to delete", "doc to keep"],
        ids=["delete_me", "keep_me"]
    )
    empty_collection.delete(ids=["delete_me"])
    assert empty_collection.count() == 1
    remaining = empty_collection.get(ids=["keep_me"])
    assert remaining["ids"] == ["keep_me"]

def test_query_returns_n_results(populated_collection):
    for k in [1, 2, 3]:
        results = populated_collection.query(
            query_texts=["search index"],
            n_results=k
        )
        assert len(results["ids"][0]) == k

def test_distances_are_bounded_for_cosine(populated_collection):
    """Cosine distance in Chroma is 1 - cosine_similarity, range [0, 2]."""
    results = populated_collection.query(
        query_texts=["vector similarity"],
        n_results=5,
        include=["distances"]
    )
    for dist in results["distances"][0]:
        assert 0.0 <= dist <= 2.0, f"Unexpected cosine distance: {dist}"

```

These tests run in milliseconds — no network calls, no model inference (using the deterministic hash embedder), no external dependencies. They test the logic of your collection schema, filtering predicates, and upsert behavior. The deterministic embedding function is a deliberate choice: real embedding models are non-deterministic under different runtime environments (GPU vs. CPU, model version updates), so pinning embedding behavior for unit tests produces stable CI pipelines.

For integration tests that actually exercise the embedding model, use a separate fixture that marks the tests with `@pytest.mark.integration` and skip them in standard CI runs:

```

# Python
import pytest

@pytest.mark.integration
def test_semantic_retrieval_quality(chroma_client):
    """
    Integration test: verifies that semantic retrieval returns topically
    relevant results using the real embedding model. Requires OPENAI_API_KEY.
    Skipped in standard CI; run in pre-deployment integration stage.
    """
    from chromadb.utils import embedding_functions
    import os

    api_key = os.environ.get("OPENAI_API_KEY")
    if not api_key:
        pytest.skip("OPENAI_API_KEY not set")

    ef = embedding_functions.OpenAIEmbeddingFunction(
        api_key=api_key,
        model_name="text-embedding-3-small"
    )
    col = chroma_client.get_or_create_collection(
        name="integration_test",
        embedding_function=ef,
        metadata={"hnsw:space": "cosine"}
    )

    try:
        col.add(
            documents=[
                "Index rebuild reclaims space from fragmented indexes in SQL
Server",
                "HNSW indexing uses a multi-layer graph for approximate search",
                "Connection pooling reduces latency by reusing database
connections",
            ],
            ids=["defrag", "hnsw", "pooling"]
        )

        results = col.query(
            query_texts=["how to reclaim dead row storage?"],
            n_results=1
        )
        top_id = results["ids"][0][0]
        assert top_id == "defrag", (
            f"Expected 'defrag' as top result, got '{top_id}'. "
            "Embedding model may have changed behavior."
        )
    finally:
        chroma_client.delete_collection("integration_test")

```

Key Takeaways

- → **Chroma is an embedded database first.** Its default operating mode — in-process, single-file, no daemon — makes it the fastest path from idea to working vector search, following the same philosophy as SQLite for relational data. Client/server mode extends this to multi-process deployments without changing the API surface.
- → **The embedding function is a schema constraint.** Attaching an embedding function to a collection fixes both the dimensionality and the semantic space of that collection. Changing the model requires rebuilding the collection. Treat the embedding function choice with the same deliberateness you would apply to choosing a column type.
- → **HNSW parameters set at construction time are permanent.** ``hnsw:construction_ef`` and ``hnsw:M`` cannot be changed without rebuilding the index. Set them based on your target recall before ingesting production data; ``hnsw:search_ef`` can be tuned at query time without rebuilding.
- → **Graduation from Chroma is a data volume and operational maturity decision, not a capability gap.** The filtering, embedding, and retrieval patterns you build against Chroma are directly portable to SQL Server native vectors, Pinecone, and Weaviate. The migration scripts in this chapter preserve stored embeddings so you do not pay re-embedding costs at migration time.
- → **Chroma's ephemeral client makes vector search testable in CI without infrastructure.** A deterministic embedding function combined with ``EphemeralClient`` produces fast, stable unit tests for collection schema, metadata filtering, and upsert logic. Reserve real embedding model calls for integration tests that run in pre-deployment stages.

Chapter 19: Milvus: Open-Source Vector Search at Enterprise Scale

Milvus occupies a distinctive position in the vector database landscape: it is the most fully-featured open-source vector search engine designed from the ground up for production workloads at scale. While SQL Server native vectors are the natural extension for teams already on SQL Server, and Pinecone offers managed simplicity, Milvus targets organizations that need full control over their infrastructure, support for billions of vectors, and enterprise-grade operational features — without paying managed-service premiums or accepting vendor lock-in. This chapter covers Milvus architecture, deployment patterns, collection and index design, filtering and hybrid search, and the operational patterns that separate a proof-of-concept cluster from a system you can put on-call. Whether you are migrating from a simpler vector store that has started showing its limits, or designing a greenfield AI platform, understanding Milvus at the engineering level gives you the vocabulary and judgment to decide when it is the right tool and how to run it correctly.

19.1 Milvus Architecture: Disaggregated Storage and Compute

Milvus 2.x was rebuilt around a cloud-native, disaggregated architecture. Understanding this design is essential before you write a single line of code, because it directly determines how you size clusters, recover from failures, and tune for throughput versus latency.

The Four Layers

Milvus separates concerns into four functional layers that can scale independently:

Layer	Components	Responsibility
Access	Proxy	Request routing, load balancing, query planning
Coordinator	Root, Query, Data, Index coordinators	Metadata management, segment assignment, topology
Worker	Query Nodes, Data Nodes, Index Nodes	Actual compute: search, ingest, index build

| Storage | Object storage (S3/MinIO) + etcd + message bus (Pulsar/Kafka) | Durability, WAL, metadata |

This is architecturally analogous to separating the SQL Server query optimizer, execution engine, storage engine, and transaction log into independently scalable services — something relational databases have only recently begun doing in products like Aurora and Azure SQL Hyperscale. For vector workloads, this disaggregation matters because index building is CPU-and-memory-intensive work that would otherwise contend with query serving. A Milvus Index Node can build an IVF_FLAT or HNSW index on a 100M-vector segment while Query Nodes continue serving search requests without interference.

```
graph TD
    Client[Client SDK]
    Proxy[Access Layer: Proxy  
routing, load balancing]
    Coord[Coordinator Layer  
Root / Query / Data / Index coordinators]
    QN[Query Nodes  
serve search]
    DN[Data Nodes  
ingest, persist]
    IN[Index Nodes  
build indexes]
    Storage[(Storage Layer  
Object storage + etcd + Pulsar/Kafka)]

    Client --> Proxy
    Proxy --> Coord
    Coord --> QN
    Coord --> DN
    Coord --> IN
    QN <--> Storage
    DN <--> Storage
    IN <--> Storage
```

The four layers scale independently: Query Nodes handle search load, Data Nodes handle ingestion, and Index Nodes build indexes in the background — none of them contend with each other for CPU or memory.

Segment Model

Data in Milvus is organized into segments, the fundamental unit of indexing and distribution. Segments start as **growing** (in-memory, accepting inserts), then seal to **sealed** (immutable), and eventually get compacted and persisted to object storage. This is structurally similar to LSM-tree compaction in RocksDB or SQL Server's ghost-record cleanup cycle — data flows through a pipeline of mutability states, with index building happening asynchronously on sealed segments.

A sealed segment with an HNSW index on S3 is loaded into Query Node memory on demand. This lazy-load model enables a cluster to host more data than fits in aggregate RAM, at the cost of cold-query latency when a segment must be fetched from object storage. Tune the `dataCoord.segment.maxSize` parameter (default 512 MB) to control segment granularity — smaller segments reduce memory pressure per load but increase metadata overhead.

Message Bus and WAL

Milvus uses Apache Pulsar (or Kafka in enterprise configurations) as its internal message bus. Every insert, delete, and schema change is published as a message and consumed by Data Nodes for persistence and by Query Nodes for real-time visibility. This gives Milvus a **streaming-first** ingestion model: you do not insert into a table and wait for a commit; you publish a record and it becomes queryable within milliseconds once the Query Node consumes the message off the channel. The practical consequence is that Milvus offers **eventual consistency** by default, not the snapshot isolation you get from SQL Server. For RAG pipelines where documents are indexed asynchronously after upload, this is usually acceptable. For applications that insert a record and immediately query for it, you must call `collection.flush()` to force segment sealing.

19.2 Deployment: From Docker Compose to Kubernetes

Milvus ships in two operational modes: **Milvus Lite** (in-process, for development only), **Milvus Standalone** (single-node, suitable for up to ~50M vectors), and **Milvus Distributed** (the full multi-node cluster). Zilliz Cloud is the managed SaaS version if you want to outsource operations.

Local Development with Docker Compose

```
# shell
# Download the official Milvus standalone compose file
wget https://github.com/milvus-io/milvus/releases/download/v2.4.0/milvus-standalone-docker-compose.yml \
    -O docker-compose.yml

# Start Milvus standalone with embedded MinIO and etcd
docker-compose up -d

# Verify all services are healthy
docker-compose ps
# Expected: milvus-standalone, milvus-minio, milvus-etcd all running

# Check Milvus version and connectivity
docker exec milvus-standalone milvus --version
```

Python Environment Setup

```
# Python
# Install the Milvus client and supporting libraries
# pip install pymilvus==2.4.0 sentence-transformers openai numpy

from pymilvus import connections, utility

# Connect to local standalone instance
connections.connect(
    alias="default",
    host="localhost",
    port="19530"
)

# Verify connection and list existing collections
print("Milvus version:", utility.get_server_version())
print("Existing collections:", utility.list_collections())
```

Kubernetes Deployment with Helm

For production, use the official Helm chart. The following configuration is a starting point for a medium-scale deployment handling ~200M vectors with mixed read/write workloads:

```

# shell
# Add Milvus Helm repository
helm repo add milvus https://zilliztech.github.io/milvus-helm/
helm repo update

# Create a values override file for production sizing
cat > milvus-values.yaml << 'EOF'
cluster:
  enabled: true

# Proxy handles client connections and query routing
proxy:
  replicas: 2
  resources:
    requests:
      cpu: "2"
      memory: "4Gi"
    limits:
      cpu: "4"
      memory: "8Gi"

# Query nodes serve search requests – size for your working set
queryNode:
  replicas: 4
  resources:
    requests:
      cpu: "8"
      memory: "32Gi"
    limits:
      cpu: "16"
      memory: "64Gi"

# Data nodes handle ingestion and segment persistence
dataNode:
  replicas: 2
  resources:
    requests:
      cpu: "4"
      memory: "8Gi"

# Index nodes build vector indexes asynchronously
indexNode:
  replicas: 2
  resources:
    requests:
      cpu: "16"
      memory: "32Gi"

# External object storage – replace with your S3 bucket
minio:
  enabled: false # Use external S3

externalS3:

```

```

enabled: true
host: "s3.amazonaws.com"
port: 443
useSSL: true
bucketName: "milvus-production"
accessKey: ""    # Use IAM roles or secrets manager
secretKey: ""

# External etcd for metadata durability
etcd:
  enabled: false

externalEtcd:
  enabled: true
  endpoints:
    - "etcd-0.etcd-headless:2379"
    - "etcd-1.etcd-headless:2379"
    - "etcd-2.etcd-headless:2379"

# Pulsar for message bus
pulsar:
  enabled: true
  broker:
    replicaCount: 2

EOF

# Deploy to the milvus namespace
kubectl create namespace milvus
helm install milvus milvus/milvus \
  --namespace milvus \
  --values milvus-values.yaml \
  --version 4.1.0

```

The key sizing principle for Query Nodes: your working dataset (the collection of segments actively loaded) must fit in aggregate Query Node memory. If your total index size is 160 GB and each Query Node has 32 GB available for data, you need at least 5 Query Nodes. Monitor the `milvus_querynode_cache_hit_rate` Prometheus metric — anything below 85% indicates memory pressure and cold-load latency will dominate your p99.

19.3 Collection Design, Indexing, and Schema Modeling

A Milvus *collection* is the conceptual equivalent of a table. It has a schema with a primary key, scalar fields for metadata, and one or more vector fields. Getting schema design right before you load 500M records saves you from a costly dump-and-reload cycle.

Schema Design and Collection Creation

```

# Python
from pymilvus import (
    connections, Collection, CollectionSchema,
    FieldSchema, DataType, utility
)
import numpy as np

connections.connect(alias="default", host="localhost", port="19530")

# Define schema for a document chunk collection
# Analogous to CREATE TABLE with column definitions
fields = [
    FieldSchema(
        name="chunk_id",
        dtype=DataType.INT64,
        is_primary=True,
        auto_id=True          # Milvus generates IDs, like IDENTITY in SQL Server
    ),
    FieldSchema(
        name="document_id",
        dtype=DataType.VARCHAR,
        max_length=128        # Scalar field for filtering
    ),
    FieldSchema(
        name="chunk_index",
        dtype=DataType.INT32   # Position within document
    ),
    FieldSchema(
        name="content_type",
        dtype=DataType.VARCHAR,
        max_length=32         # "paragraph", "table", "code", etc.
    ),
    FieldSchema(
        name="created_at",
        dtype=DataType.INT64   # Unix timestamp – Milvus has no native TIMESTAMP
    ),
    FieldSchema(
        name="embedding",
        dtype=DataType.FLOAT_VECTOR,
        dim=1536               # OpenAI text-embedding-3-small dimensions
    ),
    FieldSchema(
        name="sparse_embedding", # For hybrid dense+sparse search
        dtype=DataType.SPARSE_FLOAT_VECTOR
    )
]

schema = CollectionSchema(
    fields=fields,
    description="Document chunks with dense and sparse embeddings",
    enable_dynamic_field=True # Allows schemaless extra fields, like JSONB
)

```



```

# Create the collection with 2 shards
# More shards = better write parallelism, but higher metadata overhead
collection = Collection(
    name="document_chunks",
    schema=schema,
    shards_num=2
)

print(f"Collection created: {collection.name}")
print(f"Schema fields: {[f.name for f in collection.schema.fields]}")

# Build index on the dense vector field
# HNSW is the right default for most workloads under 500M vectors
index_params = {
    "metric_type": "COSINE",
    "index_type": "HNSW",
    "params": {
        "M": 16,                # Neighbors per node. Range: 4-64. Higher = better
                                # recall, more memory
        "efConstruction": 200  # Build-time search depth. Higher = better index
                                # quality, slower build
    }
}

collection.create_index(
    field_name="embedding",
    index_params=index_params,
    index_name="dense_hnsw"
)

# Build index on sparse vector field for BM25-style keyword matching
sparse_index_params = {
    "metric_type": "IP",
    "index_type": "SPARSE_INVERTED_INDEX",
    "params": {"drop_ratio_build": 0.2} # Drop bottom 20% of weights to save space
}

collection.create_index(
    field_name="sparse_embedding",
    index_params=sparse_index_params,
    index_name="sparse_inverted"
)

# Build scalar index on document_id for filtered searches
# Equivalent to CREATE INDEX on a relational column
collection.create_index(
    field_name="document_id",
    index_name="doc_id_index"
)

print("Indexes created successfully")

```

Index Type Selection

Milvus supports a broader range of index types than most vector databases. The selection criteria map cleanly to database index selection reasoning:

Index Type	Best For	Memory	Build Time	Recall	Notes
------------	----------	--------	------------	--------	-------

---	---	---	---	---	---
-----	-----	-----	-----	-----	-----

FLAT	<1M vectors, exact recall required	High	Fast	100%	Brute force, no approximation
------	------------------------------------	------	------	------	-------------------------------

IVF_FLAT	1M–100M, balanced	Medium	Medium	~98%	<code>nlist</code> controls cluster count
----------	-------------------	--------	--------	------	---

IVF_SQ8	Memory-constrained, 8-bit quantization	Low	Medium	~95%	Good throughput/memory tradeoff
---------	--	-----	--------	------	---------------------------------

HNSW	<500M, low-latency queries	High	Slow	~99%	Best recall/latency tradeoff
------	----------------------------	------	------	------	------------------------------

DISKANN	>500M, cannot fit in RAM	Disk	Slow	~97%	Serves from SSD, lower cost
---------	--------------------------	------	------	------	-----------------------------

GPU_IVF_FLAT	Batch throughput workloads	GPU VRAM	Fast (GPU)	~98%	Requires CUDA-capable hardware
--------------	----------------------------	----------	------------	------	--------------------------------

SCANN	High-recall, Google-developed	Medium	Medium	~99%	Competitive with HNSW at scale
-------	-------------------------------	--------	--------	------	--------------------------------

For HNSW, the `M` parameter is the most impactful tunable. `M=16` is a good default — `M=32` increases memory usage by roughly 2x but improves recall for high-dimensional spaces (>768 dimensions). The `ef` parameter at query time (not index build time) is the runtime recall/latency lever: start at `ef=64` and increase until recall meets your SLA, then cap it.

19.4 Querying: ANN Search, Filtered Search, and Hybrid Search

Milvus search is richer than the typical "find K nearest neighbors" interface. Understanding its query primitives lets you build sophisticated retrieval patterns that would require multiple round trips in simpler systems.

Basic ANN Search and Filtered Search

```

# Python
from pymilvus import connections, Collection
from sentence_transformers import SentenceTransformer
import time

connections.connect(alias="default", host="localhost", port="19530")
collection = Collection("document_chunks")

# Load collection into Query Node memory before searching
# This is the equivalent of a warm cache – cold collections have high first-query
latency
collection.load()

# Initialize embedding model
encoder = SentenceTransformer("all-MiniLM-L6-v2")

def search_documents(
    query: str,
    document_ids: list[str] | None = None,
    content_type: str | None = None,
    top_k: int = 10,
    ef: int = 64
) -> list[dict]:
    """
    Search with optional pre-filtering on scalar fields.
    Pre-filtering in Milvus happens before ANN search (unlike post-filtering),
    which maintains recall guarantees but requires scalar indexes.
    """
    query_embedding = encoder.encode([query])[0].tolist()

    # Build filter expression – syntax is similar to SQL WHERE clause
    filter_parts = []
    if document_ids:
        # IN operator for set membership
        ids_str = ', '.join(document_ids)
        filter_parts.append(f'document_id in [{"ids_str}"]')
    if content_type:
        filter_parts.append(f'content_type == "{content_type}"')

    filter_expr = " and ".join(filter_parts) if filter_parts else ""

    search_params = {
        "metric_type": "COSINE",
        "params": {"ef": ef} # Runtime recall parameter for HNSW
    }

    start = time.perf_counter()
    results = collection.search(
        data=[query_embedding],
        anns_field="embedding",
        param=search_params,
        limit=top_k,
        expr=filter_expr if filter_expr else None,

```

```

        output_fields=["document_id", "chunk_index", "content_type", "created_at"]
    )
    latency_ms = (time.perf_counter() - start) * 1000

    hits = []
    for hit in results[0]:
        hits.append({
            "chunk_id": hit.id,
            "score": hit.score,
            "document_id": hit.entity.get("document_id"),
            "chunk_index": hit.entity.get("chunk_index"),
            "content_type": hit.entity.get("content_type"),
            "latency_ms": latency_ms
        })

    return hits

# Example: Search within specific documents with content type filter
results = search_documents(
    query="how to configure connection pooling",
    document_ids=["sqlserver-docs-v17", "read-committed-snapshot-guide"],
    content_type="paragraph",
    top_k=5
)

for r in results:
    print(f"Score: {r['score']:.4f} | Doc: {r['document_id']} | Chunk: {r['chunk_index']}")

```

Hybrid Search: Dense + Sparse Fusion

Hybrid search — combining dense semantic embeddings with sparse keyword-matching vectors — is one of Milvus's strongest differentiators. The rationale is well-established: dense retrieval handles paraphrase and semantic similarity, sparse retrieval handles exact keyword matching. Combining them with Reciprocal Rank Fusion (RRF) or weighted scoring consistently outperforms either alone, particularly for technical documentation where exact terms matter.

```

# Python
from pymilvus import (
    connections, Collection, AnnSearchRequest,
    RRFRanker, WeightedRanker
)
from pymilvus.model.hybrid import BGEM3EmbeddingFunction
import numpy as np

connections.connect(alias="default", host="localhost", port="19530")
collection = Collection("document_chunks")
collection.load()

# BGE-M3 generates both dense and sparse embeddings in one pass
# This is more efficient than separate dense and sparse encoder calls
embedding_fn = BGEM3EmbeddingFunction(
    model_name="BAAI/bge-m3",
    device="cpu",          # Use "cuda" if GPU available
    use_fp16=False
)

def hybrid_search(
    query: str,
    top_k: int = 10,
    dense_weight: float = 0.7,
    sparse_weight: float = 0.3,
    use_rrf: bool = True
) -> list[dict]:
    """
    Hybrid dense+sparse search with configurable fusion strategy.
    RRF is rank-based and robust to score scale differences.
    WeightedRanker requires scores to be on the same scale.
    """
    # Generate both dense and sparse embeddings
    embeddings = embedding_fn([query])
    dense_vec = embeddings["dense"][0]
    sparse_vec = embeddings["sparse"][[0]] # scipy sparse matrix row

    # Dense search request – HNSW index
    dense_request = AnnSearchRequest(
        data=[dense_vec.tolist()],
        anns_field="embedding",
        param={"metric_type": "COSINE", "params": {"ef": 100}},
        limit=top_k * 2 # Over-fetch to allow fusion reranking
    )

    # Sparse search request – inverted index
    sparse_request = AnnSearchRequest(
        data=[sparse_vec],
        anns_field="sparse_embedding",
        param={"metric_type": "IP", "params": {"drop_ratio_search": 0.2}},
        limit=top_k * 2
    )

```

```

# Fusion strategy: RRF (rank-based) vs Weighted (score-based)
if use_rrf:
    ranker = RRFRanker(k=60)    # k=60 is the standard RRF constant
else:
    ranker = WeightedRanker(dense_weight, sparse_weight)

results = collection.hybrid_search(
    reqs=[dense_request, sparse_request],
    ranker=ranker,
    limit=top_k,
    output_fields=["document_id", "chunk_index", "content_type"]
)

hits = []
for hit in results[0]:
    hits.append({
        "chunk_id": hit.id,
        "fused_score": hit.score,
        "document_id": hit.entity.get("document_id"),
        "chunk_index": hit.entity.get("chunk_index"),
    })

return hits

# Compare dense-only vs hybrid for a keyword-sensitive query
query = "sql server create vector index diskann syntax"

hybrid_results = hybrid_search(query, use_rrf=True)
print("Hybrid search results (RRF fusion):")
for r in hybrid_results[:5]:
    print(f"  Score: {r['fused_score']:.4f} | {r['document_id']}: {r['chunk_index']}")

```

Range Search and Grouping

Two additional search primitives worth knowing: **range search** returns all vectors within a distance threshold (rather than top-K), which is useful for deduplication and similarity clustering. **Grouping search** returns the best match per group — equivalent to `ROW_NUMBER() OVER (PARTITION BY document_id ...)` filtering in SQL Server — preventing a single long document from dominating results with many similar chunks.

```

# Python
from pymilvus import connections, Collection

connections.connect(alias="default", host="localhost", port="19530")
collection = Collection("document_chunks")
collection.load()

# Range search: find all chunks within a cosine similarity threshold
# Useful for deduplication – flag chunks with similarity > 0.98 as near-duplicates
def range_search_for_deduplication(embedding: list[float], threshold: float = 0.98):
    results = collection.search(
        data=[embedding],
        anns_field="embedding",
        param={
            "metric_type": "COSINE",
            "params": {
                "ef": 200,
                "radius": 1.0 - threshold,      # Convert similarity threshold to
distance                                     # Include everything from 0 to
radius                                     radius
            }
        },
        limit=100,
        output_fields=["document_id", "chunk_index"]
    )
    return results[0]

# Grouping search: best match per document, prevent any single doc from dominating
def search_with_document_diversity(query_embedding: list[float], top_k: int = 10):
    results = collection.search(
        data=[query_embedding],
        anns_field="embedding",
        param={"metric_type": "COSINE", "params": {"ef": 200}},
        limit=top_k,
        group_by_field="document_id",    # One result per unique document_id
        group_size=1,                   # How many results per group
        output_fields=["document_id", "chunk_index", "content_type"]
    )
    return results[0]

```

19.5 Ingestion Pipelines and Data Management

Ingesting data into Milvus at scale requires attention to batching, consistency semantics, and the operational lifecycle of collections. Unlike a relational database where you `INSERT` rows and they are immediately visible to subsequent queries, Milvus's streaming architecture means you need to manage the pipeline explicitly.

Batch Ingestion with OpenAI Embeddings


```

# Python
import os
import time
import numpy as np
from typing import Iterator
from pymilvus import connections, Collection, utility
from openai import OpenAI

connections.connect(alias="default", host="localhost", port="19530")
openai_client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])
collection = Collection("document_chunks")

def generate_embeddings_batch(texts: list[str], model: str = "text-embedding-3-small") -> list[list[float]]:
    """
    Generate embeddings with automatic retry and rate-limit handling.
    OpenAI allows up to 2048 texts per batch call for embedding models.
    """
    response = openai_client.embeddings.create(
        input=texts,
        model=model
    )
    return [item.embedding for item in response.data]

def chunk_iterator(documents: list[dict], chunk_size: int = 512) -> Iterator[dict]:
    """
    Yield individual chunks from a list of documents.
    Each document is a dict with 'id', 'content_type', and 'text' fields.
    """
    for doc in documents:
        words = doc["text"].split()
        for i in range(0, len(words), chunk_size):
            chunk_text = " ".join(words[i:i + chunk_size])
            yield {
                "document_id": doc["id"],
                "chunk_index": i // chunk_size,
                "content_type": doc.get("content_type", "paragraph"),
                "created_at": int(time.time()),
                "text": chunk_text
            }

def ingest_documents(
    documents: list[dict],
    batch_size: int = 200,
    flush_every_n_batches: int = 10
) -> dict:
    """
    Ingest documents into Milvus with batched embedding generation.
    batch_size=200 balances OpenAI API throughput with memory usage.
    flush_every_n_batches triggers segment sealing to make data queryable.
    """
    total_inserted = 0
    batch_count = 0

```

```

chunks = list(chunk_iterator(documents))

for batch_start in range(0, len(chunks), batch_size):
    batch = chunks[batch_start:batch_start + batch_size]
    texts = [c["text"] for c in batch]

    # Generate embeddings for this batch
    embeddings = generate_embeddings_batch(texts)

    # Prepare columnar data – Milvus insert expects lists per field
    insert_data = {
        "document_id": [c["document_id"] for c in batch],
        "chunk_index": [c["chunk_index"] for c in batch],
        "content_type": [c["content_type"] for c in batch],
        "created_at": [c["created_at"] for c in batch],
        "embedding": embeddings
    }

    result = collection.insert(insert_data)
    total_inserted += len(result.primary_keys)
    batch_count += 1

    # Periodically flush to seal growing segments and make data queryable
    # Without flush, inserted data is only visible to searches after
    # the message bus consumer processes it – usually within seconds,
    # but flush guarantees visibility
    if batch_count % flush_every_n_batches == 0:
        collection.flush()
        print(f"Flushed after {total_inserted} insertions")

# Final flush to ensure all data is persisted and queryable
collection.flush()

return {
    "total_chunks": len(chunks),
    "total_inserted": total_inserted,
    "batches_processed": batch_count
}

# Sample usage
sample_documents = [
    {
        "id": "pg-docs-v16-ch01",
        "content_type": "paragraph",
        "text": "SQL Server is a relational database management system from
Microsoft..."
    },
    {
        "id": "pg-docs-v16-ch02",
        "content_type": "paragraph",
        "text": "Connection pooling reduces the overhead of establishing new
database connections..."
    }
]

```

```
]

```

```
stats = ingest_documents(sample_documents, batch_size=200)
print(f"Ingestion complete: {stats}")
```

Upsert and Deletion

Milvus supports upserts (insert or replace by primary key) and soft-delete semantics. Deleted records are marked with a tombstone and excluded from search results; the physical space is reclaimed during compaction. This is analogous to SQL Server's version store, where old row versions are cleaned up asynchronously by the ghost cleanup task.

```
# Python
from pymilvus import connections, Collection

connections.connect(alias="default", host="localhost", port="19530")
collection = Collection("document_chunks")

# Upsert: replaces existing record if primary key exists
# Useful when re-processing a document with updated content
def upsert_chunks(chunks_data: dict) -> int:
    result = collection.upsert(chunks_data)
    collection.flush()
    return len(result.primary_keys)

# Delete by primary key – immediate for query purposes after flush
def delete_chunks_by_id(chunk_ids: list[int]):
    id_list = ", ".join(str(i) for i in chunk_ids)
    expr = f"chunk_id in [{id_list}]"
    collection.delete(expr)
    collection.flush()

# Delete all chunks belonging to a document – filter-based delete
def delete_document(document_id: str):
    expr = f'document_id == "{document_id}"'
    collection.delete(expr)
    collection.flush()
    print(f"Deleted all chunks for document: {document_id}")

# Example: remove a document that has been superseded
delete_document("pg-docs-v15-ch01")
```

Compaction and Index Rebuild

After heavy deletes, Milvus segment files contain tombstoned records that waste memory when loaded. Trigger manual compaction to physically reclaim space, then optionally rebuild indexes if segment fragmentation is high.

```

# Python
from pymilvus import connections, Collection, utility
import time

connections.connect(alias="default", host="localhost", port="19530")
collection = Collection("document_chunks")

# Trigger compaction – runs asynchronously in the background
compaction_id = collection.compact()
print(f"Compaction triggered, job ID: {compaction_id}")

# Poll until compaction completes
while True:
    state = collection.get_compaction_state()
    print(f"Compaction state: {state.state}")
    if str(state.state) in ("Completed", "Failed"):
        break
    time.sleep(5)

# Check collection statistics after compaction
stats = utility.get_query_segment_info("document_chunks")
print(f"Segment count after compaction: {len(stats)}")

```

19.6 Multi-Tenancy Patterns

Multi-tenancy in vector databases is a first-class design concern that does not have a single right answer. Milvus offers three patterns, each with different isolation, performance, and operational complexity characteristics — an almost exact parallel to the multi-tenancy patterns in relational database design.

Pattern 1: Collection-per-Tenant

Each tenant has a dedicated collection. This gives maximum isolation: tenant data is physically separate, indexes are independent, and you can drop a tenant's collection without affecting others.

```

# Python
from pymilvus import connections, Collection, CollectionSchema, FieldSchema,
DataType, utility

connections.connect(alias="default", host="localhost", port="19530")

def create_tenant_collection(tenant_id: str, vector_dim: int = 1536) -> Collection:
    """
    Create a dedicated collection for a tenant.
    Collection names must be alphanumeric – sanitize tenant IDs.
    """
    collection_name = f"tenant_{tenant_id.replace('-', '_)}"

    if utility.has_collection(collection_name):
        return Collection(collection_name)

    fields = [
        FieldSchema(name="chunk_id", dtype=DataType.INT64, is_primary=True,
auto_id=True),
        FieldSchema(name="document_id", dtype=DataType.VARCHAR, max_length=128),
        FieldSchema(name="embedding", dtype=DataType.FLOAT_VECTOR, dim=vector_dim)
    ]
    schema = CollectionSchema(fields=fields)
    collection = Collection(name=collection_name, schema=schema)

    collection.create_index(
        field_name="embedding",
        index_params={"metric_type": "COSINE", "index_type": "HNSW", "params": {"M":
16, "efConstruction": 200}}
    )
    return collection

def drop_tenant(tenant_id: str):
    """Remove a tenant's collection entirely – instant, no residual data."""
    collection_name = f"tenant_{tenant_id.replace('-', '_)}"
    if utility.has_collection(collection_name):
        utility.drop_collection(collection_name)
        print(f"Dropped collection for tenant: {tenant_id}")

```

The limitation: Milvus has a practical limit on the number of simultaneously loaded collections (typically in the hundreds, bounded by Query Node memory). This pattern does not scale beyond a few hundred active tenants without implementing a load/unload scheduling layer.

Pattern 2: Partition-per-Tenant

A single collection with tenant-specific partitions. Partitions in Milvus are physical subdivisions of a collection — search can be scoped to one or more partitions, isolating tenant data at query time while sharing a single collection's overhead.

```

# Python
from pymilvus import connections, Collection, Partition

connections.connect(alias="default", host="localhost", port="19530")
collection = Collection("shared_document_chunks")

def get_or_create_tenant_partition(tenant_id: str) -> Partition:
    partition_name = f"tenant_{tenant_id}"
    if not collection.has_partition(partition_name):
        collection.create_partition(partition_name)
    return Partition(collection, partition_name)

def insert_for_tenant(tenant_id: str, data: dict) -> list[int]:
    partition = get_or_create_tenant_partition(tenant_id)
    result = partition.insert(data)
    return result.primary_keys

def search_for_tenant(tenant_id: str, query_embedding: list[float], top_k: int =
10):
    partition_name = f"tenant_{tenant_id}"
    # Load only this tenant's partition – other tenants' data stays unloaded
    collection.load(partition_names=[partition_name])

    results = collection.search(
        data=[query_embedding],
        anns_field="embedding",
        param={"metric_type": "COSINE", "params": {"ef": 64}},
        limit=top_k,
        partition_names=[partition_name]
    )
    return results[0]

```

Partitions scale to around 1,024 per collection in Milvus 2.4. This pattern is appropriate for SaaS applications with up to several hundred tenants that have predictable, moderate-sized datasets per tenant.

Pattern 3: Field-based Tenant Isolation with Filtering

All tenant data lives in a single collection with a `tenant_id` scalar field. Every query includes a filter expression scoping results to the requesting tenant. This is the simplest operationally but requires rigorous application-layer enforcement — there is no database-level guarantee that a query cannot accidentally omit the tenant filter.

```

# Python
from pymilvus import connections, Collection

connections.connect(alias="default", host="localhost", port="19530")
collection = Collection("shared_document_chunks")
collection.load()

def search_tenant_strict(
    tenant_id: str,
    query_embedding: list[float],
    top_k: int = 10
) -> list[dict]:
    """
    Always inject tenant_id filter – never allow unfiltered search.
    Enforce at this layer so application code cannot accidentally bypass tenant
    isolation.
    """
    results = collection.search(
        data=[query_embedding],
        anns_field="embedding",
        param={"metric_type": "COSINE", "params": {"ef": 64}},
        limit=top_k,
        expr=f'tenant_id == "{tenant_id}"',
        output_fields=["document_id", "chunk_index", "tenant_id"]
    )
    # Verify no cross-tenant leakage – defensive check in high-compliance contexts
    for hit in results[0]:
        assert hit.entity.get("tenant_id") == tenant_id, \
            f"Cross-tenant data leakage detected: got {hit.entity.get('tenant_id')}}"
    return results[0]

```

This pattern scales to unlimited tenants with no collection or partition count limits, but recall degrades proportionally with the fraction of the collection that belongs to each tenant. A tenant with 0.01% of records in a 1B-vector collection will experience poor recall because the ANN graph was built over all tenants' data, and HNSW pruning may skip relevant neighbors that happen to be separated by other tenants' records in the graph.

19.7 Operational Concerns: Monitoring, Backup, and Upgrades

Running Milvus in production means owning the same operational surface area as any stateful distributed system. This section covers the monitoring signals, backup strategy, and upgrade path that keep a Milvus cluster healthy over time.

Prometheus and Grafana Integration

Milvus exposes a rich set of Prometheus metrics. The following are the highest-signal metrics for production dashboards:

```
# prometheus-scrape-config.yaml snippet
scrape_configs:
  - job_name: 'milvus'
    static_configs:
      - targets:
        - 'milvus-proxy:9091'
        - 'milvus-querynode-0:9091'
        - 'milvus-querynode-1:9091'
        - 'milvus-datanode-0:9091'
        - 'milvus-indexnode-0:9091'
    metrics_path: '/metrics'
```

Key metrics to alert on:

Metric	Alert Threshold	Meaning
--------	-----------------	---------

---	---	---
-----	-----	-----

<code>milvus_proxy_search_latency_p99</code>	>500ms	Search latency SLA breach
--	--------	---------------------------

<code>milvus_querynode_cache_hit_rate</code>	<85%	Insufficient Query Node memory
--	------	--------------------------------

<code>milvus_querynode_sq_req_count</code>	Rate drop >20%	Query Node crash or overload
--	----------------	------------------------------

<code>milvus_datanode_consume_msg_count</code>	Rate stall	Message bus backpressure
--	------------	--------------------------

<code>milvus_indexnode_index_task_count</code>	Queue depth >100	Index build falling behind ingestion
--	------------------	--------------------------------------

<code>etcd_server_has_leader</code>	0	etcd quorum lost — cluster is unavailable
-------------------------------------	---	---

Backup Strategy

Milvus does not have a built-in `BACKUP DATABASE` equivalent. Backup is a combination of:

- Object storage versioning:** Enable S3 versioning on your Milvus bucket. Segment files are immutable once written; versioning gives you point-in-time recovery without a dedicated backup job.
- etcd snapshot:** etcd holds all collection metadata, index definitions, and segment assignments. Snapshot it on a schedule using `etcdctl snapshot save`.
- Milvus Backup tool:** The official `milvus-backup` CLI can take logical backups of specific collections, streaming segment files to a separate S3 location.


```

# shell
# Install milvus-backup CLI
wget https://github.com/zilliztech/milvus-backup/releases/latest/download/milvus-
backup_linux_amd64.tar.gz
tar -xzf milvus-backup_linux_amd64.tar.gz

# Create a backup configuration
cat > backup-config.yaml << 'EOF'
milvus:
  address: localhost
  port: 19530

minio:
  address: localhost
  port: 9000
  bucketName: milvus-production
  backupBucketName: milvus-backups
  rootPath: files

backup:
  maxSegmentGroupSize: 2G
EOF

# Backup a specific collection
./milvus-backup create \
  --config backup-config.yaml \
  --name "document_chunks_backup_$(date +%Y%m%d)" \
  --colls document_chunks

# List available backups
./milvus-backup list --config backup-config.yaml

# Restore a collection from backup
./milvus-backup restore \
  --config backup-config.yaml \
  --name document_chunks_backup_20240115 \
  --suffix "_restored" # Restores as document_chunks_restored to avoid name
collision

```

Rolling Upgrades

Milvus uses a rolling upgrade process for Kubernetes deployments: update the Helm chart version, and Kubernetes will cycle Pods one at a time. The critical constraint is that coordinators (Root, Query, Data, Index) must be upgraded before worker nodes, because coordinators control the protocol version negotiation. The Milvus compatibility matrix guarantees that one minor version of difference between coordinators and workers is supported, enabling zero-downtime rolling upgrades within a minor version family.

```
# shell
# Check current deployment version
helm list --namespace milvus

# Upgrade Milvus with rolling restart – coordinator pods upgrade first
# because the Helm chart orders Pod updates by deployment dependency
helm upgrade milvus milvus/milvus \
  --namespace milvus \
  --values milvus-values.yaml \
  --version 4.2.0 \
  --set coordinator.activeStandby.enabled=true # Enable HA mode for zero-downtime

# Monitor rollout progress
kubectl rollout status deployment/milvus-querynode --namespace milvus
kubectl rollout status deployment/milvus-datanode --namespace milvus
```

19.8 Milvus vs. Alternatives: When to Choose What

With a full picture of Milvus's architecture and operational model, we can revisit the positioning question with engineering precision rather than marketing language.

Milvus vs. SQL Server Native Vectors

Choose SQL Server native vectors when: your vector dataset fits comfortably in SQL Server's working memory (under ~10M vectors for interactive latency), you need transactional consistency between vector and relational data, or your team's operational experience is entirely centered on SQL Server. The DiskANN index is competitive with Milvus HNSW up to this scale, and the operational simplicity of a single database cannot be overstated.

Choose Milvus when: you have more than ~50M vectors, you need multi-modal search (dense + sparse hybrid) as a first-class feature, or you need independent scaling of ingestion and query compute. Milvus also supports GPU-accelerated index building natively, which SQL Server does not.

Milvus vs. Pinecone

Pinecone is a managed service with a deliberately simple operational surface. If your organization lacks Kubernetes expertise or cannot dedicate engineering time to operating a distributed system, Pinecone's operational simplicity has real value. Milvus's advantages over Pinecone are: no data leaves your infrastructure (compliance), no per-vector storage costs (cost at scale), and full control over index parameters and hardware.

Milvus vs. Weaviate

Weaviate bundles vectorization, schema management, and a GraphQL query interface alongside vector search. It is a better fit for applications that want a single system to own the object store, the embedding pipeline, and the search layer. Milvus is a pure vector search engine — it expects you to bring your own embedding pipeline and metadata store. Teams that have already built embedding pipelines and want maximum search performance and scalability without a bundled object store tend to prefer Milvus.

Decision Matrix

Criterion	SQL Server vectors	Milvus	Pinecone	Weaviate
Operational complexity	Low	High	Very Low	Medium
Max practical scale	~50M vectors	Billions	Billions (managed)	Hundreds of millions
Hybrid dense+sparse	Limited	Native	Available	Available
Transactional consistency	Full ACID	Eventual	Eventual	Eventual
GPU acceleration	No	Yes	Managed	No
Self-hosted	Yes	Yes	No	Yes
Multi-tenancy support	Schema/RLS	Partition/Collection	Namespace	Tenant field

Key Takeaways

- → **Disaggregated architecture enables independent scaling**: Milvus separates proxy, coordinator, query, data, and index roles into independently scalable components. This means you can add Query Nodes to handle more concurrent searches without touching ingestion capacity, or add Index Nodes to accelerate index builds during a bulk load without competing with live query serving — a design that monolithic vector stores cannot match at the same scale.
- → **Index selection is a first-class engineering decision**: HNSW is the right default for most workloads under 500M vectors where low latency is the priority, IVF variants suit memory-constrained environments, and DISKANN enables billion-scale deployments that cannot fit in RAM. Treat the `M` and `ef` parameters as tunable knobs analogous to SQL Server's `MAXDOP` and buffer pool sizing — start with documented defaults, measure recall and latency, then adjust.

- → **Hybrid dense+sparse search consistently outperforms either alone**: For knowledge retrieval over technical content, combining HNSW-indexed dense embeddings with sparse inverted indexes and fusing results via RRF produces materially better recall than dense search alone. Milvus's native support for ``SPARSE_FLOAT_VECTOR`` fields and ``AnnSearchRequest`` fusion makes this pattern straightforward to implement without external reranking infrastructure.
- → **Multi-tenancy pattern selection has lasting consequences**: Collection-per-tenant gives the strongest isolation but limits active tenant count; partition-per-tenant scales to hundreds of tenants with selective loading; filter-based isolation scales to any tenant count but relies entirely on application-layer enforcement and degrades recall for small tenants in large shared collections. Choose based on your tenant count, data size per tenant, and compliance requirements — and choose before your first bulk load.
- → **Operational maturity requires owning the full stack**: Milvus's power comes with the full operational responsibility of a distributed system: Prometheus monitoring with latency and cache-hit-rate alerts, etcd snapshot backups alongside object storage versioning, and a disciplined Helm-based upgrade path that respects coordinator-before-worker ordering. Teams that underestimate this surface area typically encounter their first production incident before they have instrumentation in place to diagnose it.

Chapter 20: Benchmarking Vector Databases: Throughput, Latency, Recall

Choosing a vector database without benchmarking it is the equivalent of selecting a relational engine based on marketing collateral alone — you will eventually regret it. The vector database ecosystem is young, vendor claims are aggressive, and the gap between synthetic benchmark numbers and production behavior is wide enough to cause real incidents at scale. This chapter builds a rigorous framework for measuring what actually matters in vector search systems: query throughput, tail latency, and recall accuracy. You will learn how these three metrics interact (improving one often degrades another), how to construct benchmarks that reflect your actual workload rather than a vendor's best-case scenario, and how to interpret results from the most common tools in the ecosystem — SQL Server native vectors, Pinecone, Weaviate, Qdrant, and Milvus. By the end, you will have runnable harnesses, concrete thresholds to target, and a vocabulary for communicating vector database performance the same way you already communicate IOPS, latency percentiles, and cache hit ratios.

The Three Metrics That Define Vector Search Performance

Every vector database benchmark ultimately reduces to three numbers: **throughput**, **latency**, and **recall**. Understanding what each measures — and where the bodies are buried — is the prerequisite to designing any useful test.

Throughput is queries per second (QPS). It measures how many similarity searches the system can complete in a unit of time under sustained load. Like any throughput metric, it is meaningless without specifying concurrency. A system that delivers 500 QPS at 1 concurrent client and 480 QPS at 50 concurrent clients is a fundamentally different beast from one that delivers 500 QPS at 1 client and 12,000 QPS at 50 clients. Always measure throughput under the concurrency your application actually generates.

Latency is the time to complete a single query. In production systems, what matters is not mean latency but tail latency — specifically p95 and p99. A vector database with a 5ms mean and a 2,000ms p99 will make your application feel broken during load spikes. Most vector workloads are in the critical path of user-facing requests, which makes p99 the governing metric.

Recall is the fraction of true nearest neighbors that a query actually returns. It is the metric unique to approximate nearest neighbor (ANN) systems, and it has no direct analogue in traditional databases. If you ask for the 10 most similar vectors to a query, and the system returns 10 results but only 7 of them are genuinely in the true top-10, your recall@10 is 0.7. Recall is what separates ANN search from exact k-NN search — and it is what makes benchmarking vector databases harder than benchmarking relational systems.

The fundamental tension is this: **recall, throughput, and latency form a triangle where you can optimize any two by sacrificing the third.** Tightening recall requires searching more of the index graph, which increases latency and reduces throughput. Reducing latency requires searching less, which damages recall. This is not an implementation deficiency — it is the mathematical reality of ANN algorithms, and any honest benchmark must measure all three simultaneously.

The recall-latency curve is the most important diagnostic tool in vector database performance analysis. It plots recall (x-axis) against query latency (y-axis) at fixed concurrency, with each point representing a different index parameter setting. The database with the curve closest to the lower-right corner of the plot — high recall at low latency — wins that workload.

Practical thresholds for production systems:

| Metric | Acceptable | Good | Excellent |

| --- | --- | --- | --- |

| Recall@10 | > 0.90 | > 0.95 | > 0.99 |

| p99 latency (single node) | < 100ms | < 20ms | < 5ms |

| QPS (per node, 768-dim) | > 500 | > 2,000 | > 10,000 |

These numbers are for 768-dimensional embeddings (a common size from models like `text-embedding-3-small` and `all-MiniLM-L6-v2`) on a dataset of 1–10 million vectors.

Dimensionality, dataset size, and filter complexity all shift these thresholds dramatically.

Constructing a Realistic Benchmark Dataset

Vendor benchmarks almost always use ANN-benchmarks datasets — SIFT1M, GIST1M, or GloVe — because they are public, reproducible, and make their systems look good. Your workload is probably none of those. Building a benchmark that predicts production behavior requires matching four properties of your real dataset: dimensionality, distribution, dataset size, and filter selectivity.

Dimensionality directly determines index build time, memory consumption, and query latency. OpenAI's `text-embedding-3-large` produces 3072-dimensional vectors. Cohere's `embed-english-v3.0` produces 1024-dimensional vectors. Most open-source models produce 384–768 dimensions. Do not benchmark at 128 dimensions if you are going to deploy at 1536.

Distribution matters more than most engineers expect. ANN index structures (HNSW, IVF, ScaNN) exploit clustering in the vector space to skip regions during search. Randomly generated vectors are uniformly distributed in high-dimensional space and do not cluster — they are the worst case for ANN indices and the best case for demonstrating that an index provides no benefit. Real embedding vectors cluster by semantic topic. If your documents are all from a single domain (medical literature, legal contracts, software documentation), your vectors will cluster differently than a general-purpose corpus, and your recall numbers will differ from published benchmarks.

Filtered search is the single biggest gap between published benchmarks and production behavior. Most real applications combine vector similarity with metadata filters: "find the 10 most similar documents to this query, but only from this tenant's data, written after 2023-01-01, tagged as 'approved'." Pre-filtering (apply metadata filter first, then search remaining vectors) destroys recall when the filter is selective. Post-filtering (search vectors first, then apply filter) destroys recall when the filter eliminates most results. How a database handles this tradeoff — Qdrant's filterable HNSW, Weaviate's inverted-index integration, Pinecone's metadata filtering — is one of its most important production characteristics.

The following script generates a synthetic benchmark dataset that mirrors real embedding distributions by using actual sentence-transformers embeddings on sampled text:

```

# Python
# benchmark_dataset_generator.py
# Generates a realistic benchmark dataset using real embedding models
# Requires: sentence-transformers, numpy, h5py, datasets

import numpy as np
import h5py
import time
from sentence_transformers import SentenceTransformer
from datasets import load_dataset

def generate_benchmark_dataset(
    n_corpus: int = 100_000,
    n_queries: int = 1_000,
    model_name: str = "sentence-transformers/all-MiniLM-L6-v2",
    output_path: str = "benchmark_dataset.h5",
    batch_size: int = 512
):
    """
    Generates an HDF5 benchmark dataset in the ANN-benchmarks format.
    Uses real text embeddings for realistic vector distributions.

    HDF5 schema:
        /train    (n_corpus, dim)  float32 - corpus vectors
        /test     (n_queries, dim) float32 - query vectors
        /neighbors (n_queries, 100) int32  - ground truth (computed exactly)
        /distances (n_queries, 100) float32 - ground truth distances
    """
    print(f>Loading model: {model_name}")
    model = SentenceTransformer(model_name)
    dim = model.get_sentence_embedding_dimension()
    print(f>Embedding dimension: {dim}")

    # Load a real text corpus for realistic distributions
    print("Loading corpus text from Wikipedia sample...")
    dataset = load_dataset(
        "wikipedia", "20220301.en",
        split="train",
        streaming=True,
        trust_remote_code=True
    )

    texts = []
    for item in dataset:
        # Extract first 256 characters of each article as a passage
        texts.append(item["text"][:256])
        if len(texts) >= n_corpus + n_queries:
            break

    corpus_texts = texts[:n_corpus]
    query_texts = texts[n_corpus:n_corpus + n_queries]

    print(f>Encoding {n_corpus} corpus documents...")

```



```

t0 = time.time()
corpus_vectors = model.encode(
    corpus_texts,
    batch_size=batch_size,
    show_progress_bar=True,
    normalize_embeddings=True # Cosine similarity -> dot product
).astype(np.float32)
encode_time = time.time() - t0
print(f"Corpus encoding: {encode_time:.1f}s ({n_corpus/encode_time:.0f} docs/
sec)")

print(f"Encoding {n_queries} query vectors...")
query_vectors = model.encode(
    query_texts,
    batch_size=batch_size,
    show_progress_bar=True,
    normalize_embeddings=True
).astype(np.float32)

# Compute exact ground truth using brute-force cosine similarity
# For normalized vectors, cosine similarity = dot product
print("Computing exact ground truth (brute-force)...")
k_gt = 100 # Store top-100 ground truth for evaluating recall@1 through @100

neighbors = np.zeros((n_queries, k_gt), dtype=np.int32)
distances = np.zeros((n_queries, k_gt), dtype=np.float32)

# Process in batches to avoid OOM on large datasets
query_batch_size = 100
for i in range(0, n_queries, query_batch_size):
    batch = query_vectors[i:i + query_batch_size]
    # Matrix multiply: (batch_size, dim) x (dim, n_corpus) -> (batch_size,
n_corpus)
    sims = np.dot(batch, corpus_vectors.T)
    # Get top-k indices (descending similarity)
    top_k_idx = np.argpartition(sims, -k_gt, axis=1)[: , -k_gt:]
    for j, row_idx in enumerate(range(i, min(i + query_batch_size, n_queries))):
        top_k_sorted = top_k_idx[j][np.argsort(sims[j, top_k_idx[j]])[::-1]]
        neighbors[row_idx] = top_k_sorted
        distances[row_idx] = sims[j, top_k_sorted]

    if i % 1000 == 0:
        print(f" Ground truth: {i}/{n_queries} queries")

# Write HDF5 compatible with ann-benchmarks tooling
print(f"Writing dataset to {output_path}...")
with h5py.File(output_path, "w") as f:
    f.create_dataset("train", data=corpus_vectors, compression="gzip")
    f.create_dataset("test", data=query_vectors, compression="gzip")
    f.create_dataset("neighbors", data=neighbors, compression="gzip")
    f.create_dataset("distances", data=distances, compression="gzip")
    f.attrs["dimension"] = dim
    f.attrs["n_corpus"] = n_corpus

```

```

        f.attrs["n_queries"] = n_queries
        f.attrs["model"] = model_name
        f.attrs["metric"] = "cosine"

    print(f"Dataset written: {corpus_vectors.nbytes / 1e6:.1f} MB corpus, "
          f"{query_vectors.nbytes / 1e6:.1f} MB queries")
    return output_path

if __name__ == "__main__":
    generate_benchmark_dataset(
        n_corpus=100_000,
        n_queries=1_000,
        model_name="sentence-transformers/all-MiniLM-L6-v2",
        output_path="minilm_100k_benchmark.h5"
    )

```

This produces an HDF5 file in the same format as ANN-benchmarks, so you can drop it into standard tooling or write your own evaluation loop. The critical design choice here is using real embeddings on real text: your benchmark dataset should come from the same distribution as your production data, or your recall numbers will be wrong by 5–15 percentage points.

Building a Multi-Database Benchmark Harness

With a dataset in hand, the next step is a consistent measurement harness that runs the same queries against multiple databases and captures the metrics that matter. The harness must be fair: each database should be given its best-case index configuration, not a default one. Default configurations almost always favor index build time over query performance, which is the opposite of production priorities.

```

# Python
# vector_benchmark_harness.py
# Multi-database benchmark harness for SQL Server, Qdrant, and Pinecone
# Requires: pyodbc, qdrant-client, pinecone-client, numpy, h5py, scipy

import time
import numpy as np
import h5py
import statistics
import concurrent.futures
from dataclasses import dataclass, field
from typing import Callable, List, Optional
from contextlib import contextmanager

import pyodbc
from qdrant_client import QdrantClient
from qdrant_client.models import (
    Distance, VectorParams, PointStruct,
    HnswConfigDiff, OptimizersConfigDiff, SearchRequest
)
from pinecone import Pinecone, ServerlessSpec

# -----
# Data structures
# -----

@dataclass
class BenchmarkResult:
    database: str
    index_config: dict
    n_corpus: int
    n_queries: int
    concurrency: int
    k: int # Number of results requested
    # Throughput
    total_queries: int = 0
    total_time_sec: float = 0.0
    qps: float = 0.0
    # Latency (milliseconds)
    latencies_ms: List[float] = field(default_factory=list)
    p50_ms: float = 0.0
    p95_ms: float = 0.0
    p99_ms: float = 0.0
    # Recall
    recall_at_k: float = 0.0
    # Index build
    index_build_sec: float = 0.0
    index_size_mb: float = 0.0

def compute_recall(
    returned_ids: List[List[int]],

```

```

        ground_truth: np.ndarray,
        k: int
    ) -> float:
        """
        Computes recall@k: fraction of true top-k neighbors found in returned results.
        ground_truth shape: (n_queries, k_gt) – rows are sorted by similarity
        descending.
        """
        n_queries = len(returned_ids)
        recall_sum = 0.0
        for i in range(n_queries):
            true_top_k = set(ground_truth[i, :k].tolist())
            returned = set(returned_ids[i][:k])
            if len(true_top_k) > 0:
                recall_sum += len(true_top_k & returned) / len(true_top_k)
        return recall_sum / n_queries

# -----
# Database adapters
# -----

class SqlServerBenchmark:
    """
    SQL Server 2025 native vector benchmark adapter.
    Compares exact VECTOR_DISTANCE search (ground truth, 100% recall)
    against approximate VECTOR_SEARCH over a DiskANN index.
    """

    def __init__(self, connection_string: str, table_name: str =
"benchmark_vectors"):
        self.connection_string = connection_string
        self.table = table_name
        self.dim = 0

    @contextmanager
    def _conn(self):
        conn = pyodbc.connect(self.connection_string)
        try:
            yield conn
        finally:
            conn.close()

    def load_data(self, vectors: np.ndarray) -> float:
        """Loads corpus vectors and returns seconds elapsed."""
        n, dim = vectors.shape
        self.dim = dim
        t0 = time.time()
        with self._conn() as conn:
            cur = conn.cursor()
            cur.execute(f"IF OBJECT_ID('{self.table}', 'U') IS NOT NULL DROP TABLE
{self.table}")
            cur.execute(f"""

```

```

        CREATE TABLE {self.table} (
            id          INT NOT NULL CONSTRAINT PK_{self.table} PRIMARY KEY,
            embedding VECTOR({dim}) NOT NULL
        )
    """
    # Bulk insert: fast_executemany batches the parameter arrays
    cur.fast_executemany = True
    rows = [
        (i, "[" + ",".join(f"{v:.6f}" for v in vec) + "]")
        for i, vec in enumerate(vectors)
    ]
    cur.executemany(
        f"INSERT INTO {self.table} (id, embedding) "
        f"VALUES (?, CAST(? AS VECTOR({dim})))",
        rows
    )
    conn.commit()
    return time.time() - t0

def build_diskann_index(self, maxdop: int = 4) -> float:
    """Builds the DiskANN vector index, returns build time in seconds."""
    t0 = time.time()
    with self._conn() as conn:
        cur = conn.cursor()
        cur.execute(
            f"IF EXISTS (SELECT 1 FROM sys.indexes "
            f"WHERE name = '{self.table}_diskann_idx') "
            f"DROP INDEX {self.table}_diskann_idx ON {self.table}"
        )
        cur.execute(f"""
            CREATE VECTOR INDEX {self.table}_diskann_idx
            ON {self.table} (embedding)
            WITH (METRIC = 'cosine', TYPE = 'DiskANN', MAXDOP = {maxdop})
        """)
        conn.commit()
    return time.time() - t0

def run_queries(
    self,
    query_vectors: np.ndarray,
    k: int = 10,
    concurrency: int = 1,
    mode: str = "ann",
    top_n: int = 50
) -> tuple:
    """
    Returns (latencies_ms, all_returned_ids).
    mode='exact': VECTOR_DISTANCE ORDER BY – ground truth, 100% recall.
    mode='ann': VECTOR_SEARCH over the DiskANN index with TOP_N over-fetch.
    """
    n_queries = len(query_vectors)
    latencies = []
    all_ids = []

```

```

def single_query(i: int):
    vec = query_vectors[i]
    vec_lit = "[" + ",".join(f"{v:.6f}" for v in vec) + "]"
    conn = pyodbc.connect(self.connection_string)
    try:
        cur = conn.cursor()
        t0 = time.perf_counter()
        if mode == "exact":
            cur.execute(f"""
                SELECT TOP ({k}) id
                FROM {self.table}
                WHERE embedding IS NOT NULL
                ORDER BY VECTOR_DISTANCE('cosine', embedding,
                                          CAST(? AS VECTOR({self.dim})))
            """, (vec_lit,))
        else:
            # Join back to the table: VECTOR_SEARCH returns the key
            # columns plus s.distance (see Chapter 2)
            cur.execute(f"""
                SELECT TOP ({k}) t2.id
                FROM VECTOR_SEARCH(
                    TABLE = {self.table} AS t,
                    COLUMN = embedding,
                    SIMILAR_TO = CAST(? AS VECTOR({self.dim})),
                    METRIC = 'cosine',
                    TOP_N = ?
                ) AS s
                JOIN {self.table} AS t2 ON t2.id = s.id
                ORDER BY s.distance
            """, (vec_lit, top_n))
            ids = [row[0] for row in cur.fetchall()]
            elapsed = (time.perf_counter() - t0) * 1000
            return elapsed, ids
    finally:
        conn.close()

with concurrent.futures.ThreadPoolExecutor(max_workers=concurrency) as
executor:
    futures = [executor.submit(single_query, i) for i in range(n_queries)]
    for future in concurrent.futures.as_completed(futures):
        lat, ids = future.result()
        latencies.append(lat)
        all_ids.append(ids)

    return latencies, all_ids

class QdrantBenchmark:
    """
    Qdrant benchmark adapter.
    Qdrant is particularly strong at filtered ANN – test with and without filters.
    """

```

```

def __init__(
    self,
    url: str = "http://localhost:6333",
    collection_name: str = "benchmark"
):
    self.client = QdrantClient(url=url, timeout=60)
    self.collection = collection_name

def load_data(self, vectors: np.ndarray, m: int = 16, ef_construction: int =
100) -> float:
    """Loads corpus and builds HNSW index simultaneously."""
    n, dim = vectors.shape

    # Delete existing collection
    try:
        self.client.delete_collection(self.collection)
    except Exception:
        pass

    t0 = time.time()
    self.client.create_collection(
        collection_name=self.collection,
        vectors_config=VectorParams(
            size=dim,
            distance=Distance.COSINE
        ),
        hnsw_config=HnswConfigDiff(
            m=m,
            ef_construct=ef_construction,
            full_scan_threshold=10_000 # Use HNSW for collections > 10k vectors
        ),
        optimizers_config=OptimizersConfigDiff(
            indexing_threshold=0 # Index immediately, don't batch
        )
    )

    # Batch upload
    batch_size = 1000
    for start in range(0, n, batch_size):
        end = min(start + batch_size, n)
        batch = vectors[start:end]
        points = [
            PointStruct(
                id=start + i,
                vector=batch[i].tolist()
            )
            for i in range(len(batch))
        ]
        self.client.upsert(collection_name=self.collection, points=points)

    # Wait for indexing to complete
    while True:

```

```

        info = self.client.get_collection(self.collection)
        if info.status.value == "green":
            break
        time.sleep(1.0)

    return time.time() - t0

def run_queries(
    self,
    query_vectors: np.ndarray,
    k: int = 10,
    concurrency: int = 1,
    ef_search: int = 128
) -> tuple:
    latencies = []
    all_ids = []

    def single_query(i: int):
        vec = query_vectors[i].tolist()
        t0 = time.perf_counter()
        results = self.client.search(
            collection_name=self.collection,
            query_vector=vec,
            limit=k,
            search_params={"hnsf_ef": ef_search}
        )
        elapsed = (time.perf_counter() - t0) * 1000
        ids = [int(r.id) for r in results]
        return elapsed, ids

    with concurrent.futures.ThreadPoolExecutor(max_workers=concurrency) as
executor:
        futures = [executor.submit(single_query, i) for i in
range(len(query_vectors))]
        for future in concurrent.futures.as_completed(futures):
            lat, ids = future.result()
            latencies.append(lat)
            all_ids.append(ids)

    return latencies, all_ids

# -----
# Benchmark runner
# -----

def run_benchmark(
    dataset_path: str,
    sqlserver_conn: Optional[str] = None,
    qdrant_url: Optional[str] = None,
    concurrencies: List[int] = [1, 4, 16, 32],
    k: int = 10,
    top_n_values: List[int] = [10, 50, 100]

```



```

) -> List[BenchmarkResult]:
    """
    Runs the full benchmark suite against all configured databases.
    Returns a list of BenchmarkResult objects, one per (database, concurrency,
    TOP_N) combination.
    """
    # Load dataset
    print(f"Loading dataset from {dataset_path}...")
    with h5py.File(dataset_path, "r") as f:
        corpus = f["train"][:]
        queries = f["test"][:]
        ground_truth = f["neighbors"][:]

    n_corpus, dim = corpus.shape
    n_queries = len(queries)
    print(f"Dataset: {n_corpus:,} corpus vectors, {n_queries:,} queries, dim={dim}")

    results = []

    # — SQL Server native vectors —————
    if sqlserver_conn:
        print("\n=== SQL Server native vectors benchmark ===")
        mssql = SqlServerBenchmark(sqlserver_conn)

        print("Loading data...")
        load_time = mssql.load_data(corpus)
        print(f"  Data load: {load_time:.1f}s")

        print("Building DiskANN index (MAXDOP=4)...")
        build_time = mssql.build_diskann_index(maxdop=4)
        print(f"  Index build: {build_time:.1f}s")

        # Exact search gives the ground truth for recall measurement
        print("Measuring exact-search baseline (ground truth)...")
        _, exact_ids = mssql.run_queries(queries, k=k, concurrency=4, mode="exact")
        exact_recall = compute_recall(exact_ids, ground_truth, k)
        print(f"  Exact baseline recall: {exact_recall:.4f} (should be 1.0000)")

        for top_n in top_n_values:
            for concurrency in concurrencies:
                print(f"  Querying: TOP_N={top_n}, concurrency={concurrency}")
                latencies, returned_ids = mssql.run_queries(
                    queries, k=k,
                    concurrency=concurrency,
                    mode="ann",
                    top_n=top_n
                )
                recall = compute_recall(returned_ids, ground_truth, k)
                latencies_sorted = sorted(latencies)

                result = BenchmarkResult(
                    database="sqlserver",
                    index_config={"type": "DiskANN", "metric": "cosine", "top_n":

```

```

top_n},

        n_corpus=n_corpus,
        n_queries=n_queries,
        concurrency=concurrency,
        k=k,
        total_queries=n_queries,
        total_time_sec=sum(latencies) / 1000 / concurrency,
        qps=n_queries / (sum(latencies) / 1000 / concurrency),
        latencies_ms=latencies,
        p50_ms=latencies_sorted[int(0.50 * len(latencies_sorted))],
        p95_ms=latencies_sorted[int(0.95 * len(latencies_sorted))],
        p99_ms=latencies_sorted[int(0.99 * len(latencies_sorted))],
        recall_at_k=recall,
        index_build_sec=build_time
    )
    results.append(result)
    print(f"      QPS={result.qps:.0f}   p99={result.p99_ms:.1f}ms
recall={recall:.4f}")

# — Qdrant —————
if qdrant_url:
    print("\n=== Qdrant benchmark ===")
    qd = QdrantBenchmark(qdrant_url)

    for m, ef_c in [(16, 100), (32, 128)]:
        print(f"  Loading data and building HNSW (m={m},
ef_construction={ef_c})...")
        build_time = qd.load_data(corpus, m=m, ef_construction=ef_c)
        print(f"  Index build+load: {build_time:.1f}s")

        for ef_s in ef_search_values:
            for concurrency in concurrencies:
                print(f"  Querying: ef_search={ef_s},
concurrency={concurrency}")
                latencies, returned_ids = qd.run_queries(
                    queries, k=k,
                    concurrency=concurrency,
                    ef_search=ef_s
                )
                recall = compute_recall(returned_ids, ground_truth, k)
                latencies_sorted = sorted(latencies)

                result = BenchmarkResult(
                    database="qdrant",
                    index_config={"m": m, "ef_construction": ef_c, "ef_search":
ef_s},

                    n_corpus=n_corpus,
                    n_queries=n_queries,
                    concurrency=concurrency,
                    k=k,
                    total_queries=n_queries,
                    total_time_sec=sum(latencies) / 1000 / concurrency,
                    qps=n_queries / (sum(latencies) / 1000 / concurrency),

```

```

        latencies_ms=latencies,
        p50_ms=latencies_sorted[int(0.50 * len(latencies_sorted))],
        p95_ms=latencies_sorted[int(0.95 * len(latencies_sorted))],
        p99_ms=latencies_sorted[int(0.99 * len(latencies_sorted))],
        recall_at_k=recall,
        index_build_sec=build_time
    )
    results.append(result)
    print(f"    QPS={result.qps:.0f}  p99={result.p99_ms:.1f}ms
recall={recall:.4f}")

    return results

def print_summary_table(results: List[BenchmarkResult]):
    """Prints a formatted comparison table sorted by recall descending."""
    print("\n" + "=" * 90)
    print(f"{'Database':<12} {'Config':<30} {'Conc':>5} {'QPS':>8} {'p50ms':>7} "
          f"{'p99ms':>7} {'Recall':>8}")
    print("-" * 90)
    for r in sorted(results, key=lambda x: (-x.recall_at_k, x.p99_ms)):
        cfg = f"top_n={r.index_config.get('top_n','-')}"
        print(f"{r.database:<12} {cfg:<30} {r.concurrency:>5} {r.qps:>8.0f} "
              f"{r.p50_ms:>7.1f} {r.p99_ms:>7.1f} {r.recall_at_k:>8.4f}")
    print("=" * 90)

if __name__ == "__main__":
    import argparse
    parser = argparse.ArgumentParser()
    parser.add_argument("--dataset", default="minilm_100k_benchmark.h5")
    parser.add_argument("--sqlserver_conn", default=None)
    parser.add_argument("--qdrant_url", default=None)
    parser.add_argument("--concurrency", nargs="+", type=int, default=[1, 4, 16,
32])
    parser.add_argument("--k", type=int, default=10)
    args = parser.parse_args()

    results = run_benchmark(
        dataset_path=args.dataset,
        sqlserver_conn=args.sqlserver_conn,
        qdrant_url=args.qdrant_url,
        concurrencies=args.concurrencies,
        k=args.k
    )
    print_summary_table(results)

```

This harness is intentionally modular. Adding a Weaviate or Milvus adapter means implementing `load_data()` and `run_queries()` following the same contract, then registering the adapter in `run_benchmark()`. The key design choice is measuring latency from the client's perspective — including network round-trip and serialization — not from inside the database engine. That is the number your users experience.

SQL Server Native Vectors: Index Tuning for Production Throughput

SQL Server 2025 native vectors are the most common starting point for teams already on the Microsoft stack, which means you inherit connection pooling via `SqlConnection`, availability groups, ACID transactions, and decades of operational tooling. They are not the fastest vector option in pure QPS terms, but they are frequently the right answer when your dataset fits on your existing instances and your team already operates SQL Server.

Two access paths govern SQL Server vector performance: exact search with `VECTOR_DISTANCE` (a scan, 100% recall) and approximate search through the DiskANN index with `VECTOR_SEARCH`. The levers that matter most:

- → **Index metric match.** The `METRIC` in `CREATE VECTOR INDEX` must equal the metric argument in every `VECTOR_DISTANCE` / `VECTOR_SEARCH` call. A mismatch silently disables the index and drops you to a full scan.
- → **`TOP_N` over-fetch in `VECTOR_SEARCH`.** Higher `TOP_N` → better recall, higher latency. This is the recall-latency knob you sweep, analogous to `ef_search` in HNSW engines.
- → **`MAXDOP` at index build time.** DiskANN builds are CPU-intensive; set `MAXDOP` in the `WITH` clause to use multiple cores, then schedule builds in maintenance windows.
- → **Plan choice under filters.** With selective `WHERE` clauses the optimizer may choose a scan over the DiskANN index. Inspect actual execution plans and treat unexpected plan flips as the vector equivalent of a parameter-sniffing incident.

The relationship between `TOP_N` and the recall-latency tradeoff is the most important control knob during production tuning. The following T-SQL sweeps it against the exact-search baseline:

```

-- SQL Server 2025: sweep TOP_N to find the recall-latency knee point
DECLARE @q VECTOR(1536) = '[0.1, 0.2, ...]';

-- TOP_N = 10: fast but lower recall
SELECT t2.id, s.distance
FROM VECTOR_SEARCH(
    TABLE = dbo.benchmark_vectors AS t,
    COLUMN = embedding,
    SIMILAR_TO = @q,
    METRIC = 'cosine',
    TOP_N = 10
) AS s
JOIN dbo.benchmark_vectors AS t2 ON t2.id = s.id
ORDER BY s.distance;

-- TOP_N = 50: balanced
SELECT t2.id, s.distance
FROM VECTOR_SEARCH(
    TABLE = dbo.benchmark_vectors AS t,
    COLUMN = embedding,
    SIMILAR_TO = @q,
    METRIC = 'cosine',
    TOP_N = 50
) AS s
JOIN dbo.benchmark_vectors AS t2 ON t2.id = s.id
ORDER BY s.distance;

-- TOP_N = 200: high recall, slower
SELECT t2.id, s.distance
FROM VECTOR_SEARCH(
    TABLE = dbo.benchmark_vectors AS t,
    COLUMN = embedding,
    SIMILAR_TO = @q,
    METRIC = 'cosine',
    TOP_N = 200
) AS s
JOIN dbo.benchmark_vectors AS t2 ON t2.id = s.id
ORDER BY s.distance;

-- Compare each run's result set against the exact ground truth:
SELECT TOP (10) id
FROM dbo.benchmark_vectors
WHERE embedding IS NOT NULL
ORDER BY VECTOR_DISTANCE('cosine', embedding, @q);

```

One SQL Server behavior that surprises engineers coming from pure vector databases: **the query optimizer can choose not to use the vector index**. If your query includes a highly selective WHERE clause, the optimizer may decide a scan is cheaper than an index seek followed by a filter. This is generally correct behavior for small result sets, but it means your p99 latency can spike when a predicate causes plan flips. The remedy is to examine actual

execution plans for queries that combine vector ranking with metadata filters, and to compare plans in a controlled test (the benchmark harness above makes this repeatable) to find the index-vs-scan crossover point.

Parallelism is another SQL Server lever that most engineers do not configure for vector workloads. DiskANN index builds accept `MAXDOP` directly, so a build can use multiple CPU cores:

```
-- SQL Server 2025: parallel DiskANN index build
CREATE VECTOR INDEX IX_benchmark_vectors_embedding
ON dbo.benchmark_vectors (embedding)
WITH (METRIC = 'cosine', TYPE = 'DiskANN', MAXDOP = 8);

-- Verify build parallelism and timing via the actual plan / Query Store
-- (Chapter 14 covers vector-specific wait and plan analysis)
```

The tradeoff with build parallelism is predictable: it shortens the maintenance window at the cost of CPU pressure on the instance. Schedule builds off-hours, and at high query concurrency prefer letting concurrency itself saturate CPU cores rather than over-parallelizing individual operations.

Weaviate and Milvus: Distributed Vector Search Benchmarking

When your dataset exceeds single-node capacity — typically above 50–100 million vectors at 768 dimensions on a well-provisioned machine — you need a distributed vector database. Weaviate and Milvus both offer horizontal scaling, but with different architectural philosophies that produce different benchmark characteristics.

Weaviate stores vectors alongside a rich object store and a built-in GraphQL/REST API. Its HNSW index is built per-shard, with shards distributed across nodes. The benchmarking implication is that inter-shard query fan-out adds latency that does not appear in single-node tests. When benchmarking Weaviate in a multi-node configuration, always include the scatter-gather latency by targeting the coordinator endpoint, not an individual shard node directly.

Milvus separates compute and storage more aggressively. Its architecture includes query nodes (which hold loaded segments in memory), data nodes (which manage write-ahead logs and compaction), and an S3-compatible object store for persistence. The benchmarking implication is that Milvus latency is bimodal: queries against loaded segments are fast (all data in DRAM), but queries that trigger segment loading from object storage are dramatically slower. Always warm the query nodes before recording benchmark numbers.

A minimal Weaviate benchmark query using the Python client:

```
# Python – Weaviate v4 client benchmark query
import weaviate
import time
import numpy as np

def benchmark_weaviate(
    host: str,
    collection_name: str,
    query_vectors: np.ndarray,
    k: int = 10,
    ef_search: int = 128
) -> tuple:
    """
    Returns (latencies_ms, all_returned_ids) for a Weaviate collection.
    ef parameter controls HNSW search depth at query time.
    """

    client = weaviate.connect_to_local(host=host)
    collection = client.collections.get(collection_name)

    latencies = []
    all_ids = []

    try:
        for vec in query_vectors:
            t0 = time.perf_counter()
            results = collection.query.near_vector(
                near_vector=vec.tolist(),
                limit=k,
                return_properties=[],          # Only need the ID for recall
                additional_properties=["id"],
                query_properties=None,
                # ef is set at collection level via schema, not per-query in v4
            )
            elapsed_ms = (time.perf_counter() - t0) * 1000
            latencies.append(elapsed_ms)
            all_ids.append([obj.uuid for obj in results.objects])
    finally:
        client.close()

    return latencies, all_ids
```

One Weaviate-specific tuning parameter deserves explicit attention: the `ef` parameter in the collection's HNSW configuration controls search depth at query time, similar to the `TOP_N` over-fetch in SQL Server's `VECTOR_SEARCH`. Unlike SQL Server, where `TOP_N` is a per-query argument, Weaviate requires updating the collection's `efConfig` via the schema API to

change it without restarting. This makes dynamic ef sweeping more cumbersome in Weaviate — plan your benchmark parameter grid before starting rather than adjusting on the fly.

Filtered Search Benchmarking: The Production Reality Check

The scenario that breaks most vendor benchmark claims is filtered search. A realistic e-commerce or SaaS application rarely searches all vectors — it searches vectors belonging to a specific user, product category, date range, or tenant. The interaction between the vector index and the metadata filter is where architectural differences between databases become most visible.

To measure filtered search performance, extend the benchmark dataset with synthetic metadata and test at several selectivity levels:


```

# Python
# filtered_benchmark.py
# Tests recall and latency under metadata filtering at multiple selectivity levels

import numpy as np
import time
from qdrant_client import QdrantClient
from qdrant_client.models import (
    FieldCondition, Filter, MatchValue,
    Range, PointStruct, PayloadSchemaType
)

def load_qdrant_with_metadata(
    client: QdrantClient,
    collection_name: str,
    vectors: np.ndarray,
    n_categories: int = 100
):
    """
    Assigns each vector a random category_id (0..n_categories-1).
    Creates a payload index for fast filtered search.
    """
    n = len(vectors)
    categories = np.random.randint(0, n_categories, size=n)

    batch_size = 1000
    for start in range(0, n, batch_size):
        end = min(start + batch_size, n)
        points = [
            PointStruct(
                id=start + i,
                vector=vectors[start + i].tolist(),
                payload={"category_id": int(categories[start + i])}
            )
            for i in range(end - start)
        ]
        client.upsert(collection_name=collection_name, points=points)

    # Create payload index so Qdrant can use filterable HNSW
    client.create_payload_index(
        collection_name=collection_name,
        field_name="category_id",
        field_schema=PayloadSchemaType.INTEGER
    )

    return categories

def benchmark_filtered_query(
    client: QdrantClient,
    collection_name: str,
    query_vector: np.ndarray,

```

```

        category_id: int,
        k: int = 10,
        ef_search: int = 128
    ) -> tuple:
        """
        Returns (latency_ms, result_ids) for a filtered query.
        """
        query_filter = Filter(
            must=[
                FieldCondition(
                    key="category_id",
                    match=MatchValue(value=category_id)
                )
            ]
        )
        t0 = time.perf_counter()
        results = client.search(
            collection_name=collection_name,
            query_vector=query_vector.tolist(),
            query_filter=query_filter,
            limit=k,
            search_params={"hnsw_ef": ef_search}
        )
        elapsed_ms = (time.perf_counter() - t0) * 1000
        ids = [int(r.id) for r in results]
        return elapsed_ms, ids

def run_filtered_benchmark(
    client: QdrantClient,
    collection_name: str,
    query_vectors: np.ndarray,
    categories: np.ndarray,
    n_categories_list: list = [10, 100, 1000],
    k: int = 10
):
    """
    Tests how latency and result count vary with filter selectivity.
    With 100k vectors and n_categories=10, each bucket has ~10k vectors (low
    selectivity).
    With n_categories=1000, each bucket has ~100 vectors (high selectivity – hard
    case).
    """
    print(f"\n{'Selectivity':<20} {'n_categories':<15} {'Avg Latency':>12} "
          f"{'p99 Latency':>12} {'Avg Results':>12}")
    print("-" * 75)

    for n_cat in n_categories_list:
        # Re-assign categories at this granularity
        n_vectors = len(categories)
        cat_ids = categories % n_cat # Map existing categories to smaller range

        latencies = []

```

```

result_counts = []

for i, qvec in enumerate(query_vectors):
    target_cat = int(cat_ids[i % n_vectors])
    lat, ids = benchmark_filtered_query(
        client, collection_name, qvec, target_cat, k=k
    )
    latencies.append(lat)
    result_counts.append(len(ids))

selectivity = f"~{n_vectors // n_cat:}, vectors/cat"
p99 = sorted(latencies)[int(0.99 * len(latencies))]
avg_lat = sum(latencies) / len(latencies)
avg_results = sum(result_counts) / len(result_counts)

print(f"{selectivity:<20} {n_cat:<15} {avg_lat:>11.1f}ms "
      f"{p99:>11.1f}ms {avg_results:>12.1f}")

```

When you run this across databases, the output reveals something that synthetic benchmarks hide entirely: **highly selective filters (small buckets) cause different failure modes in different databases**. In SQL Server, a highly selective filter may cause the optimizer to skip the DiskANN index in favor of a scan — which is slow but returns perfect recall. In Weaviate, the `ef` parameter drives how aggressively the system oversamples before filtering, so high selectivity requires increasing `ef` dramatically to maintain recall, which increases latency. In Qdrant, the filterable HNSW implementation tracks which vectors satisfy payload predicates inside the graph structure itself, which generally maintains recall at lower latency penalty — but requires the payload index to be created before ingest, not after.

These are the kinds of behavioral differences that do not appear in any vendor benchmark and that will determine which database actually fits your production query pattern.

Interpreting Benchmark Results: Common Traps

Running a benchmark is easier than interpreting one. There are several systematic traps that cause engineers to make wrong decisions from correct data.

The warm-cache trap. Most vector databases cache recently accessed graph nodes or segments in memory. Running your benchmark query set repeatedly will produce increasingly optimistic latency numbers as the cache warms. Always measure latency on a cold or realistically warm cache, which means either clearing the OS page cache before each run (`echo 3 > /proc/sys/vm/drop_caches` on Linux) or pre-warming with a separate query set that does not overlap your test queries.

The single-client trap. If your application generates concurrent queries — and production applications almost always do — measuring at concurrency=1 gives you latency numbers that are optimistic by a factor of 2–5x for CPU-bound workloads. Measure at concurrency levels matching your peak application concurrency, not your baseline.

The index-not-built trap. Some databases defer index construction until a threshold of writes is reached, or until a compaction job runs. Milvus, for example, will serve queries from growing segments using brute-force until a segment reaches a size threshold and gets indexed. If you run your benchmark immediately after loading, you may be benchmarking brute-force search, not ANN. Always verify index status before measuring. For Milvus, call `utility.index_building_progress()`. For Qdrant, check that `collection_info.status == "green"`. For SQL Server, confirm the vector index exists in `sys.indexes` and that row counts match the expected corpus size before measuring.

The metric distance trap. Cosine similarity and Euclidean distance produce different nearest neighbor orderings, and recall numbers computed against ground truth using the wrong metric will be meaningless. Always confirm that your ground truth was computed with the same distance metric as your index. The benchmark harness above uses cosine similarity (dot product on normalized vectors) throughout — verify this is consistent if you adapt it.

The dimensionality compression trap. Some databases support scalar quantization (SQ8, SQ4) or product quantization (PQ) to compress vectors and reduce memory usage. These techniques trade recall for memory efficiency, sometimes dramatically. When comparing a quantized Milvus index against an unquantized SQL Server DiskANN index, you are not comparing like for like — you are comparing a memory-efficient approximate index against a full-precision one. Measure both the quantized and unquantized configurations if your target system uses quantization.

Continuous Benchmarking in CI/CD

A benchmark run before database selection is necessary but insufficient. Vector database performance degrades as data grows, as index parameters drift, and as query distributions shift when your application evolves. Embedding continuous benchmarks in your CI/CD pipeline catches regressions before they reach production.

The practical approach is a lightweight nightly benchmark job that runs a fixed 1,000-query set against your staging database and emits three metrics to your observability platform: recall@10, p99 latency, and QPS at your standard concurrency. When any of these crosses a threshold — recall drops below 0.95, p99 exceeds 50ms, QPS falls below 1,000 — the pipeline fires an alert.

```

# Python
# ci_benchmark.py
# Lightweight benchmark for CI integration – exits non-zero on regression

import sys
import h5py
import numpy as np
import pyodbc
import time

RECALL_THRESHOLD = 0.95
P99_LATENCY_THRESHOLD_MS = 50.0
MIN_QPS = 500.0

def run_ci_benchmark(dsn: str, dataset_path: str, k: int = 10) -> bool:
    """
    Returns True if all thresholds pass, False on regression.
    """
    with h5py.File(dataset_path, "r") as f:
        queries = f["test"][:100] # Use 100 queries for speed in CI
        ground_truth = f["neighbors"][:]

    latencies = []
    returned_ids = []

    DIM = 1536 # must match benchmark_vectors.embedding
    conn = pyodbc.connect(dsn)
    cur = conn.cursor()
    for vec in queries:
        vec_str = "[" + ",".join(f"{v:.6f}" for v in vec) + "]"
        t0 = time.perf_counter()
        cur.execute(
            f"SELECT TOP ({k}) t2.id "
            f"FROM VECTOR_SEARCH( "
            f"      TABLE = benchmark_vectors AS t, "
            f"      COLUMN = embedding, "
            f"      SIMILAR_TO = CAST(? AS VECTOR({DIM})), "
            f"      METRIC = 'cosine', TOP_N = 50) AS s "
            f"JOIN benchmark_vectors AS t2 ON t2.id = s.id "
            f"ORDER BY s.distance",
            (vec_str,)
        )
        ids = [row[0] for row in cur.fetchall()]
        latencies.append((time.perf_counter() - t0) * 1000)
        returned_ids.append(ids)
    conn.close()

    # Compute metrics
    n = len(queries)
    recall = sum(
        len(set(ground_truth[i, :k]) & set(returned_ids[i][:k])) / k
        for i in range(n)
    ) / n

```

```

latencies_sorted = sorted(latencies)
p99 = latencies_sorted[int(0.99 * n)]
qps = n / (sum(latencies) / 1000)

print(f"Recall@{k}: {recall:.4f} (threshold: {RECALL_THRESHOLD})")
print(f"p99 latency: {p99:.1f}ms (threshold: {P99_LATENCY_THRESHOLD_MS}ms)")
print(f"QPS: {qps:.0f} (threshold: {MIN_QPS})")

passed = True
if recall < RECALL_THRESHOLD:
    print(f"FAIL: recall {recall:.4f} below threshold {RECALL_THRESHOLD}")
    passed = False
if p99 > P99_LATENCY_THRESHOLD_MS:
    print(f"FAIL: p99 {p99:.1f}ms exceeds threshold {P99_LATENCY_THRESHOLD_MS}
ms")
    passed = False
if qps < MIN_QPS:
    print(f"FAIL: QPS {qps:.0f} below threshold {MIN_QPS}")
    passed = False

return passed

if __name__ == "__main__":
    import argparse
    parser = argparse.ArgumentParser()
    parser.add_argument("--dsn", required=True)
    parser.add_argument("--dataset", required=True)
    args = parser.parse_args()

    passed = run_ci_benchmark(args.dsn, args.dataset)
    sys.exit(0 if passed else 1)

```

Embed this in your CI pipeline as a step that runs after migrations and before promotion to staging. The dataset file is committed to artifact storage (not to the repository, as it can be hundreds of megabytes) and fetched at CI runtime. The thresholds are checked into source control alongside your application code, which means recall regressions are treated with the same severity as functional test failures.

Choosing Between Databases: A Decision Framework

After running benchmarks across SQL Server native vectors, Pinecone, Weaviate, Qdrant, and Milvus, most teams face the same question: given similar recall numbers at similar latency, how do I choose? Pure throughput numbers rarely decide the question. The decision rests on four operational dimensions that benchmarks illuminate but do not automatically answer.

Dataset size and growth trajectory. If your dataset is under 5 million vectors today and is expected to grow slowly, SQL Server native vectors on a well-provisioned instance handle the workload with no additional infrastructure. The operational simplicity of a single system is worth significant throughput headroom. If your dataset is already in the tens of millions and growing at a rate that will require horizontal scaling within twelve months, start with a distributed system (Qdrant cluster, Weaviate with multiple shards, or Milvus) even if a single node would technically suffice today. Migration from single-node to distributed is painful; choosing the right architecture at 20M vectors is far cheaper than migrating at 80M.

Filter complexity. If your queries are predominantly unfiltered — similarity search against the full corpus — most databases perform comparably at equivalent hardware spend. If your queries are predominantly filtered by high-cardinality metadata (per-user, per-tenant, per-category with thousands of categories), Qdrant's filterable HNSW is worth measuring carefully, because the recall-under-filter numbers differ meaningfully from unfiltered recall at the same ef parameter values.

Operational maturity of your team. Milvus is architecturally sophisticated — it has separate query nodes, index nodes, data nodes, and a message queue — and running it reliably requires understanding all of those components. If your team has not operated distributed systems with complex topology before, the operational surface area is a real risk. Pinecone eliminates that risk entirely by being a managed service at the cost of query customization and egress pricing. There is no architecture decision that is universally correct; the right choice accounts for the humans who will be on call when it fails at 2am.

Latency budget and tail behavior. Managed services like Pinecone have network round-trips to their API endpoints that add fixed latency regardless of index parameters. If your application requires sub-10ms p99 and it is co-located in the same cloud region as the vector database, self-hosted Qdrant or SQL Server native vectors will consistently outperform managed Pinecone on raw latency, because you eliminate the inter-service network hop. If your latency budget is 100ms and reliability is more important than raw speed, the managed operational model wins on total cost of ownership.

Key Takeaways

- → ****Recall, latency, and throughput form an inseparable triangle:**** you can optimize any two by sacrificing the third. Every benchmark must measure all three simultaneously at the same index configuration, or the results are misleading. The recall-latency curve — not a single headline QPS number — is the authoritative diagnostic.

- → **Benchmark datasets must match production distributions:** vectors generated by real embedding models on domain-relevant text cluster differently from synthetic random vectors, and recall numbers differ by 5–15 percentage points between synthetic and realistic datasets. Always build benchmarks from actual embeddings of actual text.
- → **Filtered search is where vendor benchmarks diverge most from production behavior:** the interaction between vector ANN indices and metadata filters produces dramatically different recall and latency outcomes across databases. Measure filtered recall at multiple selectivity levels before trusting any performance claim for a workload that combines similarity search with attribute filtering.
- → **Index state validation is a prerequisite for valid measurements:** systems like Milvus, Qdrant, and SQL Server can serve queries without using the ANN index (exact-scan fallback), producing benchmark numbers that reflect brute-force search rather than ANN search. Always confirm index build completion and warm query caches with a separate, non-overlapping query set before recording measurements.
- → **Continuous benchmarking in CI/CD converts a one-time evaluation into an ongoing regression signal:** committing recall thresholds, p99 latency limits, and minimum QPS values alongside application code ensures that schema migrations, index parameter changes, data growth, and dependency upgrades are caught before they degrade production search quality.

Chapter 21: What is RAG and Why It Needs a Vector Database

21.1 The Problem RAG Solves

Large language models are trained on data that ends at a fixed point in time, and they have no access to your organization's private documents, your product catalog, or last week's incident reports. Ask a model about your company's refund policy and it will either refuse, answer from stale public knowledge, or — most dangerously — invent a confident-sounding policy that does not exist. This is the grounding problem: LLMs generate fluent text, but fluency is not knowledge.

Retrieval-Augmented Generation (RAG) solves it with an architectural trick rather than a bigger model. Instead of asking the model to remember your data, you retrieve the relevant documents at query time and hand them to the model as context: "Here are the three most relevant passages from our knowledge base. Answer the user's question using only these." The model stops inventing and starts summarizing. Accuracy on domain-specific questions jumps from unreliable to production-grade, answers become citable (you can show which documents were used), and updating knowledge means updating rows in a table — not retraining a model.

The retrieval step is where the vector database earns its place. Keyword search retrieves documents that share vocabulary with the question; a vector index retrieves documents that share *meaning* with it. For a support chatbot, a runbook search, or a code assistant, meaning is what you need.

21.2 RAG Architecture: The Three-Stage Pipeline

Every production RAG system has the same three stages, and as a DBA you will own most of the first two:

- → **Stage 1 — Ingest and index.** Documents are chunked (Chapter 22), each chunk is embedded by a model (Chapter 23), and the chunks plus vectors are stored in SQL Server with a DiskANN index. This is an ETL pipeline with all the usual operational concerns: idempotency, incremental updates, monitoring.

- → **Stage 2 — Retrieve.** The user's question is embedded with the same model, and the vector index returns the top-k most similar chunks — optionally fused with full-text results (Chapter 11) and filtered by tenant or permissions (Chapter 12). This is a database query with a latency SLA.
- → **Stage 3 — Generate.** The retrieved chunks are assembled into a prompt and sent to the LLM, which produces the answer. This is the only stage that is not really a database problem — which is why the first two determine whether your RAG system succeeds.

```
graph LR
    A[Documents] --> B[Chunk + Embed  
ingest pipeline]
    B --> C[(SQL Server  
VECTOR + DiskANN)]
    D[User question] --> E[Embed question]
    E --> C
    C --> F[Top-k chunks  
+ full-text fusion]
    F --> G[LLM prompt  
with context]
    G --> H[Answer + citations]
```

Notice the asymmetry: Stage 1 runs in batch or trickle-feed and can tolerate minutes of latency; Stage 2 runs per user question and must answer in tens of milliseconds. The database design in this book — chunk tables, vector indexes, hybrid fusion queries — is all about making Stage 2 fast and correct while keeping Stage 1 operable.

21.3 Embeddings: The Translation Layer Between Text and Geometry

An embedding model translates text into a point in high-dimensional space such that texts with similar meaning land near each other. "How do I reset my password?" and "I forgot my login credentials" produce vectors with small cosine distance despite sharing almost no words. The model is the translation layer; the vector column is where the translation is stored; the index is how you search the translations.

A hard rule with embeddings: **the model used during ingestion and the model used at query time must be identical.** Querying a SQL Server vector column populated with OpenAI `text-embedding-3-small` vectors using Cohere embeddings produces meaningless results. This is the vector-database equivalent of a character encoding mismatch — the numbers exist but represent different things. Store the model name and version alongside every vector (a column, not a comment), and treat model upgrades as full re-ingestion events (Chapter 23 covers the migration pattern).

The dimension must also match the column: a `VECTOR(1536)` column rejects 1024-dimension vectors at write time. That strictness is a feature — it turns a silent quality corruption into a loud, catchable error.

21.4 Vector Search Mechanics and Why It Beats Full-Text for Retrieval

Full-text search in SQL Server (`CONTAINS` , `FREETEXTTABLE`) is excellent at what it does — stemming, ranking by term frequency, phrase proximity — but it cannot close the semantic gap. A vector query retrieves chunks by geometric proximity in embedding space, which encodes meaning rather than vocabulary. Documents with similar meaning cluster together regardless of their exact words.

In SQL Server, you express the similarity ranking with `VECTOR_DISTANCE` :

```
-- SQL Server 2025: RAG retrieval query (exact search path)
DECLARE @q VECTOR(1536) = '[0.12, 0.87, ...]'; -- embedded user question

SELECT TOP (5)
    ChunkId, DocTitle, ChunkText,
    VECTOR_DISTANCE('cosine', Embedding, @q) AS Distance
FROM dbo.RagChunks
WHERE Embedding IS NOT NULL
ORDER BY Distance;
-- Feed ChunkText of the TOP 5 rows to the LLM as context
```

`VECTOR_DISTANCE` returns distance, so `ORDER BY` ascending returns the most similar chunks first. For production RAG at scale, the same logic runs through the DiskANN index via `VECTOR_SEARCH` (Chapter 2), and the hybrid fusion with full-text from Chapter 11 catches the exact-term cases — error numbers, SKUs, proper nouns — where pure semantic search is sloppy.

The practical scaling guidance: for RAG corpora under ~1 million chunks, exact search or a modest DiskANN index on SQL Server handles retrieval with room to spare. Beyond ~10 million chunks with aggressive latency SLAs, evaluate the dedicated engines (Chapters 15–19) — but keep the chunk store and the retrieval procedure in the same shape, so a migration changes the access path, not the architecture.

21.5 Building a Production RAG Pipeline on SQL Server

Here is a minimal but complete RAG pipeline: OpenAI embeddings, SQL Server storage, and a retrieval function you can call from any application. The ingest side chunks documents, embeds them, and upserts with `MERGE` so re-runs are idempotent; the query side embeds the question and retrieves the top chunks for the LLM prompt.

```

# Python – minimal RAG pipeline on SQL Server
# Requirements: openai, pyodbc, tiktoken
import openai, pyodbc, tiktoken

openai_client = openai.OpenAI()
conn = pyodbc.connect(
    "DRIVER={ODBC Driver 18 for SQL Server};SERVER=localhost;"
    "DATABASE=ragdb;Trusted_Connection=yes;Encrypt=yes;"
    "TrustServerCertificate=yes"
)

SCHEMA = """
CREATE TABLE dbo.RagChunks (
    ChunkId      BIGINT IDENTITY(1,1) NOT NULL CONSTRAINT PK_RagChunks PRIMARY KEY,
    DocId        NVARCHAR(100) NOT NULL,
    DocTitle     NVARCHAR(300) NULL,
    ChunkIndex   INT NOT NULL,
    ChunkText    NVARCHAR(MAX) NOT NULL,
    Embedding    VECTOR(1536) NOT NULL,
    ModelName    NVARCHAR(100) NOT NULL CONSTRAINT DF_RagChunks_Model
                DEFAULT ('text-embedding-3-small'),
    UpdatedAt    DATETIME2(0) NOT NULL CONSTRAINT DF_RagChunks_Updated
                DEFAULT SYSUTCDATETIME(),
    CONSTRAINT UQ_RagChunks_Doc_Chunk UNIQUE (DocId, ChunkIndex)
);
CREATE VECTOR INDEX IX_RagChunks_Embedding
    ON dbo.RagChunks (Embedding)
    WITH (METRIC = 'cosine', TYPE = 'DiskANN', MAXDOP = 4);
"""

def vec_lit(emb): return "[" + ",".join(str(v) for v in emb) + "]"

def ingest_document(doc_id, title, text, model="text-embedding-3-small"):
    enc = tiktoken.encoding_for_model(model)
    tokens, chunks, i = enc.encode(text), [], 0
    while i < len(tokens):
        # 512-token chunks, 50 overlap
        chunks.append(enc.decode(tokens[i:i + 512])); i += 462
    embs = [e.embedding for e in
            openai_client.embeddings.create(input=chunks, model=model).data]
    cur = conn.cursor()
    for idx, (chunk, emb) in enumerate(zip(chunks, embs)):
        cur.execute("""MERGE dbo.RagChunks AS t
            USING (SELECT ? AS DocId, ? AS ChunkIndex, ? AS DocTitle,
                ? AS ChunkText, CAST(? AS VECTOR(1536)) AS Embedding,
                ? AS ModelName) AS s
            ON t.DocId = s.DocId AND t.ChunkIndex = s.ChunkIndex
            WHEN MATCHED THEN UPDATE SET ChunkText = s.ChunkText,
                Embedding = s.Embedding, ModelName = s.ModelName,
                UpdatedAt = SYSUTCDATETIME()
            WHEN NOT MATCHED THEN INSERT
                (DocId, ChunkIndex, DocTitle, ChunkText, Embedding, ModelName)
            VALUES (s.DocId, s.ChunkIndex, s.DocTitle, s.ChunkText,
                s.Embedding, s.ModelName);""",

```

```

        (doc_id, idx, title, chunk, vec_lit(emb), model))
    conn.commit()
    return len(chunks)

def retrieve(question, top_k=5, model="text-embedding-3-small"):
    q = vec_lit(openai_client.embeddings.create(
        input=[question], model=model).data[0].embedding)
    cur = conn.cursor()
    cur.execute("""SELECT TOP (?) DocTitle, ChunkText,
                    VECTOR_DISTANCE('cosine', Embedding,
                    CAST(? AS VECTOR(1536))) AS Distance
                FROM dbo.RagChunks
                WHERE Embedding IS NOT NULL AND ModelName = ?
                ORDER BY Distance""", (top_k, q, model))
    return cur.fetchall()  # hand ChunkText rows to your LLM as context

```

Two details worth noting. The `ModelName` filter in `retrieve()` enforces the same-model rule at query time — during a model migration, old and new vectors can coexist in the table without cross-contamination (Chapter 23). And `MERGE` makes ingestion idempotent: re-running the pipeline on an unchanged document updates timestamps and embeddings instead of duplicating chunks.

21.6 RAG Failure Modes and What the Database Can Fix

Most RAG failures are data problems, not model problems, and the database layer is where you fix them:

- → **Stale context.** The model answers from last quarter's policy because nobody re-ingested the new one. Fix: treat ingestion as a streaming ETL pipeline, not a batch job. Use document change detection (file modification timestamps, CDC events, webhook triggers) to re-embed only changed documents, and monitor the gap between source-document freshness and chunk freshness as an SLI.
- → **Retrieved the wrong chunks.** The similarity search returned loosely related text. Fix: hybrid fusion with full-text (Chapter 11), metadata pre-filters for tenant and document type (Chapter 12), and a distance threshold that refuses to answer when the best match is still far away — "I don't have information about that" beats a hallucinated answer.
- → **Chunks too coarse or too fine.** A 2,000-token chunk dilutes the answer; a 50-token chunk lacks context. Fix: chunking strategy is a database design decision (Chapter 22) — store chunk size and overlap as metadata so you can measure which granularity serves your queries.

- → **Permission leakage.** The retriever returns a chunk the user is not allowed to see, and the LLM happily summarizes it. Fix: enforce row-level security or tenant predicates in the retrieval query itself, never in application code after retrieval (Chapter 12).
- → **Model drift.** Someone swaps the embedding model and retrieval quality silently degrades. Fix: the `ModelName` column plus a startup check that the query-time model matches the indexed model.

21.7 Choosing Your Vector Database for RAG

SQL Server 2025 native vectors

Best when: your corpus fits on your existing SQL Server infrastructure (under ~5 million chunks for comfortable DiskANN performance), your team already operates SQL Server, and you want transactional consistency between vector search and relational data — chunk text, metadata, permissions, and audit trails in one database, one backup, one security model.

Operational model: built into the engine, also available on Azure SQL Database and Azure SQL Managed Instance. Full ACID guarantees. JOINS between vector results and relational tables work naturally, and hybrid full-text-plus-vector search (Chapter 11) needs no second system.

Limitations: single-engine scale; the DiskANN index is in preview with a rebuild-to-ingest write path. ANN throughput at very high QPS will not match a purpose-built distributed engine.

Pinecone

Best when: you want zero vector infrastructure to operate and your scale or QPS justifies a managed service. Namespaces give you cheap multi-tenancy; the sparse-plus-dense hybrid model is native.

Limitations: managed-service cost at scale, network hop latency, and your chunk text lives elsewhere — keep SQL Server as the authoritative store and Pinecone IDs as foreign keys (Chapter 15).

Weaviate / Qdrant / Milvus

Best when: you need self-hosted scale into the tens or hundreds of millions of vectors, first-class hybrid search (Weaviate), sophisticated payload filtering (Qdrant), or billion-scale disaggregated architecture (Milvus). More operationally complex than SQL Server native vectors — the GraphQL or gRPC APIs have a learning curve for teams fluent in T-SQL, and HNSW memory requirements at scale are significant.

21.8 Chunking Strategies and Their Database Implications

Chunking — how you split documents before embedding — is the highest-leverage retrieval decision after the embedding model itself, and it is fundamentally a database design problem: chunk size determines row counts, row width determines page density and index size, and overlap determines how much of your storage is redundant by design. Chapter 22 covers the full taxonomy (fixed-size, overlap design, structural and semantic splitting, parent-child retrieval) with the sizing math you need before you create the table. The one rule to carry forward: store chunking parameters as metadata on every chunk, because the day you want to measure which granularity serves your queries best, you will need them.

Key Takeaways

- → RAG grounds LLMs in your data: retrieve relevant chunks at query time and hand them to the model as context. The retrieval stages — ingest/index and retrieve — are database problems, and they determine whether the system succeeds.
- → The ingestion model and the query-time model must be identical; enforce it with a `ModelName` column and a startup check. Dimension mismatches fail loudly at the `VECTOR(n)` column — that strictness is a feature.
- → Vector search retrieves by meaning where full-text retrieves by vocabulary; production RAG uses both, fused (Chapter 11), with tenant and permission predicates enforced in the retrieval query (Chapter 12).
- → Treat ingestion as streaming ETL with idempotent upserts (`MERGE`), monitor chunk freshness as an SLI, and refuse to answer when the best match is too distant.
- → SQL Server 2025 native vectors are the right RAG store when your corpus fits your existing infrastructure and you want one transactional system for chunks, metadata, and permissions; graduate to Pinecone, Weaviate, Qdrant, or Milvus when scale or QPS demands it — the chunk schema and retrieval shape transfer.

Chapter 22: Chunking Strategies for RAG

22.1 Why Chunking Is a Database Design Problem

Embedding models have context windows, but the reason you chunk documents goes deeper than model limits: a chunk is the unit of retrieval, and retrieval quality is dominated by chunk granularity. A 5,000-token document embedded as a single vector averages its meaning into mush — the vector for a runbook chapter says "this is vaguely about failover" instead of pointing at the exact step the user needs. A 20-token fragment, on the other hand, may be so context-free ("click OK to continue") that it matches everything and explains nothing.

Chunking is also a storage design problem, and you should think of it like page layout. Chunk size determines row count; row width (chunk text plus a 1,536-dimension vector is roughly 6 KB before overhead) determines page density; overlap determines how much of your storage is redundant by design; and chunk metadata columns determine what you can filter on at query time. Every decision in this chapter shows up in the table definition, the index size, and the retrieval latency.

The performance characteristics are well-understood: for a corpus of one million documents averaging 2,000 tokens, 512-token chunks with 64-token overlap produce approximately 4.3 million chunks. At 1,536 dimensions, that is roughly 25 GB of raw vector data before indexing overhead. These numbers matter when sizing your SQL Server vector table and planning the DiskANN index build.

22.2 Fixed-Size Chunking: Mechanics and Limits

Fixed-size chunking splits text every N tokens — typically 256 or 512 — regardless of content boundaries. It is the simplest strategy, the easiest to reason about for capacity planning, and the correct default when you are standing up your first RAG pipeline. Its weakness is that it cuts mid-sentence and mid-thought: a procedure split across a chunk boundary becomes two vectors that each half-answer the question.

```
# Python – fixed-size chunking by token count (tiktoken)
import tiktoken
enc = tiktoken.get_encoding("cl100k_base")

def fixed_chunks(text, size=512, overlap=64):
    toks = enc.encode(text)
    step = size - overlap
    return [enc.decode(toks[i:i + size])
            for i in range(0, len(toks), step)
            if toks[i:i + size]]
```

Size the chunks to your retrieval budget, not your model's context window. The retrieved chunks must fit in the LLM prompt alongside the question — five 512-token chunks consume ~2,500 tokens of context before the model writes a word. If your answers need tight precision (error codes, configuration values), prefer 256-token chunks and retrieve more of them; if your answers need narrative context (runbook procedures, design docs), prefer 512.

22.3 Overlap Design: What Gets Repeated and Why

Overlap exists for one reason: boundary sentences. Without overlap, the sentence straddling a chunk boundary belongs fully to neither chunk's vector, and a question about exactly that sentence retrieves two mediocre matches instead of one good one. A 10–15% overlap (50–64 tokens on a 512-token chunk) captures boundary context at a modest storage cost.

Overlap is redundant storage by design — budget for it. A 512-token chunk with 64-token overlap stores ~12.5% more vectors than the non-overlapping equivalent, and the DiskANN index grows with it. That is a fair price for retrieval quality, but do not let overlap creep to 50%: at that point you are paying for an index that is half duplicate, and the near-duplicate vectors distort the similarity ranking by crowding out genuinely different content.

22.4 Structural and Semantic Splitting

When your documents have structure, use it. Markdown headers, HTML sections, PDF outline levels, and code function boundaries are natural chunk edges — a chunk that starts at a heading and ends before the next heading is a self-contained unit of meaning, which is exactly what you want a vector to represent. Recursive character splitting (split on double newline, then single newline, then sentence, then word) approximates this without a real parser and is a solid middle ground.

Semantic splitting goes one step further: embed sentences individually, then group consecutive sentences whose embeddings are similar, cutting where the topic shifts. It produces chunks that respect meaning rather than formatting, at the cost of an extra embedding pass per document. Reserve it for high-value corpora where retrieval quality is the product — support knowledge bases, legal documents — not for bulk log ingestion where fixed-size chunking is fine.

Whichever strategy you choose, store the strategy as metadata. A `ChunkStrategy` column ('fixed-512', 'recursive-md', 'semantic') lets you A/B chunking strategies in production — serve 90% of traffic from one strategy, 10% from another, and measure which retrieves better answers. You cannot run that experiment if the strategy is only in your ETL script's git history.

22.5 Ingesting Chunks into SQL Server and Pinecone

The following pipeline chunks a document, embeds each chunk with OpenAI, and writes to SQL Server as the authoritative store — with the option to mirror vectors into Pinecone for pure-vector workloads at scale. SQL Server holds chunk text, metadata, and vectors in one transaction; Pinecone, when used, holds vectors plus a chunk ID foreign key.

```
-- SQL Server: chunk table with strategy metadata and versioning
CREATE TABLE dbo.DocumentChunks (
    ChunkId          BIGINT IDENTITY(1,1) NOT NULL
        CONSTRAINT PK_DocumentChunks PRIMARY KEY,
    DocId            NVARCHAR(100) NOT NULL,
    DocVersion       INT NOT NULL CONSTRAINT DF_DocumentChunks_Version DEFAULT (1),
    ChunkIndex       INT NOT NULL,
    ChunkStrategy    NVARCHAR(40) NOT NULL,          -- 'fixed-512', 'recursive-md', ...
    ChunkSize        INT NOT NULL,                  -- tokens, for sizing analysis
    OverlapTokens    INT NOT NULL,
    ChunkText        NVARCHAR(MAX) NOT NULL,
    Embedding        VECTOR(1536) NOT NULL,
    ModelName        NVARCHAR(100) NOT NULL,
    TenantId         INT NOT NULL,
    CreatedAt        DATETIME2(0) NOT NULL
        CONSTRAINT DF_DocumentChunks_Created DEFAULT SYSUTCDATETIME(),
    CONSTRAINT UQ_DocumentChunks_DocVer_Chunk
        UNIQUE (DocId, DocVersion, ChunkIndex)
);
CREATE INDEX IX_DocumentChunks_Tenant_Doc
    ON dbo.DocumentChunks (TenantId, DocId, DocVersion);
-- Build the DiskANN index AFTER the bulk ingest, not before
-- (preview write constraints favor index-after-load):
CREATE VECTOR INDEX IX_DocumentChunks_Embedding
    ON dbo.DocumentChunks (Embedding)
    WITH (METRIC = 'cosine', TYPE = 'DiskANN', MAXDOP = 8);
```

```

# Python – chunk, embed, upsert to SQL Server (pyodbc), mirror to Pinecone
import openai, pyodbc, tiktoken
from pinecone import Pinecone # optional: only if you mirror vectors

openai_client = openai.OpenAI()
pc = Pinecone(api_key="...") if MIRROR_TO_PINECONE else None
conn = pyodbc.connect("DRIVER={ODBC Driver 18 for SQL Server};"
                      "SERVER=localhost;DATABASE=ragdb;Trusted_Connection=yes;"
                      "Encrypt=yes;TrustServerCertificate=yes")

def vec_lit(emb): return "[" + ",".join(str(v) for v in emb) + "]"

def ingest(doc_id, version, text, tenant_id,
           strategy="fixed-512", size=512, overlap=64,
           model="text-embedding-3-small"):
    enc = tiktoken.get_encoding("cl100k_base")
    toks, step = enc.encode(text), size - overlap
    chunks = [enc.decode(toks[i:i + size])
               for i in range(0, len(toks), step) if toks[i:i + size]]
    embs = [e.embedding for e in
            openai_client.embeddings.create(input=chunks, model=model).data]
    cur = conn.cursor()
    for idx, (chunk, emb) in enumerate(zip(chunks, embs)):
        cur.execute("""MERGE dbo.DocumentChunks AS t
            USING (SELECT ? AS DocId, ? AS DocVersion, ? AS ChunkIndex,
                          ? AS ChunkStrategy, ? AS ChunkSize, ? AS OverlapTokens,
                          ? AS ChunkText, CAST(? AS VECTOR(1536)) AS Embedding,
                          ? AS ModelName, ? AS TenantId) AS s
            ON t.DocId = s.DocId AND t.DocVersion = s.DocVersion
            AND t.ChunkIndex = s.ChunkIndex
            WHEN MATCHED THEN UPDATE SET ChunkText = s.ChunkText,
            Embedding = s.Embedding, ModelName = s.ModelName
            WHEN NOT MATCHED THEN INSERT (DocId, DocVersion, ChunkIndex,
            ChunkStrategy, ChunkSize, OverlapTokens, ChunkText,
            Embedding, ModelName, TenantId)
            VALUES (s.DocId, s.DocVersion, s.ChunkIndex, s.ChunkStrategy,
            s.ChunkSize, s.OverlapTokens, s.ChunkText,
            s.Embedding, s.ModelName, s.TenantId);""",
            (doc_id, version, idx, strategy, size, overlap,
            chunk, vec_lit(emb), model, tenant_id))
    if pc: # mirror: vector + metadata only, text stays in SQL Server
        pc.Index("rag-vectors").upsert([
            {"id": f"{doc_id}:v{version}:{idx}",
             "values": emb,
             "metadata": {"tenant": tenant_id, "chunk_id_hint": idx}}])
    conn.commit()
    return len(chunks)

```

One Pinecone-specific concern worth flagging: metadata stored alongside vectors is not free — each vector's metadata counts toward index size. Storing full chunk text in Pinecone metadata avoids a round-trip to SQL Server but inflates costs for large corpora. The pattern above keeps authoritative text in SQL Server and only a chunk ID hint in Pinecone metadata, so retrieval does one key lookup back to the database for the rows it actually uses.

22.6 Querying Across Chunk Strategies: Parent-Child Retrieval

Small chunks retrieve precisely; large chunks provide context. Parent-child retrieval gets both: index small child chunks (128–256 tokens) for the similarity search, but return their larger parent chunks (1,024+ tokens) to the LLM. The retrieval is precise, the context is rich.

```
-- SQL Server: parent-child retrieval
-- ChildChunks(ChunkId, ParentId, Embedding VECTOR(1536), ...)
-- ParentChunks(ParentId PK, ParentText NVARCHAR(MAX), ...)
DECLARE @q VECTOR(1536) = '[0.12, 0.87, ...]'; -- embedded question

SELECT DISTINCT p.ParentId, p.ParentText
FROM (
    SELECT TOP (20) c.ParentId
    FROM dbo.ChildChunks AS c
    WHERE c.TenantId = @tenant
        AND VECTOR_DISTANCE('cosine', c.Embedding, @q) < @maxDistance
    ORDER BY VECTOR_DISTANCE('cosine', c.Embedding, @q)
) AS hits
JOIN dbo.ParentChunks AS p ON p.ParentId = hits.ParentId;
-- DISTINCT collapses multiple child hits under the same parent
```

Chunk-level filtering: the metadata columns you attach at ingestion become filter dimensions at query time. In SQL Server this is a standard T-SQL `WHERE` clause combined with the vector similarity predicate:

```
-- SQL Server: retrieve chunks from a specific document version,
-- tenant-scoped, with similarity ranking
DECLARE @q VECTOR(1536) = '[0.12, 0.87, ...]';

SELECT TOP (10) ChunkId, ChunkText,
    VECTOR_DISTANCE('cosine', Embedding, @q) AS Distance
FROM dbo.DocumentChunks
WHERE TenantId = @tenant
    AND DocId = @doc
    AND DocVersion = (SELECT MAX(DocVersion)
        FROM dbo.DocumentChunks WHERE DocId = @doc)
ORDER BY Distance;
```

Pre-filtering on metadata before the vector similarity scan is an important performance decision. With a DiskANN index, a filtered ANN search that is highly selective can be inefficient — for high-selectivity filters (eliminating more than ~90% of rows), an exact scan over the filtered subset via `VECTOR_DISTANCE` with `ORDER BY` often outperforms the ANN path. Measure both on your data (Chapter 20) rather than assuming the index always wins.

22.7 Choosing a Chunking Strategy by Document Type

- → **Runbooks and procedures:** structural splitting on headings. Steps must stay together; a boundary cut inside a procedure is the most damaging failure mode.
- → **Q&A and support articles:** one chunk per question-answer pair. The natural unit of the content is the natural unit of retrieval.
- → **Long narrative docs and design records:** fixed 512-token chunks with 64-token overlap, or parent-child with 256-token children for precision.
- → **Code:** split on function/class boundaries, keep signatures with bodies, and store the file path and symbol name as metadata columns. Never split mid-function.
- → **Tables and structured data:** one chunk per row-group with the column headers repeated in each chunk's text — a vector of bare numbers without headers retrieves on noise.

Match chunk granularity to query granularity. If your users ask narrow factual questions ("What is the default `max server memory` for this workload?"), small precise chunks (128–256 tokens) will outperform large ones. If your users ask synthesis questions ("Summarize the tradeoffs between rowstore and columnstore for this workload"), larger chunks (512+ tokens) serve better. When one application serves both, the parent-child pattern from Section 22.6 is the production-grade solution.

22.8 Measuring Chunking Quality

Chunking quality is measured the same way retrieval quality is measured: with a labeled evaluation set of questions and the chunks that should answer them. Build 50–100 representative questions against your corpus, record the expected answer fragment for each, then score strategies by the fraction of questions whose expected fragment appears in the top-k retrieved chunks.

```
-- SQL Server: evaluation samples live next to the chunks
CREATE TABLE dbo.ChunkEvalSamples (
    SampleId INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_ChunkEvalSamples PRIMARY KEY,
    Question NVARCHAR(1000) NOT NULL,
    QuestionEmbedding VECTOR(1536) NOT NULL, -- embedded with the same model
    ExpectedFragment NVARCHAR(500) NOT NULL, -- must appear in a top-k chunk
    StrategyUnderTest NVARCHAR(40) NOT NULL
);
-- Score: for each sample, retrieve top-5 chunks of that strategy and check
-- whether ExpectedFragment appears; aggregate the hit rate per strategy.
```

Run the evaluation per strategy using the `ChunkStrategy` column to partition, and re-run it whenever you change chunk size, overlap, or the embedding model. A strategy change that improves one query type often degrades another — the eval set is how you see the tradeoff before your users do.

22.9 Operational Considerations: Re-Chunking and Schema Evolution

Chunking strategies evolve: you start with fixed-512, discover your runbooks need structural splitting, and now half your corpus uses the old strategy. The `DocVersion` column in the schema above makes this manageable — re-chunk a document as a new version, backfill in batches, and switch the retrieval query to `MAX(DocVersion)` only when the new version is complete. Old versions stay queryable during the cutover and provide an instant rollback.

The practical response to a model upgrade is the same pattern one level up: store the model name and version alongside each chunk and treat model upgrades as full re-ingestion events. Pinecone's approach is to create a new index for the upgraded model and switch query traffic only after full re-ingestion, using a feature flag. In SQL Server, add a second vector column for the new model and backfill it in batches before switching the query path:

```
-- SQL Server: side-by-side model migration
ALTER TABLE dbo.DocumentChunks ADD Embedding_v2 VECTOR(3072) NULL;
-- Backfill in batches (online, throttled):
-- UPDATE TOP (10000) dbo.DocumentChunks
-- SET Embedding_v2 = ... WHERE Embedding_v2 IS NULL;
CREATE VECTOR INDEX IX_DocumentChunks_Embedding_v2
    ON dbo.DocumentChunks (Embedding_v2)
    WITH (METRIC = 'cosine', TYPE = 'DiskANN', MAXDOP = 8);
-- Switch retrieval to Embedding_v2, then:
-- ALTER TABLE dbo.DocumentChunks DROP COLUMN Embedding;
-- DROP INDEX IX_DocumentChunks_Embedding ON dbo.DocumentChunks;
```

Key Takeaways

- → Chunking is a database design decision: chunk size sets row counts, row width sets page density, overlap is redundant storage by design, and metadata columns become query-time filters.
- → Start with fixed-size chunking (256–512 tokens, 10–15% overlap); graduate to structural splitting where documents have headings, and semantic splitting where retrieval quality is the product.
- → Store chunking strategy, size, overlap, model name, and document version as columns — they are the dimensions you will filter, evaluate, and migrate on.
- → Use parent-child retrieval when one application serves both precise factual questions and synthesis questions: small children for ranking, large parents for context.
- → For highly selective metadata filters, an exact `VECTOR_DISTANCE` scan over the filtered subset can beat the DiskANN path — measure both (Chapter 20).
- → Re-chunk as new document versions and migrate embedding models side-by-side with a second vector column; never mix models or strategies in one query path without a version predicate.

Chapter 23: Embedding Model Selection

What an Embedding Model Actually Does (and Why It Matters to a DBA)

An embedding model is a function from text to a fixed-length array of floats. Everything downstream — your `VECTOR(n)` column width, your storage footprint, your index build time, your retrieval quality — is determined by two properties of that function: its output dimension and its semantic quality on your domain. The model is chosen once per corpus and lives in your schema as a column width and a `ModelName` value, which makes model selection a schema decision with the permanence of a data type choice. Get it wrong and you face cascading problems: mismatched dimensions between stored and query vectors, poor retrieval quality on domain-specific content, unnecessary API costs at query time, or latency profiles that make the application unusable.

This chapter walks through the three major embedding ecosystems — OpenAI, Cohere, and Sentence Transformers — examining their output dimensions, quality characteristics, cost structures, and operational tradeoffs. You will leave with a framework for matching model to workload and production-ready integration code for each provider against SQL Server.

OpenAI Embeddings: Production Workhorse

OpenAI's `text-embedding-3-small` (1,536 dimensions) and `text-embedding-3-large` (3,072 dimensions) are the default choice for most production RAG systems: strong general quality, simple API, and per-token pricing that is cheap at ingest scale. The 3-series supports dimension truncation — you can request fewer dimensions than the native output (e.g., 512 from the large model) with modest quality loss, which lets you trade storage and index size against retrieval quality without changing models.

```
# Python – batch-embed documents and store in SQL Server
import openai, pyodbc

openai_client = openai.OpenAI()
conn = pyodbc.connect("DRIVER={ODBC Driver 18 for SQL Server};"
                     "SERVER=localhost;DATABASE=ragdb;Trusted_Connection=yes;"
                     "Encrypt=yes;TrustServerCertificate=yes")

def load_documents_to_sqlserver(docs, model="text-embedding-3-small",
                               dimensions=None):
    """Batch-embed documents and store in SQL Server native VECTOR."""
    kwargs = {"model": model}
    if dimensions:
        # 3-series supports truncation
        kwargs["dimensions"] = dimensions
    resp = openai_client.embeddings.create(input=[d["text"] for d in docs],
                                           **kwargs)
    dim = dimensions or (3072 if "large" in model else 1536)
    cur = conn.cursor()
    for doc, e in zip(docs, resp.data):
        vec = "[" + ",".join(str(v) for v in e.embedding) + "]"
        cur.execute("""INSERT INTO dbo.Documents
                      (DocId, Title, BodyText, Embedding, ModelName)
                      VALUES (?, ?, ?, CAST(? AS VECTOR(%d)), ?)""" % dim,
                    (doc["id"], doc["title"], doc["text"], vec, model))
    conn.commit()
    return len(docs)
```

The SQL Server schema for this setup:

```
-- SQL Server 2025
CREATE TABLE dbo.Documents (
    DocKey    BIGINT IDENTITY(1,1) NOT NULL
        CONSTRAINT PK_Documents PRIMARY KEY,
    DocId     NVARCHAR(100) NOT NULL,
    Title     NVARCHAR(300) NULL,
    BodyText  NVARCHAR(MAX) NOT NULL,
    Embedding VECTOR(1536) NOT NULL,          -- matches text-embedding-3-small
    ModelName NVARCHAR(100) NOT NULL
        CONSTRAINT DF_Documents_Model DEFAULT ('text-embedding-3-small'),
    CreatedAt DATETIME2(0) NOT NULL
        CONSTRAINT DF_Documents_Created DEFAULT SYSUTCDATETIME()
);
-- DiskANN index for ANN search (exact VECTOR_DISTANCE for smaller corpora):
CREATE VECTOR INDEX IX_Documents_Embedding
    ON dbo.Documents (Embedding)
    WITH (METRIC = 'cosine', TYPE = 'DiskANN', MAXDOP = 4);
-- To drop and rebuild after a model change:
-- DROP INDEX IX_Documents_Embedding ON dbo.Documents;
```

Operational notes: OpenAI is an external API dependency on your ingest path — batch aggressively (the API accepts large input arrays), retry with backoff, and checkpoint progress so a failed batch does not restart a million-document ingest. At query time, embedding the question adds one API round-trip to retrieval latency; for latency-sensitive paths, cache question embeddings or use a local model for the query side while keeping OpenAI vectors for the corpus — never mixing the two in one query.

Cohere Embeddings: Retrieval-Optimized with Input Type Awareness

Cohere's `embed-v4` family is purpose-built for retrieval: it accepts an `input_type` parameter (`search_document` vs `search_query`) that tunes the embedding for its role, which measurably improves retrieval on asymmetric tasks — short question against long document — where generic embeddings underperform. It also offers native quantized outputs, reducing storage and memory without a separate compression step.

```
# Python – Cohere embeddings with input-type awareness into SQL Server
import cohere, pyodbc

co = cohere.ClientV2(api_key="...")
conn = pyodbc.connect("DRIVER={ODBC Driver 18 for SQL Server};"
                     "SERVER=localhost;DATABASE=ragdb;Trusted_Connection=yes;"
                     "Encrypt=yes;TrustServerCertificate=yes")

def load_with_cohere(docs, model="embed-v4.0", dim=1536):
    resp = co.embed(texts=[d["text"] for d in docs], model=model,
                    input_type="search_document",
                    embedding_types=["float"])
    cur = conn.cursor()
    for doc, emb in zip(docs, resp.embeddings.float_):
        vec = "[" + ", ".join(str(v) for v in emb[:dim]) + "]"
        cur.execute("""INSERT INTO dbo.Documents
                      (DocId, Title, BodyText, Embedding, ModelName)
                      VALUES (?, ?, ?, CAST(? AS VECTOR(1536)), ?)""",
                    (doc["id"], doc["title"], doc["text"], vec, model))
    conn.commit()

# Query side uses the matching input type:
# co.embed(texts=[question], model=model,
#           input_type="search_query", embedding_types=["float"])
```

The input-type distinction is a contract you must honor in both directions: documents embedded as `search_document` must be queried with `search_query` vectors. Enforce it with the `ModelName` column plus an input-type column if you serve both roles from one table.

Sentence Transformers: Local Embeddings for Privacy and Cost Control

Sentence Transformers (e.g., `all-MiniLM-L6-v2` at 384 dimensions, `bge-large-en-v1.5` at 1,024) run on your own infrastructure — no API dependency, no per-token cost, no data leaving your network. For regulated industries and air-gapped environments, that settles the decision by itself. The tradeoff is quality on general-domain queries (top commercial models still lead on broad benchmarks) and the operational burden of GPU inference at ingest scale.

```
# Python – local Sentence Transformers embeddings into SQL Server
from sentence_transformers import SentenceTransformer
import pyodbc

st_model = SentenceTransformer("BAAI/bge-large-en-v1.5") # 1024 dims
conn = pyodbc.connect("DRIVER={ODBC Driver 18 for SQL Server};"
                      "SERVER=localhost;DATABASE=ragdb;Trusted_Connection=yes;"
                      "Encrypt=yes;TrustServerCertificate=yes")

def load_with_st(docs, batch_size=256):
    cur = conn.cursor()
    for i in range(0, len(docs), batch_size):
        batch = docs[i:i + batch_size]
        embs = st_model.encode([d["text"] for d in batch],
                                normalize_embeddings=True) # cosine-ready
        for doc, emb in zip(batch, embs):
            vec = "[" + ",".join(str(v) for v in emb) + "]"
            cur.execute("""INSERT INTO dbo.Documents
                           (DocId, Title, BodyText, Embedding, ModelName)
                           VALUES (?, ?, ?, CAST(? AS VECTOR(1024)), ?)""",
                        (doc["id"], doc["title"], doc["text"], vec,
                         "bge-large-en-v1.5"))
    conn.commit() # checkpoint per batch: resumable ingest
```

Note the column width changes with the model: 384-dimension MiniLM needs `VECTOR(384)`, 1,024-dimension BGE needs `VECTOR(1024)`. The dimension is part of the schema, so a model change is a schema migration — plan for the side-by-side pattern in the versioning section below rather than an in-place ALTER.

Comparing Models: Quality, Cost, and Latency at Production Scale

Compare on three axes, in this order:

- → **Quality on your data.** Public benchmarks (MTEB) rank models on generic corpora; your corpus is not generic. Build the evaluation harness from Chapter 22.8 — 50–100 questions with expected fragments — and score candidate models on it. A smaller model that understands your domain vocabulary (medical, legal, industrial) routinely beats a larger general model.
- → **Total cost.** API models charge per token at ingest and per question at query time; local models charge GPU-hours once plus engineering time. At 100M documents, API ingest cost dominates; at 10K questions per second, API query cost and latency dominate. Model the full three-year cost, not the per-1K-token price.
- → **Latency profile.** API embedding adds a network round-trip to every retrieval; local inference adds GPU scheduling latency but no network. Measure p99 end-to-end (embed question + vector search + LLM), not embedding latency in isolation.

Query to estimate current vector storage in a SQL Server table:

```
-- SQL Server: estimate vector storage per model dimension
SELECT ModelName,
       COUNT(*) AS Vectors,
       COUNT(*) * 1536 * 4 / 1024 / 1024 AS ApproxVectorMB_1536d,
       COUNT(*) * 1024 * 4 / 1024 / 1024 AS ApproxVectorMB_1024d
FROM dbo.Documents
GROUP BY ModelName;
-- float32 = 4 bytes/dimension; refine with actual dimension per model.
-- Add sys.dm_db_partition_stats reserved pages for the full picture
-- including row overhead, indexes, and allocation.
```

If storage is the binding constraint, consider quantization: Cohere provides native quantized outputs, and Sentence Transformers float32 outputs can be post-processed into lower-precision representations before storage. Quantization trades recall for memory — measure both configurations on your eval set (Chapter 20) before committing the schema to a narrower representation.

Model Versioning and Migration Strategy

Embedding models get updated — vendors release new versions, and your own fine-tunes evolve. Because vectors from different models are mutually meaningless, a model change is a full re-ingestion event, not an in-place update. The production pattern is side-by-side: add a second vector column, backfill in batches, switch the query path, then drop the old column.

```
-- SQL Server: side-by-side embedding model migration
ALTER TABLE dbo.Documents ADD Embedding_v2 VECTOR(3072) NULL;
-- Backfill in throttled batches during off-peak:
-- UPDATE TOP (50000) dbo.Documents
-- SET Embedding_v2 = ... -- computed by the re-embedding job
-- WHERE Embedding_v2 IS NULL;
CREATE VECTOR INDEX IX_Documents_Embedding_v2
  ON dbo.Documents (Embedding_v2)
  WITH (METRIC = 'cosine', TYPE = 'DiskANN', MAXDOP = 8);
-- Point retrieval at Embedding_v2 + ModelName = 'text-embedding-3-large',
-- verify eval scores, then:
-- DROP INDEX IX_Documents_Embedding ON dbo.Documents;
-- ALTER TABLE dbo.Documents DROP COLUMN Embedding;
-- EXEC sp_rename 'dbo.Documents.Embedding_v2', 'Embedding', 'COLUMN';
```

During the backfill, the `ModelName` predicate in the retrieval query guarantees old and new vectors never mix in one result set. Size the backfill batches to your maintenance window and monitor transaction log growth — re-embedding a large corpus is one of the heaviest write workloads the table will ever see.

Choosing the Right Model: A Decision Framework

- → **Starting out, general domain?** OpenAI `text-embedding-3-small` at 1,536 dimensions. Best quality-per-dollar, simplest operations, and the dimension truncation option gives you a storage escape hatch.
- → **Asymmetric retrieval (short queries, long documents)?** Cohere with `search_document / search_query` input types. The role-aware embeddings earn their keep on Q&A workloads.
- → **Regulated data or no external APIs?** Sentence Transformers on your own GPUs. Accept the quality gap on generic queries or close it with domain fine-tuning.
- → **Massive scale, cost-dominated?** Truncated OpenAI dimensions (512) or quantized Cohere outputs; validate recall on your eval set before committing.
- → **Sub-50ms retrieval SLA?** Local model for the query path (no API round-trip) regardless of which model embedded the corpus — as long as query and corpus models match.

End-to-End Example: Evaluating Two Models Side by Side

The decision framework narrows the field; the eval set makes the call. Embed your Chapter 22.8 evaluation questions with both candidate models, retrieve top-5 chunks per model from side-by-side columns, and score hit rate plus p99 retrieval latency:

```

-- SQL Server: A/B retrieval quality between two embedding models
-- EvalQuestions(QuestionId, QuestionText, ExpectedFragment,
--               QSmall VECTOR(1536), QLarge VECTOR(3072))
-- Documents has Embedding (small model) and Embedding_v2 (large model)
DECLARE @qid INT = 42;

SELECT 'small' AS Model,
      (SELECT COUNT(*) FROM (
        SELECT TOP (5) d.BodyText
        FROM dbo.Documents AS d
        WHERE d.ModelName = 'text-embedding-3-small'
        ORDER BY VECTOR_DISTANCE('cosine', d.Embedding, q.QSmall)
      ) AS r WHERE r.BodyText LIKE '%' + q.ExpectedFragment + '%') AS Hit
FROM dbo.EvalQuestions AS q WHERE q.QuestionId = @qid
UNION ALL
SELECT 'large',
      (SELECT COUNT(*) FROM (
        SELECT TOP (5) d.BodyText
        FROM dbo.Documents AS d
        WHERE d.ModelName = 'text-embedding-3-large'
        ORDER BY VECTOR_DISTANCE('cosine', d.Embedding_v2, q.QLarge)
      ) AS r WHERE r.BodyText LIKE '%' + q.ExpectedFragment + '%')
FROM dbo.EvalQuestions AS q WHERE q.QuestionId = @qid;
-- Aggregate Hit over all questions per model: higher hit rate wins,
-- with p99 latency from Chapter 20's harness as the tiebreaker.

```

Run this over the full eval set, not a handful of samples — model differences that are invisible at $n=10$ become decisive at $n=100$. Keep the eval set versioned alongside the schema: when the vendor ships the next model version, you re-run the same harness instead of re-litigating the decision from scratch.

Key Takeaways

- → Model selection is a schema decision: dimension becomes the `VECTOR(n)` column width, and the model name belongs in a column, not a comment.
- → OpenAI is the default for general workloads, Cohere earns its keep on asymmetric Q&A retrieval with input-type awareness, and Sentence Transformers win wherever data cannot leave your network.
- → Never mix models in one query path: enforce the same-model rule with a `ModelName` predicate, and honor input-type contracts (Cohere) in both directions.
- → Decide on your data, not public benchmarks: score candidates on a versioned eval set of your own questions, with p99 latency as the tiebreaker.

- → Migrate models side-by-side — second vector column, throttled backfill, new DiskANN index, verified eval scores, then drop the old column. A model change is a re-ingestion event, never an in-place update.

Chapter 24: Building a RAG Pipeline on SQL Server

Retrieval-augmented generation is the workload that made vector databases famous: instead of trusting a language model to recall facts from its training data, you retrieve the relevant documents from your own corpus and put them in the prompt. The model's job shrinks from "know everything" to "reason over this provided context" — hallucinations drop, answers become citable, and private data never needs to enter model training. When the corpus lives in SQL Server, the entire RAG pipeline — chunk storage, embedding ingestion, similarity retrieval, permission filtering, and prompt assembly — can run inside the database engine, with the LLM call as the only external step. This chapter builds that pipeline end to end.

Pipeline Architecture: Five Stages, One Database

A production RAG pipeline has five stages. Four of them belong in SQL Server; only the last one leaves it:

- → **1. Ingest and chunk.** Source documents land in `RagDocuments`; a chunking process splits them into retrievable units in `RagChunks` (schema from Chapter 8, strategies from Chapter 22).
- → **2. Embed.** An ingestion pipeline generates embeddings and stores them in the `VECTOR(1536)` column — batched, with the staging-and-quarantine pattern from Chapter 8.
- → **3. Retrieve.** At query time, embed the user's question, run similarity search (exact or DiskANN), apply permission and tenant filters, and return the top-k chunks. This is a stored procedure — the heart of the chapter.
- → **4. Assemble.** Format the retrieved chunks into a prompt with citations, staying within the model's context window. Also a stored procedure — or application code, depending on taste.
- → **5. Generate.** Send the prompt to the LLM (Azure OpenAI, or any model endpoint) and return the answer with its citations. This is the only stage that leaves the database.

```

graph LR
    A[Documents] --> B[Chunk
T-SQL / app]
    B --> C[Embed
batched pipeline]
    C --> D[(RagChunks
VECTOR column)]
    E[User question] --> F[Embed question]
    F --> G[Retrieve
usp_RetrieveChunks]
    D --> G
    G --> H[Assemble prompt
+ citations]
    H --> I[LLM call
external]
    I --> J[Answer + sources]

```

Keeping stages 1–4 in SQL Server is not just convenience — it is correctness. Permission filtering inside the retrieval procedure means the LLM never sees chunks the user is not allowed to see; there is no post-generation filtering step to forget. Transactional ingestion means a document and its chunks are never half-present. And the audit trail — which chunks grounded which answer — is a query away.

The Retrieval Procedure: Similarity Plus Permission Filtering

The retrieval procedure is the load-bearing wall of the pipeline. It takes an embedded question and a user context, and returns ranked, permission-filtered chunks with their source metadata. Here is the production shape — DiskANN approximate search with over-fetch, joined back for enrichment and filtered by document permissions:

```

-- The heart of RAG: permission-filtered vector retrieval in one procedure
CREATE OR ALTER PROCEDURE dbo.usp_RetrieveChunks
    @QuestionEmbedding VECTOR(1536),
    @UserId            INT,
    @TopK              INT = 8
AS
BEGIN
    SET NOCOUNT ON;

    -- 1. Approximate retrieval: over-fetch 5x to survive permission filtering
    -- 2. Join back for document metadata + permission check
    -- 3. Trim to TopK AFTER filtering (never before)
    SELECT TOP (@TopK)
        c.ChunkId, c.ChunkText, d.Title, d.SourceUri,
        s.distance AS Distance
    FROM VECTOR_SEARCH(
        TABLE = dbo.RagChunks AS t,
        COLUMN = Embedding,
        SIMILAR_TO = @QuestionEmbedding,
        METRIC = 'cosine',
        TOP_N = @TopK * 5
    ) AS s
    JOIN dbo.RagChunks AS c    ON c.ChunkId = s.ChunkId
    JOIN dbo.RagDocuments AS d ON d.DocumentId = c.DocumentId
    WHERE d.IsPublished = 1
        AND EXISTS (SELECT 1 FROM dbo.DocumentPermissions AS p
                     WHERE p.DocumentId = d.DocumentId
                           AND p.UserId = @UserId) -- row-level permission check
    ORDER BY s.distance;
END;
GO

```

Three details make this production-grade rather than demo-grade. **Over-fetch before filtering:** `TOP_N = @TopK * 5` because permission and publication filters consume candidates — fetching exactly 8 and filtering to 3 is the classic RAG recall bug. **Filter after retrieval, rank after filtering:** the `ORDER BY s.distance` applies to the filtered set, so the final ranking reflects what the user may actually see. **Permissions as data, not as application logic:** the `EXISTS` check against a permissions table means access control is enforced by the same query that retrieves — auditable, testable, and impossible for a future application rewrite to bypass. (Row-level security from Chapter 12 can enforce this even more strongly.)

For corpora under ~50K vectors, replace the `VECTOR_SEARCH` call with exact `VECTOR_DISTANCE` ranking — same procedure shape, same filters, 100% recall, no index to maintain.

Hybrid Retrieval: Vector Plus Full-Text in One Query

Pure vector search misses exact-term matches ("error 0x80070005" is a string, not a concept), and pure full-text search misses paraphrases. Production RAG uses both. SQL Server composes them naturally — run the full-text predicate and the vector ranking in one statement and fuse the scores:

```
-- Hybrid retrieval: full-text candidates UNION vector candidates, fused
DECLARE @q VECTOR(1536) = '...';
DECLARE @keywords NVARCHAR(4000) = '"password reset" OR "account recovery"';

WITH VectorHits AS (
    SELECT TOP (20) c.ChunkId, s.distance AS VecDist,
        ROW_NUMBER() OVER (ORDER BY s.distance) AS VecRank
    FROM VECTOR_SEARCH(TABLE = dbo.RagChunks AS t, COLUMN = Embedding,
        SIMILAR_TO = @q, METRIC = 'cosine', TOP_N = 20) AS s
    JOIN dbo.RagChunks AS c ON c.ChunkId = s.ChunkId
),
TextHits AS (
    SELECT TOP (20) c.ChunkId,
        ROW_NUMBER() OVER (ORDER BY k.RANK DESC) AS TextRank
    FROM CONTAINSTABLE(dbo.RagChunks, ChunkText, @keywords) AS k
    JOIN dbo.RagChunks AS c ON c.ChunkId = k.[KEY]
)
-- Reciprocal rank fusion: 1/(60+rank) per list, summed
SELECT TOP (10) c.ChunkId, c.ChunkText,
    SUM(1.0 / (60 + COALESCE(v.VecRank, 1000))
        + 1.0 / (60 + COALESCE(t.TextRank, 1000))) AS RRFScore
FROM dbo.RagChunks AS c
LEFT JOIN VectorHits AS v ON v.ChunkId = c.ChunkId
LEFT JOIN TextHits AS t ON t.ChunkId = c.ChunkId
WHERE v.ChunkId IS NOT NULL OR t.ChunkId IS NOT NULL
GROUP BY c.ChunkId, c.ChunkText
ORDER BY RRFScore DESC;
```

Reciprocal rank fusion (RRF) needs no score normalization between the two lists — it works purely on ranks, which is why it is the standard fusion method. The constant 60 dampens the influence of very high ranks; the `COALESCE(..., 1000)` gives chunks appearing in only one list a small participation score rather than zero. This is Chapter 11's hybrid search applied to the RAG retrieval path.

Prompt Assembly and the Generation Call

With chunks retrieved, prompt assembly is string formatting with a budget: the model's context window. Number each chunk, include its source, and instruct the model to cite chunk numbers — citations are what make RAG answers trustworthy and debuggable:

```
-- Assemble a cited prompt from retrieved chunks (application or T-SQL)
-- Prompt template:
--   "Answer the question using ONLY the context below. Cite sources like [1], [2].
--   If the context does not contain the answer, say so explicitly.
--
--   Context:
--   [1] (Password reset procedure – https://intranet/help/password-reset)
--   To reset your password, open Settings > Security > ...
--   [2] (Account recovery FAQ – https://intranet/help/account-faq)
--   If you are locked out, contact the service desk at ...
--
--   Question: How do I reset my password?"
```

The generation call itself goes to the LLM — Azure OpenAI via

`sp_invoke_external_rest_endpoint` for in-database orchestration, or from application code for production scale. Either way, log the answer alongside the chunk IDs that grounded it: that log is your evaluation dataset (Chapter 27), your audit trail, and your debugging lifeline when a user reports a wrong answer. "Which chunks did the model see?" should always be a query away, never a mystery.

Keeping the Pipeline Honest: Freshness and Evaluation Hooks

A RAG pipeline rots in two ways: stale content and degrading retrieval quality. Build the countermeasures into the pipeline rather than bolting them on later. For freshness, the `DocHash` change detection from Chapter 8 drives incremental re-chunking, and the embedding backlog query tells you whether ingestion is keeping up. For quality, the recall history table from Chapter 14 trends retrieval health, and the answer log feeds the evaluation metrics in Chapter 27.

One more practice that pays for itself: a **golden question set** — twenty to fifty representative questions with known-good answers, run through the full pipeline on a schedule, with answer quality scored (by an LLM judge or by human spot-check). It is the end-to-end test of the entire system: chunking, embeddings, index, retrieval procedure, prompt template, and model. When the golden set degrades, you know something changed; the per-stage metrics tell you where.

Key Takeaways

- → A SQL Server RAG pipeline keeps four of five stages in the database — chunking, embedding ingestion, retrieval, and prompt assembly — with only the LLM generation call external. Permission filtering inside the retrieval procedure means the model never sees unauthorized chunks.
- → The retrieval procedure over-fetches from `VECTOR_SEARCH ( TOP_N = @TopK * 5 )`, joins back for metadata, applies publication and permission filters, then trims to `@TopK` — filtering before trimming is what keeps recall intact.
- → Hybrid retrieval combines `VECTOR_SEARCH` with `CONTAINSTABLE` full-text results fused by reciprocal rank fusion — no score normalization needed, and exact-term matches stop falling through the cracks.
- → Assemble prompts with numbered, sourced chunks and explicit cite-or-abstain instructions; log every answer with the chunk IDs that grounded it — that log is your evaluation dataset, audit trail, and debugging record.
- → Fight the two rots with built-in countermeasures: hash-based change detection and backlog monitoring for freshness, recall history plus a scheduled golden-question set for end-to-end retrieval quality.

Chapter 25: Reranking — Improving Retrieval Quality

25.1 Why Vector Search Alone Is Not Enough

Vector search is a recall machine: it finds candidates fast across millions of rows, but its ranking is coarse. A bi-encoder embedding compresses a 512-token document into a single 1,536-dimensional point, and that compression loses information. Two documents with different specific claims can land at nearly identical distances from a query vector because their general topics match.

Embedding compression loss. A question about "SQL Server index fragmentation thresholds" may return documents about "SQL Server index performance" ranked higher than a document that actually explains when to `REORGANIZE` vs `REBUILD` — because "index performance" is a broader, more common phrase in the embedding model's training data. The vector distance says "topically close"; the user's question needed "specifically answering."

Reranking fixes this with a two-stage retrieval: Stage 1 (the vector index) retrieves 50–200 candidates cheaply for recall; Stage 2 (the reranker) scores each candidate against the query precisely and re-orders them for precision. You pay the expensive scoring only on the candidates, not the corpus — which is why reranking routinely delivers the biggest quality-per-millisecond improvement in a RAG pipeline.

25.2 Cross-Encoder Architecture and Why It Works Better

A bi-encoder embeds the query and each document independently, then compares vectors. A cross-encoder takes the query and one document *together* as input and outputs a relevance score directly — the model sees the actual words of both, so it can reward exact answer matches, penalize topic-only overlap, and understand negation ("this does NOT apply to Standard Edition"). That joint attention is why cross-encoders outrank bi-encoders on precision, and why they are too slow to run over a whole corpus: each query-document pair is a full model forward pass.


```
# Python – cross-encoder reranking (ms-marco-MiniLM-L-6-v2)
from sentence_transformers import CrossEncoder

reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

def rerank(query, candidates, top_n=5):
    """candidates: list of dicts with 'text'. Returns top_n, re-ordered."""
    pairs = [[query, c["text"]] for c in candidates]
    scores = reranker.predict(pairs)
    ranked = sorted(zip(candidates, scores),
                    key=lambda x: x[1], reverse=True)
    return [{"text": c["text"], "rerank_score": float(s),
            "vector_distance": c.get("distance")}
            for c, s in ranked[:top_n]]
```

The canonical example: a SQL Server DBA knowledge base where the query is "How do I reduce index fragmentation on a high-write table?"

```
query = "How do I reduce index fragmentation on a high-write table?"
# Candidate documents from ANN retrieval (top-20 from SQL Server)
candidates = [
    {"id": "doc_001", "text":
        "sys.dm_db_index_physical_stats reports fragmentation. Query it regularly "
        "to monitor avg_fragmentation_in_percent across your indexes."},
    {"id": "doc_002", "text":
        "Use ALTER INDEX ... REORGANIZE for fragmentation between 5 and 30 "
        "percent, and ALTER INDEX ... REBUILD for fragmentation above 30 percent. "
        "On high-write tables, schedule rebuilds during maintenance windows."},
    {"id": "doc_003", "text":
        "Fill factor reserves free space on index pages. Setting FILLFACTOR to 80 "
        "on high-write tables reduces page splits and slows fragmentation growth."},
    {"id": "doc_004", "text":
        "Index performance in SQL Server depends on statistics. Running UPDATE "
        "STATISTICS after bulk loads ensures the optimizer uses indexes correctly."},
    {"id": "doc_005", "text":
        "Partitioning a large table lets you rebuild individual partitions. For "
        "high-write partitioned tables, rebuild only the hot partitions nightly."},
]
ranked = rerank(query, candidates, top_n=3)
for r in ranked:
    print(f"{r['rerank_score']:.3f} {r['text'][:80]}...")
# Expected: doc_002 (REORGANIZE vs REBUILD thresholds) rises to rank 1,
# doc_003 (fill factor) to rank 2 – both directly answering fragmentation
# mitigation – while doc_001 (monitoring) and doc_004 (statistics) fall.
```

25.3 Re-ranking with Cohere and Production API Integration

Running a cross-encoder locally costs you GPU inference infrastructure on the query path. Cohere Rerank is the managed alternative: send the query and up to hundreds of candidate texts, get relevance scores back. Fifty documents through Cohere Rerank typically completes in 80–150ms round-trip from a co-located cloud instance — often faster than local CPU inference, with no model to host.

```
# Python – Cohere Rerank over SQL Server retrieval candidates
import cohere, pyodbc

co = cohere.ClientV2(api_key="...")
conn = pyodbc.connect("DRIVER={ODBC Driver 18 for SQL Server};"
                    "SERVER=localhost;DATABASE=ragdb;Trusted_Connection=yes;"
                    "Encrypt=yes;TrustServerCertificate=yes")

def sqlserver_retrieve(question_emb, top_k=50, tenant_id=1):
    vec = "[" + ",".join(str(v) for v in question_emb) + "]"
    cur = conn.cursor()
    cur.execute("""SELECT TOP (?) ChunkId, ChunkText,
                    VECTOR_DISTANCE('cosine', Embedding,
                                   CAST(? AS VECTOR(1536))) AS Distance
                    FROM dbo.DocumentChunks
                    WHERE TenantId = ? AND Embedding IS NOT NULL
                    ORDER BY Distance""", (top_k, vec, tenant_id))
    return [{"id": r[0], "text": r[1], "distance": r[2]}
            for r in cur.fetchall()]

def retrieve_and_rerank(question, question_emb, top_n=5, tenant_id=1):
    candidates = sqlserver_retrieve(question_emb, top_k=50,
                                     tenant_id=tenant_id)
    rr = co.rerank(model="rerank-v3.5", query=question,
                  documents=[c["text"] for c in candidates],
                  top_n=top_n, return_documents=False)
    return [{"id": candidates[r.index]["id"],
            "text": candidates[r.index]["text"],
            "rerank_score": r.relevance_score,
            "vector_distance": candidates[r.index]["distance"]}
            for r in rr.results]
```

For latency budgets: retrieving 50 rows from SQL Server takes 5–15ms on typical hardware. The `ms-marco-MiniLM-L-6-v2` cross-encoder scores 50 pairs on CPU in approximately 120–200ms; Cohere Rerank does it in 80–150ms over the network. Either way, total two-stage retrieval stays under 300ms — well within a 500ms SLA for interactive search.

25.4 Implementing Re-ranking in SQL Server Pipelines

When your vector store is SQL Server, the re-ranking step happens in the application layer after the T-SQL retrieval. The database returns candidate rows with their distance values; Python applies the cross-encoder and returns the re-ordered set. Keep the boundary clean: SQL Server owns recall (top-50 candidates, tenant filtering, distance thresholding), the reranker owns precision (final top-5 order).

```
-- SQL Server: candidate retrieval tuned for reranking
-- Over-fetch (TOP 50) because the reranker re-orders; the final cut is top-5.
-- The distance threshold keeps obvious non-matches out of paid rerank calls.
DECLARE @q VECTOR(1536) = '[0.12, 0.87, ...]'; -- embedded question

SELECT TOP (50) ChunkId, ChunkText,
               VECTOR_DISTANCE('cosine', Embedding, @q) AS Distance
FROM dbo.DocumentChunks
WHERE TenantId = @tenant
      AND ModelName = 'text-embedding-3-small'
      AND VECTOR_DISTANCE('cosine', Embedding, @q) < 0.45 -- threshold: tune on evals
ORDER BY Distance;
```

```

# Python – SQL Server retrieve + local cross-encoder rerank
import pyodbc
from sentence_transformers import CrossEncoder

conn = pyodbc.connect("DRIVER={ODBC Driver 18 for SQL Server};"
                      "SERVER=localhost;DATABASE=ragdb;Trusted_Connection=yes;"
                      "Encrypt=yes;TrustServerCertificate=yes")
reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

def sqlserver_retrieve_and_rerank(question, question_emb,
                                  candidate_k=50, top_n=5,
                                  max_distance=0.45, tenant_id=1):
    vec = "[" + ",".join(str(v) for v in question_emb) + "]"
    cur = conn.cursor()
    cur.execute("""SELECT TOP (?) ChunkId, ChunkText,
                    VECTOR_DISTANCE('cosine', Embedding,
                                   CAST(? AS VECTOR(1536))) AS Distance
                    FROM dbo.DocumentChunks
                    WHERE TenantId = ? AND ModelName = 'text-embedding-3-small'
                    AND VECTOR_DISTANCE('cosine', Embedding,
                                   CAST(? AS VECTOR(1536))) < ?
                    ORDER BY Distance""",
                (candidate_k, vec, tenant_id, vec, max_distance))
    candidates = [{"id": r[0], "text": r[1], "distance": r[2]}
                  for r in cur.fetchall()]
    if not candidates:
        return [] # threshold refused: better no answer than a bad one
    scores = reranker.predict([[question, c["text"]] for c in candidates])
    ranked = sorted(zip(candidates, scores),
                    key=lambda x: x[1], reverse=True)[:top_n]
    return [{"id": c["id"], "text": c["text"],
              "rerank_score": float(s), "vector_distance": c["distance"]}
            for c, s in ranked]

```

The distance threshold deserves emphasis: it is your cheapest quality control. Candidates beyond the threshold are so dissimilar that reranking them wastes inference budget and risks promoting a fluent-but-irrelevant chunk. Tune the threshold on your eval set (Chapter 22.8) — it is the one number that most directly trades "answered questions" against "answer quality."

25.5 LLM-Based Re-ranking and Reciprocal Rank Fusion

A cross-encoder scores query-document pairs; an LLM reranker goes further and judges them. Prompt the LLM with the query plus numbered candidates and ask it to return the best ordering with brief justifications. Quality is excellent — the LLM understands nuance no

cross-encoder captures — but cost and latency are an order of magnitude higher, so reserve LLM reranking for the final ordering of an already-reranked top-10, or for offline eval-set grading.

Reciprocal Rank Fusion (RRF) solves a different problem: combining rankings from *multiple* retrieval paths. Your hybrid search (Chapter 11) produces a vector ranking and a full-text ranking that disagree; RRF merges them without needing comparable scores:

```
-- SQL Server: Reciprocal Rank Fusion over vector + full-text rankings
-- Each CTE ranks the same candidate pool by a different signal (1 = best).
-- RRF score = SUM(1 / (k + rank)); k = 60 is the standard constant.
DECLARE @q VECTOR(1536) = '[0.12, 0.87, ...]';

WITH VectorRank AS (
    SELECT ChunkId, ROW_NUMBER() OVER (
        ORDER BY VECTOR_DISTANCE('cosine', Embedding, @q)) AS rnk
    FROM dbo.DocumentChunks WHERE TenantId = @tenant
),
TextRank AS (
    SELECT ChunkId, ROW_NUMBER() OVER (
        ORDER BY RANK DESC) AS rnk
    FROM CONTAINSTABLE(dbo.DocumentChunks, ChunkText, @keywords)
)
SELECT COALESCE(v.ChunkId, t.ChunkId) AS ChunkId,
       1.0 / (60 + COALESCE(v.rnk, 1000))
       + 1.0 / (60 + COALESCE(t.rnk, 1000)) AS RrfScore
FROM VectorRank AS v
FULL OUTER JOIN TextRank AS t ON t.ChunkId = v.ChunkId
ORDER BY RrfScore DESC;
-- Feed the top RRF results into the cross-encoder for final ordering.
```

The pipeline order that works in production: hybrid retrieval → RRF fusion → cross-encoder rerank of the fused top-50 → top-5 to the LLM. Each stage narrows the candidate set while increasing the scoring precision, so total latency stays bounded.

25.6 Production Patterns: Latency, Caching, and Model Selection

- → **Budget the stages.** Retrieval 5–15ms, rerank 80–200ms, generation seconds. Reranking is never the bottleneck — but cap candidate count (50, not 200) so a slow reranker cannot blow the SLA.
- → **Cache rerank decisions.** Frequent questions rerank the same candidates identically. Cache the final top-5 per (question hash, corpus version) — Chapter 33 covers semantic caching, which extends this to paraphrased questions.

- → **Choose the reranker by volume.** Under ~100 QPS, local cross-encoder on CPU is cheapest. Above that, compare GPU inference cost against Cohere Rerank API pricing at your actual candidate counts — the crossover surprises most teams.
- → **Version the reranker with the corpus.** A reranker upgrade changes ordering; a corpus re-chunk changes candidates. Deploy them behind the same feature flag and measure the eval set before and after.

25.7 Evaluating Re-ranking Quality

Measure reranking with the same eval set you use for chunking (Chapter 22.8), scored at two cutoffs: hit rate at top-5 *before* reranking (vector retrieval alone) and *after* (reranked order). The delta is the reranker's contribution. Track it per query type — rerankers help most on ambiguous queries where the vector ranking is flat, and least on exact-match queries where the vector ranking is already correct.

```
# Python – measure reranker lift on an eval set
def evaluate_reranker(eval_set, embed_fn, top_n=5):
    pre_hits = post_hits = 0
    for q in eval_set:
        q_emb = embed_fn(q["question"])
        candids = sqlserver_retrieve(q_emb, top_k=50)          # Stage 1
        pre_hits += any(q["expected"] in c["text"] for c in candids[:top_n])
        ranked = rerank(q["question"], candids, top_n=top_n)  # Stage 2
        post_hits += any(q["expected"] in r["text"] for r in ranked)
    n = len(eval_set)
    return {"vector_only": pre_hits / n, "reranked": post_hits / n,
            "lift": (post_hits - pre_hits) / n}

# Typical result: vector_only=0.62, reranked=0.78, lift=+0.16
# If lift < 0.05, your reranker is not earning its latency: check the
# candidate count, the distance threshold, and whether the eval questions
# match production query distribution.
```

25.8 End-to-End Pipeline: SQL Server + Cross-Encoder + RAG Generation

Putting it together — a complete RAG query path with per-stage latency instrumentation:

```

# Python – full RAG pipeline: SQL Server retrieval, cross-encoder
# rerank, LLM generation, with per-stage timing
import time, pyodbc, openai
from sentence_transformers import CrossEncoder

openai_client = openai.OpenAI()
conn = pyodbc.connect("DRIVER={ODBC Driver 18 for SQL Server};"
                     "SERVER=localhost;DATABASE=ragdb;Trusted_Connection=yes;"
                     "Encrypt=yes;TrustServerCertificate=yes")
reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

def rag_answer(question, tenant_id=1, top_n=5):
    stages = {}
    # --- Stage 1: embed the question ---
    t0 = time.perf_counter()
    q_emb = openai_client.embeddings.create(
        input=[question], model="text-embedding-3-small").data[0].embedding
    stages["embed_ms"] = (time.perf_counter() - t0) * 1000
    # --- Stage 2: ANN retrieval from SQL Server ---
    t0 = time.perf_counter()
    candidates = sqlserver_retrieve_and_rerank(
        question, q_emb, candidate_k=50, top_n=top_n, tenant_id=tenant_id)
    stages["retrieve_rerank_ms"] = (time.perf_counter() - t0) * 1000
    if not candidates:
        return {"answer": "I don't have information about that.",
                "stages": stages, "sources": []}
    # --- Stage 3: generate with retrieved context ---
    context = "\n\n".join(f"[{c['id']}] {c['text']}" for c in candidates)
    t0 = time.perf_counter()
    completion = openai_client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system",
             "content": "Answer using only the provided context. "
                        "Cite sources as [id]. If the context does not "
                        "contain the answer, say so."},
            {"role": "user",
             "content": f"Context:\n{context}\n\nQuestion: {question}"}])
    stages["generate_ms"] = (time.perf_counter() - t0) * 1000
    return {"answer": completion.choices[0].message.content,
            "stages": stages,
            "sources": [c["id"] for c in candidates]}

result = rag_answer(
    "What is the difference between REORGANIZE and REBUILD for index maintenance?",
    tenant_id=7)
print(result["stages"]) # e.g. embed_ms=180, retrieve_rerank_ms=210, ...
print(result["answer"])

```

This pipeline produces an answer with cited sources, full latency instrumentation per stage, and a clean separation between retrieval, re-ranking, and generation that allows each component to be swapped independently — for example, replacing the local cross-encoder with Cohere Rerank, or replacing SQL Server with a dedicated vector engine, without touching the generation layer.

Key Takeaways

- → Reranking is a two-stage retrieval: the vector index optimizes recall over the corpus, the reranker optimizes precision over 50–200 candidates. It is the highest quality-per-millisecond improvement in most RAG pipelines.
- → SQL Server owns recall (tenant-filtered top-50 with a distance threshold); the cross-encoder or Cohere Rerank owns the final ordering. Keep that boundary clean.
- → Fuse hybrid rankings with Reciprocal Rank Fusion in T-SQL before reranking: RRF merges vector and full-text orderings without comparable scores.
- → Tune the distance threshold on your eval set — it is the cheapest quality control you have, and it keeps irrelevant candidates out of paid rerank calls.
- → Measure reranker lift (hit rate before vs after) per query type; if lift is under ~5 points, fix candidate count, threshold, or eval distribution before blaming the model.

Chapter 26: Caching Embeddings

Why Embeddings Are Unusually Cacheable

Embedding inference is expensive — an API call costs money and 100–300ms, a local model costs GPU time — and it is also perfectly deterministic: the same text through the same model always produces the same vector. Deterministic plus expensive is the textbook definition of cacheable. Unlike most application caches, an embedding cache never goes stale on its own: the vector for a text does not change unless the model changes, so invalidation is a model-versioning problem, not a TTL problem.

Not all text is equally cacheable. Query strings from users cluster heavily around common questions — support tickets, product searches, FAQ lookups. Document chunks in a RAG corpus are almost always worth caching because they are embedded once during ingestion and compared repeatedly at query time. If you are already storing document embeddings in SQL Server native vectors or Pinecone, the document side of the cache is essentially already solved. The query side is where the additional caching layer pays off.

Cache Architecture: Where Embeddings Live

Layer 1: In-process LRU cache

Lives in the application process, holds the hottest few thousand question embeddings. Sub-microsecond lookup, zero network. Lost on restart — acceptable, because it refills in minutes under production traffic.

Layer 2: Redis (shared, fast)

Shared across application instances, sub-millisecond latency, survives individual app restarts. The workhorse layer: every question embedding check hits Redis before it hits an API or GPU.

Layer 3: Persistent database cache (SQL Server)

Survives server restarts and infrastructure events. Slower than Redis but queryable. Useful when you need to audit which embeddings have been generated, track model versions, or join embedding cache data with other relational data. SQL Server native `VECTOR` is a natural fit here because the cache table can double as a lightweight vector store.

```

graph TD
    Q[Question text] --> L1{L1: in-process LRU}
    L1 -->|hit| R[Return vector]
    L1 -->|miss| L2{L2: Redis}
    L2 -->|hit| R
    L2 -->|miss| L3{L3: SQL Server}
    L3 -->|hit| R
    L3 -->|miss| E[Embed via API/GPU]
    E --> W[Write through]
    W --> R
    L1 + L2 + L3

```

Implementing a Redis Embedding Cache

```

# Python – Redis L2 embedding cache with model-versioned keys
import hashlib, struct
import redis

r = redis.Redis(host="localhost", port=6379, decode_responses=False)
MODEL_ID = "text-embedding-3-small"
DIM = 1536

def cache_key(text):
    digest = hashlib.sha256(text.encode("utf-8")).hexdigest()
    return f"emb:{MODEL_ID}:{digest}".encode()

def get_cached(text):
    raw = r.get(cache_key(text))
    if raw is None:
        return None
    return list(struct.unpack(f"<{DIM}f", raw)) # float32 little-endian

def set_cached(text, embedding):
    raw = struct.pack(f"<{DIM}f", *embedding)
    r.set(cache_key(text), raw) # no TTL: vectors don't decay

def get_or_embed(text, embed_fn):
    vec = get_cached(text)
    if vec is None:
        vec = embed_fn(text)
        set_cached(text, vec)
    return vec

```

Two design decisions matter here. The model ID is part of the key — a model upgrade must never read another model's vectors, and key namespacing makes that impossible by construction. And there is no TTL: embedding vectors do not decay, so expiry would only cause pointless recomputation. Eviction, if Redis memory pressure demands it, should use an allkeys-lru policy rather than time-based expiry.

SQL Server as a Persistent Embedding Cache

When you need durability, auditability, or the ability to query cache contents alongside relational data, SQL Server with native `VECTOR` is a natural fit. The same table that stores cached embeddings can participate in vector similarity queries, making it a dual-purpose structure — cache and lightweight vector store in one.

```
-- SQL Server 2025: persistent embedding cache table
CREATE TABLE dbo.EmbeddingCache (
    CacheKey    CHAR(64) NOT NULL          -- SHA-256 of the text
    CONSTRAINT PK_EmbeddingCache PRIMARY KEY,
    ModelId     NVARCHAR(100) NOT NULL,    -- part of the logical key
    DimCount    INT NOT NULL,
    Embedding   VECTOR(1536) NOT NULL,
    TextPreview NVARCHAR(200) NULL,        -- for auditing, not retrieval
    HitCount    BIGINT NOT NULL CONSTRAINT DF_EmbeddingCache_Hits DEFAULT (0),
    CreatedAt   DATETIME2(0) NOT NULL
    CONSTRAINT DF_EmbeddingCache_Created DEFAULT SYSUTCDATETIME(),
    LastHitAt   DATETIME2(0) NULL
);
CREATE INDEX IX_EmbeddingCache_Model_Hits
ON dbo.EmbeddingCache (ModelId, HitCount DESC);
```

```
-- SQL Server: cache read (with hit accounting) and write
DECLARE @key CHAR(64) = '...sha256...';
DECLARE @model NVARCHAR(100) = 'text-embedding-3-small';

UPDATE dbo.EmbeddingCache
SET HitCount = HitCount + 1, LastHitAt = SYSUTCDATETIME()
WHERE CacheKey = @key AND ModelId = @model;

SELECT Embedding FROM dbo.EmbeddingCache
WHERE CacheKey = @key AND ModelId = @model;
-- If no row: embed, then:
-- INSERT INTO dbo.EmbeddingCache
--     (CacheKey, ModelId, DimCount, Embedding, TextPreview)
-- VALUES (@key, @model, 1536, CAST(@vec AS VECTOR(1536)), LEFT(@text, 200));
```

Here is a Python integration layer that uses the SQL Server cache with a Redis L2 in front of it:

```

# Python – two-tier embedding cache: Redis (fast) -> SQL Server (persistent)
import hashlib, os, struct
import pyodbc, redis

r = redis.Redis(host="localhost", port=6379, decode_responses=False)
conn = pyodbc.connect(os.environ["SQLSERVER_CONN"])
MODEL_ID = "text-embedding-3-small"
DIM = 1536

def _key(text):
    return hashlib.sha256(text.encode()).hexdigest()

def get_or_embed_tiered(text, embed_fn):
    """Two-tier embedding cache: Redis (fast) -> SQL Server (persistent)."""
    digest = _key(text)
    rkey = f"emb:{MODEL_ID}:{digest}".encode()
    # L2: Redis check
    raw = r.get(rkey)
    if raw:
        return list(struct.unpack(f"<{DIM}f", raw))
    # L3: SQL Server check
    cur = conn.cursor()
    cur.execute("""UPDATE dbo.EmbeddingCache
                   SET HitCount = HitCount + 1, LastHitAt = SYSUTCDATETIME()
                   WHERE CacheKey = ? AND ModelId = ?""", (digest, MODEL_ID))
    row = cur.execute("""SELECT CAST(Embedding AS NVARCHAR(MAX))
                       FROM dbo.EmbeddingCache
                       WHERE CacheKey = ? AND ModelId = ?""",
                      (digest, MODEL_ID)).fetchone()
    if row:
        vec = [float(x) for x in row[0].strip("[]").split(",")]
        r.set(rkey, struct.pack(f"<{DIM}f", *vec)) # backfill Redis
        conn.commit()
        return vec
    # Miss everywhere: embed and write through both layers
    vec = embed_fn(text)
    vec_lit = "[" + ",".join(str(v) for v in vec) + "]"
    cur.execute("""INSERT INTO dbo.EmbeddingCache
                   (CacheKey, ModelId, DimCount, Embedding, TextPreview)
                   VALUES (?, ?, ?, CAST(? AS VECTOR(1536)), ?)""",
                (digest, MODEL_ID, DIM, vec_lit, text[:200]))
    conn.commit()
    r.set(rkey, struct.pack(f"<{DIM}f", *vec))
    return vec

```

The `HitCount` column is more useful than it looks: it tells you which cached embeddings earn their storage, which questions dominate your traffic, and — when joined against retrieval logs — whether your cache is actually covering the expensive calls.

Cache Invalidation: The Hard Part

Model version tagging is the primary defense. Every cache entry must include the model identifier as part of the cache key and as a stored attribute. The examples above use `MODEL_ID` in both the Redis key and the SQL Server row. A model migration procedure looks like this:

```
-- SQL Server: model migration for the embedding cache
-- 1. New rows are written under the new ModelId automatically
--    (the application passes the new MODEL_ID).
-- 2. Retire the old model's rows after the cutover + grace period:
DELETE FROM dbo.EmbeddingCache
WHERE ModelId = 'text-embedding-3-small'
  AND LastHitAt < DATEADD(day, -30, SYSUTCDATETIME());
-- 3. In Redis, either FLUSHDB the emb: namespace or let allkeys-lru
--    evict old keys naturally; they are namespaced by model and harmless.
```

Never reuse a cache key across models, never "migrate" vectors from one model to another by copying rows, and never let a query path read a vector whose `ModelId` does not match the model that will consume it. Cross-model vector reads are silent corruption — the query runs, the numbers look plausible, and retrieval quality degrades without an error.

LangChain and LlamaIndex Cache Integration

If you use LangChain, its `CacheBackedEmbeddings` wraps any embedding model with a byte-store cache — point it at Redis and you get the L2 layer with no custom code. For LlamaIndex, the ingestion pipeline's `IngestionCache` deduplicates document embeddings across pipeline runs, which covers the document side. Neither replaces the SQL Server L3 layer: framework caches are key-value conveniences, while the database cache gives you auditability, hit analytics, and joins against your retrieval logs.

Semantic Cache: A Step Beyond Exact Match

Exact-match caching only helps when the same question text repeats. A semantic cache goes further: embed the incoming question, search the cache for a *similar* previously answered question, and reuse its answer when similarity is high enough. This turns the cache from a key-value lookup into a vector search problem — and SQL Server can serve it directly:

```
-- SQL Server 2025: semantic answer cache
CREATE TABLE dbo.SemanticAnswerCache (
    EntryId          BIGINT IDENTITY(1,1) NOT NULL
        CONSTRAINT PK_SemanticAnswerCache PRIMARY KEY,
    QuestionText     NVARCHAR(1000) NOT NULL,
    QuestionEmb      VECTOR(1536) NOT NULL,
    AnswerText       NVARCHAR(MAX) NOT NULL,
    ModelId          NVARCHAR(100) NOT NULL,
    HitCount         BIGINT NOT NULL CONSTRAINT DF_SemCache_Hits DEFAULT (0),
    CreatedAt        DATETIME2(0) NOT NULL
        CONSTRAINT DF_SemCache_Created DEFAULT SYSUTCDATETIME()
);
CREATE VECTOR INDEX IX_SemanticAnswerCache_Q
    ON dbo.SemanticAnswerCache (QuestionEmb)
    WITH (METRIC = 'cosine', TYPE = 'DiskANN');

-- Lookup: reuse an answer when a near-duplicate question was seen before
DECLARE @q VECTOR(1536) = '[0.12, 0.87, ...]'; -- embedded incoming question
SELECT TOP (1) EntryId, AnswerText,
    VECTOR_DISTANCE('cosine', QuestionEmb, @q) AS Distance
FROM dbo.SemanticAnswerCache
WHERE ModelId = 'text-embedding-3-small'
    AND VECTOR_DISTANCE('cosine', QuestionEmb, @q) < 0.08 -- near-duplicate only
ORDER BY Distance;
-- On hit: UPDATE HitCount, return AnswerText, skip the LLM entirely.
-- On miss: run the full RAG pipeline, then INSERT the Q/A pair.
```

The threshold is everything: too loose and paraphrased-but-different questions get wrong answers; too tight and the cache never hits. Start at 0.08 cosine distance, measure the false-hit rate on your eval set, and only widen it with evidence. Chapter 33 covers semantic caching patterns in full, including multi-tenant isolation of cached answers.

Monitoring Cache Performance

```
-- SQL Server: embedding cache effectiveness dashboard query
SELECT ModelId,
    COUNT(*) AS CachedVectors,
    SUM(HitCount) AS TotalHits,
    CAST(SUM(HitCount) AS FLOAT) / NULLIF(COUNT(*), 0) AS AvgHitsPerVector,
    SUM(CASE WHEN LastHitAt > DATEADD(day, -7, SYSUTCDATETIME())
        THEN 1 ELSE 0 END) AS ActiveLast7d
FROM dbo.EmbeddingCache
GROUP BY ModelId;
-- Healthy cache: AvgHitsPerVector well above 1, ActiveLast7d a large
-- share of CachedVectors. A cache where most rows have HitCount = 0
-- is just an expensive log table – shrink what you cache.
```

Track hit rate per layer (L1/L2/L3) in your application metrics, not just in the database. A healthy deployment shows L1 absorbing the hottest questions, L2 absorbing the long tail of repeats, and L3 catching cold starts after deploys. If L3 hit rate is near zero, your Redis sizing is fine and the database layer is pure insurance — which is a legitimate architecture, as long as it is a conscious one.

Cache Warming Strategies

A cold cache after a deploy means every question pays full embedding latency until the cache refills. Warm it from history: take the top-N most frequent questions from your query logs, embed them in a background job at deploy time, and write through all three layers. For the document side, the ingest pipeline already warms the cache — every embedded chunk is a cache entry if you route ingestion through `get_or_embed_tiered`.

Schedule warming as part of the deploy pipeline, not as a manual step. A deploy that finishes with a warm cache behaves like the old version from the first request; a deploy that finishes cold creates a latency spike that your dashboards will attribute to the new code.

Key Takeaways

- → **Layer your caches:** in-process LRU for the hottest queries, Redis for cross-instance sharing with sub-millisecond latency, SQL Server native `VECTOR` for durability and auditability, and the vector store itself for document embeddings already indexed during ingestion.
- → Namespace every cache key by model ID and never set a TTL on embedding vectors — they do not decay. Invalidation is a model-versioning problem.
- → Use the `HitCount` column to prove the cache earns its storage; a cache where most rows are never hit is an expensive log table.
- → A semantic answer cache in SQL Server — near-duplicate question lookup via DiskANN — can skip the LLM entirely for repeated questions, but the similarity threshold must be tuned on evals, not guessed.
- → Warm the cache from query-log history at deploy time so the first request after a release behaves like the last request before it.

Chapter 27: Evaluating RAG Systems

The RAG Quality Problem: Why "It Feels Right" Doesn't Scale

Every RAG demo looks brilliant on the five questions its builder tried. Production is the other 50,000 questions, and the failure mode is not dramatic — it is a slow bleed of almost-right answers, missing citations, and confident hallucinations that erode trust one ticket at a time. "It feels right" does not scale; measured quality does. Evaluation is the discipline of turning answer quality into numbers you can trend, alert on, and gate deployments with.

A RAG system has two halves that fail independently: retrieval (did we fetch the right chunks?) and generation (did the LLM use them faithfully?). Measure them separately. A faithful answer built on wrong chunks is a retrieval failure; a hallucinated answer built on right chunks is a generation failure. Conflating them sends you tuning the wrong half.

Retrieval Metrics: Measuring What the Vector Search Returns

Build a labeled eval set: 100+ representative questions, each with the chunk IDs that should answer it (Chapter 22.8). Then score retrieval with the standard information-retrieval metrics:

- → **Recall@k**: fraction of questions where at least one gold chunk appears in the top-k retrieved. The headline retrieval metric.
- → **Precision@k / MRR**: how high the gold chunks rank, not just whether they appear. MRR (mean reciprocal rank) rewards putting the best chunk first.
- → **Context precision**: of the retrieved chunks, how many are actually relevant — the metric that tells you whether the LLM prompt is mostly signal or mostly noise.


```

# Python – retrieval evaluation harness on SQL Server
# Uses SQL Server native VECTOR for retrieval and a simple eval harness
import pyodbc, openai

openai_client = openai.OpenAI()
conn = pyodbc.connect("DRIVER={ODBC Driver 18 for SQL Server};"
                      "SERVER=localhost;DATABASE=ragdb;Trusted_Connection=yes;"
                      "Encrypt=yes;TrustServerCertificate=yes")

def retrieve_chunk_ids(question_emb, k=10, tenant_id=1):
    """Returns ordered list of chunk IDs from SQL Server vector search."""
    vec = "[" + ",".join(str(v) for v in question_emb) + "]"
    cur = conn.cursor()
    cur.execute("""SELECT TOP (?) ChunkId
                  FROM dbo.DocumentChunks
                  WHERE TenantId = ?
                  ORDER BY VECTOR_DISTANCE('cosine', Embedding,
                  CAST(? AS VECTOR(1536)))""",
                (k, tenant_id, vec))
    return [r[0] for r in cur.fetchall()]

def recall_at_k(eval_set, k=10):
    hits = 0
    for q in eval_set:
        q_emb = openai_client.embeddings.create(
            input=[q["question"]],
            model="text-embedding-3-small").data[0].embedding
        retrieved = set(retrieve_chunk_ids(q_emb, k=k))
        if retrieved & set(q["gold_chunk_ids"]):
            hits += 1
    return hits / len(eval_set)

# eval_set entries: {"question": ..., "gold_chunk_ids": [...]}
# e.g. question="How does log shipping differ from mirroring?",
#      gold chunks from the HA/DR runbook chapter
print("Recall@10:", recall_at_k(eval_set, k=10))

```

When retrieval scores are low, the diagnostic path mirrors index tuning: check embedding model alignment with your query distribution, examine chunk size and overlap, verify that the DiskANN index's `TOP_N` over-fetch and distance thresholds match your recall target, and consider hybrid search (vector + full-text, Chapter 11) for queries with strong keyword signals.

Generation Metrics: Faithfulness, Relevance, and Hallucination Detection

Retrieval metrics stop at the chunk list; generation metrics judge the answer. The three that matter:

- → **Faithfulness**: is every claim in the answer supported by the retrieved context? Decompose the answer into atomic claims and verify each against the chunks. A single unsupported claim is a hallucination.
- → **Answer relevance**: does the answer actually address the question, or does it summarize related-but-irrelevant context fluently?
- → **Context utilization**: did the answer use the retrieved chunks, or did the model answer from its own parametric knowledge while ignoring the context you paid to retrieve?

The RAGAS framework implements these as LLM-judged metrics. Here is a RAGAS evaluation pipeline for a RAG system on SQL Server:

```

# Python – RAGAS evaluation pipeline for SQL Server-backed RAG
# pip install ragas langchain-openai datasets
import pyodbc, openai
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_precision
from datasets import Dataset

openai_client = openai.OpenAI()
conn = pyodbc.connect("DRIVER={ODBC Driver 18 for SQL Server};"
                      "SERVER=localhost;DATABASE=ragdb;Trusted_Connection=yes;"
                      "Encrypt=yes;TrustServerCertificate=yes")

def rag_pipeline(question):
    """Your RAG pipeline: returns (answer, retrieved contexts)."""
    q_emb = openai_client.embeddings.create(
        input=[question], model="text-embedding-3-small").data[0].embedding
    vec = "[" + ",".join(str(v) for v in q_emb) + "]"
    cur = conn.cursor()
    rows = cur.execute("""SELECT TOP (5) ChunkText
                          FROM dbo.DocumentChunks
                          WHERE TenantId = 1
                          ORDER BY VECTOR_DISTANCE('cosine', Embedding,
                          CAST(? AS VECTOR(1536)))""", (vec,)).fetchall()
    contexts = [r[0] for r in rows]
    completion = openai_client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "system",
                    "content": "Answer using only the provided context."},
                  {"role": "user",
                    "content": f"Context:\n{chr(10).join(contexts)}\n\n"
                               f"Question: {question}"}])
    return completion.choices[0].message.content, contexts

questions = [
    "What are the ACID properties and how does SQL Server enforce them?",
    "What is the difference between REORGANIZE and REBUILD?",
    # ... 100+ questions with ground-truth answers
]
records = {"question": [], "answer": [], "contexts": [],
          "ground_truth": []}
for q, gt in zip(questions, ground_truths):
    answer, contexts = rag_pipeline(q)
    records["question"].append(q); records["answer"].append(answer)
    records["contexts"].append(contexts); records["ground_truth"].append(gt)

results = evaluate(Dataset.from_dict(records),
                  metrics=[faithfulness, answer_relevancy,
                          context_precision])
print(results)

```

LLM-judged metrics cost money and have their own variance — run each eval three times and average, and spot-check a sample by hand. An eval harness nobody trusts is worse than no harness, because it produces confident wrong conclusions.

End-to-End Correctness: Answer Quality at Scale

Retrieval metrics and generation metrics diagnose components; end-to-end correctness measures the system your users experience. The practical approach: a golden set of question/answer pairs graded by an LLM judge against rubrics (correct, partially correct, incorrect, refused), run nightly against production data snapshots.

Grade on a rubric, not a vibe. "Correct" means the answer contains the expected facts with no contradictions; "partially correct" means the key fact is present but a detail is wrong or missing; "refused" is a valid outcome when the context genuinely lacks the answer — and should be scored as correct behavior, not failure, or your metrics will punish honesty and reward hallucination.

Storing and Trending Evaluation Results in SQL Server

Eval results are time-series data. Store every run in SQL Server — same database, same backup, joinable against deployment history and corpus version metadata:

```
-- SQL Server 2025: evaluation results warehouse
CREATE TABLE dbo.EvalRuns (
    RunId          BIGINT IDENTITY(1,1) NOT NULL
        CONSTRAINT PK_EvalRuns PRIMARY KEY,
    RunAt          DATETIME2(0) NOT NULL
        CONSTRAINT DF_EvalRuns_RunAt DEFAULT SYSUTCDATETIME(),
    EvalDataset    NVARCHAR(100) NOT NULL,    -- 'support-v3', 'dba-runbooks-v1'
    CorpusVersion  INT NOT NULL,
    EmbeddingModel NVARCHAR(100) NOT NULL,
    ChunkStrategy NVARCHAR(40) NOT NULL,
    IndexType      NVARCHAR(40) NOT NULL,      -- 'DiskANN', 'exact', 'hybrid'
    GitSha         CHAR(40) NULL,              -- pipeline code version
    Notes          NVARCHAR(500) NULL
);
CREATE TABLE dbo.EvalScores (
    ScoreId BIGINT IDENTITY(1,1) NOT NULL
        CONSTRAINT PK_EvalScores PRIMARY KEY,
    RunId    BIGINT NOT NULL CONSTRAINT FK_EvalScores_Run
        REFERENCES dbo.EvalRuns (RunId),
    Metric   NVARCHAR(60) NOT NULL,          -- 'recall@10', 'faithfulness', ...
    Score    FLOAT NOT NULL,
    SampleN  INT NOT NULL
);
CREATE INDEX IX_EvalScores_Run_Metric ON dbo.EvalScores (RunId, Metric);
```

```
-- SQL Server: trend faithfulness across runs for one dataset
SELECT r.RunAt, r.GitSha, r.EmbeddingModel, s.Score AS Faithfulness
FROM dbo.EvalRuns AS r
JOIN dbo.EvalScores AS s ON s.RunId = r.RunId AND s.Metric = 'faithfulness'
WHERE r.EvalDataset = 'support-v3'
ORDER BY r.RunAt;
-- A drop here after a deploy is your rollback signal.
```

```
-- SQL Server: did the chunking change help? Compare two runs metric by metric
DECLARE @before BIGINT = 41, @after BIGINT = 42;
SELECT COALESCE(b.Metric, a.Metric) AS Metric,
       b.Score AS BeforeScore, a.Score AS AfterScore,
       a.Score - b.Score AS Delta
FROM (SELECT Metric, Score FROM dbo.EvalScores WHERE RunId = @before) AS b
FULL OUTER JOIN
    (SELECT Metric, Score FROM dbo.EvalScores WHERE RunId = @after) AS a
    ON a.Metric = b.Metric
ORDER BY Metric;
```

Production Monitoring: Online Metrics and Feedback Loops

Offline evals gate deployments; online metrics catch what evals miss. Instrument the production RAG path:

- → **Retrieval telemetry**: log top-1 distance per query. A rising median distance means the corpus is drifting away from the questions — new topics, new vocabulary, stale chunks.
- → **Refusal rate**: how often the pipeline answers "I don't have information about that." A spike means a corpus gap or a threshold misconfiguration, not a model failure.
- → **User feedback**: thumbs up/down on answers, joined to the retrieval log. The highest-value eval data you will ever get — feed confirmed-bad answers back into the golden set.
- → **Latency per stage**: embed, retrieve, rerank, generate (Chapter 25.8). Regressions here are operational, not quality — but users experience them as quality.

Run the golden-set eval as a scheduled job (a SQL Server Agent job or an external scheduler invoking the Python harness) and write results into `dbo.EvalRuns` / `dbo.EvalScores`. Alert when faithfulness or recall@k drops more than a threshold below the rolling baseline — that alert is the production equivalent of a failed unit test.

Building an Evaluation-Driven Development Loop

The loop: change one thing (chunk size, reranker threshold, embedding model) → run the eval harness → compare `dbo.EvalScores` before/after → keep or revert. One variable at a time, like any experiment. The teams that improve RAG quality fastest are not the ones with the best models — they are the ones with the tightest loop between idea and measured result.

Gate deployments on it: a CI step that runs the golden set and fails the build when faithfulness drops below the baseline turns quality regressions into build failures — visible, attributable, and reversible before users ever see them (Chapter 20 covers the CI benchmarking pattern this plugs into).

A Note on Evaluation Dataset Maintenance

Eval sets rot. Questions go stale as the product changes, gold chunks get re-chunked out of existence, and the set silently stops representing production traffic. Refresh it quarterly: sample real user questions from the retrieval logs, label a fresh batch, and retire questions

the corpus can no longer answer. Version the dataset (`EvalDataset = 'support-v4'`) and never compare scores across versions — compare within a version, and re-baseline when you cut a new one.

Key Takeaways

- → Measure retrieval (recall@k, MRR, context precision) and generation (faithfulness, answer relevance) separately — they fail independently and are fixed independently.
- → Store every eval run in SQL Server (`dbo.EvalRuns` / `dbo.EvalScores`) with corpus version, model, chunk strategy, and code SHA — quality trends are time-series data.
- → Score refusals as correct when the context genuinely lacks the answer, or your metrics will punish honesty and reward hallucination.
- → Run the golden set nightly as a scheduled job, alert on drops below the rolling baseline, and gate deployments on faithfulness like a unit test.
- → Feed production feedback (thumbs down, rising top-1 distances, refusal spikes) back into the golden set — the loop between idea and measured result is what compounds.

Chapter 28: Indexing Pipelines

The Vector Indexing Lifecycle

A vector index is not a fire-and-forget artifact — it has a lifecycle driven by the data underneath it:

1. **Source data changes** — a document is added, updated, or deleted in your primary store (SQL Server tables, S3, a CMS, a product catalog).
2. **Change detected** — CDC, a trigger-based outbox, or a polling watermark picks up the change.
3. **Embedding generated** — the changed text is chunked and embedded.
4. **Index updated** — vectors are written; the DiskANN index absorbs or is rebuilt around them.
5. **Quality verified** — recall checks confirm the index still serves correctly.

The freshness-performance tradeoff mirrors something DBAs already know from indexed views: the more aggressively you refresh, the closer to current your results, but the higher the maintenance overhead. SQL Server's DiskANN index in preview favors an index-after-load pattern — bulk ingest first, build the index once — over continuous trickle writes against an existing vector index. Design your pipeline around that constraint: buffer writes, then rebuild or refresh on a schedule your freshness SLA actually requires, not on every row change.

Building the Embedding Generation Pipeline

Here is a complete pipeline using OpenAI embeddings, writing into SQL Server native `VECTOR` with idempotent upserts:


```

# Python – chunk, embed, and upsert documents into SQL Server
import hashlib, openai, pyodbc

openai_client = openai.OpenAI()
conn = pyodbc.connect("DRIVER={ODBC Driver 18 for SQL Server};"
                      "SERVER=localhost;DATABASE=ragdb;Trusted_Connection=yes;"
                      "Encrypt=yes;TrustServerCertificate=yes")
MODEL_ID = "text-embedding-3-small"
DIM = 1536

def vec_lit(emb):
    return "[" + ",".join(str(v) for v in emb) + "]"

def chunk_text(text, size=512, overlap=64):
    # Simple word-based chunking; replace with tiktoken for production
    words = text.split()
    step = size - overlap
    return [" ".join(words[i:i + size])
            for i in range(0, len(words), step) if words[i:i + size]]

def upsert_document(doc_id, text, tenant_id):
    """Chunk, embed, and upsert a document into SQL Server."""
    chunks = chunk_text(text)
    resp = openai_client.embeddings.create(input=chunks, model=MODEL_ID)
    content_hash = hashlib.sha256(text.encode()).hexdigest()
    cur = conn.cursor()
    for idx, (chunk, e) in enumerate(zip(chunks, resp.data)):
        cur.execute("""MERGE dbo.DocumentChunks AS t
            USING (SELECT ? AS DocId, ? AS ChunkIndex, ? AS ChunkText,
                CAST(? AS VECTOR(1536)) AS Embedding,
                ? AS ModelId, ? AS TenantId,
                ? AS ContentHash) AS s
            ON t.DocId = s.DocId AND t.ChunkIndex = s.ChunkIndex
            WHEN MATCHED AND t.ContentHash <> s.ContentHash THEN
                UPDATE SET ChunkText = s.ChunkText, Embedding = s.Embedding,
                    ContentHash = s.ContentHash,
                    UpdatedAt = SYSUTCDATETIME()
            WHEN NOT MATCHED THEN
                INSERT (DocId, ChunkIndex, ChunkText, Embedding,
                    ModelId, TenantId, ContentHash)
                VALUES (s.DocId, s.ChunkIndex, s.ChunkText, s.Embedding,
                    s.ModelId, s.TenantId, s.ContentHash);""",
            (doc_id, idx, chunk, vec_lit(e.embedding),
             MODEL_ID, tenant_id, content_hash))
    conn.commit()
    return len(chunks)

```

The `ContentHash` guard is the pipeline's idempotency key: re-running the pipeline on an unchanged document updates nothing, so the job is safely re-runnable and the embedding API is never billed twice for the same text.

```
-- SQL Server: chunks table with content-hash idempotency
CREATE TABLE dbo.DocumentChunks (
    ChunkId      BIGINT IDENTITY(1,1) NOT NULL
        CONSTRAINT PK_DocumentChunks PRIMARY KEY,
    DocId        NVARCHAR(100) NOT NULL,
    ChunkIndex   INT NOT NULL,
    ChunkText    NVARCHAR(MAX) NOT NULL,
    Embedding    VECTOR(1536) NOT NULL,
    ModelId      NVARCHAR(100) NOT NULL,
    TenantId     INT NOT NULL,
    ContentHash  CHAR(64) NOT NULL,
    UpdatedAt    DATETIME2(0) NOT NULL
        CONSTRAINT DF_DocumentChunks_Updated DEFAULT SYSUTCDATETIME(),
    CONSTRAINT UQ_DocumentChunks_Doc_Chunk UNIQUE (DocId, ChunkIndex)
);
```

Change Data Capture for Continuous Indexing

Approach 1: Watermark polling. For freshness SLAs measured in minutes, a `ROWVERSION` or `UpdatedAt` watermark on the source table is the simplest reliable trigger. A scheduled job selects rows newer than the last watermark, runs them through the embedding pipeline, and advances the watermark in the same transaction.

```
-- SQL Server: watermark table + polling query
CREATE TABLE dbo.IndexWatermarks (
    PipelineName NVARCHAR(100) NOT NULL
        CONSTRAINT PK_IndexWatermarks PRIMARY KEY,
    LastWatermark DATETIME2(0) NOT NULL
);
-- Poller (runs every N minutes via SQL Server Agent):
-- SELECT DocId FROM dbo.SourceDocuments
-- WHERE UpdatedAt > (SELECT LastWatermark FROM dbo.IndexWatermarks
--                     WHERE PipelineName = 'doc-embeddings')
-- ORDER BY UpdatedAt;
-- ... embed each, upsert chunks, then:
-- UPDATE dbo.IndexWatermarks
-- SET LastWatermark = @maxSeen WHERE PipelineName = 'doc-embeddings';
```

Approach 2: SQL Server Change Data Capture. For sub-minute freshness requirements, polling does not cut it. Enable CDC on the source table and have the embedding consumer read the change tables — every insert/update/delete arrives as a row in `cdc.dbo_SourceDocuments_CT` with an operation code, no triggers required on the source table itself.

```
-- SQL Server: enable CDC on the source database and table
-- EXEC sys.sp_cdc_enable_db;
-- EXEC sys.sp_cdc_enable_table
--     @source_schema = 'dbo',
--     @source_name    = 'SourceDocuments',
--     @role_name       = NULL;
--
-- Consumer query: net changes since last LSN
-- DECLARE @from_lsn BINARY(10) = ...; -- persisted per pipeline
-- DECLARE @to_lsn   BINARY(10) = sys.fn_cdc_get_max_lsn();
-- SELECT __$operation, DocId FROM cdc.dbo_SourceDocuments_CT
-- WHERE __$start_lsn > @from_lsn AND __$start_lsn <= @to_lsn;
-- __$operation: 1 = delete, 2 = insert, 3 = update (before), 4 = update (after)
```

Deletes need explicit handling in both approaches: a deleted source document must delete its chunks, or the vector index will keep serving ghost content. The CDC operation code makes this natural — map operation 1 to a chunk delete; with watermark polling, use soft deletes (`IsDeleted` flag) on the source so the poller can see them.

Index Optimization and Segment Management

DiskANN behavior under writes. In SQL Server 2025 preview, the pragmatic pattern is index-after-load: bulk ingest into the heap or clustered table first, then `CREATE VECTOR INDEX` once. For ongoing trickle writes, accumulate them in a staging table and rebuild the vector index on a schedule — nightly for most workloads, hourly only if the freshness SLA demands it.

The production pattern is to **rebuild the vector index during a low-traffic window** after bulk ingestion:

```
-- SQL Server: scheduled DiskANN rebuild after bulk ingestion
-- 1. Ingest into the base table (no vector index yet, or staged separately)
-- 2. Drop the stale vector index if one exists:
-- DROP INDEX IX_DocumentChunks_Embedding ON dbo.DocumentChunks;
-- 3. Rebuild:
CREATE VECTOR INDEX IX_DocumentChunks_Embedding
    ON dbo.DocumentChunks (Embedding)
    WITH (METRIC = 'cosine', TYPE = 'DiskANN', MAXDOP = 8);
-- 4. Verify with DMVs before serving traffic:
SELECT i.name, i.type_desc, p.rows
FROM sys.indexes AS i
JOIN sys.partitions AS p
    ON p.object_id = i.object_id AND p.index_id = i.index_id
WHERE i.object_id = OBJECT_ID('dbo.DocumentChunks');
```

Plan the rebuild window from measured build times on your hardware and vector counts — DiskANN builds are CPU-intensive and parallelize with `MAXDOP` . Until the index exists or while it is being rebuilt, the exact `VECTOR_DISTANCE` scan path still serves correct (if slower) results, so schedule the rebuild like any other maintenance: off-peak, monitored, with a rollback plan.

Handling Embedding Model Changes

A model change invalidates every vector in the index — the pipeline must re-embed the full corpus, not just new documents. This is the heaviest workload the indexing pipeline will ever run: full-corpus embedding API spend plus a full table rewrite plus a DiskANN rebuild. Treat it as a planned migration (Chapter 23), not a routine pipeline run: side-by-side column, throttled backfill, eval-gated cutover.

Recall Degradation Detection

Indexes degrade silently — recall drops a point a month and nobody notices until users complain. Detect it with a canary: a fixed set of eval questions with known-good chunks (Chapter 27), run on a schedule against production, with results trended in `dbo.EvalScores` . A falling recall@k trend after heavy ingest periods is the signal to rebuild the DiskANN index; a falling trend with no ingest correlation points at corpus drift — new topics the chunking or model handles poorly.

```
-- SQL Server: canary recall check (scheduled, trended)
-- EvalCanary(QuestionId, QuestionEmbedding VECTOR(1536), GoldChunkId)
-- For each canary question: is GoldChunkId in the top-10?
-- Log the hit rate per run into dbo.EvalScores with Metric = 'canary_recall@10'.
-- Alert when the 7-day rolling average drops > 3 points below baseline.
```

Backfill Strategies for Large Datasets

The initial load — or a full re-embed after a model change — is a bulk operation, not a trickle. The pattern that works:

- → **Stage, then swap.** Backfill into a staging table (or a new table version), build the DiskANN index on the staging table, validate recall on the canary set, then switch the application to the new table — via synonym repointing or a view — in one transaction. The old table is the instant rollback.
- → **Throttle the write side.** Batch embedding API calls at the provider's rate limits, commit the database writes in bounded batches (10–50K rows), and checkpoint progress so an interrupted backfill resumes, not restarts.
- → **Mind the log.** A full-corpus rewrite generates transaction log proportional to the data. Size log growth, consider batch commits with log backups between them, and never run the first full backfill unmonitored.

```
# Python – resumable backfill with checkpoints
import pyodbc

conn = pyodbc.connect("DRIVER={ODBC Driver 18 for SQL Server};"
                      "SERVER=localhost;DATABASE=ragdb;Trusted_Connection=yes;"
                      "Encrypt=yes;TrustServerCertificate=yes")

BATCH = 20000
cur = conn.cursor()
while True:
    rows = cur.execute(f"""SELECT TOP ({BATCH}) DocId, BodyText
                           FROM dbo.BackfillQueue
                           WHERE Status = 'pending'
                           ORDER BY DocId""").fetchall()

    if not rows:
        break
    for doc_id, body in rows:
        upsert_document(doc_id, body, tenant_id=1) # from Section 28.2
        cur.execute("UPDATE dbo.BackfillQueue SET Status = 'done' "
                    "WHERE DocId = ?", (doc_id,))
    conn.commit() # checkpoint per batch: kill-safe and resumable
print("Backfill complete.")
```

Key Takeaways

- → **Design around index-after-load.** SQL Server's DiskANN preview favors bulk ingest followed by index build over trickle writes against a live vector index. Buffer writes, then rebuild on a schedule your freshness SLA requires.
- → **Make ingestion idempotent.** Content-hash guards and `MERGE` upserts mean the pipeline is safely re-runnable and never double-bills the embedding API.

- → **Match change capture to the SLA.** Watermark polling for minutes-level freshness, SQL Server CDC for sub-minute — and handle deletes explicitly in both, or the index serves ghost content.
- → **Backfill by stage-and-swap.** New table, new index, canary-validated, synonym-switched — with the old table as instant rollback and checkpointed batches throughout.
- → **Detect degradation with a canary.** Trend recall@k on a fixed question set in `dbo.EvalScores` ; falling recall after heavy ingest means rebuild the index, falling recall without ingest means corpus drift.

Chapter 29: Real-Time vs Batch Embedding Generation

29.1 Understanding the Embedding Generation Pipeline

Every vector in your database was produced by an embedding call — an API request or a GPU inference — and that production step has its own architecture decisions: when embeddings are generated (at write time or in bulk later), how failures are retried, and how the pipeline scales with document volume. Get the timing wrong and you either pay synchronous latency on every write or serve stale search results for hours.

The two pure strategies are real-time (embed synchronously as documents arrive) and batch (queue documents, embed in bulk on a schedule). Most production systems end up hybrid. This chapter gives you the patterns for each, implemented against SQL Server, and a framework for choosing.

29.2 Real-Time Embedding Generation

The following example shows a FastAPI endpoint that ingests documents and generates embeddings synchronously using the OpenAI API and SQL Server native `VECTOR` for storage:

```

# Python – FastAPI synchronous embedding ingestion into SQL Server
import hashlib, openai, pyodbc
from fastapi import FastAPI, HTTPException

app = FastAPI()
openai_client = openai.OpenAI()
conn = pyodbc.connect("DRIVER={ODBC Driver 18 for SQL Server};"
                      "SERVER=localhost;DATABASE=ragdb;Trusted_Connection=yes;"
                      "Encrypt=yes;TrustServerCertificate=yes;MARS_Connection=yes")

@app.post("/documents")
def ingest_document(doc_id: str, text: str, tenant_id: int):
    # Step 1: embed synchronously (caller waits for the API round-trip)
    try:
        emb = openai_client.embeddings.create(
            input=[text], model="text-embedding-3-small").data[0].embedding
    except Exception as e:
        raise HTTPException(502, f"Embedding provider failed: {e}")
    vec = "[" + ",".join(str(v) for v in emb) + "]"
    content_hash = hashlib.sha256(text.encode()).hexdigest()
    # Step 2: write to SQL Server
    cur = conn.cursor()
    cur.execute("""MERGE dbo.DocumentChunks AS t
        USING (SELECT ? AS DocId, ? AS ChunkText,
                     CAST(? AS VECTOR(1536)) AS Embedding,
                     ? AS ContentHash, ? AS TenantId) AS s
        ON t.DocId = s.DocId AND t.ChunkIndex = 0
        WHEN MATCHED AND t.ContentHash <> s.ContentHash THEN
            UPDATE SET ChunkText = s.ChunkText, Embedding = s.Embedding,
                     ContentHash = s.ContentHash,
                     UpdatedAt = SYSUTCDATETIME()
        WHEN NOT MATCHED THEN
            INSERT (DocId, ChunkIndex, ChunkText, Embedding,
                  ModelId, TenantId, ContentHash)
            VALUES (s.DocId, 0, s.ChunkText, s.Embedding,
                  'text-embedding-3-small', s.TenantId, s.ContentHash);""",
        (doc_id, text, vec, content_hash, tenant_id))
    conn.commit()
    return {"doc_id": doc_id, "status": "indexed"}

```

Real-time is the right choice when freshness is the product: a support agent that must search a ticket the moment it is filed, a product catalog where new SKUs must be searchable immediately. The costs are synchronous latency (every write waits for the embedding API), coupled failure modes (embedding provider outage blocks your writes), and per-request API pricing with no batching discount.

Harden the failure mode: never let an embedding failure lose the document. Write the raw text first with a NULL embedding and an `EmbeddingStatus = 'pending'` flag, then fill it in — synchronously in the fast path, or via the batch worker in Section 29.3 when the provider is down. The document is never lost; only its searchability is delayed.

29.3 Batch Embedding Generation

Batch generation decouples document arrival from embedding: writes land instantly with `EmbeddingStatus = 'pending'`, and a worker fleet drains the queue in bulk. You get API batching discounts, retry isolation (provider outage delays searchability, not writes), and controllable load — at the cost of freshness measured in minutes instead of milliseconds.

```
-- SQL Server: embedding work queue (status-driven, no SKIP LOCKED needed)
CREATE TABLE dbo.EmbeddingQueue (
    QueueId          BIGINT IDENTITY(1,1) NOT NULL
        CONSTRAINT PK_EmbeddingQueue PRIMARY KEY,
    DocId            NVARCHAR(100) NOT NULL,
    ChunkIndex       INT NOT NULL,
    ChunkText        NVARCHAR(MAX) NOT NULL,
    TenantId         INT NOT NULL,
    Status           NVARCHAR(20) NOT NULL
        CONSTRAINT DF_EmbeddingQueue_Status DEFAULT ('pending'),
    Attempts         INT NOT NULL CONSTRAINT DF_EmbeddingQueue_Attempts DEFAULT (0),
    LastError        NVARCHAR(1000) NULL,
    EnqueuedAt       DATETIME2(0) NOT NULL
        CONSTRAINT DF_EmbeddingQueue_Enq DEFAULT SYSUTCDATETIME(),
    CONSTRAINT UQ_EmbeddingQueue_Doc_Chunk UNIQUE (DocId, ChunkIndex)
);
CREATE INDEX IX_EmbeddingQueue_Status_Id
    ON dbo.EmbeddingQueue (Status, QueueId);
-- Workers claim batches with an atomic status transition:
-- UPDATE TOP (100) dbo.EmbeddingQueue
-- SET Status = 'processing', Attempts = Attempts + 1
-- OUTPUT inserted.QueueId, inserted.DocId, inserted.ChunkText
-- WHERE Status = 'pending';
-- Dead-letter after N attempts: Status = 'failed' for manual review.
```

The `UPDATE TOP ... OUTPUT` claim pattern is SQL Server's answer to the queue-consumer problem: each worker atomically claims its batch, processes it, and writes results back. Multiple workers run in parallel without coordination — the status transition is the lock, and a crashed worker's `'processing'` rows are reclaimed by a sweeper job after a timeout.

```

# Python – batch embedding worker: claim, embed in bulk, write back
import time, openai, pyodbc

openai_client = openai.OpenAI()
conn = pyodbc.connect("DRIVER={ODBC Driver 18 for SQL Server};"
                      "SERVER=localhost;DATABASE=ragdb;Trusted_Connection=yes;"
                      "Encrypt=yes;TrustServerCertificate=yes")

BATCH = 100
MAX_ATTEMPTS = 5

def claim_batch():
    cur = conn.cursor()
    rows = cur.execute("""UPDATE TOP (%d) dbo.EmbeddingQueue
                          SET Status = 'processing',
                              Attempts = Attempts + 1
                          OUTPUT inserted.QueueId, inserted.DocId,
                              inserted.ChunkIndex, inserted.ChunkText,
                              inserted.TenantId, inserted.Attempts
                          WHERE Status = 'pending'"" % BATCH).fetchall()

    conn.commit()
    return rows

def run_worker():
    while True:
        batch = claim_batch()
        if not batch:
            time.sleep(10); continue
        try:
            resp = openai_client.embeddings.create(
                input=[r[3] for r in batch], model="text-embedding-3-small")
            cur = conn.cursor()
            for row, e in zip(batch, resp.data):
                qid, doc_id, cidx, text, tenant, attempts = row
                vec = "[" + ",".join(str(v) for v in e.embedding) + "]"
                cur.execute("""MERGE dbo.DocumentChunks AS t
                              USING (SELECT ? AS DocId, ? AS ChunkIndex, ? AS ChunkText,
                                              CAST(? AS VECTOR(1536)) AS Embedding,
                                              ? AS TenantId) AS s
                              ON t.DocId = s.DocId AND t.ChunkIndex = s.ChunkIndex
                              WHEN MATCHED THEN UPDATE SET ChunkText = s.ChunkText,
                                  Embedding = s.Embedding, UpdatedAt = SYSUTCDATETIME()
                              WHEN NOT MATCHED THEN INSERT
                                  (DocId, ChunkIndex, ChunkText, Embedding,
                                   ModelId, TenantId, ContentHash)
                                  VALUES (s.DocId, s.ChunkIndex, s.ChunkText, s.Embedding,
                                           'text-embedding-3-small', s.TenantId, '');""",
                                      (doc_id, cidx, text, vec, tenant))
                cur.execute("DELETE FROM dbo.EmbeddingQueue WHERE QueueId = ?",
                            (qid,))

            conn.commit()
        except Exception as ex:
            conn.rollback()
            cur = conn.cursor() # requeue or dead-letter per row

```

```

for row in batch:
    status = ('failed' if row[5] >= MAX_ATTEMPTS else 'pending')
    cur.execute("""UPDATE dbo.EmbeddingQueue
                  SET Status = ?, LastError = ?
                  WHERE QueueId = ?""",
                (status, str(ex)[:1000], row[0]))
conn.commit()
time.sleep(30)  # back off on provider errors

```

29.4 Hybrid Architectures: Combining Both Strategies

Production reality is hybrid: a fast path for documents that must be searchable now, and a batch path for everything else. Route by document priority — user-uploaded content and tickets go synchronous; bulk imports, re-embeds, and backfills go through the queue. Both paths write through the same `MERGE` upsert, so ordering conflicts resolve to "last writer wins" and the content hash prevents redundant API calls when both paths touch the same document.

```

graph LR
    A[Document arrives] --> B{Priority?}
    B -->|urgent| C[Sync embed  
+ upsert]
    B -->|bulk| D[Enqueue  
pending]
    D --> E[Batch worker  
claim → embed → upsert]
    C --> F[(dbo.DocumentChunks  
MERGE upsert)]
    E --> F

```

29.5 Cost Optimization and Model Selection

- → **Batch for the discount.** Embedding APIs price per token regardless, but batching lets you use cheaper async tiers, avoid per-request overhead, and deduplicate via the content hash before spending a cent.
- → **Deduplicate before embedding.** The content-hash check in the `MERGE` means re-ingested unchanged documents cost nothing. For near-duplicate detection at scale, hash normalized text (lowercased, whitespace-collapsed).
- → **Right-size the model.** A 1,536-dimension model costs roughly 3x the tokens-per-dollar of aggressive truncation to 512 dimensions (Chapter 23). Run the eval delta before committing the corpus to the cheaper representation.

- → **Local models for the query path.** If per-question API cost dominates, embed questions locally with Sentence Transformers — provided the corpus was embedded with the same model.

29.6 Monitoring and Observability

```
-- SQL Server: embedding pipeline health dashboard
-- Queue depth and oldest pending item (freshness SLA signal):
SELECT Status, COUNT(*) AS Items,
       MIN(EnqueuedAt) AS OldestEnqueued,
       AVG(Attempts) AS AvgAttempts
FROM dbo.EmbeddingQueue
GROUP BY Status;
-- Dead letters needing attention:
SELECT TOP (50) QueueId, DocId, Attempts, LastError, EnqueuedAt
FROM dbo.EmbeddingQueue
WHERE Status = 'failed'
ORDER BY EnqueuedAt DESC;
-- Ingest freshness: documents updated but not yet searchable
-- (chunks missing or stale vs source watermark) – join against
-- dbo.IndexWatermarks from Chapter 28 for the end-to-end lag number.
```

Alert on three signals: queue depth growing faster than drain rate (workers under-provisioned or provider throttling), dead-letter growth (systematic failure, not transient), and oldest-pending age exceeding the freshness SLA. The end-to-end lag number — source change to searchable vector — is the pipeline's headline SLI; trend it, not just the queue depth.

29.7 Re-Embedding and Index Migration

A model change re-embeds the entire corpus — the batch pipeline is how you execute it without an outage. Enqueue every document with a migration tag, let the workers drain at a throttled rate, and write into the side-by-side `Embedding_v2` column (Chapter 23) rather than overwriting the live vectors. Retrieval keeps serving from the old column until the eval-gated cutover flips the query path.

```
-- SQL Server: enqueue a full-corpus re-embed for model migration
INSERT INTO dbo.EmbeddingQueue (DocId, ChunkIndex, ChunkText, TenantId, Status)
SELECT DocId, ChunkIndex, ChunkText, TenantId, 'pending'
FROM dbo.DocumentChunks;
-- Workers pick these up with the NEW model id; the MERGE targets
-- Embedding_v2 (see Chapter 23). Throttle by limiting worker count
-- during business hours – the old vectors keep serving meanwhile.
```

29.8 Choosing Your Strategy: A Practical Framework

- → **Freshness SLA under 1 minute?** Real-time path. Accept the coupling and harden the failure mode with pending-status rows.
- → **Freshness SLA in minutes, high volume?** Batch workers with `UPDATE TOP ... OUTPUT` claiming. Scales horizontally, isolates provider failures.
- → **Mixed workload?** Hybrid: urgent documents synchronous, bulk through the queue, one `MERGE` upsert underneath.
- → **Cost-dominated?** Batch everything, deduplicate by content hash, and evaluate dimension truncation before paying full rate.

Key Takeaways

- → **Real-time embeds on the write path** for sub-minute freshness; never let an embedding failure lose the document — persist text first with a pending flag.
- → **Batch workers claim with** `UPDATE TOP ... OUTPUT` — atomic, parallel-safe, no external queue required. Multiple workers scale horizontally with zero coordination.
- → **Content-hash idempotency** in the `MERGE` upsert prevents double-billing the embedding API and makes every path safely re-runnable.
- → **Hybrid is the production default:** urgent documents synchronous, bulk through the queue, one upsert underneath, model migrations as throttled re-embed waves into a side-by-side column.
- → **Monitor the end-to-end lag** (source change → searchable vector) as the headline SLI, with queue depth, dead letters, and oldest-pending age as the alerting signals.

Chapter 30: Embedding Model Versioning

Why Model Changes Break Vector Search

Embedding models are versioned artifacts, and vendors update them: new training data, new architectures, deprecated endpoints. Every model change breaks vector search in one of two ways. **Dimension mismatch** is the loud one — the new model produces vectors of a different length than the old model. SQL Server's `VECTOR(1536)` column throws a hard error if you attempt to insert a 1,024-dimensional vector. This is the easy failure to catch.

Semantic drift is the quiet one — same dimension, different geometry. A new model version with 1,536 dimensions produces vectors that fit your column perfectly and mean something completely different. Queries run, distances look plausible, and retrieval quality degrades without a single error. This is the failure that costs you users, and it is why model identity must be a first-class column in your schema, not a deployment note.

Detecting Degradation Before and After Migration

Never migrate on faith. Before committing to a new model, run the canary: embed your eval question set with both models, retrieve from side-by-side columns, and compare recall@k and faithfulness (Chapters 22.8, 27). Migrate only when the new model meets or beats the old on your data — vendor benchmark improvements do not always transfer to domain corpora.

After cutover, keep the old column for one full business cycle as the control. If production metrics (refusal rate, thumbs-down rate, top-1 distance distribution) degrade, the rollback is a query-path switch, not a re-ingest — because the old vectors are still there.

```
-- SQL Server: pre-migration A/B comparison on the eval set
-- Documents(Embedding = old model, Embedding_v2 = new model backfilled)
-- EvalQuestions(QuestionId, QOld VECTOR(1536), QNew VECTOR(1536), GoldChunkId)
SELECT 'old_model' AS Model,
      AVG(CASE WHEN g.GoldChunkId IN (
        SELECT TOP (10) d.ChunkId FROM dbo.Documents AS d
        ORDER BY VECTOR_DISTANCE('cosine', d.Embedding, q.QOld))
      THEN 1.0 ELSE 0.0 END) AS RecallAt10
FROM dbo.EvalQuestions AS q JOIN dbo.GoldChunks AS g
  ON g.QuestionId = q.QuestionId
UNION ALL
SELECT 'new_model',
      AVG(CASE WHEN g.GoldChunkId IN (
        SELECT TOP (10) d.ChunkId FROM dbo.Documents AS d
        WHERE d.Embedding_v2 IS NOT NULL
        ORDER BY VECTOR_DISTANCE('cosine', d.Embedding_v2, q.QNew))
      THEN 1.0 ELSE 0.0 END)
FROM dbo.EvalQuestions AS q JOIN dbo.GoldChunks AS g
  ON g.QuestionId = q.QuestionId;
-- Migrate only if new_model RecallAt10 >= old_model on YOUR questions.
```

Migration Strategies: Zero-Downtime Approaches

The zero-downtime pattern is side-by-side: add a second vector column, backfill it with the new model, build a second DiskANN index, eval-gate the cutover, then retire the old column. Retrieval never stops serving — it just switches which column the query reads.

```
-- SQL Server: zero-downtime embedding model migration
-- Phase 1: add the new column (nullable; old column keeps serving)
ALTER TABLE dbo.Documents ADD Embedding_v2 VECTOR(3072) NULL;
-- Phase 2: backfill in throttled batches (see Section 30.4)
-- Phase 3: index the new column once backfill completes
CREATE VECTOR INDEX IX_Documents_Embedding_v2
  ON dbo.Documents (Embedding_v2)
  WITH (METRIC = 'cosine', TYPE = 'DiskANN', MAXDOP = 8);
-- Phase 4: cutover – retrieval procedure switches column + model predicate
--   (deploy behind a feature flag; eval-gate before flipping)
-- Phase 5: after one stable business cycle, retire the old column
-- DROP INDEX IX_Documents_Embedding ON dbo.Documents;
-- ALTER TABLE dbo.Documents DROP COLUMN Embedding;
-- EXEC sp_rename 'dbo.Documents.Embedding_v2', 'Embedding', 'COLUMN';
```

The retrieval procedure must be model-aware from day one — a `@modelVersion` parameter that selects the column and the `ModelName` predicate. If the model choice is hardcoded in application code, every migration becomes a code deploy instead of a flag flip, and rollbacks get slower exactly when you need them fast.

```

graph LR
    A[Phase 1  
Add Embedding_v2] --> B[Phase 2  
Throttled backfill]
    B --> C[Phase 3  
Build DiskANN v2]
    C --> D[Phase 4  
Eval-gated cutover  
feature flag]
    D --> E[Phase 5  
Retire old column  
after stable cycle]

```

Backfill Job Design: Resumability and Rate Limiting

The backfill is the heaviest write workload your vector table will ever see: full-corpus embedding API spend plus a full column rewrite. Design it like any bulk job — resumable, throttled, observable:

```

-- SQL Server: migration backfill queue (resumable, observable)
CREATE TABLE dbo.ModelMigrationQueue (
    QueueId    BIGINT IDENTITY(1,1) NOT NULL
        CONSTRAINT PK_ModelMigrationQueue PRIMARY KEY,
    ChunkId    BIGINT NOT NULL,
    Status     NVARCHAR(20) NOT NULL
        CONSTRAINT DF_MMQ_Status DEFAULT ('pending'),
    Attempts   INT NOT NULL CONSTRAINT DF_MMQ_Attempts DEFAULT (0),
    EnqueuedAt DATETIME2(0) NOT NULL
        CONSTRAINT DF_MMQ_Enq DEFAULT SYSUTCDATETIME()
);
CREATE INDEX IX_MMQ_Status ON dbo.ModelMigrationQueue (Status, QueueId);
-- Seed: one row per chunk. Workers claim with UPDATE TOP ... OUTPUT
-- (Chapter 29.3 pattern), embed with the NEW model, and:
-- UPDATE dbo.Documents SET Embedding_v2 = CAST(@vec AS VECTOR(3072))
-- WHERE ChunkId = @chunkId;
-- Throttle: run fewer workers during business hours; the old column
-- keeps serving, so backfill speed is a cost decision, not an outage.

```

```

-- SQL Server: backfill progress dashboard
SELECT Status, COUNT(*) AS Items,
       MIN(EnqueuedAt) AS OldestItem
FROM   dbo.ModelMigrationQueue
GROUP BY Status;
-- And verify no NULLs remain before building the v2 index:
-- SELECT COUNT(*) FROM dbo.Documents WHERE Embedding_v2 IS NULL;

```


Rate-limit against the embedding provider's quota, not your database's throughput — the API is always the bottleneck. Checkpoint per batch so a killed worker resumes instead of restarting, and keep the backfill's transaction scope small: one batch per transaction, never the whole corpus.

Handling Dimension Changes: The Hard Case

In SQL Server, vector columns are typed with a fixed dimension: `VECTOR(1536)` will reject a `VECTOR(1024)` insert with a hard error. You have two options:

Option A: side-by-side column with the new dimension (recommended). `ALTER TABLE ... ADD Embedding_v2 VECTOR(1024)` — the pattern from Section 30.3. Clean, explicit, and the old column remains a rollback target. The cost is temporary double storage during migration.

Option B: in-place migration with downtime. Drop the vector index, alter the column type, re-embed everything, rebuild. Only acceptable for small corpora in maintenance windows — and even then, Option A's safety margin usually justifies its storage cost.

```
-- SQL Server: dimension change via side-by-side column
-- Old: Embedding VECTOR(1536) | New model: 1024 dimensions
ALTER TABLE dbo.Documents ADD Embedding_v2 VECTOR(1024) NULL;
-- Backfill Embedding_v2 with the new model (Section 30.4 pattern),
-- then index and cut over exactly as in Section 30.3.
-- The fixed-dimension type is doing its job: a 1536-d vector can never
-- silently land in the 1024-d column. That strictness is the feature
-- that makes Option A safe.
```

Never "pad" or "truncate" vectors to force a dimension fit — zero-padding a 1024-d vector into a 1536-d column produces vectors that compare meaninglessly against true 1536-d vectors. Dimension is identity; treat it as such.

Version Pinning and Model Deprecation Tracking

Pin exact model versions in configuration — `text-embedding-3-small` today may resolve to a different snapshot tomorrow if the vendor updates the alias. Store the resolved version (model name plus version date or snapshot ID) in the `ModelName` column at ingest time, so every vector carries its provenance.

```
-- SQL Server: model deprecation tracking
CREATE TABLE dbo.EmbeddingModelRegistry (
    ModelName NVARCHAR(100) NOT NULL
        CONSTRAINT PK_EmbeddingModelRegistry PRIMARY KEY,
    Vendor NVARCHAR(60) NOT NULL,
    Dimensions INT NOT NULL,
    DeprecatedAt DATETIME2(0) NULL, -- vendor deprecation notice date
    RetireAt DATETIME2(0) NULL, -- when the API goes dark
    ReplacedBy NVARCHAR(100) NULL,
    MigrationStatus NVARCHAR(20) NOT NULL
        CONSTRAINT DF_EMR_Status DEFAULT ('current')
);
-- Ops review query: which models in production are on a deprecation clock?
SELECT r.ModelName, r.RetireAt, r.ReplacedBy,
    (SELECT COUNT(*) FROM dbo.Documents AS d
     WHERE d.ModelName = r.ModelName) AS VectorsAffected
FROM dbo.EmbeddingModelRegistry AS r
WHERE r.RetireAt IS NOT NULL
ORDER BY r.RetireAt;
-- A vendor deprecation notice starts the migration clock in Section 30.3:
-- the RetireAt date is your backfill deadline.
```

Treat vendor deprecation notices like certificate expirations: they arrive with a deadline, the migration takes longer than you expect, and the cost of missing the deadline is an outage. The registry turns a surprise into a scheduled project.

Key Takeaways

- → **Dimension mismatch fails loud; semantic drift fails silent.** The `VECTOR(n)` type catches the first; only eval-gated migrations catch the second.
- → **Migrate side-by-side, never in place.** New column, throttled resumable backfill, second DiskANN index, eval-gated flag flip, old column retained as rollback for a full business cycle.
- → **Make retrieval model-aware from day one** — a `@modelVersion` parameter selecting column and predicate turns migrations into flag flips instead of code deploys.
- → **Never pad or truncate vectors to force a dimension fit.** Dimension is identity; a mismatched vector in the wrong column is silent corruption.
- → **Track deprecations like certificate expirations.** Pin exact model versions, record vendor retirement dates in a registry, and let the retirement date set your backfill deadline.

Chapter 31: Multi-Modal Search: Text and Image Together

Most production search systems today handle one modality at a time — a text query searches text, an image query searches images. But users do not experience information in those silos. A retail customer uploads a photo of shoes and types "like these but in red." A medical researcher searches with an MRI scan and a clinical note simultaneously. A news archive needs to find articles where the image and headline tell contradictory stories. These compound queries require a database layer that understands both text and images as first-class embedding types and can combine them during retrieval. This chapter covers the embedding models that bridge the modality gap, the storage and indexing strategies for mixed-type vectors, the query architectures that fuse signals from both dimensions, and the production patterns that make multi-modal search scale reliably.

The Embedding Bridge: How Text and Images Share a Vector Space

The foundational insight behind multi-modal search is that text and images can be embedded into the *same* vector space — a shared coordinate system where semantic proximity means the same thing regardless of modality. A vector for the text "golden retriever playing fetch" and a vector for an actual photograph of that scene should land close together in this space. This is not obvious from first principles; it required learning a joint embedding model from hundreds of millions of image-caption pairs.

CLIP and Its Descendants

OpenAI's CLIP (Contrastive Language-Image Pre-training, 2021) established the practical template. CLIP trains a text encoder and an image encoder jointly using contrastive loss — during training, the model pulls matching image-text pairs together while pushing non-matching pairs apart. The result is two encoders that produce vectors in the same 512- or 768-dimensional space.

```

graph LR
    A["Text: 'red running shoes'"] --> B[Text encoder]
    C[Image: shoe photo] --> D[Image encoder]
    B --> E[(Shared CLIP  
vector space)]
    D --> E
    E --> F[Cosine similarity  
ranks both against  
each other]
    style E fill:#c8e6c9

```

Text and image encoders trained jointly land vectors in the same coordinate system — this is what makes cross-modal retrieval possible.

For production use, the relevant CLIP derivatives include:

Model	Dimensions	Notes
-------	------------	-------

openai/clip-vit-base-patch32	512	Baseline; widely benchmarked
openai/clip-vit-large-patch14	768	Better quality, higher latency
laion/CLIP-ViT-H-14-laion2B-s32B-b79K	1024	Trained on LAION-2B; strong zero-shot
sentence-transformers/clip-ViT-B-32	512	HuggingFace wrapper; easier pipeline
Google's SigLIP	1152	Sigmoid loss variant; better recall at scale
OpenAI text-embedding-3-large	3072	Text only; not image-capable

The practical implication for database architects: you are storing vectors from the *same* model family in the same table or collection. If you embed images with `clip-vit-large-patch14`, you must embed all query text with that same model's text encoder. Mixing encoders from different CLIP checkpoints produces meaningless distance comparisons — equivalent to storing lengths in meters in one column and feet in another and comparing them without conversion.

Dimensionality and Distance Metrics

CLIP vectors use cosine similarity as the natural distance metric because the encoders produce unit-normalized vectors by default. In SQL Server terms, that means the `VECTOR` type with `VECTOR_DISTANCE('cosine', ...)` rather than `'euclidean'`. Pinecone, Weaviate, and Qdrant all support cosine similarity natively and it should be your default for CLIP-family embeddings.

One architectural decision that often surprises DBAs: you can store text embeddings and image embeddings in the **same** column when they share the same model and dimensionality. There is no type system distinction between "image vector" and "text vector" at the database level. The distinction is carried by your application logic and metadata columns. This is similar to storing both customer and product records in a single `entities` table — possible, but you need a `type` discriminator column to avoid joining apples with oranges in business queries.

Storage Architecture for Mixed-Modal Data

Before writing any queries, you need a schema that accommodates multiple modalities without creating a maintenance nightmare. There are three patterns worth knowing, each with different tradeoffs.

Pattern 1: Single Table, Shared Vector Column

All embeddings live in one table. A `modality` column distinguishes text from image. This works well when text and image documents are genuinely interchangeable from a business perspective — a product catalog where each item has both a description text and one or more product images.

```

-- SQL Server 2025
-- Single-table multi-modal schema for a product catalog
CREATE TABLE dbo.ProductEmbeddings (
    Id          BIGINT IDENTITY(1,1) NOT NULL
        CONSTRAINT PK_ProductEmbeddings PRIMARY KEY,
    ProductId   BIGINT NOT NULL
        CONSTRAINT FK_ProductEmbeddings_Products
        REFERENCES dbo.Products (Id),
    Modality    NVARCHAR(10) NOT NULL
        CONSTRAINT CK_ProductEmbeddings_Modality
        CHECK (Modality IN ('text', 'image')),
    SourceRef   NVARCHAR(500) NOT NULL,    -- field name or image URL/path
    Embedding   VECTOR(512) NOT NULL,      -- CLIP ViT-B/32 dimensions
    CreatedAt   DATETIME2(0) NOT NULL
        CONSTRAINT DF_ProductEmbeddings_Created DEFAULT SYSUTCDATETIME(),
    MetadataJson NVARCHAR(MAX) NULL
        CONSTRAINT CK_ProductEmbeddings_Json CHECK (ISJSON(MetadataJson) = 1)
);
-- One DiskANN index over the shared column; modality is a query-time filter
CREATE VECTOR INDEX IX_ProductEmbeddings_Embedding
    ON dbo.ProductEmbeddings (Embedding)
    WITH (METRIC = 'cosine', TYPE = 'DiskANN', MAXDOP = 8);
-- Composite index for filtered product lookups
CREATE INDEX IX_ProductEmbeddings_Product_Modality
    ON dbo.ProductEmbeddings (ProductId, Modality);

```

On the single-table pattern, keep one DiskANN index over the shared `Embedding` column and pass `Modality = 'text'` as a query-time predicate (SQL Server has no partial vector indexes). The modality filter narrows candidates before the similarity ranking is applied; the composite index on `(ProductId, Modality)` covers the filtered product lookups. If text and image pipelines diverge in update rate or dimensionality, prefer Pattern 2 instead of fighting one shared index.

Pattern 2: Separate Tables per Modality

More explicit and easier to maintain independently. Use this when text and image pipelines have very different update rates or when the embedding models differ in dimensionality.

```
-- SQL Server 2025
-- Separate tables for text and image embeddings
CREATE TABLE dbo.DocumentTextEmbeddings (
    Id          BIGINT IDENTITY(1,1) NOT NULL
        CONSTRAINT PK_DocumentTextEmbeddings PRIMARY KEY,
    DocId       BIGINT NOT NULL,
    ChunkIndex  INT NOT NULL CONSTRAINT DF_DTE_ChunkIndex DEFAULT (0),
    Content     NVARCHAR(MAX) NOT NULL,
    Embedding   VECTOR(1536) NOT NULL,    -- OpenAI text-embedding-3-small
    MetadataJson NVARCHAR(MAX) NULL,
    CONSTRAINT UQ_DTE_Doc_Chunk UNIQUE (DocId, ChunkIndex)
);
CREATE TABLE dbo.DocumentImageEmbeddings (
    Id          BIGINT IDENTITY(1,1) NOT NULL
        CONSTRAINT PK_DocumentImageEmbeddings PRIMARY KEY,
    DocId       BIGINT NOT NULL,
    ImageIndex  INT NOT NULL CONSTRAINT DF_DIE_ImageIndex DEFAULT (0),
    ImagePath   NVARCHAR(500) NOT NULL,
    Embedding   VECTOR(512) NOT NULL,    -- CLIP ViT-B/32
    MetadataJson NVARCHAR(MAX) NULL,
    CONSTRAINT UQ_DIE_Doc_Image UNIQUE (DocId, ImageIndex)
);
CREATE VECTOR INDEX IX_DTE_Embedding
    ON dbo.DocumentTextEmbeddings (Embedding)
    WITH (METRIC = 'cosine', TYPE = 'DiskANN', MAXDOP = 8);
CREATE VECTOR INDEX IX_DIE_Embedding
    ON dbo.DocumentImageEmbeddings (Embedding)
    WITH (METRIC = 'cosine', TYPE = 'DiskANN', MAXDOP = 8);
```

The tradeoff: two tables means two search queries per multi-modal lookup, and you need an application-layer fusion step. The benefit is that you can update the image embedding model without touching the text pipeline, and each table gets its own DiskANN index tuned (via `MAXDOP` and build scheduling) to its vector space's geometry and update rate.

Pattern 3: Weaviate Multi-Modal Collections

Weaviate has native multi-modal support via its `multi2vec-clip` vectorizer module, which handles both text and image fields in a single collection and produces a single fused vector. This shifts the fusion problem from your application layer to the database layer.

```

# Python
import weaviate
from weaviate.classes.config import Configure, Property, DataType
import base64
from pathlib import Path

client = weaviate.connect_to_local() # or weaviate.connect_to_weaviate_cloud()

# Create a multi-modal collection
client.collections.create(
    name="ProductCatalog",
    vectorizer_config=Configure.Vectorizer.multi2vec_clip(
        image_fields=["product_image"],
        text_fields=["name", "description"],
        # Weight the contribution of each field to the combined vector
        weights={
            "product_image": [0.6], # 60% image signal
            "name": [0.2],
            "description": [0.2],
        }
    ),
    properties=[
        Property(name="name", data_type=DataType.TEXT),
        Property(name="description", data_type=DataType.TEXT),
        Property(name="product_image", data_type=DataType.BLOB), # base64 image
        Property(name="sku", data_type=DataType.TEXT),
        Property(name="price", data_type=DataType.NUMBER),
        Property(name="category", data_type=DataType.TEXT),
    ]
)

# Ingest a product with both text and image
def encode_image(image_path: str) -> str:
    with open(image_path, "rb") as f:
        return base64.b64encode(f.read()).decode("utf-8")

collection = client.collections.get("ProductCatalog")

collection.data.insert({
    "name": "Running Shoe X500",
    "description": "Lightweight mesh upper with cushioned midsole, ideal for road running",
    "product_image": encode_image("./images/shoe_x500.jpg"),
    "sku": "SHOE-X500-BLK-10",
    "price": 129.99,
    "category": "footwear"
})

client.close()

```


The Weaviate `multi2vec-clip` module uses field weights to combine sub-embeddings into one vector before indexing. This is architecturally different from the two-query-and-fuse pattern — you get a single HNSW index, lower query latency, and simpler application code. The cost is that you cannot independently update image weights without re-embedding the entire collection.

Ingestion Pipelines: Embedding at Scale

Getting data into a multi-modal store efficiently requires a pipeline that handles image preprocessing, batching, and error recovery. Raw images rarely arrive in a form suitable for CLIP: they need resizing, normalization, and format validation before the model sees them.

```

# Python
# Production-grade multi-modal ingestion pipeline
# Using sentence-transformers (CLIP wrapper), Qdrant, and async batching

import asyncio
import logging
from dataclasses import dataclass
from pathlib import Path
from typing import Optional
import numpy as np
from PIL import Image
from sentence_transformers import SentenceTransformer
from qdrant_client import QdrantClient, AsyncQdrantClient
from qdrant_client.models import (
    Distance, VectorParams, PointStruct,
    NamedVector, VectorsConfig
)

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# CLIP ViT-B/32 via sentence-transformers wrapper
# Note: This model produces BOTH text and image vectors in the same 512-d space
MODEL_NAME = "clip-ViT-B-32"
VECTOR_DIM = 512
BATCH_SIZE = 64
MAX_IMAGE_SIZE = (336, 336) # CLIP ViT-B/32 optimal input resolution

@dataclass
class ProductRecord:
    product_id: str
    name: str
    description: str
    image_path: Optional[str]
    category: str
    price: float

def load_and_preprocess_image(image_path: str) -> Optional[Image.Image]:
    """Load image with validation and normalization."""
    try:
        img = Image.open(image_path).convert("RGB")
        # Preserve aspect ratio while bounding to MAX_IMAGE_SIZE
        img.thumbnail(MAX_IMAGE_SIZE, Image.LANCZOS)
        return img
    except Exception as e:
        logger.warning(f"Failed to load image {image_path}: {e}")
        return None

def embed_batch(
    model: SentenceTransformer,
    records: list[ProductRecord]
) -> tuple[list[np.ndarray], list[np.ndarray]]:
    """

```

```

Embed a batch of records.
Returns (text_embeddings, image_embeddings).
image_embeddings[i] is None if the image failed to load.
"""
# Text: concatenate name + description for richer signal
texts = [f"{r.name}. {r.description}" for r in records]
text_embeddings = model.encode(
    texts,
    batch_size=BATCH_SIZE,
    normalize_embeddings=True,
    show_progress_bar=False
)

images = []
image_mask = [] # Track which records have valid images
for r in records:
    if r.image_path:
        img = load_and_preprocess_image(r.image_path)
        if img:
            images.append(img)
            image_mask.append(True)
        else:
            image_mask.append(False)
    else:
        image_mask.append(False)

image_embeddings_raw = []
if images:
    image_embeddings_raw = model.encode(
        images,
        batch_size=BATCH_SIZE,
        normalize_embeddings=True,
        show_progress_bar=False
    )

# Reconstruct image_embeddings list aligned with records
img_iter = iter(image_embeddings_raw)
image_embeddings = []
for has_image in image_mask:
    if has_image:
        image_embeddings.append(next(img_iter))
    else:
        image_embeddings.append(None)

return text_embeddings, image_embeddings

def build_qdrant_points(
    records: list[ProductRecord],
    text_embeddings: list[np.ndarray],
    image_embeddings: list[Optional[np.ndarray]]
) -> list[PointStruct]:
    """
    Build Qdrant points using named vectors.

```

```

Qdrant supports multiple named vector spaces per collection.
"""
points = []
for i, record in enumerate(records):
    vectors = {
        "text": text_embeddings[i].tolist()
    }
    if image_embeddings[i] is not None:
        vectors["image"] = image_embeddings[i].tolist()

    points.append(PointStruct(
        id=abs(hash(record.product_id)) % (2**63), # Qdrant needs uint64
        vector=vectors,
        payload={
            "product_id": record.product_id,
            "name": record.name,
            "description": record.description,
            "category": record.category,
            "price": record.price,
            "has_image": image_embeddings[i] is not None
        }
    ))
return points

async def setup_collection(client: AsyncQdrantClient, collection_name: str):
    """Create collection with named vector spaces for text and image."""
    collections = await client.get_collections()
    existing = [c.name for c in collections.collections]

    if collection_name not in existing:
        await client.create_collection(
            collection_name=collection_name,
            vectors_config={
                "text": VectorParams(size=VECTOR_DIM, distance=Distance.COSINE),
                "image": VectorParams(size=VECTOR_DIM, distance=Distance.COSINE),
            }
        )
        logger.info(f"Created collection: {collection_name}")
    else:
        logger.info(f"Collection {collection_name} already exists, skipping creation")

async def ingest_products(
    records: list[ProductRecord],
    collection_name: str = "products_multimodal",
    qdrant_url: str = "http://localhost:6333"
):
    model = SentenceTransformer(MODEL_NAME)
    client = AsyncQdrantClient(url=qdrant_url)

    await setup_collection(client, collection_name)

    total_ingested = 0

```

```

for batch_start in range(0, len(records), BATCH_SIZE):
    batch = records[batch_start:batch_start + BATCH_SIZE]

    text_embs, image_embs = embed_batch(model, batch)
    points = build_qdrant_points(batch, text_embs, image_embs)

    await client.upsert(
        collection_name=collection_name,
        points=points,
        wait=True
    )
    total_ingested += len(points)
    logger.info(
        f"Ingested batch {batch_start}-{batch_start + len(batch)}, "
        f"total: {total_ingested}"
    )

await client.close()
logger.info(f"Ingestion complete. Total records: {total_ingested}")

```

A few production details worth noting in this pipeline:

Qdrant named vectors allow multiple vector spaces per document in a single collection. This is superior to storing separate collections per modality because a single upsert keeps both vectors atomic — you cannot have a text vector without its corresponding image vector falling out of sync.

Image failure handling: Real product images fail to load regularly — corrupted files, unsupported formats, network-mounted paths that go stale. The pipeline tracks `has_image` in the payload so queries can filter to image-capable records when needed.

Normalization at embed time: CLIP vectors should be L2-normalized before storage. `normalize_embeddings=True` in `sentence-transformers` handles this. Pre-normalizing means the database computes cosine similarity as a dot product — numerically cheaper and avoids re-normalization overhead on the query path.

Query Patterns: Fusing Text and Image Signals

With data ingested, the interesting engineering happens at query time. Multi-modal search exposes four distinct query patterns, each appropriate to different use cases.

Pattern 1: Text Query Against Image Index

The simplest multi-modal query: a user types text, you retrieve images whose CLIP image embeddings are closest to the CLIP text embedding of the query. The entire "magic" is that the CLIP encoder maps both to the same space.

```

# Python
# Multi-modal query patterns using Qdrant named vectors

from qdrant_client import QdrantClient
from qdrant_client.models import (
    SearchRequest, NamedVector, Filter,
    FieldCondition, MatchValue, MatchAny
)
from sentence_transformers import SentenceTransformer
from PIL import Image
import base64
import io
from typing import Optional

model = SentenceTransformer("clip-ViT-B-32")
client = QdrantClient(url="http://localhost:6333")
COLLECTION = "products_multimodal"

def text_to_image_search(
    query_text: str,
    category_filter: Optional[str] = None,
    top_k: int = 20
) -> list[dict]:
    """
    Query: text string → retrieve closest images.
    Classic cross-modal retrieval.
    """
    query_vector = model.encode(
        query_text,
        normalize_embeddings=True
    ).tolist()

    search_filter = None
    if category_filter:
        search_filter = Filter(
            must=[
                FieldCondition(key="category",
match=MatchValue(value=category_filter)),
                FieldCondition(key="has_image", match=MatchValue(value=True))
            ]
        )

    results = client.search(
        collection_name=COLLECTION,
        query_vector=NamedVector(name="image", vector=query_vector), # text → image
space
        query_filter=search_filter,
        limit=top_k,
        with_payload=True,
        score_threshold=0.25 # CLIP cosine scores below 0.25 are typically noise
    )

    return [

```

```

        {
            "product_id": r.payload["product_id"],
            "name": r.payload["name"],
            "score": r.score,
            "has_image": r.payload.get("has_image", False)
        }
        for r in results
    ]

def image_to_image_search(
    query_image_path: str,
    top_k: int = 20,
    exclude_product_id: Optional[str] = None
) -> list[dict]:
    """
    Query: image → retrieve visually similar items.
    'Shop the look' / visual similarity pattern.
    """
    img = Image.open(query_image_path).convert("RGB")
    query_vector = model.encode(img, normalize_embeddings=True).tolist()

    search_filter = None
    if exclude_product_id:
        # Exclude the query item itself from results
        search_filter = Filter(
            must_not=[
                FieldCondition(key="product_id",
match=MatchValue(value=exclude_product_id))
            ]
        )

    results = client.search(
        collection_name=COLLECTION,
        query_vector=NamedVector(name="image", vector=query_vector),
        query_filter=search_filter,
        limit=top_k,
        with_payload=True
    )
    return [{"product_id": r.payload["product_id"], "name": r.payload["name"],
"score": r.score}
            for r in results]

def combined_text_image_search(
    query_text: str,
    query_image_path: str,
    text_weight: float = 0.4,
    image_weight: float = 0.6,
    top_k: int = 20
) -> list[dict]:
    """
    Fused query: BOTH a text query AND a reference image.
    Use case: 'Find shoes like [image] but with [text description]'.

```


Fusion strategy: weighted average of text and image query vectors.
This works because both live in the same CLIP space.

"""

```
import numpy as np
```

```
text_vector = model.encode(query_text, normalize_embeddings=True)
```

```
img = Image.open(query_image_path).convert("RGB")
```

```
image_vector = model.encode(img, normalize_embeddings=True)
```

```
# Weighted combination – still in the shared CLIP space
```

```
fused_vector = text_weight * text_vector + image_weight * image_vector
```

```
# Re-normalize after combination
```

```
fused_norm = np.linalg.norm(fused_vector)
```

```
if fused_norm > 0:
```

```
    fused_vector = fused_vector / fused_norm
```

```
# Query the image index with the fused vector
```

```
# (could also query the text index – depends on what you indexed)
```

```
results = client.search(
```

```
    collection_name=COLLECTION,
```

```
    query_vector=NamedVector(name="image", vector=fused_vector.tolist()),
```

```
    limit=top_k,
```

```
    with_payload=True,
```

```
    score_threshold=0.20
```

```
)
```

```
return [
```

```
    {
```

```
        "product_id": r.payload["product_id"],
```

```
        "name": r.payload["name"],
```

```
        "score": r.score,
```

```
        "query_mode": "fused_text_image"
```

```
    }
```

```
    for r in results
```

```
]
```

```
def late_fusion_search(
```

```
    query_text: str,
```

```
    query_image_path: str,
```

```
    text_weight: float = 0.5,
```

```
    image_weight: float = 0.5,
```

```
    candidate_pool: int = 100,
```

```
    top_k: int = 20
```

```
) -> list[dict]:
```

```
    """
```

```
Late fusion: run two separate searches, then combine scores.
```

```
More expensive but allows independent index tuning per modality.
```

```
Use when text and image are in DIFFERENT embedding spaces.
```

```
    """
```

```
import numpy as np
```

```

# Search 1: text vector → text index
text_vector = model.encode(query_text, normalize_embeddings=True).tolist()

text_results = client.search(
    collection_name=COLLECTION,
    query_vector=NamedVector(name="text", vector=text_vector),
    limit=candidate_pool,
    with_payload=True,
    score_threshold=0.15
)

# Search 2: image vector → image index
img = Image.open(query_image_path).convert("RGB")
image_vector = model.encode(img, normalize_embeddings=True).tolist()

image_results = client.search(
    collection_name=COLLECTION,
    query_vector=NamedVector(name="image", vector=image_vector),
    limit=candidate_pool,
    with_payload=True,
    score_threshold=0.15
)

# Merge scores by product_id
score_map: dict[str, dict] = {}
for r in text_results:
    pid = r.payload["product_id"]
    score_map[pid] = {
        "product_id": pid,
        "name": r.payload["name"],
        "text_score": r.score,
        "image_score": 0.0,
        "payload": r.payload
    }

for r in image_results:
    pid = r.payload["product_id"]
    if pid in score_map:
        score_map[pid]["image_score"] = r.score
    else:
        score_map[pid] = {
            "product_id": pid,
            "name": r.payload["name"],
            "text_score": 0.0,
            "image_score": r.score,
            "payload": r.payload
        }

# Weighted combined score
fused = []
for pid, data in score_map.items():
    combined = text_weight * data["text_score"] + image_weight *
data["image_score"]

```

```
fused.append({
    "product_id": pid,
    "name": data["name"],
    "combined_score": combined,
    "text_score": data["text_score"],
    "image_score": data["image_score"]
})

fused.sort(key=lambda x: x["combined_score"], reverse=True)
return fused[@top_k]
```

The late fusion pattern deserves a closer look, because it is the right choice whenever text and image embeddings come from *different* models with *different* dimensionalities. When you have a 1536-dimension OpenAI text embedding alongside a 512-dimension CLIP image embedding, you cannot average them — the vector spaces are unrelated. Late fusion avoids that requirement: each modality gets its own search against its own index, and scores are combined numerically after the fact. The cost is two round-trips to the database and a merge step in application memory.

Early fusion (the weighted average in `combined_text_image_search`) only works when both modalities are encoded by the same model family into the same shared space. CLIP's design makes this possible; most other combinations do not.

Multi-Modal Search in SQL Server: The T-SQL Perspective

For teams operating on SQL Server 2025, the fusion logic moves into T-SQL rather than application code. This has advantages for teams that want to keep retrieval logic auditable and version-controlled as migrations.

```

-- SQL Server 2025: late fusion multi-modal search using two CTEs
-- Assumes:
--   dbo.ProductTextEmbeddings (ProductId, Embedding VECTOR(1536))
--   dbo.ProductImageEmbeddings (ProductId, Embedding VECTOR(512))
-- Each has its own DiskANN index.
-- Parameters (DECLARed or passed by the application):
--   @text_query_vector  VECTOR(1536) – OpenAI embedding of the query string
--   @image_query_vector VECTOR(512)  – CLIP embedding of the reference image
--   @text_weight, @image_weight      – e.g. 0.5 / 0.5
--   @candidate_k, @top_k             – e.g. 100 / 20
WITH text_candidates AS (
    SELECT TOP (@candidate_k)
        product_id,
        1.0 - VECTOR_DISTANCE('cosine', embedding, CAST(@text_query_vector AS
VECTOR(1536)))
        AS text_score
    FROM product_text_embeddings
    ORDER BY VECTOR_DISTANCE('cosine', embedding, CAST(@text_query_vector AS
VECTOR(1536)))
),
image_candidates AS (
    SELECT TOP (@candidate_k)
        product_id,
        1.0 - VECTOR_DISTANCE('cosine', embedding, CAST(@image_query_vector AS
VECTOR(512)))
        AS image_score
    FROM product_image_embeddings
    ORDER BY VECTOR_DISTANCE('cosine', embedding, CAST(@image_query_vector AS
VECTOR(512)))
),
fused AS (
    SELECT
        COALESCE(t.product_id, i.product_id) AS product_id,
        COALESCE(t.text_score, 0.0) * @text_weight
        + COALESCE(i.image_score, 0.0) * @image_weight AS combined_score,
        COALESCE(t.text_score, 0.0) AS text_score,
        COALESCE(i.image_score, 0.0) AS image_score
    FROM text_candidates t
    FULL OUTER JOIN image_candidates i ON t.product_id = i.product_id
)
SELECT TOP (@top_k)
    f.product_id,
    p.name,
    p.category,
    p.price,
    f.combined_score,
    f.text_score,
    f.image_score
FROM fused f
JOIN products p ON p.id = f.product_id
ORDER BY f.combined_score DESC;

```

A subtlety in both queries: the `FULL OUTER JOIN` between `text_candidates` and `image_candidates` is deliberate. Products that appear in only one result set still participate in the final ranking, with a zero score contribution from the missing modality. This prevents the fusion from arbitrarily excluding products that have strong text relevance but weak image relevance or vice versa. Whether that behavior is correct depends on your use case — if you require **both** modalities to have non-trivial signal, replace the `FULL OUTER JOIN` with an `INNER JOIN`.

Score Normalization Before Fusion

A common mistake in late fusion implementations is combining raw cosine similarity scores across modalities without considering their distributions. Different models produce scores with different statistical properties: a cosine similarity of 0.7 in a text embedding space does not mean the same "closeness" as 0.7 in a CLIP image space. CLIP image-to-image similarities tend to cluster in a narrower range than text-to-text similarities because image features are more visually concrete.

The solution is score normalization within each modality's result set before combining. The two standard approaches:

Min-Max normalization: Scale each modality's scores to $[0, 1]$ relative to the candidate pool.

```

# Python
def normalize_scores_minmax(scores: list[float]) -> list[float]:
    if not scores:
        return scores
    min_s, max_s = min(scores), max(scores)
    if max_s == min_s:
        return [1.0] * len(scores)
    return [(s - min_s) / (max_s - min_s) for s in scores]

def late_fusion_with_normalization(
    text_results: list, # [(product_id, name, score), ...]
    image_results: list,
    text_weight: float = 0.5,
    image_weight: float = 0.5,
    top_k: int = 20
) -> list[dict]:
    import numpy as np

    text_scores_raw = [r.score for r in text_results]
    image_scores_raw = [r.score for r in image_results]

    text_scores_norm = normalize_scores_minmax(text_scores_raw)
    image_scores_norm = normalize_scores_minmax(image_scores_raw)

    score_map: dict[str, dict] = {}
    for r, norm_score in zip(text_results, text_scores_norm):
        pid = r.payload["product_id"]
        score_map[pid] = {"name": r.payload["name"], "text_score": norm_score,
"image_score": 0.0}

    for r, norm_score in zip(image_results, image_scores_norm):
        pid = r.payload["product_id"]
        if pid in score_map:
            score_map[pid]["image_score"] = norm_score
        else:
            score_map[pid] = {"name": r.payload["name"], "text_score": 0.0,
"image_score": norm_score}

    fused = [
        {
            "product_id": pid,
            "name": data["name"],
            "combined_score": text_weight * data["text_score"] + image_weight *
data["image_score"],
            "text_score": data["text_score"],
            "image_score": data["image_score"]
        }
        for pid, data in score_map.items()
    ]
    fused.sort(key=lambda x: x["combined_score"], reverse=True)
    return fused[:top_k]

```

Reciprocal Rank Fusion (RRF): An alternative that is rank-based rather than score-based, and therefore inherently distribution-agnostic.

```
# Python
def reciprocal_rank_fusion(
    text_results: list,
    image_results: list,
    k: int = 60, # RRF smoothing constant; 60 is the standard default
    top_n: int = 20
) -> list[dict]:
    """
    Combine two ranked lists using RRF.
    RRF score = sum over each list of: 1 / (rank_in_list + k)
    """
    rrf_scores: dict[str, float] = {}
    payload_store: dict[str, dict] = {}

    for rank, r in enumerate(text_results, start=1):
        pid = r.payload["product_id"]
        rrf_scores[pid] = rrf_scores.get(pid, 0.0) + 1.0 / (rank + k)
        payload_store[pid] = r.payload

    for rank, r in enumerate(image_results, start=1):
        pid = r.payload["product_id"]
        rrf_scores[pid] = rrf_scores.get(pid, 0.0) + 1.0 / (rank + k)
        if pid not in payload_store:
            payload_store[pid] = r.payload

    ranked = sorted(rrf_scores.items(), key=lambda x: x[1], reverse=True)[:top_n]

    return [
        {
            "product_id": pid,
            "name": payload_store[pid]["name"],
            "rrf_score": score
        }
        for pid, score in ranked
    ]
```

RRF has a practical advantage in production: it is robust to outlier scores and does not require you to tune normalization ranges as your corpus grows. The tradeoff is that you lose the interpretable per-modality score breakdown — you cannot easily tell whether a result ranked highly because of text signal, image signal, or both.

Handling Queries Without an Image

In real deployments, most users will provide text queries most of the time. The multi-modal system must degrade gracefully when only one modality is present, rather than treating a missing image as an error.


```

# Python
from enum import Enum
from dataclasses import dataclass
from typing import Optional

class QueryModality(Enum):
    TEXT_ONLY    = "text_only"
    IMAGE_ONLY   = "image_only"
    BOTH         = "both"

@dataclass
class MultiModalQuery:
    text: Optional[str]
    image_path: Optional[str]
    category_filter: Optional[str] = None
    text_weight: float = 0.5
    image_weight: float = 0.5
    top_k: int = 20

    @property
    def modality(self) -> QueryModality:
        if self.text and self.image_path:
            return QueryModality.BOTH
        elif self.text:
            return QueryModality.TEXT_ONLY
        elif self.image_path:
            return QueryModality.IMAGE_ONLY
        raise ValueError("Query must have at least one of text or image_path")

def route_multimodal_query(
    query: MultiModalQuery,
    model: SentenceTransformer,
    client: QdrantClient
) -> list[dict]:
    """
    Route query to the appropriate search strategy based on what inputs are
    available.
    """
    modality = query.modality

    if modality == QueryModality.TEXT_ONLY:
        # Standard vector search on text index; optionally also hit image index
        # for cross-modal retrieval (user typed, wants images back)
        return text_to_image_search(
            query_text=query.text,
            category_filter=query.category_filter,
            top_k=query.top_k
        )

    elif modality == QueryModality.IMAGE_ONLY:
        return image_to_image_search(
            query_image_path=query.image_path,
            top_k=query.top_k

```

```

    )

    else: # BOTH
        # Decide between early fusion and late fusion based on modality
        compatibility
        # (For CLIP, early fusion is fine; generalize with late fusion if needed)
        text_results = client.search(
            collection_name=COLLECTION,
            query_vector=NamedVector(
                name="text",
                vector=model.encode(query.text, normalize_embeddings=True).tolist()
            ),
            limit=query.top_k * 3,
            with_payload=True
        )
        from PIL import Image as PILImage
        img = PILImage.open(query.image_path).convert("RGB")
        image_results = client.search(
            collection_name=COLLECTION,
            query_vector=NamedVector(
                name="image",
                vector=model.encode(img, normalize_embeddings=True).tolist()
            ),
            limit=query.top_k * 3,
            with_payload=True
        )

        return reciprocal_rank_fusion(
            text_results=text_results,
            image_results=image_results,
            top_n=query.top_k
        )

```

The routing logic here is deliberately simple — three branches, no machine learning. In a production system you might introduce a fourth branch: a **zero-shot re-ranker** that scores the top-K candidates from the fused list against the original text query using a cross-encoder model. That adds latency (typically 50–150 ms for a list of 100 candidates on CPU) but improves precision significantly, especially for ambiguous queries where the bi-encoder CLIP search is unreliable.

Pinecone: Multi-Modal with Namespaces and Metadata Filtering

Pinecone's architecture does not have native named vector support in the same way Qdrant does. The production pattern for multi-modal search in Pinecone uses **namespaces** to separate modalities within a single index, combined with metadata filtering.

```

# Python
# Multi-modal pattern in Pinecone using namespaces
# Pinecone indexes are fixed-dimension; CLIP ViT-B/32 = 512

from pinecone import Pinecone, ServerlessSpec
from sentence_transformers import SentenceTransformer
from PIL import Image
import numpy as np
from typing import Optional

pc = Pinecone(api_key="YOUR_PINECONE_API_KEY")
model = SentenceTransformer("clip-ViT-B-32")

INDEX_NAME = "products-multimodal"
TEXT_NS = "text" # namespace for text embeddings
IMAGE_NS = "image" # namespace for image embeddings
VECTOR_DIM = 512

def create_pinecone_index():
    if INDEX_NAME not in [i.name for i in pc.list_indexes()]:
        pc.create_index(
            name=INDEX_NAME,
            dimension=VECTOR_DIM,
            metric="cosine",
            spec=ServerlessSpec(cloud="aws", region="us-east-1")
        )

def upsert_product(
    product_id: str,
    name: str,
    description: str,
    category: str,
    price: float,
    image_path: Optional[str] = None
):
    index = pc.Index(INDEX_NAME)

    # Upsert text embedding
    text_vector = model.encode(
        f"{name}. {description}", normalize_embeddings=True
    ).tolist()

    index.upsert(
        vectors=[{
            "id": f"{product_id}",
            "values": text_vector,
            "metadata": {
                "product_id": product_id,
                "name": name,
                "category": category,
                "price": price,
                "modality": "text"
            }
        }]
    )

```

```

    }],
    namespace=TEXT_NS
)

# Upsert image embedding if image is available
if image_path:
    img = Image.open(image_path).convert("RGB")
    image_vector = model.encode(img, normalize_embeddings=True).tolist()

    index.upsert(
        vectors=[{
            "id": f"{product_id}",
            "values": image_vector,
            "metadata": {
                "product_id": product_id,
                "name": name,
                "category": category,
                "price": price,
                "modality": "image"
            }
        }],
        namespace=IMAGE_NS
    )

def multimodal_query_pinecone(
    query_text: Optional[str],
    query_image_path: Optional[str],
    category_filter: Optional[str] = None,
    top_k: int = 20,
    text_weight: float = 0.5,
    image_weight: float = 0.5
) -> list[dict]:
    index = pc.Index(INDEX_NAME)

    metadata_filter = {}
    if category_filter:
        metadata_filter = {"category": {"$eq": category_filter}}

    results_text = []
    results_image = []

    if query_text:
        text_vector = model.encode(query_text, normalize_embeddings=True).tolist()
        resp = index.query(
            namespace=IMAGE_NS,          # text query → image namespace (cross-
modal)
            vector=text_vector,
            top_k=top_k * 3,
            filter=metadata_filter if metadata_filter else None,
            include_metadata=True
        )
        results_text = resp.matches

```

```

if query_image_path:
    img = Image.open(query_image_path).convert("RGB")
    image_vector = model.encode(img, normalize_embeddings=True).tolist()
    resp = index.query(
        namespace=IMAGE_NS,
        vector=image_vector,
        top_k=top_k * 3,
        filter=metadata_filter if metadata_filter else None,
        include_metadata=True
    )
    results_image = resp.matches

    if results_text and results_image:
        return reciprocal_rank_fusion_pinecone(results_text, results_image,
top_n=top_k)
    elif results_text:
        return [{"product_id": m.metadata["product_id"], "name": m.metadata["name"],
"score": m.score}
                for m in results_text[@top_k]]
    elif results_image:
        return [{"product_id": m.metadata["product_id"], "name": m.metadata["name"],
"score": m.score}
                for m in results_image[@top_k]]
    return []

def reciprocal_rank_fusion_pinecone(
    text_matches,
    image_matches,
    k: int = 60,
    top_n: int = 20
) -> list[dict]:
    rrf: dict[str, float] = {}
    meta: dict[str, dict] = {}

    for rank, m in enumerate(text_matches, start=1):
        pid = m.metadata["product_id"]
        rrf[pid] = rrf.get(pid, 0.0) + 1.0 / (rank + k)
        meta[pid] = m.metadata

    for rank, m in enumerate(image_matches, start=1):
        pid = m.metadata["product_id"]
        rrf[pid] = rrf.get(pid, 0.0) + 1.0 / (rank + k)
        if pid not in meta:
            meta[pid] = m.metadata

    ranked = sorted(rrf.items(), key=lambda x: x[1], reverse=True)[:top_n]
    return [{"product_id": pid, "name": meta[pid]["name"], "rrf_score": score}
            for pid, score in ranked]

```

One limitation of the Pinecone namespace approach: namespaces share the same dimension. Since CLIP text and image encoders produce the same dimension by design, this is not a constraint for CLIP-based systems. If your architecture mixes CLIP images (512-d) with a different text embedding model (1536-d), you would need two separate Pinecone indexes rather than two namespaces.

Production Considerations: Latency, Cost, and Quality Tradeoffs

Multi-modal search introduces compound latency: you are now encoding queries with a vision model in addition to a text model, and potentially issuing two database queries instead of one. Understanding where time is spent guides optimization.

Embedding latency is typically the largest contributor. CLIP ViT-B/32 runs at roughly 5–15 ms per image on a modern CPU with batch size 1, and under 2 ms per text query. On GPU, image embedding drops below 2 ms. If your deployment is CPU-only, consider:

- → Caching embeddings for repeated reference images (user uploads the same product photo repeatedly)
- → Quantizing the CLIP model with INT8 to reduce inference time at marginal quality cost
- → Using a lighter CLIP variant (ViT-S/32) for latency-critical paths and a heavier model for offline indexing

Index scan latency scales with collection size and HNSW parameters. A practical rule: keep `ef` (the HNSW search parameter at query time) at 2–4× your `top_k`. For `top_k=20`, `ef=64` is a reasonable starting point. Increasing `ef` improves recall at the cost of latency — profile this against your SLA.

Quality measurement for multi-modal search requires a labeled evaluation set with both modalities. The standard metric is Recall@K: for a set of known-relevant items, what fraction appear in the top-K results? Building this dataset means collecting (query text, reference image, known-relevant product IDs) triples — time-consuming but necessary before tuning fusion weights.

Cost management in cloud-hosted vector databases (Pinecone, Weaviate Cloud) is driven by stored vector count and query volume. Multi-modal doubles your vector count per document when storing separate text and image embeddings. If cost is a constraint, consider:

- → Storing only image embeddings and using the CLIP text encoder at query time for cross-modal retrieval (text query → image index). This halves storage at the cost of losing text-to-text retrieval as a fallback.

- → Tiering: use a smaller vector dimension (256-d via PCA projection of CLIP) for the first-pass ANN search, then re-score top candidates with the full 512-d embedding.

A Complete End-to-End Example: Fashion Visual Search

To bring the patterns together, consider a fashion retail application that supports three query types: "type a description," "upload a photo," and "type a description while holding a photo." The following shows the complete server-side handler.

```

# Python
# Fashion visual search: end-to-end query handler
# FastAPI endpoint integrating all multi-modal patterns

from fastapi import FastAPI, File, Form, UploadFile, HTTPException
from fastapi.responses import JSONResponse
import tempfile, os, shutil
from typing import Optional
from sentence_transformers import SentenceTransformer
from qdrant_client import QdrantClient
from qdrant_client.models import NamedVector, Filter, FieldCondition, MatchValue

app = FastAPI(title="Fashion Visual Search API")

# Initialize at startup – avoid re-loading per request
_model: Optional[SentenceTransformer] = None
_qdrant: Optional[QdrantClient] = None

@app.on_event("startup")
async def startup():
    global _model, _qdrant
    _model = SentenceTransformer("clip-ViT-B-32")
    _qdrant = QdrantClient(url=os.getenv("QDRANT_URL", "http://localhost:6333"))

@app.post("/search")
async def search(
    query_text: Optional[str] = Form(None),
    category: Optional[str] = Form(None),
    top_k: int = Form(20),
    text_weight: float = Form(0.4),
    image_weight: float = Form(0.6),
    image: Optional[UploadFile] = File(None)
):
    if not query_text and not image:
        raise HTTPException(status_code=400, detail="Provide at least query_text or an image.")

    # Persist uploaded image to temp file for PIL
    tmp_image_path = None
    if image:
        suffix = os.path.splitext(image.filename)[-1] or ".jpg"
        with tempfile.NamedTemporaryFile(delete=False, suffix=suffix) as tmp:
            shutil.copyfileobj(image.file, tmp)
            tmp_image_path = tmp.name

    try:
        q = MultiModalQuery(
            text=query_text,
            image_path=tmp_image_path,
            category_filter=category,
            text_weight=text_weight,
            image_weight=image_weight,
            top_k=top_k

```



```

    )
    results = route_multimodal_query(q, _model, _qdrant)
    return JSONResponse(content={"results": results, "modality":
q.modality.value})

    finally:
        if tmp_image_path and os.path.exists(tmp_image_path):
            os.unlink(tmp_image_path) # always clean up temp files

@app.get("/health")
async def health():
    return {"status": "ok", "model": "clip-ViT-B-32"}

```

The FastAPI handler deliberately isolates temp-file cleanup in a `finally` block — a failure during embedding or Qdrant search must not leave orphaned image files filling the server's `/tmp` directory. In a container environment, `/tmp` is often an in-memory tmpfs with limited capacity.

Key Takeaways

- → **Shared embedding spaces** are the foundation of multi-modal search. CLIP-family models encode text and images into the same vector space, enabling cross-modal queries where a text string can retrieve images and vice versa without any additional translation layer.
- → **Storage architecture choices** — single table with modality column, separate tables per modality, or database-native multi-modal collections (Weaviate) — each carry different tradeoffs for indexing flexibility, update independence, and operational complexity. Choose based on whether text and image pipelines change at the same rate.
- → **Early fusion** (averaging query vectors) works only when text and image embeddings share the same model and dimensionality. **Late fusion** (two searches, score combination) is the safe default when embedding models differ, and RRF is the most robust score combination method because it is rank-based and distribution-agnostic.
- → **Score normalization before combining modality scores** is not optional in production. Raw cosine similarities from different models have different statistical distributions; combining them without normalization biases fusion toward whichever modality happens to produce larger raw values.
- → **Graceful degradation** — routing single-modality queries through the appropriate index without treating a missing image or text as an error — is required for any system that faces real user traffic. Wrap modality detection in an explicit routing layer so that adding new query types later does not require restructuring the retrieval core.

Chapter 32: Graph and Vector: Combining Relationship and Semantic Search

Semantic search answers the question **what does this mean?** Graph traversal answers the question **how are these things connected?** Neither question alone is sufficient for many real-world AI applications. A knowledge graph without semantic understanding forces exact string matches and brittle schema design. A vector index without relationship awareness loses the structural context that determines whether two semantically similar documents are actually related, authored by the same person, part of the same regulatory framework, or connected through a chain of citations. This chapter explores how to combine graph and vector capabilities into a unified retrieval layer — covering the architectural patterns, the specific tools that support native graph-vector hybrid search, and the practical tradeoffs that determine when you need both versus when one is sufficient.

Why Graph Structure and Vector Similarity Serve Different Retrieval Needs

Start with a concrete example: a pharmaceutical knowledge base containing drug compounds, clinical trials, regulatory approvals, adverse event reports, and the researchers who authored each document. Pure vector search can find documents semantically similar to "cardiovascular side effects in elderly patients." But it cannot tell you that two apparently similar adverse event reports refer to different formulations of the same drug, or that the most semantically relevant paper was authored by a researcher whose institution's data has been retracted. That structural context lives in the graph.

Graph databases model entities and relationships as first-class citizens. In property graph terms, nodes hold attributes and edges hold typed, directional relationships with their own properties. Neo4j's Cypher, Amazon Neptune's Gremlin and SPARQL, and Weaviate's cross-references all express this structure. The query planner traverses the graph using index-free adjacency — following pointers directly from one node to its neighbors rather than performing joins across tables. This gives graph databases their characteristic performance on multi-hop traversal queries: $O(\text{depth} \times \text{branching_factor})$ rather than $O(n \log n)$ for a join-based equivalent.

Vector databases answer a different kind of query. Given a query embedding, find the k nearest vectors in high-dimensional space. The performance envelope is defined by approximate nearest neighbor (ANN) algorithms — HNSW, IVF, ScaNN — that trade recall for

speed. A well-tuned HNSW index on a 1536-dimensional embedding space with 10 million vectors returns 10 nearest neighbors in under 10ms at 99% recall. No graph traversal is involved; the structure being searched is a proximity graph of embeddings, not an entity-relationship graph of domain objects.

The architectural question is where these two search modes intersect. Three patterns emerge in production:

Pre-filter by graph, rank by vector. Traverse the graph to identify a candidate set of entity IDs, then perform vector similarity search over that restricted set. This uses graph traversal to answer structural questions ("all drugs approved by the FDA in the last three years") and vector search to rank within that set.

Post-filter by graph, retrieve by vector. Retrieve top-k semantically similar documents, then apply graph traversal to expand, filter, or re-rank based on structural relationships. A recommendation system might retrieve 50 semantically similar items, then use the user's purchase graph to remove already-purchased items and boost items purchased by structurally similar users.

Graph-guided vector traversal. Use graph relationships to guide which regions of the vector space to explore. This is the approach taken by graph-aware RAG systems that follow citation or entity linkage graphs to gather context before embedding retrieval.

```
graph TD
    subgraph "Pattern 1: Pre-filter by graph"
        A1[Graph traversal:  
candidate entity IDs] --> A2[Vector search  
within candidate set]
    end
    subgraph "Pattern 2: Post-filter by graph"
        B1[Vector search:  
top-k similar] --> B2[Graph traversal:  
filter / expand / re-rank]
    end
    subgraph "Pattern 3: Graph-guided traversal"
        C1[Entity / citation graph] --> C2[Selects regions  
of vector space] --> C3[Embedding retrieval]
    end
```

The three ways graph structure and vector similarity combine in production retrieval systems.

Native Graph-Vector Databases: Weaviate, Neo4j Vector, and Amazon Neptune Analytics

Several databases now offer native support for both paradigms within a single query engine, eliminating the need for application-level orchestration between separate systems.

Weaviate was designed from the start around the idea that objects have both vector representations and references to other objects. Its schema system defines cross-references between classes, and its GraphQL query layer lets you traverse those references while filtering by vector similarity in the same request.

```
# Python
import weaviate
import weaviate.classes as wvc

client = weaviate.connect_to_local()

# Schema: Drug nodes reference ClinicalTrial nodes
# This query retrieves trials semantically similar to a query
# that are linked to a specific drug compound
query_text = "cardiovascular adverse events in elderly patients"

response = (
    client.collections.get("ClinicalTrial")
    .query.near_text(
        query=query_text,
        limit=10,
        return_metadata=wvc.query.MetadataQuery(distance=True),

filters=wvc.query.Filter.by_ref("hasDrug").by_property("name").equal("Atorvastatin")
,
    )
)

for obj in response.objects:
    print(f"Trial ID: {obj.properties['trial_id']}")
    print(f"Title: {obj.properties['title']}")
    print(f"Distance: {obj.metadata.distance:.4f}")
    print()

client.close()
```

The `Filter.by_ref()` call translates to a graph traversal: follow the `hasDrug` reference edge and filter by the target node's property. Weaviate executes this as a pre-filter before the vector search, meaning the ANN index is queried only over the subset of trials linked to

Atorvastatin. At scale this matters: if 10% of your 10M trial documents are linked to the target drug, you are running ANN over 1M vectors rather than 10M, which is roughly a 10x improvement in query latency with no change in recall.

Neo4j added native vector indexing in version 5.11, allowing a `CALL db.index.vector.queryNodes()` procedure alongside standard Cypher graph traversal. This enables a query that first does vector retrieval and then traverses the graph from the results.

```

# Python - Neo4j combined graph + vector query
from neo4j import GraphDatabase
from openai import OpenAI

openai_client = OpenAI()
driver = GraphDatabase.driver("bolt://localhost:7687", auth=("neo4j", "password"))

def get_embedding(text: str) -> list[float]:
    response = openai_client.embeddings.create(
        input=text,
        model="text-embedding-3-small"
    )
    return response.data[0].embedding

def graph_vector_search(query: str, hop_depth: int = 2, top_k: int = 5):
    embedding = get_embedding(query)

    cypher = """
    // Step 1: Vector similarity search to find seed nodes
    CALL db.index.vector.queryNodes('document_embeddings', $top_k, $embedding)
    YIELD node AS doc, score AS similarity_score

    // Step 2: Traverse the graph from each seed node
    // Follow AUTHORED_BY and CITES relationships up to hop_depth
    CALL apoc.path.subgraphNodes(doc, {
        relationshipFilter: 'AUTHORED_BY>|CITES>',
        maxLevel: $hop_depth
    })
    YIELD node AS related

    // Step 3: Collect and return with structural context
    WITH doc, similarity_score,
        collect(DISTINCT related) AS graph_neighbors
    RETURN
        doc.title AS title,
        doc.document_id AS doc_id,
        similarity_score,
        [n IN graph_neighbors WHERE n:Author | n.name] AS authors,
        [n IN graph_neighbors WHERE n:Document | n.title] AS cited_docs,
        size(graph_neighbors) AS graph_connectivity
    ORDER BY similarity_score DESC
    """

    with driver.session() as session:
        result = session.run(
            cypher,
            embedding=embedding,
            top_k=top_k,
            hop_depth=hop_depth
        )
        return [record.data() for record in result]

results = graph_vector_search(

```

```

    "machine learning approaches for drug interaction prediction",
    hop_depth=2,
    top_k=5
)

for r in results:
    print(f"Title: {r['title']}")
    print(f"Similarity: {r['similarity_score']:.4f}")
    print(f"Authors: {' , '.join(r['authors'])}")
    print(f"Cites: {len(r['cited_docs'])} documents")
    print(f"Graph Connectivity: {r['graph_connectivity']} related nodes")
    print()

driver.close()

```

The `graph_connectivity` metric here is not just informational. Documents with high graph connectivity in a citation network are generally more influential and more likely to represent canonical sources rather than derivative work. This is a structural signal that pure vector similarity cannot capture.

Amazon Neptune Analytics (not Neptune Database) introduced vector similarity search in 2024, enabling openCypher queries that combine graph traversal with approximate nearest neighbor search over stored embeddings. The design pattern follows the same logic: use Neptune's property graph for structural filtering, Neptune Analytics' vector index for semantic ranking.

SQL Server Graph Tables Meet VECTOR: Graph-Vector Hybrid in T-SQL

If your stack is already on SQL Server, you may not need a dedicated graph database for moderate-complexity relationship modeling. SQL Server's native graph tables (`CREATE TABLE AS NODE / AS EDGE` , since 2017) handle traversal via `MATCH` predicates, and the 2025 `VECTOR` type handles the similarity layer. The combination is not as expressive as Cypher for deep traversal, but for depth-limited graph queries it is entirely production-viable and keeps your stack consolidated.

```

-- SQL Server 2025
-- Schema: documents with embeddings, plus a relationship graph
CREATE TABLE dbo.Documents (
    DocId      UNIQUEIDENTIFIER NOT NULL
        CONSTRAINT PK_Documents PRIMARY KEY
        DEFAULT NEWSEQUENTIALID(),
    Title      NVARCHAR(300) NOT NULL,
    Content    NVARCHAR(MAX) NULL,
    Embedding  VECTOR(1536) NULL,
    CreatedAt  DATETIME2(0) NOT NULL
        CONSTRAINT DF_Documents_Created DEFAULT SYSUTCDATETIME()
);
CREATE TABLE dbo.DocRelationships (
    SourceId   UNIQUEIDENTIFIER NOT NULL
        CONSTRAINT FK_DR_Source REFERENCES dbo.Documents (DocId),
    TargetId   UNIQUEIDENTIFIER NOT NULL
        CONSTRAINT FK_DR_Target REFERENCES dbo.Documents (DocId),
    RelType    NVARCHAR(40) NOT NULL, -- 'CITES', 'AUTHORED_BY_SAME',
    'PART_OF_SERIES'
    Weight     FLOAT NOT NULL CONSTRAINT DF_DR_Weight DEFAULT (1.0),
    CONSTRAINT PK_DocRelationships PRIMARY KEY (SourceId, TargetId, RelType)
);
CREATE VECTOR INDEX IX_Documents_Embedding
    ON dbo.Documents (Embedding)
    WITH (METRIC = 'cosine', TYPE = 'DiskANN', MAXDOP = 8);
CREATE INDEX IX_DocRelationships_Source ON dbo.DocRelationships (SourceId);
CREATE INDEX IX_DocRelationships_Target ON dbo.DocRelationships (TargetId);

-- Graph-vector hybrid query:
-- 1. Find top-10 semantically similar documents (vector layer)
-- 2. Traverse 2 hops of the citation graph from those seeds (graph layer)
-- 3. Re-rank the expanded set by a combined score
DECLARE @q VECTOR(1536) = '[0.1, 0.2, ...]'; -- embedded query

WITH
-- Step 1: Vector similarity search - seed nodes
VectorSeeds AS (
    SELECT TOP (10) d.DocId, d.Title,
        1 - VECTOR_DISTANCE('cosine', d.Embedding, @q) AS CosineSimilarity
    FROM dbo.Documents AS d
    WHERE d.Embedding IS NOT NULL
    ORDER BY VECTOR_DISTANCE('cosine', d.Embedding, @q)
),
-- Step 2: Graph traversal from seed nodes (up to 2 hops)
GraphExpansion AS (
    SELECT DocId, DocId AS OriginId, 0 AS Hops FROM VectorSeeds
    UNION ALL
    SELECT dr.TargetId AS DocId, ge.OriginId, ge.Hops + 1
    FROM GraphExpansion AS ge
    JOIN dbo.DocRelationships AS dr
        ON dr.SourceId = ge.DocId AND dr.RelType = 'CITES'
    WHERE ge.Hops < 2
),

```



```

ExpandedCandidates AS (
    SELECT DocId, MIN(Hops) AS MinHops
    FROM GraphExpansion
    GROUP BY DocId
),
FinalRanking AS (
    SELECT d.DocId, d.Title,
        1 - VECTOR_DISTANCE('cosine', d.Embedding, @q) AS CosineSimilarity,
        ec.MinHops,
        -- Graph proximity penalty: seeds (hop=0) get full score;
        -- each hop reduces score by 15%
        (1 - VECTOR_DISTANCE('cosine', d.Embedding, @q))
        * POWER(0.85, ec.MinHops) AS CombinedScore,
        CASE WHEN vs.DocId IS NOT NULL THEN 1 ELSE 0 END AS IsSeedNode
    FROM ExpandedCandidates AS ec
    JOIN dbo.Documents AS d ON d.DocId = ec.DocId
    LEFT JOIN VectorSeeds AS vs ON vs.DocId = ec.DocId
    WHERE d.Embedding IS NOT NULL
)
SELECT TOP (20) DocId, Title,
    ROUND(CAST(CosineSimilarity AS FLOAT), 4) AS SemanticScore,
    MinHops AS GraphHops,
    ROUND(CAST(CombinedScore AS FLOAT), 4) AS CombinedScore,
    IsSeedNode
FROM FinalRanking
ORDER BY CombinedScore DESC
OPTION (MAXRECURSION 10);

```

The hop penalty (0.85 per hop) is a tunable parameter. In citation networks where relevance decays sharply with distance, values between 0.75 and 0.85 work well. In social graphs where second-degree connections are highly relevant, you might use 0.95 or no penalty at all. Benchmark this against your human-labeled relevance set before deploying.

Performance characteristics to expect: with 1M documents and a DiskANN index, the vector seed retrieval runs in ~5ms. The recursive CTE traversal over 10 seeds at depth 2 with average branching factor 5 will generate ~250 candidate nodes, and the secondary embedding lookups add ~1ms. Total query time is typically 10–30ms for this pattern at this scale — well within interactive response time budgets.

Graph-Aware RAG: Using Relationship Context in LLM Pipelines

Retrieval-Augmented Generation pipelines typically operate in a flat retrieval model: embed the query, retrieve top-k chunks, stuff them into the context window. This loses structural information. Graph-aware RAG incorporates relationship traversal into the retrieval step so the context passed to the LLM includes not just semantically similar content but contextually connected content.

Microsoft Research published GraphRAG in 2024, which builds a community detection graph over extracted entities and uses that graph to enable global queries (summarizing broad themes across a corpus) that flat RAG cannot handle. The architecture is more complex than simple hybrid search but the principle is the same: use graph structure to decide what to retrieve.

A more immediately deployable pattern is entity-linked RAG: extract named entities from the query, look up those entities in a knowledge graph to gather structured context, then combine that structured context with vector-retrieved chunks.

```

# Python
# Entity-linked Graph-aware RAG with Neo4j + SQL Server + OpenAI

from neo4j import GraphDatabase
from openai import OpenAI
import pyodbc
import json
from typing import Optional

openai_client = OpenAI()
neo4j_driver = GraphDatabase.driver("bolt://localhost:7687", auth=("neo4j",
"password"))
sql_conn = pyodbc.connect("DRIVER={ODBC Driver 18 for SQL Server};"
                          "SERVER=localhost;DATABASE=knowledge_db;"
                          "Trusted_Connection=yes;Encrypt=yes;"
                          "TrustServerCertificate=yes")

def extract_entities_from_query(query: str) -> list[str]:
    """Use LLM to extract named entities from the query."""
    response = openai_client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {
                "role": "system",
                "content": "Extract named entities (people, organizations, drugs,
diseases, locations) "
                    "from the query. Return a JSON array of entity names
only."
            },
            {"role": "user", "content": query}
        ],
        response_format={"type": "json_object"}
    )
    result = json.loads(response.choices[0].message.content)
    return result.get("entities", [])

def get_graph_context(entities: list[str]) -> dict:
    """Traverse knowledge graph for entity context."""
    if not entities:
        return {}

    cypher = """
UNWIND $entity_names AS name
MATCH (e:Entity)
WHERE e.name =~ ('(?i)' + name)

// Get direct relationships (1 hop)
OPTIONAL MATCH (e)-[r1]->(neighbor1:Entity)
OPTIONAL MATCH (e)<-[r2]-(neighbor2:Entity)

RETURN
    e.name AS entity_name,
    e.type AS entity_type,

```

```

        e.description AS entity_description,
        collect(DISTINCT {
            name: neighbor1.name,
            type: neighbor1.type,
            relationship: type(r1),
            direction: 'outgoing'
        }) AS outgoing_relations,
        collect(DISTINCT {
            name: neighbor2.name,
            type: neighbor2.type,
            relationship: type(r2),
            direction: 'incoming'
        }) AS incoming_relations
    """

with neo4j_driver.session() as session:
    result = session.run(cypher, entity_names=entities)
    graph_context = {}
    for record in result:
        entity = record["entity_name"]
        graph_context[entity] = {
            "type": record["entity_type"],
            "description": record["entity_description"],
            "relations": record["outgoing_relations"] +
record["incoming_relations"]
        }
    return graph_context

def vector_search_with_graph_filter(
    query: str,
    entity_ids: Optional[list[str]] = None,
    top_k: int = 5
) -> list[dict]:
    """Vector similarity search in SQL Server, optionally filtered by entity
graph."""
    embedding = openai_client.embeddings.create(
        input=query,
        model="text-embedding-3-small"
    ).data[0].embedding

    embedding_str = "[" + ",".join(str(x) for x in embedding) + "]"

    vec_param = "[" + ",".join(str(x) for x in embedding) + "]"
    cur = sql_conn.cursor()
    if entity_ids:
        placeholders = ",".join("?" for _ in entity_ids)
        rows = cur.execute(f"""
            SELECT TOP (?) c.ChunkId, c.Content, c.SourceDoc, c.SectionTitle,
                1 - VECTOR_DISTANCE('cosine', c.Embedding,
                    CAST(? AS VECTOR(1536))) AS Similarity
            FROM dbo.DocumentChunks AS c
            JOIN dbo.ChunkEntities AS ce ON ce.ChunkId = c.ChunkId
            WHERE ce.EntityId IN ({placeholders})
        """)
    else:
        rows = cur.execute(f"""
            SELECT TOP (?) c.ChunkId, c.Content, c.SourceDoc, c.SectionTitle,
                1 - VECTOR_DISTANCE('cosine', c.Embedding,
                    CAST(? AS VECTOR(1536))) AS Similarity
            FROM dbo.DocumentChunks AS c
            WHERE c.Embedding = {embedding_str}
        """)
    return rows

```

```

        ORDER BY VECTOR_DISTANCE('cosine', c.Embedding,
                                   CAST(? AS VECTOR(1536)))
        """", (top_k, vec_param, *entity_ids, vec_param)).fetchall()
    else:
        rows = cur.execute("""
            SELECT TOP (?) ChunkId, Content, SourceDoc, SectionTitle,
                1 - VECTOR_DISTANCE('cosine', Embedding,
                                   CAST(? AS VECTOR(1536))) AS Similarity
            FROM dbo.DocumentChunks
            ORDER BY VECTOR_DISTANCE('cosine', Embedding,
                                   CAST(? AS VECTOR(1536)))
            """, (top_k, vec_param, vec_param)).fetchall()

        return [
            {
                "chunk_id": r[0],
                "content": r[1],
                "source_doc": r[2],
                "section_title": r[3],
                "similarity": float(r[4])
            }
            for r in rows
        ]

def graph_aware_rag(query: str) -> str:
    """Full graph-aware RAG pipeline."""

    # Step 1: Extract entities from query
    entities = extract_entities_from_query(query)
    print(f"Extracted entities: {entities}")

    # Step 2: Get structured graph context for those entities
    graph_context = get_graph_context(entities)
    print(f"Graph context retrieved for {len(graph_context)} entities")

    # Step 3: Vector search (filtered if we have entity context)
    # In production, translate entity names to IDs via a lookup table
    retrieved_chunks = vector_search_with_graph_filter(query, top_k=5)
    print(f"Retrieved {len(retrieved_chunks)} chunks via vector search")

    # Step 4: Construct enriched context for LLM
    graph_context_text = ""
    for entity, ctx in graph_context.items():
        graph_context_text += f"\n[Entity: {entity} ({ctx['type']})]\n"
        if ctx["description"]:
            graph_context_text += f"Description: {ctx['description']}\n"
        if ctx["relations"]:
            relations_summary = [
                f" - {rel['relationship']} -> {rel['name']}"
                for rel in ctx["relations"][:5] # Limit to top 5 relations
            ]
            graph_context_text += "Relationships:\n" + "\n".join(relations_summary)
    + "\n"

```

```

chunks_text = "\n\n".join([
    f"[Source: {c['source_doc']} | Section: {c['section_title']} | Similarity: {c['similarity']:.3f}]\n{c['content']}"
    for c in retrieved_chunks
])

# Step 5: LLM generation with structured + semantic context
prompt = f"""Answer the following question using the provided context.

KNOWLEDGE GRAPH CONTEXT (Structured relationships):
{graph_context_text if graph_context_text else "No graph context available."}

DOCUMENT CONTEXT (Semantically similar passages):
{chunks_text}

QUESTION: {query}

Provide a comprehensive answer grounded in the provided context."""

response = openai_client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "system", "content": "You are a precise research assistant. Ground your answers in the provided context."},
        {"role": "user", "content": prompt}
    ],
    temperature=0.1
)

return response.choices[0].message.content

# Example usage
answer = graph_aware_rag(
    "What are the known drug interactions between statins and blood thinners in elderly patients?"
)
print(answer)

```

The separation of graph context (structured, relational facts) from vector context (semantically similar text passages) in the prompt is intentional. LLMs attend differently to structured relational facts than to free-text passages, and keeping them visually distinct in the prompt tends to improve attribution accuracy in the generated answer.

Choosing Between Graph-Vector Integration Strategies

The decision tree for graph-vector architecture is driven by three factors: relationship depth, data volume, and team operational capacity.

Relationship depth is the primary driver. If your graph queries require more than three hops to answer meaningful questions, T-SQL recursive CTEs become slow and operationally awkward. Neo4j, Amazon Neptune, or ArangoDB handle this naturally. At one to two hops, SQL Server with a well-indexed adjacency table performs comparably to a dedicated graph database and eliminates operational complexity.

Data volume determines whether you can afford native hybrid query engines or need to separate the systems. Weaviate performs well at 50M+ vectors with cross-references, but its graph capabilities are limited to direct object references — it is not a property graph database. Neo4j's vector index, as of version 5.x, handles tens of millions of vectors with HNSW but is not optimized for the billion-vector scale that Pinecone or Milvus serve. If you need billion-scale vectors with complex multi-hop graph traversal, you are building a hybrid architecture with two separate systems communicating at the application layer.

Team operational capacity is the factor that gets underweighted in architecture decisions. Running two production database systems — a graph database and a vector database — means two monitoring setups, two backup procedures, two capacity planning models, and two teams who need to understand both. Unless the retrieval quality improvement from the native graph-vector approach is significant and measurable, the simpler architecture wins.

A practical decision matrix:

Scenario	Recommended Architecture
<10M vectors, shallow relationships (≤ 2 hops)	SQL Server native graph tables + VECTOR + recursive CTEs
<50M vectors, moderate relationship complexity	Weaviate with cross-references
Any scale, deep graph traversal (> 3 hops)	Neo4j 5.x or Amazon Neptune Analytics
>500M vectors + complex graphs	Separate vector DB (Pinecone/Milvus) + graph DB (Neo4j/Neptune), application-layer orchestration
Regulated environment, single-vendor preference	Amazon Neptune Analytics (unified)

Implementing the Application-Layer Orchestration Pattern

When you cannot fit both concerns into a single database, the application layer becomes the join engine. The pattern requires careful attention to consistency, latency, and the ordering of operations.


```

# Python
# Application-layer orchestration: Pinecone (vector) + Neo4j (graph)

from pinecone import Pinecone
from neo4j import GraphDatabase
from openai import OpenAI
from dataclasses import dataclass
from typing import Optional
import asyncio

@dataclass
class HybridSearchResult:
    doc_id: str
    title: str
    vector_score: float
    graph_score: float
    combined_score: float
    graph_hops: int
    structural_context: dict

class HybridSearchOrchestrator:
    def __init__(
        self,
        pinecone_api_key: str,
        pinecone_index_name: str,
        neo4j_uri: str,
        neo4j_auth: tuple,
    ):
        self.pc = Pinecone(api_key=pinecone_api_key)
        self.index = self.pc.Index(pinecone_index_name)
        self.neo4j_driver = GraphDatabase.driver(neo4j_uri, auth=neo4j_auth)
        self.openai_client = OpenAI()

    def _get_embedding(self, text: str) -> list[float]:
        return self.openai_client.embeddings.create(
            input=text,
            model="text-embedding-3-small"
        ).data[0].embedding

    def _vector_search(
        self,
        embedding: list[float],
        top_k: int = 20,
        namespace: str = "",
        filter_dict: Optional[dict] = None
    ) -> list[dict]:
        """Pinecone vector search returning candidate IDs and scores."""
        response = self.index.query(
            vector=embedding,
            top_k=top_k,
            namespace=namespace,
            filter=filter_dict,
            include_metadata=True

```

```

    )
    return [
        {
            "doc_id": match.id,
            "vector_score": match.score,
            "metadata": match.metadata
        }
        for match in response.matches
    ]

def _graph_score_and_expand(
    self,
    seed_doc_ids: list[str],
    expansion_depth: int = 2
) -> dict[str, dict]:
    """
    For each seed document, compute a graph centrality score and
    expand to neighboring nodes. Returns a dict keyed by doc_id.
    """
    cypher = """
    UNWIND $seed_ids AS seed_id
    MATCH (d:Document {doc_id: seed_id})

    // Compute in-degree as a structural importance proxy
    OPTIONAL MATCH (d)<-[:CITES]-(citing:Document)
    WITH d, seed_id, count(citing) AS citation_count

    // Expand neighbors up to expansion_depth
    CALL apoc.path.subgraphNodes(d, {
        relationshipFilter: 'CITES>|RELATED_TO>',
        maxLevel: $depth,
        limit: 50
    })
    YIELD node AS neighbor

    WITH
        seed_id,
        d.title AS title,
        citation_count,
        collect(DISTINCT {
            doc_id: neighbor.doc_id,
            title: neighbor.title,
            hops: 1
        }) AS neighbors

    RETURN
        seed_id,
        title,
        citation_count,
        neighbors,
        // Normalize citation count to [0, 1] range using log scale
        // We use a soft normalization; tune the divisor for your corpus
        (1.0 - exp(-citation_count / 10.0)) AS graph_score
    """

```

```

"""

with self.neo4j_driver.session() as session:
    result = session.run(
        cypher,
        seed_ids=seed_doc_ids,
        depth=expansion_depth
    )
    graph_data = {}
    for record in result:
        graph_data[record["seed_id"]] = {
            "title": record["title"],
            "citation_count": record["citation_count"],
            "graph_score": record["graph_score"],
            "neighbors": record["neighbors"]
        }
    return graph_data

def _fetch_neighbor_vectors(
    self,
    neighbor_ids: list[str],
    query_embedding: list[float]
) -> list[dict]:
    """Fetch vector scores for graph-expanded nodes not in original results."""
    if not neighbor_ids:
        return []

    # Fetch by ID from Pinecone
    fetch_response = self.index.fetch(ids=neighbor_ids)
    neighbor_results = []
    for doc_id, vector_data in fetch_response.vectors.items():
        if vector_data.values:
            # Compute cosine similarity manually
            import numpy as np
            qv = np.array(query_embedding)
            dv = np.array(vector_data.values)
            similarity = float(np.dot(qv, dv) / (np.linalg.norm(qv) *
np.linalg.norm(dv)))
            neighbor_results.append({
                "doc_id": doc_id,
                "vector_score": similarity,
                "metadata": vector_data.metadata or {}
            })
    return neighbor_results

def search(
    self,
    query: str,
    top_k_vector: int = 20,
    expansion_depth: int = 2,
    vector_weight: float = 0.6,
    graph_weight: float = 0.4,
    final_top_k: int = 10

```

```

) -> list[HybridSearchResult]:
    """
    Full hybrid search:
    1. Vector search for seed candidates
    2. Graph scoring and expansion
    3. Fetch vector scores for expanded nodes
    4. Combined scoring and final ranking
    """

    # Step 1: Vector search
    embedding = self._get_embedding(query)
    vector_results = self._vector_search(embedding, top_k=top_k_vector)
    seed_ids = [r["doc_id"] for r in vector_results]
    vector_score_map = {r["doc_id"]: r["vector_score"] for r in vector_results}

    # Step 2: Graph scoring and expansion
    graph_data = self._graph_score_and_expand(seed_ids, expansion_depth)

    # Collect all neighbor IDs not already in vector results
    all_neighbor_ids = set()
    for seed_id, data in graph_data.items():
        for neighbor in data.get("neighbors", []):
            n_id = neighbor.get("doc_id")
            if n_id and n_id not in vector_score_map:
                all_neighbor_ids.add(n_id)

    # Step 3: Fetch vector scores for expanded nodes
    expanded_vector_results = self._fetch_neighbor_vectors(
        list(all_neighbor_ids), embedding
    )
    for r in expanded_vector_results:
        vector_score_map[r["doc_id"]] = r["vector_score"]

    # Build graph score map from all known nodes
    graph_score_map: dict[str, float] = {}
    for seed_id, data in graph_data.items():
        graph_score_map[seed_id] = data["graph_score"]

    # Step 4: Combined scoring
    all_candidate_ids = set(vector_score_map.keys())
    results = []

    for doc_id in all_candidate_ids:
        v_score = vector_score_map.get(doc_id, 0.0)
        g_score = graph_score_map.get(doc_id, 0.0)
        combined = (vector_weight * v_score) + (graph_weight * g_score)

        # Determine graph hops (0 = seed, 1+ = expanded)
        hops = 0 if doc_id in {r["doc_id"] for r in vector_results} else 1

        g_data = graph_data.get(doc_id, {})
        results.append(HybridSearchResult(
            doc_id=doc_id,
            title=g_data.get("title", "Unknown"),

```

```

        vector_score=v_score,
        graph_score=g_score,
        combined_score=combined,
        graph_hops=hops,
        structural_context={
            "citation_count": g_data.get("citation_count", 0),
            "neighbor_count": len(g_data.get("neighbors", []))
        }
    ))

    results.sort(key=lambda x: x.combined_score, reverse=True)
    return results[:final_top_k]

def close(self):
    self.neo4j_driver.close()

# Usage
orchestrator = HybridSearchOrchestrator(
    pinecone_api_key="your-key",
    pinecone_index_name="research-docs",
    neo4j_uri="bolt://localhost:7687",
    neo4j_auth=("neo4j", "password")
)

results = orchestrator.search(
    query="statin therapy and cognitive decline risk factors",
    top_k_vector=20,
    expansion_depth=2,
    vector_weight=0.65,
    graph_weight=0.35,
    final_top_k=10
)

for r in results:
    print(f"[{r.graph_hops} hops] {r.title}")
    print(f"  Vector: {r.vector_score:.4f} | Graph: {r.graph_score:.4f} | Combined: {r.combined_score:.4f}")
    print(f"  Citations: {r.structural_context['citation_count']}")

orchestrator.close()

```

Two implementation details here deserve attention. First, the graph score uses a log-normalized citation count rather than raw citation count. Raw citation count creates a rich-get-richer dynamic where the most-cited papers dominate regardless of semantic relevance. The formula `1.0 - exp(-citation_count / 10.0)` maps zero citations to a graph score of 0 and asymptotically approaches 1 as citations grow, with a reasonable inflection point around 10 citations for most academic corpora. Adjust the divisor based on the citation distribution of your specific dataset.

Second, the `_fetch_neighbor_vectors` step introduces a latency cost: a Pinecone `fetch` call for potentially hundreds of IDs. Whether this is acceptable depends on your latency budget. If the latency budget is tight, consider capping expansion to 1 hop, limiting neighbors per seed to the top 5 by graph edge weight, or pre-materializing graph-neighbor embeddings into a secondary index.

SQL Server Compatibility: Graph Tables and Vector Search

SQL Server 2017 introduced native graph table support (`CREATE TABLE AS NODE` and `CREATE TABLE AS EDGE`), and SQL Server 2025 (and Azure SQL Database's preview features) added vector similarity functions. This combination allows graph-vector hybrid queries within a single SQL Server instance, following the same graph-plus-vector pattern, expressed entirely in T-SQL.

```

-- SQL Server
-- Graph tables combined with vector similarity search

-- Node tables
CREATE TABLE dbo.Documents (
    doc_id        UNIQUEIDENTIFIER DEFAULT NEWSEQUENTIALID() PRIMARY KEY,
    title         NVARCHAR(500) NOT NULL,
    content        NVARCHAR(MAX),
    embedding      VARBINARY(MAX),    -- stored as binary; cast for similarity functions
    created_at     DATETIME2 DEFAULT SYSUTCDATETIME(),
    $node_id      -- implicit SQL Server graph node column
) AS NODE;

CREATE TABLE dbo.Authors (
    author_id     UNIQUEIDENTIFIER DEFAULT NEWSEQUENTIALID() PRIMARY KEY,
    name          NVARCHAR(200),
    institution    NVARCHAR(200),
    $node_id
) AS NODE;

-- Edge tables
CREATE TABLE dbo.Cites (
    citation_date  DATE,
    $edge_id,      -- implicit edge columns
    $from_id,
    $to_id
) AS EDGE;

CREATE TABLE dbo.AuthoredBy (
    $edge_id,
    $from_id,
    $to_id
) AS EDGE;

-- Graph-vector hybrid query using MATCH syntax (SQL Server graph)
-- and VECTOR_DISTANCE (available in SQL Server 2025 / Azure SQL)

-- Step 1: Find semantically similar documents using vector search
-- Step 2: Use MATCH to traverse citation graph from those seeds
-- Step 3: Re-rank by combined score

DECLARE @query_embedding VARBINARY(MAX) = <your_embedding_binary>;
DECLARE @hop_depth INT = 2;
DECLARE @top_k INT = 10;

WITH VectorSeeds AS (
    SELECT TOP (@top_k)
        doc_id,
        title,
        embedding,
        VECTOR_DISTANCE('cosine', embedding, @query_embedding) AS cosine_distance
    FROM dbo.Documents
    ORDER BY VECTOR_DISTANCE('cosine', embedding, @query_embedding) ASC

```

```

),

-- One-hop expansion via SQL Server MATCH clause
OneHopExpansion AS (
    SELECT DISTINCT
        cited.doc_id          AS expanded_doc_id,
        seed.doc_id           AS origin_doc_id,
        1                     AS hop_count
    FROM VectorSeeds seed
    JOIN dbo.Documents AS citing
        ON seed.doc_id = citing.doc_id
    JOIN dbo.Cites AS c
        ON TRUE
    JOIN dbo.Documents AS cited
        ON TRUE
    WHERE MATCH(citing-(c)->cited)
),

-- Two-hop expansion
TwoHopExpansion AS (
    SELECT DISTINCT
        cited2.doc_id         AS expanded_doc_id,
        one.origin_doc_id,
        2                     AS hop_count
    FROM OneHopExpansion one
    JOIN dbo.Documents AS intermediate
        ON one.expanded_doc_id = intermediate.doc_id
    JOIN dbo.Cites AS c2
        ON TRUE
    JOIN dbo.Documents AS cited2
        ON TRUE
    WHERE MATCH(intermediate-(c2)->cited2)
),

AllCandidates AS (
    SELECT doc_id AS candidate_id, 0 AS min_hops
    FROM VectorSeeds
    UNION ALL
    SELECT expanded_doc_id, 1 FROM OneHopExpansion
    UNION ALL
    SELECT expanded_doc_id, 2 FROM TwoHopExpansion
),

Deduped AS (
    SELECT candidate_id, MIN(min_hops) AS min_hops
    FROM AllCandidates
    GROUP BY candidate_id
)

SELECT TOP 20
    d.doc_id,
    d.title,
    ROUND(1.0 - VECTOR_DISTANCE('cosine', d.embedding, @query_embedding), 4) AS

```



```

semantic_score,
    dj.min_hops                                AS
graph_hops,
    ROUND(
        (1.0 - VECTOR_DISTANCE('cosine', d.embedding, @query_embedding))
        * POWER(0.85, CAST(dj.min_hops AS FLOAT)),
        4
    )
combined_score
FROM Deduped dj
JOIN dbo.Documents d ON d.doc_id = dj.candidate_id
ORDER BY combined_score DESC;

```

The SQL Server `MATCH` clause follows the ASCII-art arrow syntax

(from_node-(edge)->to_node) , which is unique to SQL Server's graph implementation. Unlike Cypher, it cannot express variable-length path traversal in a single clause — hence the explicit one-hop and two-hop CTE expansion above. For deeper traversal, this approach becomes repetitive. SQL Server 2022 introduced `SHORTEST_PATH` which handles variable-length traversal more cleanly:

```

-- SQL Server 2022+: Variable-length path with SHORTEST_PATH

SELECT
    source.doc_id      AS source_doc_id,
    target.doc_id      AS target_doc_id,
    COUNT(edge.$edge_id) AS path_length
FROM dbo.Documents AS source,
     dbo.Cites FOR PATH AS edge,
     dbo.Documents FOR PATH AS target
WHERE MATCH(SHORTEST_PATH(source(-(edge)->target)+))
     AND source.doc_id IN (SELECT doc_id FROM VectorSeeds)
HAVING COUNT(edge.$edge_id) <= 3

```

`SHORTEST_PATH` finds the minimum hop count between source and target, which is preferable to the manual CTE expansion when you want the true graph distance rather than a fixed-depth expansion.

Evaluating Hybrid Retrieval Quality

Adding graph traversal to a vector search pipeline should be justified by measurable retrieval quality improvement. The standard evaluation framework applies: build a test set of queries with human-labeled relevant documents, compute Precision@k and NDCG@k for pure vector search, then recompute after adding graph expansion.

A useful heuristic to check before running a full evaluation: compute the overlap between your graph-expanded candidate set and the human-labeled relevant set. If the graph expansion surfaces relevant documents that vector search missed, the architecture is earning its complexity. If the overlap is near zero — graph expansion consistently surfaces documents that human raters do not find relevant — your graph edges may not encode the right relationships for the retrieval task.

The graph connectivity metric from the Neo4j example earlier (`graph_connectivity`) can serve as a proxy relevance signal during development. Documents with zero graph connectivity in a well-constructed knowledge graph are either orphans (data quality problem) or highly specialized documents with narrow applicability. High-connectivity documents are structurally central and tend to be more broadly relevant. Track this distribution in your test set before tuning score weights.

```

# Python
# Offline evaluation: compare pure vector vs. graph-vector hybrid retrieval

from sklearn.metrics import ndcg_score
import numpy as np

def evaluate_retrieval(
    test_queries: list[dict], # [{"query": str, "relevant_ids": list[str]}]
    searcher_fn,              # callable(query) -> list[str] of doc_ids
    k: int = 10
) -> dict:
    """
    Compute Precision@k and NDCG@k over a labeled test set.

    test_queries: list of dicts with 'query' and 'relevant_ids' keys
    searcher_fn: function that takes a query string and returns ordered list of
doc_ids
    k: cutoff rank
    """
    precisions = []
    ndcg_scores = []

    for item in test_queries:
        query = item["query"]
        relevant = set(item["relevant_ids"])

        retrieved = searcher_fn(query)[:k]

        # Precision@k
        hits = sum(1 for doc_id in retrieved if doc_id in relevant)
        precisions.append(hits / k)

        # NDCG@k: binary relevance
        relevance_true = np.array([[1 if doc_id in relevant else 0
                                   for doc_id in retrieved]])

        # Ideal ranking: all relevant docs at top
        ideal = np.array([[1] * min(len(relevant), k) +
                           [0] * max(0, k - len(relevant))])

        if relevance_true.sum() > 0:
            ndcg_scores.append(ndcg_score(ideal, relevance_true, k=k))
        else:
            ndcg_scores.append(0.0)

    return {
        f"precision_at_{k}": np.mean(precisions),
        f"ndcg_at_{k}": np.mean(ndcg_scores),
        "n_queries": len(test_queries)
    }

# Compare strategies
def pure_vector_search(query: str) -> list[str]:
    # Returns doc_ids from vector-only search
    ...

```

```
def hybrid_search(query: str) -> list[str]:
    # Returns doc_ids from graph+vector hybrid search
    ...

# Load your labeled test set
test_set = [
    {"query": "statin drug interactions elderly", "relevant_ids": ["doc_001",
"doc_045", "doc_102"]},
    # ... more test cases
]

vector_metrics = evaluate_retrieval(test_set, pure_vector_search, k=10)
hybrid_metrics = evaluate_retrieval(test_set, hybrid_search, k=10)

print("Pure Vector Search:")
print(f" Precision@10: {vector_metrics['precision_at_10']:.4f}")
print(f" NDCG@10:      {vector_metrics['ndcg_at_10']:.4f}")

print("\nGraph-Vector Hybrid Search:")
print(f" Precision@10: {hybrid_metrics['precision_at_10']:.4f}")
print(f" NDCG@10:      {hybrid_metrics['ndcg_at_10']:.4f}")

improvement = (hybrid_metrics['ndcg_at_10'] - vector_metrics['ndcg_at_10']) /
vector_metrics['ndcg_at_10'] * 100
print(f"\nNDCG@10 improvement: {improvement:+.1f}%")
```

Run this evaluation before committing to the hybrid architecture. If the improvement is under a few percentage points in NDCG@10, the operational complexity of maintaining a graph database alongside your vector store is likely not justified. If you see meaningful gains — particularly on queries that require structural disambiguation — the hybrid approach is paying for itself.

Key Takeaways

- → **Graph and vector search solve orthogonal retrieval problems.** Vector similarity finds semantically proximate content; graph traversal finds structurally related content. Many production AI applications require both, and the combination surfaces results that neither approach finds alone.
- → **Three integration patterns cover most production needs:** pre-filter by graph then rank by vector, post-retrieve by vector then expand by graph, and graph-guided RAG where entity relationships shape what context the LLM receives. Choose the pattern based on whether structural constraints are known before or after the semantic query.

- → **SQL Server native graph tables + VECTOR + recursive CTEs is a viable graph-vector stack** for shallow relationship graphs (one to two hops) and data volumes below the tens of millions. It eliminates operational complexity at the cost of expressiveness for deep traversal queries.
- → **Native graph-vector engines (Neo4j 5.x, Weaviate, Amazon Neptune Analytics) reduce application-layer complexity** but impose scale ceilings and operational learning curves. Evaluate them when your graph queries exceed two hops or when application-layer orchestration between two separate systems creates unacceptable latency or consistency challenges.
- → **Measure before you build.** Graph expansion should be justified by Precision@k and NDCG@k improvements on a labeled test set. If the graph structure of your data does not encode relationships that are semantically meaningful for your retrieval task, adding a graph layer adds latency and operational burden without improving answer quality.

Chapter 33: Semantic Caching

33.1 How Semantic Caching Differs from Conventional Caching

LLM API calls are expensive, slow, and deterministic enough that many questions asked of a system are semantically equivalent even when worded differently. A user asking "What's the refund policy?" and another asking "How do I get my money back?" want the same answer. Conventional caching based on exact string matching would treat these as cache misses. Semantic caching uses vector similarity to recognize that two queries mean roughly the same thing and return a cached response — cutting latency from seconds to milliseconds and reducing token costs by 40–80% in production workloads with high query repetition. This chapter covers how semantic caches work, how to build one on SQL Server 2025 native vectors with a Redis tier, how to integrate caching into LangChain and LlamaIndex pipelines, and how to operate a semantic cache in production without sacrificing answer quality.

The core operation is a nearest-neighbor lookup over cached *questions*: embed the incoming question, find the most similar previously asked question, and if the distance is below a strict threshold, return its cached answer. The threshold is the entire safety story — too loose and paraphrased-but-different questions get wrong answers; too tight and the cache never hits.

```
graph LR
    Q[Incoming question] --> E[Embed question]
    E --> L{Distance to nearest  
cached question?}
    L -->|< threshold| H[Return cached answer]
    L -->|>= threshold| M[Full RAG pipeline]
    M --> S[Store Q/A pair  
in cache]
```

33.2 Building a Semantic Cache on SQL Server

SQL Server 2025 with native `VECTOR` is a natural fit for a semantic cache if your application already runs on SQL Server. You avoid introducing a new infrastructure component, benefit from transactional consistency, and can use standard T-SQL for cache management queries.

```

-- SQL Server 2025: semantic answer cache
CREATE TABLE dbo.SemanticAnswerCache (
    EntryId          BIGINT IDENTITY(1,1) NOT NULL
        CONSTRAINT PK_SemanticAnswerCache PRIMARY KEY,
    Namespace        NVARCHAR(60) NOT NULL,      -- 'support', 'sales', 'codegen'
    TenantId         INT NOT NULL,               -- never share answers across tenants
    QuestionText     NVARCHAR(1000) NOT NULL,
    QuestionEmb      VECTOR(1536) NOT NULL,
    AnswerText       NVARCHAR(MAX) NOT NULL,
    ModelId          NVARCHAR(100) NOT NULL,
    HitCount         BIGINT NOT NULL CONSTRAINT DF_SAC_Hits DEFAULT (0),
    CreatedAt        DATETIME2(0) NOT NULL
        CONSTRAINT DF_SAC_Created DEFAULT SYSUTCDATETIME(),
    LastHitAt        DATETIME2(0) NULL
);
CREATE INDEX IX_SAC_Namespace_Tenant
    ON dbo.SemanticAnswerCache (Namespace, TenantId);
CREATE VECTOR INDEX IX_SAC_QuestionEmb
    ON dbo.SemanticAnswerCache (QuestionEmb)
    WITH (METRIC = 'cosine', TYPE = 'DiskANN', MAXDOP = 4);

```

Python client wrapping the SQL Server cache

```

# Python – semantic cache backed by SQL Server native VECTOR
import pyodbc, openai

class SqlServerSemanticCache:
    """Semantic cache backed by SQL Server native VECTOR."""

    def __init__(self, conn_str, model="text-embedding-3-small",
                  threshold=0.08):
        self.conn = pyodbc.connect(conn_str)
        self.client = openai.OpenAI()
        self.model = model
        self.threshold = threshold # cosine distance: lower = stricter

    def _embed(self, text):
        return self.client.embeddings.create(
            input=[text], model=self.model).data[0].embedding

    def lookup(self, question, namespace="support", tenant_id=1):
        q = "[" + ",".join(str(v) for v in self._embed(question)) + "]"
        cur = self.conn.cursor()
        row = cur.execute("""SELECT TOP (1) EntryId, AnswerText,
            VECTOR_DISTANCE('cosine', QuestionEmb,
                CAST(? AS VECTOR(1536))) AS Distance
        FROM dbo.SemanticAnswerCache
        WHERE Namespace = ? AND TenantId = ? AND ModelId = ?
            AND VECTOR_DISTANCE('cosine', QuestionEmb,
                CAST(? AS VECTOR(1536))) < ?
        ORDER BY Distance""",
            (q, namespace, tenant_id, self.model, q, self.threshold)
        ).fetchone()
        if row is None:
            return None
        cur.execute("""UPDATE dbo.SemanticAnswerCache
            SET HitCount = HitCount + 1,
                LastHitAt = SYSUTCDATETIME()
            WHERE EntryId = ?""", (row[0],))
        self.conn.commit()
        return {"answer": row[1], "distance": row[2],
            "cached": True}

    def store(self, question, answer, namespace="support", tenant_id=1):
        q = "[" + ",".join(str(v) for v in self._embed(question)) + "]"
        cur = self.conn.cursor()
        cur.execute("""INSERT INTO dbo.SemanticAnswerCache
            (Namespace, TenantId, QuestionText, QuestionEmb,
                AnswerText, ModelId)
            VALUES (?, ?, ?, CAST(? AS VECTOR(1536)), ?, ?)""",
            (namespace, tenant_id, question, q, answer, self.model))
        self.conn.commit()

```


Two non-negotiable predicates in the lookup: `TenantId` (cached answers must never leak across tenants — Chapter 34) and `ModelId` (an answer generated from one embedding model's geometry must not be served for another's). The threshold of 0.08 cosine distance is a starting point — tune it on your eval set, measuring false-hit rate, not just hit rate.

33.3 Semantic Caching with Redis and a Dedicated Vector Store

For applications that need sub-5ms cache lookups and cannot afford a full SQL Server query round-trip, Redis with the `RediSearch` module (or `Redis Stack`) provides an in-memory vector cache. The tradeoff is that Redis is bounded by available memory; entries must be kept small or the cache must be tiered.

The production pattern is a tiered cache: Redis holds the hottest question/answer pairs as exact-match entries keyed by question hash (sub-millisecond, no vector math), while SQL Server holds the full semantic cache for near-duplicate lookup. The request path checks Redis first, then SQL Server semantic lookup, then the full RAG pipeline — each tier an order of magnitude cheaper than the next to miss.

```
# Python – tiered lookup: Redis exact-match -> SQL Server semantic -> RAG
import hashlib, json
import redis

r = redis.Redis(host="localhost", port=6379, decode_responses=True)
sem_cache = SqlServerSemanticCache(conn_str="...") # Section 33.2

def answer_question(question, namespace="support", tenant_id=1):
    # Tier 1: Redis exact match on normalized question text
    norm = " ".join(question.lower().split())
    key = f"semcache:{namespace}:{tenant_id}:" \
        f"{hashlib.sha256(norm.encode()).hexdigest()}"
    hit = r.get(key)
    if hit:
        return {"answer": json.loads(hit)["a"], "tier": "redis"}
    # Tier 2: SQL Server semantic (near-duplicate) lookup
    sem = sem_cache.lookup(question, namespace, tenant_id)
    if sem:
        r.setex(key, 3600, json.dumps({"a": sem["answer"]}))
        return {"answer": sem["answer"], "tier": "semantic"}
    # Tier 3: full RAG pipeline, then populate both tiers
    result = rag_answer(question, tenant_id) # Chapters 21/25 pipeline
    sem_cache.store(question, result["answer"], namespace, tenant_id)
    r.setex(key, 3600, json.dumps({"a": result["answer"]}))
    return {"answer": result["answer"], "tier": "rag"}
```

33.4 LangChain and LlamaIndex Integration

LangChain with GPTCache and a SQL Server backend. GPTCache supports custom storage backends — implement its storage interface over `dbo.SemanticAnswerCache` and LangChain's `set_llm_cache()` picks it up with no pipeline changes. The key detail is the similarity evaluation function: GPTCache's default is exact-match-friendly, so configure its cosine-distance threshold to match the 0.08 starting point from Section 33.2 and tune from there.

LlamaIndex. Wrap the query engine: check `SqlServerSemanticCache.lookup()` before calling `query_engine.query()`, and `store()` after. For LlamaIndex's agent workloads, cache at the tool-result level rather than the final-answer level (Section 33.7) — tool calls repeat far more often than full agent trajectories.

In production, keep the framework integration thin: the cache class from Section 33.2 owns storage, thresholds, and tenant isolation; the framework glue is a few lines that call it. When you tune the threshold or change the schema, you change one place.

33.5 Cache Hit Rate Optimization

Hit rate is driven by question repetition, which you can engineer:

- → **Normalize before hashing.** Lowercase, collapse whitespace, strip punctuation for the Redis exact-match tier — "What's the refund policy?" and "what's the refund policy" are the same question.
- → **Segment by namespace.** A single cache namespace mixing product questions, support queries, and code generation requests produces false positive hits — code questions semantically overlapping with English descriptions. Segmenting by namespace (which maps to the `Namespace` column predicate in SQL Server) keeps the search space focused and reduces false positives.
- → **Tune the threshold per namespace.** Support FAQs tolerate a looser threshold (answers are stable, paraphrases are common); code generation needs a strict one (small wording changes alter the correct answer).
- → **Seed from history.** Pre-populate the cache from query logs: take the top-N frequent questions, run them through the RAG pipeline once, and store the pairs. Day-one hit rate jumps from zero to the steady-state value.

```
-- SQL Server: hit-rate and false-hit analysis
SELECT Namespace,
       COUNT(*) AS Entries,
       SUM(HitCount) AS TotalHits,
       CAST(SUM(HitCount) AS FLOAT) / NULLIF(COUNT(*), 0) AS AvgHitsPerEntry
FROM dbo.SemanticAnswerCache
GROUP BY Namespace
ORDER BY AvgHitsPerEntry DESC;
-- Namespaces with high entries but ~0 hits are mis-tuned:
-- threshold too strict, or questions that never repeat.
-- Namespaces with rising user complaints need threshold tightening,
-- not hit-rate celebration.
```

33.6 Production Operations: Monitoring, Eviction, and Invalidation

Unlike memory-bounded caches like Redis, a SQL Server-backed semantic cache grows without bound unless you actively evict entries. Three practical eviction strategies:

```
-- SQL Server: eviction strategies for the semantic cache
-- 1. Age-based: drop entries never hit in 90 days
DELETE FROM dbo.SemanticAnswerCache
WHERE LastHitAt < DATEADD(day, -90, SYSUTCDATETIME())
  AND CreatedAt < DATEADD(day, -90, SYSUTCDATETIME());
-- 2. Hit-count-based: keep only entries earning their storage
-- DELETE FROM dbo.SemanticAnswerCache
-- WHERE HitCount = 0 AND CreatedAt < DATEADD(day, -30, SYSUTCDATETIME());
-- 3. Capacity-based: keep the top-N per namespace by hits
-- WITH ranked AS (
--     SELECT EntryId, ROW_NUMBER() OVER (
--         PARTITION BY Namespace ORDER BY HitCount DESC) AS rn
--     FROM dbo.SemanticAnswerCache)
-- DELETE FROM ranked WHERE rn > 100000;
```

Invalidation is event-driven: when a source document changes, flush cache entries whose answers derived from it. Track provenance — store the source `DocId` list alongside each cached answer — so a document update can `DELETE` exactly the affected entries instead of flushing the namespace.

```
# Python – event-driven invalidation
def handle_document_updated_event(event, cache):
    """Flush cached answers derived from the changed document."""
    doc_id = event["doc_id"]
    cur = cache.conn.cursor()
    cur.execute("""DELETE FROM dbo.SemanticAnswerCache
                  WHERE EntryId IN (
                      SELECT EntryId FROM dbo.SemanticAnswerProvenance
                      WHERE DocId = ?)""", (doc_id,))
    deleted = cur.rowcount
    cache.conn.commit()
    return deleted

def handle_namespace_flush_event(event, cache):
    """Nuclear option: flush a whole namespace (e.g., policy rewrite)."""
    cur = cache.conn.cursor()
    cur.execute("DELETE FROM dbo.SemanticAnswerCache WHERE Namespace = ?",
                (event["namespace"],))
    cache.conn.commit()
```

Monitor hit rate, false-hit complaints, and p99 lookup latency per tier. A semantic cache whose hit rate climbs while user complaints climb is a threshold problem — tighten it. A cache whose latency approaches the RAG pipeline's latency has lost its purpose — check the DiskANN index health and the Redis tier.

33.7 Semantic Caching for Agentic and Multi-Turn Workloads

Agents break the single-question assumption: a multi-turn conversation has context, tool calls, and intermediate reasoning. Cache at the right granularity:

- → **Tool-result caching.** Agent tool calls (database lookups, API fetches) repeat across conversations far more than full trajectories. Cache tool results keyed by (tool name, normalized arguments) in the Redis exact-match tier — deterministic, high hit rate, trivially safe.
- → **Turn-level caching with conversation context.** For follow-up questions ("and what about the enterprise plan?"), embed the question *plus* a summary of the conversation so far. The cache key captures the context the answer depends on; without it, follow-ups collide with unrelated same-text questions.
- → **Never cache across different tool states.** An answer that depended on a tool result must be invalidated when the underlying data changes — extend the provenance pattern from Section 33.6 to tool outputs.

33.8 Failure Modes and Mitigations

- → **False hits (wrong answer served confidently).** The catastrophic mode. Mitigate with strict per-namespace thresholds, user feedback signals that auto-tighten the threshold, and a "report wrong answer" path that deletes the offending entry immediately.
- → **Stale answers.** The source changed; the cache didn't hear about it. Mitigate with provenance-tracked invalidation (Section 33.6), not TTLs — TTLs either flush too aggressively (killing hit rate) or too lazily (serving stale answers).
- → **Tenant leakage.** A cached answer from tenant A served to tenant B. Mitigate by making `TenantId` a mandatory predicate in every lookup — enforced in the cache class, not left to callers.
- → **Threshold drift.** What was strict enough at 10K entries may be too loose at 10M, as denser space means closer near-duplicates. Re-tune the threshold quarterly against the eval set.

Key Takeaways

- → Semantic caching trades LLM calls for vector lookups: embed the question, near-duplicate-match against cached questions in SQL Server, and skip the pipeline on a hit.
- → The similarity threshold is the entire safety story — tune it per namespace on evals, measuring false-hit rate, not just hit rate.
- → `TenantId` and `ModelId` are mandatory lookup predicates — tenant leakage and cross-model reads are silent corruption.
- → Tier the cache: Redis exact-match for sub-millisecond hot hits, SQL Server semantic for near-duplicates, RAG pipeline as the fallback that repopulates both.
- → Invalidate by provenance (document → affected entries), evict by age/hits/capacity, and never rely on TTLs for correctness.

Chapter 34: Access Control for Vector Data

Why Vector Access Control Is Different

Vector databases introduce a category of access control problems that traditional RBAC models weren't designed to solve. A row in a relational table is discrete and bounded — you can grant `SELECT` on a specific table, filter rows by tenant ID, and call it done. A vector embedding is a compressed semantic fingerprint of content that may span topics, contain implicit references to sensitive material, or be retrievable through queries that have no obvious surface-level connection to the underlying data. When a similarity search returns the top-k nearest neighbors, you cannot predict at query time which documents will surface. This chapter covers how to apply SQL Server Row-Level Security, namespace isolation, API key scoping, and query-time filtering to vector data across SQL Server native vectors, Pinecone, Weaviate, and Qdrant — and how to build a unified authorization layer that treats vector search as a first-class security boundary.

Row-Level Security on SQL Server Vector Tables

SQL Server's Row-Level Security (RLS) applies directly to vector tables: a security policy with a predicate function filters rows before the similarity ranking ever sees them. Every security primitive SQL Server exposes — roles, schemas, column-level grants, views — applies to vector data exactly as it does to relational data.

```

-- SQL Server 2025: RLS on a vector chunk table
-- 1. Predicate function: which tenant is calling?
CREATE FUNCTION dbo.fn_TenantPredicate(@TenantId INT)
RETURNS TABLE
WITH SCHEMABINDING
AS RETURN SELECT 1 AS AccessGranted
WHERE @TenantId = CAST(SESSION_CONTEXT(N'TenantId') AS INT)
      OR CAST(SESSION_CONTEXT(N'TenantId') AS INT) = 0; -- 0 = service admin
GO
-- 2. Security policy binding the predicate to the vector table
CREATE SECURITY POLICY dbo.TenantFilterPolicy
ADD FILTER PREDICATE dbo.fn_TenantPredicate(TenantId)
      ON dbo.DocumentChunks,
ADD BLOCK PREDICATE dbo.fn_TenantPredicate(TenantId)
      ON dbo.DocumentChunks AFTER INSERT;
GO
-- 3. Application sets context once per session:
-- EXEC sp_set_session_context 'TenantId', @tenantId;
-- Every vector query below now silently filters to the tenant:
DECLARE @q VECTOR(1536) = '[0.12, 0.87, ...]';
SELECT TOP (10) ChunkId, ChunkText,
      VECTOR_DISTANCE('cosine', Embedding, @q) AS Distance
FROM dbo.DocumentChunks
ORDER BY Distance;
-- No WHERE TenantId needed: the policy injects it. Forget the
-- sp_set_session_context call and the query returns zero rows –
-- fail-closed by design.

```

A frequent performance pitfall: when RLS adds a `TenantId = X` predicate to a DiskANN scan, the filter applies after the ANN index returns candidates, not before. If the index returns 200 candidates but only 3 belong to the current tenant, effective recall collapses and you waste I/O on irrelevant rows.

The solution is tenant-aware physical design: partition the chunk table by tenant (partitioned table with one DiskANN index per partition is not supported in preview — so instead maintain separate chunk tables per tenant tier, or over-fetch aggressively with `TOP_N` and filter in the outer query):

```

-- SQL Server: tenant-safe ANN retrieval with over-fetch + filter
-- RLS narrows to the tenant; TOP_N over-fetch compensates for
-- post-index filtering so recall survives.
DECLARE @q VECTOR(1536) = '[0.12, 0.87, ...]';
EXEC sp_set_session_context 'TenantId', @tenantId;

SELECT TOP (10) s.ChunkId, c.ChunkText, s.Distance
FROM (
    SELECT TOP (200) ChunkId,
           VECTOR_DISTANCE('cosine', Embedding, @q) AS Distance
    FROM dbo.DocumentChunks          -- RLS predicate applies here
    ORDER BY Distance
) AS s
JOIN dbo.DocumentChunks AS c ON c.ChunkId = s.ChunkId
ORDER BY s.Distance;
-- Rule of thumb: over-fetch 10–20x the final k when a tenant filter
-- follows the ANN scan; measure recall@k per tenant (Chapter 20).

```

For Azure SQL Database, the same RLS pattern works identically — set the session context from your API's authenticated claims (Microsoft Entra ID) so the tenant flows from login to predicate without application code in between.

Namespace Isolation in Pinecone and Weaviate

Pinecone namespaces partition one index into isolated vector spaces: queries against namespace `tenant-42` can never return vectors from `tenant-43`, because the namespace is part of the storage key. Map one tenant to one namespace and the isolation is structural, not predicate-based — there is no filter to forget.

Weaviate offers the same shape via tenant-aware collections (multi-tenancy feature): each tenant gets isolated shards within a collection, with per-tenant activation so cold tenants consume no memory. Qdrant's payload filtering achieves tenant isolation with a mandatory `tenant_id` payload filter — effective, but predicate-based, so enforce it in a wrapper client rather than trusting every caller.

The hierarchy of assurance, strongest first: separate indexes per tenant → namespaces/tenant-shards → mandatory metadata filters → RLS predicates. Choose based on tenant count and isolation requirements; a hundred tenants with strict data residency get separate indexes, ten thousand SaaS tenants get namespaces or RLS.

Metadata Filtering as a Security Predicate

Beyond tenant isolation, document-level permissions (this user may see these documents) belong in the retrieval query as metadata predicates. Store the access control list as columns — `AllowedRoles`, `OwnerId`, `Classification` — and apply them in the same query as the similarity ranking:

```
-- SQL Server: document-level permission predicate in the retrieval query
DECLARE @q VECTOR(1536) = '[0.12, 0.87, ...]';
DECLARE @userRoles NVARCHAR(400) = '|analyst|auditor|'; -- from auth context

SELECT TOP (10) ChunkId, ChunkText,
               VECTOR_DISTANCE('cosine', Embedding, @q) AS Distance
FROM dbo.DocumentChunks
WHERE TenantId = CAST(SESSION_CONTEXT(N'TenantId') AS INT)
  AND (Classification <> 'Restricted'
       OR OwnerId = CAST(SESSION_CONTEXT(N'UserId') AS INT)
       OR EXISTS (SELECT 1 FROM STRING_SPLIT(AllowedRoles, ',') AS r
                  WHERE @userRoles LIKE '%' + LTRIM(RTRIM(r.value)) + '%'))
ORDER BY Distance;
-- Permission logic lives in the query, not in post-retrieval application
-- code: a chunk the user may not see is never returned, never embedded
-- in a prompt, never summarized by the LLM.
```

The critical rule: **enforce permissions before retrieval, never after**. Filtering the top-k in application code is both a security hole (the LLM already saw the text if you re-rank first) and a quality bug (you return fewer than k results). The predicate belongs in the database query.

API Key Management and Least-Privilege Design

- → **Separate keys per trust boundary**. Ingest workers get write-only credentials; the retrieval API gets read-only; humans get nothing — they authenticate through the application. A leaked read-only key cannot poison the index.
- → **Scope managed-service keys**. Pinecone API keys should be scoped to the project and rotated on a schedule; store them in Azure Key Vault, not in application configuration.
- → **SQL Server: use contained database users or Entra ID**. The retrieval service connects as a low-privilege database user with SELECT on the chunk tables and nothing else — no DDL, no access to the embedding queue or eval tables.
- → **Rotate on a schedule, not on incident**. Quarterly rotation with dual-key support (accept N and N+1 during rollover) turns rotation into a routine instead of a fire drill.

Audit Logging for Vector Queries

Vector queries need auditing like any data access — which tenant asked what, which chunks were returned, and whether permission predicates were active. Log at the retrieval-procedure boundary, not in the application:

```
-- SQL Server: audit log table for vector retrieval
CREATE TABLE dbo.VectorQueryAudit (
    AuditId      BIGINT IDENTITY(1,1) NOT NULL
        CONSTRAINT PK_VectorQueryAudit PRIMARY KEY,
    OccurredAt   DATETIME2(0) NOT NULL
        CONSTRAINT DF_VQA_Occurred DEFAULT SYSUTCDATETIME(),
    TenantId     INT NOT NULL,
    UserId       INT NULL,
    Namespace    NVARCHAR(60) NOT NULL,
    TopK         INT NOT NULL,
    ReturnedIds  NVARCHAR(MAX) NOT NULL, -- JSON array of ChunkIds served
    MinDistance  FLOAT NULL,             -- top-1 distance: quality signal
    ClientIp     NVARCHAR(45) NULL
);
CREATE INDEX IX_VQA_Tenant_Time ON dbo.VectorQueryAudit (TenantId, OccurredAt);
-- Write one row per retrieval call from the stored procedure that serves
-- vector search. Partition by month; retention per compliance policy.
-- The ReturnedIds JSON lets incident response answer "who saw chunk X?"
-- with a single query.
```

```
# Python – audited retrieval wrapper (pyodbc)
import json, pyodbc

conn = pyodbc.connect("DRIVER={ODBC Driver 18 for SQL Server};"
                      "SERVER=localhost;DATABASE=ragdb;Trusted_Connection=yes;"
                      "Encrypt=yes;TrustServerCertificate=yes")

def audited_retrieve(question_emb, tenant_id, user_id, namespace, top_k=10):
    vec = "[" + ",".join(str(v) for v in question_emb) + "]"
    cur = conn.cursor()
    cur.execute("EXEC sp_set_session_context 'TenantId', ?",
                (tenant_id,)) # RLS context: fail-closed per Section 34.2
    rows = cur.execute("""SELECT TOP (?) ChunkId, ChunkText,
                          VECTOR_DISTANCE('cosine', Embedding,
                          CAST(? AS VECTOR(1536))) AS Distance
                          FROM dbo.DocumentChunks
                          ORDER BY Distance""", (top_k, vec)).fetchall()
    cur.execute("""INSERT INTO dbo.VectorQueryAudit
                  (TenantId, UserId, Namespace, TopK, ReturnedIds,
                  MinDistance)
                  VALUES (?, ?, ?, ?, ?, ?)""",
                (tenant_id, user_id, namespace, top_k,
                 json.dumps([r[0] for r in rows]),
                 rows[0][2] if rows else None))
    conn.commit()
    return rows
```

Handling Sensitive Data in Embedding Pipelines

Embeddings are not anonymized data. A vector preserves enough semantic content that sensitive text can be partially reconstructed, and the chunk text stored alongside it is the original sensitive content in full. Treat the vector store with the same classification as the source documents:

- → **Encrypt at rest and in transit.** TDE for the database, TLS for every connection — including the embedding API calls that carry raw text off-network.
- → **Redact before embedding.** PII scrubbing belongs in the ingest pipeline, before the embedding call — once PII is embedded and stored, every copy (cache, backup, replica) carries it.
- → **Prefer local embedding models for regulated data.** Sentence Transformers on your own infrastructure keeps text in-network; API embedding sends it to a third party under their retention policy.

- → **Backups inherit classification.** A backup of the vector database is a backup of the sensitive corpus — same encryption, same access controls, same retention policy (Chapter 36).

A Unified Authorization Layer

Production systems rarely use a single vector database. A typical architecture might use SQL Server native vectors for low-latency operational queries, Pinecone for high-scale semantic search, and Weaviate for knowledge graph traversal with vector similarity. Each database has its own access control model. Building authorization logic independently for each creates inconsistency and maintenance burden.

The unified layer is a policy decision point in front of all of them: one function that maps (user, tenant, roles) to (allowed namespaces, filters, top-k limits), called by every retrieval path regardless of backend:

```
# Python – unified authorization layer across vector backends
from dataclasses import dataclass

@dataclass
class AuthzDecision:
    tenant_id: int
    namespace: str          # Pinecone namespace / Weaviate tenant / SQL predicate
    max_top_k: int
    classification_ceiling: str # highest classification the user may see
    sql_tenant_filter: str     # fragment for SQL Server queries

def authorize(user, backend_name: str) -> AuthzDecision:
    """Single policy decision point. backend_name: "sqlserver" | "pinecone"
    | "qdrant" | "weaviate". Every retrieval path calls this first."""
    tenant_id = user.tenant_id
    roles = set(user.roles)
    ceiling = "Restricted" if "auditor" in roles else "Internal"
    max_k = 50 if "analyst" in roles else 10
    namespace = f"tenant-{tenant_id}" if backend_name == "pinecone" \
        else "default"
    return AuthzDecision(
        tenant_id=tenant_id, namespace=namespace, max_top_k=max_k,
        classification_ceiling=ceiling,
        sql_tenant_filter=f"TenantId = {tenant_id}")
# SQL Server path: sp_set_session_context + RLS (Section 34.2)
# Pinecone path: namespace=decision.namespace, top_k=min(k, decision.max_top_k)
# Weaviate path: tenant filter + top_k cap
# One policy, every backend – audit once, at this layer.
```

Key Takeaways

- → **SQL Server RLS integrates directly with vector tables** via security policies and `SESSION_CONTEXT` — but ANN index performance degrades when tenant predicates apply post-scan, so over-fetch 10–20x and measure recall@k per tenant.
- → **Enforce permissions before retrieval, never after.** Predicates belong in the database query; post-retrieval filtering is a security hole and a quality bug.
- → **Choose isolation strength by tenant reality:** separate indexes for strict residency, namespaces/tenant-shards for SaaS scale, RLS predicates where the database is SQL Server.
- → **Audit at the retrieval boundary** — tenant, user, returned chunk IDs, top-1 distance — so incident response can answer "who saw chunk X?" with one query.
- → **Treat embeddings as sensitive as source text:** redact before embedding, prefer local models for regulated data, and give backups the corpus's classification.
- → **Unify authorization in one policy decision point** across backends — one function mapping user to namespaces, filters, and limits, audited once.

Chapter 35: Disaster Recovery for Vector Stores

The Unique DR Challenges of Vector Stores

In a relational database, an index is disposable — you can drop and rebuild it without losing data, because the table is the source of truth. A DiskANN vector index in SQL Server 2025 holds the navigational graph over your vectors; the vectors themselves live in the table, but the index is what makes retrieval fast. After a restore, the index structure must be rebuilt from the table data — at significant cost for large tables. This shifts backup philosophy: you cannot plan vector DR as "restore the database and go" — the DiskANN rebuild is a first-class RTO component.

Vector stores rarely stand alone. They contain embeddings derived from records in SQL Server, document stores, or S3-compatible object stores. A complete DR plan must account for the synchronization point between source data and the vector index. If your source database is restored to a point-in-time 6 hours before your vector store backup, you now have semantic drift between your records and their embeddings — queries return answers grounded in documents that no longer exist.

```
graph TB
    SRC[Source documents  
SQL Server / object store] --> PIPE[Embedding pipeline]
    PIPE --> VEC[Vector tables  
dbo.DocumentChunks]
    VEC --> IDX[DiskANN index]
    BK[Backups] -.-> VEC
    BK -.-> SRC
    AG[Availability Group  
secondary replica] --> VEC
    VEC --> FAIL[Failover target]
```

Backup Strategies for SQL Server Vector Tables

SQL Server vector tables are ordinary tables with a `VECTOR` column — every native backup mechanism works without modification: full/differential/log backups, backup to URL, and managed backup in Azure. The DiskANN index definition is part of the database metadata, so it restores with the database — but the index *contents* are rebuilt from the vector data during recovery, not copied byte-for-byte. Plan for that rebuild.

```

-- SQL Server 2025: backup strategy for a vector database
-- Weekly full, daily differential, 15-minute log backups
BACKUP DATABASE VectorRagDb
TO URL = 'https://vecbackups.blob.core.windows.net/sqlbackups/VectorRagDb_full.bak'
WITH COMPRESSION, CHECKSUM, STATS = 10;
-- Differential (nightly)
BACKUP DATABASE VectorRagDb
TO URL = 'https://vecbackups.blob.core.windows.net/sqlbackups/VectorRagDb_diff.bak'
WITH DIFFERENTIAL, COMPRESSION, CHECKSUM;
-- Log (every 15 minutes via Agent job)
BACKUP LOG VectorRagDb
TO URL = 'https://vecbackups.blob.core.windows.net/sqlbackups/VectorRagDb_log.trn'
WITH COMPRESSION, CHECKSUM;
-- Record the watermark so post-restore catch-up knows where to start:
-- SELECT MAX(EmbeddedAt) FROM dbo.DocumentChunks; -- store with the backup

```

For very large vector tables where full backup windows are painful, consider a two-tier strategy: native SQL Server backups for the whole database, plus a periodic bulk export of `DocId → vector` pairs to Parquet in object storage. The export is your escape hatch — it lets you rebuild the vector tables on any SQL Server instance (or a different platform) without replaying the entire backup chain.

The critical insight for SQL Server vector DR: after a restore, the database comes online first, then the DiskANN index rebuilds from the vector column data. On a 50M-row table with 1536-dimensional DiskANN indexes, the rebuild can take several hours depending on MAXDOP and memory. Your RTO must account for this separately from the restore time — measure it on a restore test, don't estimate it.

Replication and High Availability Topologies

Always On availability groups are the primary HA story for SQL Server vector workloads. Vectors in `VECTOR` columns replicate through the log stream exactly like any other data, and the DiskANN index on the secondary stays in sync through redo — you are not rebuilding it, you are streaming physical changes. A synchronous-commit secondary gives you zero-data-loss failover with the vector index warm and queryable immediately.

```
-- SQL Server: verify AG replica health for the vector database
SELECT ag.name AS AgName, ar.replica_server_name AS Replica,
       ars.role_desc AS Role, ars.synchronization_state_desc AS SyncState,
       ars.synchronization_health_desc AS Health,
       drs.log_send_queue_size AS SendQueueKB,
       drs.redo_queue_size AS RedoQueueKB
FROM sys.dm_hadr_availability_replica_states AS ars
JOIN sys.availability_replicas AS ar
    ON ars.replica_id = ar.replica_id
JOIN sys.availability_groups AS ag
    ON ar.group_id = ag.group_id
JOIN sys.dm_hadr_database_replica_states AS drs
    ON ars.replica_id = drs.replica_id AND ars.group_id = drs.group_id
WHERE drs.database_id = DB_ID(N'VectorRagDb');
-- Watch RedoQueueKB after bulk embedding loads: a growing redo queue
-- means the secondary is falling behind the ingest rate.
```

For cross-region DR, use an asynchronous-commit secondary in the distant region: RPO of seconds-to-minutes, no write-latency penalty on the primary. Combine with distributed availability groups when the DR site is a separate Windows cluster or a different domain. For Azure SQL Database, active geo-replication or auto-failover groups provide the same shape as a managed service — the vector index replicates with the database, and failover is a DNS/connection-string flip.

Cross-Region Failover Runbook

The runbook below assumes an Always On availability group with a synchronous local secondary and an asynchronous remote secondary, with SQL Server as both source system and vector store. Adapt the connection strings for Azure SQL failover groups — the steps are the same, the promotion is a portal/PowerShell call instead of T-SQL.


```

-- SQL Server: planned cross-region failover runbook (T-SQL core)
-- Step 1: Quiesce the embedding pipeline – stop writers first
-- (run from the pipeline orchestrator, not the database)
--   pipelinctl pause --queue embedding-jobs
-- Step 2: Verify the remote secondary is caught up
--   On the primary: check log_send_queue_size / redo_queue_size ~ 0
--   (query from the previous section)
-- Step 3: Fail over the availability group to the remote replica
ALTER AVAILABILITY GROUP [VectorAG]
    FAILOVER; -- run connected to the remote secondary
-- Step 4: Record the watermark for pipeline catch-up
--   SELECT MAX(EmbeddedAt) AS failover_watermark FROM dbo.DocumentChunks;
-- Step 5: Point the application connection strings at the new primary
--   (DNS CNAME flip or config change + rolling restart)
-- Step 6: Resume the embedding pipeline from the watermark
--   pipelinctl resume --queue embedding-jobs --since <watermark>
-- Step 7: Validate – run the recall canary (Chapter 20):
--   recall@10 must be within 2 points of pre-failover baseline
--   before declaring the failover complete.

```

The step teams most often skip is Step 1 — quiescing writers. Failing over with the embedding pipeline still writing means in-flight batches land on the old primary after the flip and are silently lost to the new primary. Pause first, verify the queue is drained, then fail over.

Recovery Point Objective and Recovery Time Objective Planning

Three clocks must align: the source data RPO, the vector store RPO, and the embedding pipeline's processing lag. If your source database recovers to T-minus-2-hours, your vector store recovers to T-minus-6-hours, and your embedding queue was processing documents at the time of failure, you have semantic gaps no automatic process will identify.

Budget RTO in components, measured from restore tests — not vendor documentation:

- → Infrastructure recovery (VMs, networking, SQL Server online)
- → Database restore time (full + differential + log replay)
- → DiskANN index rebuild time — the hidden multiplier; hours at tens of millions of rows
- → Embedding catch-up (documents newer than the watermark, re-embedded at pipeline throughput)
- → Validation (recall canary + smoke queries before traffic returns)

The DiskANN rebuild time is the most significant operational difference versus a pure relational workload. For large deployments, this drives a strong architectural recommendation: maintain a synchronous or near-synchronous secondary as your primary HA mechanism, and treat backup-and-restore as the last resort — the secondary already has a warm index.

```
-- SQL Server: track DiskANN index build progress during recovery
-- (run while the index rebuilds after a restore)
SELECT OBJECT_NAME(object_id) AS TableName, name AS IndexName,
       index_id, type_desc
FROM sys.indexes
WHERE type_desc LIKE '%VECTOR%';
-- Monitor index build phases via the SQL Server error log and
-- extended events (no built-in progress DMV for DiskANN builds);
-- and time full rebuilds during DR tests to calibrate RTO budgets.
```

The Re-Embedding Recovery Path

Sometimes the fastest recovery isn't restoring vectors at all — it's re-embedding from source. If the source documents survived (or restore faster than the vector database), and your embedding pipeline can sustain high throughput, regenerating embeddings from the watermark forward can beat a multi-hour DiskANN rebuild plus log replay.

The re-embedding path requires three things prepared *before* the disaster: the source documents restorable independently, the embedding model version pinned (re-embedding with a different model version silently changes the geometry — Chapter 30), and pipeline throughput measured so you can compute catch-up time honestly. A pipeline that embeds 50 documents/second needs 5.5 hours per million documents — do that math before you promise a 2-hour RTO.

```
-- SQL Server: watermark-based re-embedding catch-up after recovery
-- Documents newer than the last good watermark get re-embedded;
-- everything older is trusted from the restored vector tables.
DECLARE @watermark DATETIME2(0) =
    (SELECT WatermarkValue FROM dbo.DrWatermarks
     WHERE WatermarkName = 'LastGoodEmbedding');

SELECT DocId
FROM dbo.SourceDocuments
WHERE LastModified > @watermark
ORDER BY LastModified;
-- Feed these DocIds to the embedding pipeline (Chapter 28),
-- then update the watermark. The retrieval path serves restored
-- vectors for old docs and fresh vectors for new docs throughout.
```

Testing Your DR Plan

An untested DR plan is a hope, not a plan. Test quarterly at minimum, and test the vector-specific parts — not just "did the database come back":

- → **Restore to an isolated instance** and time the full sequence: restore, DiskANN rebuild, watermark catch-up, recall canary. Record every duration — these are your RTO budget inputs.
- → **Fail over the AG** in a maintenance window and run the recall canary against the new primary before failing back.
- → **Test the re-embedding path** end-to-end on a subset: pick a watermark, re-embed 10K documents, verify recall matches the restored baseline.
- → **Verify the pipeline quiesce.** The most common real-failover bug is writers still running — rehearse the pause/drain/resume sequence, not just the database steps.

Synchronizing Vector Stores with Source Systems After Recovery

After any recovery, the vector store and the source system disagree about "now." Close the gap systematically: compare the source watermark (max document modification time) with the vector watermark (max embedded time), re-embed the delta, and delete vectors for source documents that were hard-deleted during the outage window.

```
-- SQL Server: post-recovery reconciliation
-- 1. Find vectors whose source documents changed or vanished
SELECT c.ChunkId, c.DocId
INTO #StaleCandidates
FROM dbo.DocumentChunks AS c
LEFT JOIN dbo.SourceDocuments AS s
  ON s.DocId = c.DocId AND s.LastModified <= c.EmbeddedAt
WHERE s.DocId IS NULL           -- source deleted during outage
      OR s.LastModified > c.EmbeddedAt; -- source changed after embedding
-- 2. Re-embed the changed docs via the pipeline (Chapter 28),
--    then delete vectors for docs gone from the source:
-- DELETE c FROM dbo.DocumentChunks AS c
-- WHERE NOT EXISTS (SELECT 1 FROM dbo.SourceDocuments AS s
--                   WHERE s.DocId = c.DocId);
-- 3. Re-run the recall canary; only then return traffic.
```

DR Runbook Checklist

- → Quiesce embedding pipeline writers; verify the queue is drained.
- → Check secondary redo/send queues are at zero before failover.
- → Fail over the availability group (or geo-failover group); record the watermark.
- → Flip application connection strings; rolling restart.
- → Resume the pipeline from the recorded watermark.
- → Run the recall canary — within 2 points of baseline before declaring done.
- → Reconcile source vs vector watermarks; re-embed the delta.
- → Update the DR log with actual durations; feed them back into the RTO budget.

Key Takeaways

- → **DiskANN rebuild is a hidden RTO multiplier.** The index rebuilds from vector data after a restore — hours at tens of millions of rows. Measure it in DR tests; budget it explicitly.
- → **Prefer a warm secondary over backup-and-restore.** Always On AG secondaries keep the DiskANN index in sync through redo — failover is minutes, not hours.
- → **Quiesce writers before failover.** In-flight embedding batches landing on the old primary are silently lost — pause, drain, then flip.
- → **Align three clocks:** source RPO, vector RPO, and pipeline lag — then reconcile with watermarks after recovery.
- → **Keep the re-embedding path warm:** pinned model version, measured throughput, independent source restore — it's sometimes faster than restoring vectors.

- → **Test quarterly, time everything.** Restore, rebuild, catch-up, canary — recorded durations are your RTO budget, not estimates.

Chapter 36: Backing Up Embedding Data

The Three-Artifact Problem

Embedding data presents a deceptively familiar backup challenge. The vectors themselves are just floating-point arrays — and any experienced DBA knows how to back up a table of floats. What makes embedding backups genuinely different is the relationship between three distinct artifacts that must remain in sync: the raw source data, the embeddings derived from that data, and the model that produced those embeddings. Back up only two of the three and you have a restoration problem that won't reveal itself until you're already in a production incident. This chapter covers the full backup and recovery strategy for vector workloads on SQL Server — from native snapshots to managed exports, from model versioning to the operational patterns that keep RAG pipelines recoverable at scale.

Consider a document chunk stored in SQL Server:

```

-- SQL Server 2025: the three artifacts live in one database
-- Artifact 1: source data
CREATE TABLE dbo.SourceDocuments (
    DocId          INT IDENTITY(1,1) NOT NULL
        CONSTRAINT PK_SourceDocuments PRIMARY KEY,
    Title          NVARCHAR(400) NOT NULL,
    Body           NVARCHAR(MAX) NOT NULL,
    LastModified   DATETIME2(0) NOT NULL
);
-- Artifact 2: embeddings derived from it (note the provenance columns)
CREATE TABLE dbo.DocumentChunks (
    ChunkId        BIGINT IDENTITY(1,1) NOT NULL
        CONSTRAINT PK_DocumentChunks PRIMARY KEY,
    DocId          INT NOT NULL
        CONSTRAINT FK_Chunks_Docs REFERENCES dbo.SourceDocuments (DocId),
    ChunkText      NVARCHAR(MAX) NOT NULL,
    Embedding      VECTOR(1536) NOT NULL,
    ModelId        NVARCHAR(100) NOT NULL, -- Artifact 3: which model made this
    ModelDims      SMALLINT NOT NULL,
    EmbeddedAt     DATETIME2(0) NOT NULL
        CONSTRAINT DF_Chunks_Embedded DEFAULT SYSUTCDATETIME()
);
-- Artifact 3: the model registry (what each ModelId means)
CREATE TABLE dbo.EmbeddingModels (
    ModelId        NVARCHAR(100) NOT NULL
        CONSTRAINT PK_EmbeddingModels PRIMARY KEY,
    Provider       NVARCHAR(60) NOT NULL,
    ModelName      NVARCHAR(120) NOT NULL,
    Dimensions     SMALLINT NOT NULL,
    RetiredAt      DATETIME2(0) NULL
);
-- A backup that captures DocumentChunks but not SourceDocuments or
-- EmbeddingModels restores vectors you cannot regenerate or interpret.
-- Back up all three, or back up none.

```

- → **Source data** — the documents. Without these, embeddings cannot be regenerated.
- → **Embeddings** — the vector arrays plus model provenance (`ModelId` , `ModelDims`). Without provenance, restored vectors are uninterpretable numbers.
- → **Model registry** — which model version produced each vector. Without it, re-embedding silently mixes geometries (Chapter 30).

Backup Strategies for SQL Server Vector Tables

SQL Server treats a table with a `VECTOR` column like any other table for backup purposes. The difference is scale: 10 million document chunks at 1536 dimensions (4 bytes per dimension in the native vector type) hold roughly **58 GB** of vector data before indexes. The DiskANN index adds further storage for the navigational graph. Plan backup windows and storage accordingly — and test restore throughput, not just backup throughput.

```
-- SQL Server: tiered backup schedule for the vector database
-- Weekly FULL (Sunday 02:00)
BACKUP DATABASE VectorRagDb
TO URL = 'https://vecbackups.blob.core.windows.net/sqlbackups/full.bak'
WITH COMPRESSION, CHECKSUM, STATS = 10;
-- Daily DIFFERENTIAL (02:00) – vector tables change mostly by INSERT,
-- so differentials stay small relative to the base
BACKUP DATABASE VectorRagDb
TO URL = 'https://vecbackups.blob.core.windows.net/sqlbackups/diff.bak'
WITH DIFFERENTIAL, COMPRESSION, CHECKSUM;
-- LOG every 15 minutes – keeps RPO tight for the source + chunk tables
BACKUP LOG VectorRagDb
TO URL = 'https://vecbackups.blob.core.windows.net/sqlbackups/log.trn'
WITH COMPRESSION, CHECKSUM;
-- Native backups capture the DiskANN index *definition*; the index
-- contents rebuild from vector data during recovery. Budget the rebuild
-- into RTO (Chapter 35) – it is not part of the restore timer.
```

For very large vector tables, add a bulk-export tier: periodically export `DocId`, `ChunkId`, `Embedding` to Parquet in object storage via `bcp` or Azure Data Factory. The export is platform-independent — it rebuilds the vector tables on any SQL Server instance without replaying the backup chain, and it doubles as the seed for a second vector platform if you ever migrate.

```
-- SQL Server Agent job: weekly vector export to Parquet-adjacent CSV
-- (bcp native format preserves the VECTOR column; CSV via a view that
-- renders the vector as text for portability)
CREATE VIEW dbo.vw_ChunkExport AS
SELECT ChunkId, DocId, ModelId, ModelDims,
       CAST(Embedding AS NVARCHAR(MAX)) AS EmbeddingText, EmbeddedAt
FROM dbo.DocumentChunks;
GO
-- bcp "SELECT * FROM VectorRagDb.dbo.vw_ChunkExport" queryout
--     "\\backupshare\vectors\chunks_20260927.csv" -S . -T -c -t "|"
-- Schedule weekly; verify row counts match:
-- SELECT COUNT(*) FROM dbo.DocumentChunks; -- compare to exported rows
```

Exporting from Managed Vector Databases

If part of your estate lives in Pinecone, Weaviate, or Qdrant Cloud, their export story is an API, not a backup file. Schedule weekly full exports of (id, vector, metadata) to object storage you control — the export is your only defense against a managed-service incident, a billing dispute that locks the account, or a migration you didn't plan.

The operational rule: **keep authoritative source data in SQL Server or object storage you control**, so a managed export failure degrades into a re-embedding event rather than data loss. The managed store is a cache with an SLA; the source of truth is yours.

```
-- SQL Server: track managed-service exports alongside native backups
CREATE TABLE dbo.BackupAudit (
    BackupId      BIGINT IDENTITY(1,1) NOT NULL
        CONSTRAINT PK_BackupAudit PRIMARY KEY,
    BackupType    NVARCHAR(20) NOT NULL,    -- 'full','diff','log','export'
    TargetSystem  NVARCHAR(50) NOT NULL,    -- 'sqlserver','pinecone','weaviate'
    StartedAt     DATETIME2(0) NOT NULL,
    FinishedAt    DATETIME2(0) NULL,
    RowCount      BIGINT NULL,
    Location      NVARCHAR(500) NOT NULL,   -- URL or path
    ChecksumOk    BIT NULL
);
-- One audit table for every backup and export across every system.
-- During an incident, "what was the consistent state of our entire
-- data layer at 03:00 on June 15?" is answered here – not by
-- reconstructing it from three different consoles.
```

Model Versioning as a Backup Dependency

A backup of embeddings without the model registry is a backup of uninterpretable numbers. The `dbo.EmbeddingModels` table from Section 36.1 must be backed up on the same schedule as the vector tables — it lives in the same database, so native backups already capture it. The failure mode this prevents: restoring vectors, then re-embedding new documents with a *different* model version because nobody recorded which version produced the restored ones. Distances between old and new vectors become meaningless, and retrieval quality collapses without a single error in the log.

- → Pin the model version in the pipeline configuration, and store the pin in the database — not just in a deployment script.
- → On restore, verify every `ModelId` in `DocumentChunks` has a row in `EmbeddingModels` before serving traffic.

- → Retired models stay in the registry forever — a retired row is documentation, not dead weight (Chapter 30).

Recovery Procedures and RTO Planning

Recovery has three phases, each with its own timer:

- → **Restore** — replay the backup chain to the target point in time. Standard SQL Server restore sequence: full, latest differential, log chain with `NORECOVERY` until the last log.
- → **Index rebuild** — the DiskANN index reconstructs from the restored vector data. This is the hidden RTO multiplier (Chapter 35): hours at tens of millions of rows.
- → **Catch-up** — re-embed documents newer than the restore point from the watermark, then run the recall canary before returning traffic.

```
-- SQL Server: point-in-time restore sequence for the vector database
-- (run on the recovery instance)
RESTORE DATABASE VectorRagDb
FROM URL = 'https://vecbackups.blob.core.windows.net/sqlbackups/full.bak'
WITH NORECOVERY, REPLACE, STATS = 10;
RESTORE DATABASE VectorRagDb
FROM URL = 'https://vecbackups.blob.core.windows.net/sqlbackups/diff.bak'
WITH NORECOVERY;
RESTORE LOG VectorRagDb
FROM URL = 'https://vecbackups.blob.core.windows.net/sqlbackups/log.trn'
WITH RECOVERY, STOPAT = '2026-09-26T14:30:00';
-- Then: allow the DiskANN index to rebuild (monitor via error log /
-- extended events), verify model registry completeness, re-embed the
-- post-STOPAT delta from the watermark, run the recall canary.
```

Backup Schedules and Retention Policies

- → **Full weekly, differential daily, log every 15 minutes.** Vector tables are insert-heavy, so differentials stay proportionally small — but verify this monthly; a re-embedding backfill turns the differential into a near-full.
- → **Retain 4 weekly fulls + 30 days of logs** as the baseline; extend for regulated data (the vector store inherits the source corpus's retention policy — Chapter 34).
- → **Export weekly to object storage** independently of native backups — different failure domain, different restore path.
- → **Test-restore monthly** to an isolated instance and record all three phase timers. A backup you've never restored is a backup you don't have.

Disaster Recovery Runbook

```
## SQL Server Vector Database – Full Restore Runbook
### Prerequisites
- Recovery SQL Server 2025 instance online (no user databases)
- Backup URLs + SAS/credentials in the vault
- Pinned embedding model version + API access confirmed
- Recall canary script ready (Chapter 20)

### Step 1: Restore the backup chain (T-SQL from previous section)
### Step 2: Verify the three artifacts
-- All ModelIds resolve:
SELECT DISTINCT c.ModelId
FROM dbo.DocumentChunks AS c
LEFT JOIN dbo.EmbeddingModels AS m ON m.ModelId = c.ModelId
WHERE m.ModelId IS NULL; -- must return zero rows
-- Source coverage: every chunk has a source document
SELECT COUNT(*) FROM dbo.DocumentChunks AS c
WHERE NOT EXISTS (SELECT 1 FROM dbo.SourceDocuments AS s
                  WHERE s.DocId = c.DocId); -- investigate any hits

### Step 3: Wait for / trigger the DiskANN rebuild; time it
### Step 4: Re-embed the post-restore delta from the watermark
### Step 5: Run the recall canary – within 2 points of baseline
### Step 6: Return traffic; log all phase durations to dbo.BackupAudit
```

Key Takeaways

- → **Back up three artifacts, not one:** source documents, embeddings with model provenance, and the model registry — restore any two and you have a silent corruption incident.
- → **Native SQL Server backups work unchanged** on vector tables — but the DiskANN index rebuilds from data during recovery, so budget the rebuild into RTO.
- → **Export weekly to object storage** as a platform-independent tier — different failure domain, different restore path, migration seed.
- → **Managed-service exports are the minimum viable posture** where you can't run native backups — and the authoritative source stays in systems you control.
- → **Audit every backup and export in one table** so incident response can answer "what was our consistent state at 03:00?" in one query.
- → **Test-restore monthly and time all three phases** — restore, index rebuild, catch-up. Untested backups are hopes, not plans.

Chapter 37: Cost Optimization: Dimensionality Reduction and Quantization

Storage and query costs in vector databases grow linearly with two variables: the number of vectors and the size of each vector. A production RAG system ingesting documents at scale can easily accumulate tens of millions of 1536-dimensional float32 vectors — roughly 90 GB of raw embedding data before any indexing overhead. This chapter covers the two primary techniques for controlling that growth without sacrificing retrieval quality: dimensionality reduction, which shrinks the number of dimensions per vector, and quantization, which shrinks the number of bits per dimension. Both approaches trade a small, measurable amount of recall for significant savings in storage, memory, and query latency.

Understanding exactly where those tradeoffs land — and how to measure them — is the engineering work this chapter addresses.

The Cost Structure of Vector Storage

Before optimizing anything, understand what you're paying for. Vector databases carry costs along four axes that compound each other.

Raw storage is the floor. A float32 value occupies 4 bytes. A 1536-dimensional OpenAI `text-embedding-3-small` vector is 6,144 bytes (6 KB) per document. At 10 million documents, that's 58 GB before indexes. Switch to `text-embedding-3-large` at 3072 dimensions and you're at 117 GB. Pinecone, Weaviate, and Qdrant all charge for storage either directly or through node sizing, so this matters immediately.

Index memory multiplies the base storage. HNSW indexes — the most common structure across all major vector databases — maintain neighbor graphs that consume roughly 1.5× to 4× the raw vector memory depending on the `M` parameter (graph connectivity). A 58 GB float32 dataset can easily require 200+ GB of RAM to serve with HNSW at reasonable recall.

Query throughput cost ties to memory pressure. When working sets don't fit in RAM, queries hit disk, and latency spikes from single-digit milliseconds to hundreds. Cloud providers size instances to fit vector indexes in memory, so every unnecessary byte in your vectors translates directly to a larger, more expensive instance.

Embedding API costs sit upstream. OpenAI charges per token for embedding generation. If you're re-embedding during pipeline changes or re-indexing after dimension changes, those API calls add up fast.

The table below shows approximate storage requirements for common embedding models at 10 million vectors:

Model	Dimensions	float32 Storage	int8 Storage	Binary Storage
text-embedding-3-small	1536	58 GB	14.6 GB	1.8 GB
text-embedding-3-large	3072	117 GB	29.3 GB	3.7 GB
text-embedding-ada-002	1536	58 GB	14.6 GB	1.8 GB
all-MiniLM-L6-v2	384	14.6 GB	3.7 GB	0.46 GB
Cohere embed-english-v3.0	1024	39 GB	9.8 GB	1.2 GB

The binary column is not a typo. That's the endgame of quantization: 1 bit per dimension instead of 32.

Dimensionality Reduction: PCA, Matryoshka, and Native Truncation

Dimensionality reduction eliminates dimensions from vectors, either by training a projection (PCA) or by using embedding models designed to work well at reduced sizes.

PCA-based reduction is the classical approach. You collect a representative sample of your vectors, fit a PCA transformation matrix, and project all vectors into a lower-dimensional space that preserves maximum variance. The projection matrix must be stored and applied to every new query vector at search time — this is the runtime cost of PCA.

```

# Python
import numpy as np
from sklearn.decomposition import PCA
from sklearn.preprocessing import normalize
import pickle

def fit_pca_reducer(sample_vectors: np.ndarray, target_dims: int) -> PCA:
    """
    Fit a PCA reducer on a sample of your corpus vectors.
    sample_vectors: shape (n_samples, original_dims)
    target_dims: number of dimensions to keep
    """
    pca = PCA(n_components=target_dims, random_state=42)
    pca.fit(sample_vectors)

    explained = pca.explained_variance_ratio_.sum()
    print(f"Explained variance with {target_dims} dims: {explained:.3f}")

    return pca

def reduce_and_normalize(vectors: np.ndarray, pca: PCA) -> np.ndarray:
    """
    Project and L2-normalize. Normalization is required after PCA
    if you use cosine similarity for search.
    """
    reduced = pca.transform(vectors)
    return normalize(reduced, norm='l2')

# Workflow: sample 100k vectors from your corpus
# (representative spread – don't use only recent documents)
sample = np.load("corpus_sample_100k.npy") # shape (100000, 1536)

pca = fit_pca_reducer(sample, target_dims=512)

# Save projection matrix for query-time use
with open("pca_reducer_1536_to_512.pkl", "wb") as f:
    pickle.dump(pca, f)

# Process your full corpus in batches
def process_batch(raw_batch: np.ndarray, pca: PCA) -> np.ndarray:
    return reduce_and_normalize(raw_batch, pca)

# Check how many dimensions you actually need
for dims in [128, 256, 512, 768, 1024]:
    test_pca = PCA(n_components=dims, random_state=42)
    test_pca.fit(sample)
    variance = test_pca.explained_variance_ratio_.sum()
    print(f" {dims} dims → {variance:.4f} explained variance ({variance*100:.1f}
%)")

```

Running this on `text-embedding-3-small` vectors typically yields 95%+ explained variance at 512 dimensions. The problem is that explained variance doesn't directly map to recall — you need to measure recall separately, which means running benchmark queries before committing to a target dimension count.

Matryoshka Representation Learning (MRL) changes the game. OpenAI's newer models (`text-embedding-3-small` , `text-embedding-3-large`) and Cohere's v3 models are trained with MRL, meaning the first N dimensions of the full vector are themselves a valid, high-quality embedding. You can truncate rather than project. No projection matrix required, no transformation at query time, and the quality at reduced dimensions is better than PCA because the model was explicitly trained for it.

```

# Python
from openai import OpenAI
import numpy as np

client = OpenAI()

def embed_with_truncation(
    texts: list[str],
    model: str = "text-embedding-3-small",
    dimensions: int = 512  # native truncation – MRL models only
) -> np.ndarray:
    """
    OpenAI supports native dimension truncation via the `dimensions` parameter.
    Valid only for text-embedding-3-* models.
    """
    response = client.embeddings.create(
        model=model,
        input=texts,
        dimensions=dimensions  # API truncates and re-normalizes for you
    )
    return np.array([item.embedding for item in response.data])

# Compare quality across dimension counts
test_queries = [
    "database indexing strategies",
    "vector similarity search performance",
    "cost optimization for embeddings"
]

for dims in [256, 512, 768, 1536]:
    embeddings = embed_with_truncation(test_queries, dimensions=dims)
    print(f"dims={dims}: shape={embeddings.shape}, "
          f"norm={np.linalg.norm(embeddings[0]):.4f}")

# For text-embedding-3-large: max 3072, usable at 256, 512, 1024, 3072
# OpenAI's benchmarks show 512-dim 3-small outperforms 1536-dim ada-002
# on MTEB – smaller is sometimes better when MRL-trained

```

The operational advantage here is significant: you can change your target dimension count by updating the API parameter and re-indexing, with no separate transformation pipeline to maintain.

Choosing a target dimension is an empirical question. The practical process:

1. Generate embeddings at your target dimension and at full dimension for a held-out query set (minimum 500 queries with known relevant documents).
2. Run nearest-neighbor retrieval at both sizes with identical HNSW parameters.
3. Compute Recall@10 at both sizes.

4. Accept the reduction if recall drop is under 3–5% for your use case.

For most RAG applications using MRL-trained models, 512 dimensions achieves recall within 2% of 1536 while cutting storage and index memory by two thirds.

Scalar Quantization: float32 to int8

Quantization reduces the number of bits per dimension. Scalar quantization maps each float32 value to an integer within a fixed range — typically int8 (256 levels) or even uint8. The loss in precision is real but bounded, and the memory savings are immediate: 4 bytes becomes 1 byte per dimension, a 4× reduction.

The math is straightforward. For a vector where values span [-1.0, 1.0], each float32 is mapped to an int8 using:

```
quantized = round(float_value × 127)
dequantized = quantized / 127
```

Most vector databases handle this transparently. Qdrant's scalar quantization is production-grade and simple to configure:

```

# Python
from qdrant_client import QdrantClient
from qdrant_client.models import (
    VectorParams, Distance,
    ScalarQuantizationConfig, ScalarType,
    QuantizationSearchParams
)

client = QdrantClient(url="http://localhost:6333")

# Create collection with int8 scalar quantization
client.create_collection(
    collection_name="documents_sq",
    vectors_config=VectorParams(
        size=1536,
        distance=Distance.COSINE
    ),
    quantization_config=ScalarQuantizationConfig(
        type=ScalarType.INT8,
        quantile=0.99,      # clip outliers: 99th percentile maps to max int8
        always_ram=True     # keep quantized index in RAM, fetch originals from disk
    )
)

# always_ram=True is the key production setting:
# - Search uses quantized (1-byte) vectors for fast candidate selection
# - Top candidates are re-scored with original float32 vectors
# - Original vectors can live on disk (SSD), quantized in RAM
# This is "quantization with rescoring" – near full recall at fraction of memory

# Search with rescoring enabled (default when quantization is configured)
results = client.search(
    collection_name="documents_sq",
    query_vector=[0.1] * 1536,  # your actual query embedding here
    limit=10,
    search_params=QuantizationSearchParams(
        ignore=False,          # use quantization for initial search
        rescore=True,          # rescore top candidates with originals
        oversampling=2.0       # fetch 20 candidates, rescore to return 10
    )
)

# oversampling controls the recall/speed tradeoff:
# 1.0 → fastest, lowest recall
# 3.0 → slower, highest recall (approaches float32 quality)
# 2.0 is the practical default for most production use cases

print(f"Retrieved {len(results)} results")
for r in results:
    print(f"  id={r.id}, score={r.score:.4f}")

```

Quantization on SQL Server. The native `VECTOR` type in SQL Server 2025 stores float32 — there is no built-in float16 or binary-quantized column type. Your compression levers are (a) storing fewer dimensions via MRL truncation (Section 37.2), (b) quantizing at the application layer before insert, and (c) the two-phase pattern: a compact representation for candidate selection, full float32 for rescoring.

```
-- SQL Server 2025: two-phase pattern – compact candidates, exact rescore
-- Phase 1 storage: truncated 512-dim vectors in a smaller VECTOR column
CREATE TABLE dbo.DocumentsCompact (
    ChunkId    BIGINT IDENTITY(1,1) NOT NULL
        CONSTRAINT PK_DocumentsCompact PRIMARY KEY,
    ChunkText  NVARCHAR(MAX) NOT NULL,
    Emb512     VECTOR(512) NOT NULL    -- 512 x 4 bytes = 2 KB vs 6 KB for 1536
);
CREATE VECTOR INDEX IX_DocCompact_Emb512
    ON dbo.DocumentsCompact (Emb512)
    WITH (METRIC = 'cosine', TYPE = 'DiskANN', MAXDOP = 4);
-- Phase 2: full 1536-dim vectors for exact rescoring of the candidates
CREATE TABLE dbo.DocumentsFull (
    ChunkId    BIGINT NOT NULL
        CONSTRAINT PK_DocumentsFull PRIMARY KEY,
    EmbFull     VECTOR(1536) NOT NULL
);
GO
-- Two-phase search: DiskANN over compact vectors, exact rescore over full
DECLARE @q512  VECTOR(512) = '[0.1, ...]';    -- truncated query embedding
DECLARE @qfull VECTOR(1536) = '[0.1, ...]';    -- full query embedding
WITH CompactCandidates AS (
    SELECT TOP (100) ChunkId,
        VECTOR_DISTANCE('cosine', Emb512, @q512) AS approx_dist
    FROM dbo.DocumentsCompact
    ORDER BY approx_dist
)
SELECT TOP (10) c.ChunkId,
    VECTOR_DISTANCE('cosine', f.EmbFull, @qfull) AS exact_dist
FROM CompactCandidates AS c
JOIN dbo.DocumentsFull AS f ON f.ChunkId = c.ChunkId
ORDER BY exact_dist;
-- 3x storage reduction on the indexed column, recall within 1-3% of full
-- float32 search when validated on your eval set (Section 37.5).
```

The two-phase pattern — coarse binary search, exact float32 reranking — is the same pattern Qdrant calls "rescoring." It's the standard production approach for binary quantization and delivers recall within 1–3% of full float32 search at a fraction of the memory cost.

Product Quantization and Vector Database Native Support

Product quantization (PQ) goes further than scalar quantization by dividing each high-dimensional vector into subvectors and quantizing each subvector against a learned codebook. The result is a compact code — often 8–32 bytes for a 1536-dimensional vector — that approximates the original with reasonable accuracy.

PQ is more complex than scalar quantization: it requires a training phase to build codebooks, and the codebooks must be stored and used at query time. The payoff is compression ratios of 32× to 96× with recall that holds up better than naive binary quantization for unoptimized models.

Faiss, the underlying library in many vector databases, exposes PQ directly. Weaviate and Milvus expose it through their configuration APIs:

```

# Python – Milvus PQ configuration via pymilvus
from pymilvus import (
    connections, FieldSchema, CollectionSchema,
    DataType, Collection, utility
)

connections.connect(host="localhost", port="19530")

fields = [
    FieldSchema(name="id", dtype=DataType.INT64, is_primary=True, auto_id=True),
    FieldSchema(name="text", dtype=DataType.VARCHAR, max_length=4096),
    FieldSchema(name="embedding", dtype=DataType.FLOAT_VECTOR, dim=1536)
]

schema = CollectionSchema(fields=fields, description="PQ quantized documents")
collection = Collection(name="documents_pq", schema=schema)

# IVF_PQ index: inverted file + product quantization
# This is the standard high-compression option in Milvus/Faiss
index_params = {
    "metric_type": "IP",          # inner product (use normalized vectors for
    cosine)
    "index_type": "IVF_PQ",
    "params": {
        "nlist": 1024,          # number of IVF clusters (scale with
sqrt(n_vectors))
        "m": 16,                # number of subvectors (must divide dim evenly:
1536/16=96)
        "nbits": 8              # bits per subvector code (8 = 256 centroids per
subquantizer)
    }
}

# Memory per vector: m * (nbits/8) bytes = 16 * 1 = 16 bytes vs 6144 bytes float32
# Compression ratio: 384x

collection.create_index(field_name="embedding", index_params=index_params)
collection.load()

# Search – nprobe controls how many IVF clusters to check
# More clusters → higher recall, higher latency
search_params = {
    "metric_type": "IP",
    "params": {"nprobe": 64}    # check 64 of 1024 clusters; 6.25% of index
}

results = collection.search(
    data=[[0.1] * 1536],        # query embedding (normalized)
    anns_field="embedding",
    param=search_params,
    limit=10,
    output_fields=["text"]
)

```

```

for hits in results:
    for hit in hits:
        print(f"id={hit.id}, score={hit.score:.4f}, text={hit.entity.get('text')[:80]}")

# PQ recall benchmarking guide:
# nprobe=10 → ~85% recall@10, fastest
# nprobe=64 → ~93% recall@10, good default
# nprobe=256 → ~97% recall@10, slowest (approaching brute force)
# Tune nprobe to hit your latency SLA, then evaluate recall

```

Weaviate exposes its own PQ interface through its schema configuration, which is worth knowing if you're running Weaviate in production:

```

# Python – Weaviate PQ configuration via weaviate-client v4
import weaviate
from weaviate.classes.config import (
    Configure, Property, DataType, VectorDistances
)

client = weaviate.connect_to_local()

# Create collection with PQ enabled
client.collections.create(
    name="DocumentsPQ",
    vectorizer_config=Configure.Vectorizer.none(), # bring your own vectors
    vector_index_config=Configure.VectorIndex.hnsw(
        distance_metric=VectorDistances.COSINE,
        quantizer=Configure.VectorIndex.Quantizer.pq(
            training_limit=100_000, # vectors needed before PQ kicks in
            segments=96,           # 1536 dims / 96 segments = 16 dims per
segment
            centroids=256           # 2^8 centroids (8-bit codes)
        )
    ),
    properties=[
        Property(name="content", data_type=DataType.TEXT),
        Property(name="source", data_type=DataType.TEXT)
    ]
)

# Weaviate trains PQ automatically once training_limit vectors are loaded
# Before training_limit: stores float32 flat (temporary, high memory)
# After training_limit: compresses existing vectors + new vectors

# For production: pre-populate with training_limit vectors before going live
# Or set a lower training_limit if your corpus starts small

client.close()

```

The practical PQ tradeoff: you're trading recall and query-time CPU for memory. PQ requires decoding compact codes during distance computation, which is CPU-intensive. On modern hardware with SIMD support, Faiss makes this fast — but it's not free. Benchmark both latency and recall before committing.

Measuring the Tradeoffs: A Recall Benchmarking Framework

Every compression decision requires recall validation. "Recall@K" measures what fraction of the true top-K nearest neighbors you retrieve when using an approximate or compressed index. This is your primary metric.

```

# Python
import numpy as np
from typing import Callable
import time

def compute_recall_at_k(
    queries: np.ndarray,
    corpus: np.ndarray,
    search_fn: Callable,    # function: query_vector -> list of retrieved IDs
    k: int = 10,
    batch_size: int = 100
) -> dict:
    """
    Computes Recall@K by comparing approximate search results
    against brute-force exact neighbors.

    queries:      shape (n_queries, dims)
    corpus:       shape (n_corpus, dims) – the indexed vectors
    search_fn:    your vector DB search call, returns list of IDs
    """

    # Ground truth: exact brute-force neighbors
    # For large corpora, compute this once and cache it
    print("Computing ground truth (brute force)...")
    ground_truth = []

    # Batch to avoid OOM on large corpora
    for q_start in range(0, len(queries), batch_size):
        q_batch = queries[q_start:q_start + batch_size]

        # Cosine similarity via dot product (assumes normalized vectors)
        similarities = q_batch @ corpus.T          # (batch, n_corpus)
        top_k_indices = np.argsort(-similarities, axis=1)[:k]
        ground_truth.extend(top_k_indices.tolist())

    # Approximate search with the system under test
    print("Running approximate search...")
    retrieved = []
    latencies = []

    for i, query in enumerate(queries):
        start = time.perf_counter()
        result_ids = search_fn(query)
        elapsed = time.perf_counter() - start
        latencies.append(elapsed * 1000)    # ms
        retrieved.append(result_ids[:k])

    # Recall@K: what fraction of ground truth top-K did we find?
    recall_scores = []
    for true_ids, ret_ids in zip(ground_truth, retrieved):
        true_set = set(true_ids)
        ret_set = set(ret_ids)
        recall = len(true_set & ret_set) / len(true_set)

```



```

        recall_scores.append(recall)

    return {
        "recall_at_k": np.mean(recall_scores),
        "recall_p50": np.percentile(recall_scores, 50),
        "recall_p10": np.percentile(recall_scores, 10),    # worst queries
        "latency_p50_ms": np.percentile(latencies, 50),
        "latency_p95_ms": np.percentile(latencies, 95),
        "latency_p99_ms": np.percentile(latencies, 99),
        "k": k
    }

# Example: compare float32 vs int8 quantized search in Qdrant
from qdrant_client import QdrantClient
from qdrant_client.models import QuantizationSearchParams

qdrant = QdrantClient(url="http://localhost:6333")

def search_float32(query: np.ndarray) -> list:
    """Search without quantization."""
    results = qdrant.search(
        collection_name="documents_float32",
        query_vector=query.tolist(),
        limit=10, search_params=QuantizationSearchParams(ignore=True) # disable
quantization
    )
    return [r.id for r in results]

def search_int8_rescore(query: np.ndarray) -> list:
    """Search with int8 quantization + rescoreing."""
    results = qdrant.search(
        collection_name="documents_sq",
        query_vector=query.tolist(),
        limit=10,
        search_params=QuantizationSearchParams(
            ignore=False,
            rescore=True,
            oversampling=2.0
        )
    )
    return [r.id for r in results]

def search_int8_no_rescore(query: np.ndarray) -> list:
    """Search with int8 quantization only – maximum speed."""
    results = qdrant.search(
        collection_name="documents_sq",
        query_vector=query.tolist(),
        limit=10,
        search_params=QuantizationSearchParams(
            ignore=False,
            rescore=False
        )
    )

```

```

    return [r.id for r in results]

# Load your benchmark queries
queries = np.load("benchmark_queries.npy")          # shape (1000, 1536)
corpus  = np.load("corpus_vectors_sample.npy")      # shape (50000, 1536) — same IDs
as DB

configs = {
    "float32_exact":      search_float32,
    "int8_with_rescore":  search_int8_rescore,
    "int8_no_rescore":    search_int8_no_rescore,
}

results = {}
for name, fn in configs.items():
    print(f"\nBenchmarking: {name}")
    results[name] = compute_recall_at_k(queries, corpus, fn, k=10)

# Print comparison table
print(f"\n{'Config':<25} {'Recall@10':>10} {'P10 Recall':>10} {'P50 Lat ms':>12}
{'P95 Lat ms':>12}")
print("-" * 75)
for name, metrics in results.items():
    print(f"{name:<25} {metrics['recall_at_k']:>10.4f} "
          f"{metrics['recall_p10']:>10.4f} "
          f"{metrics['latency_p50_ms']:>12.2f} "
          f"{metrics['latency_p95_ms']:>12.2f}")

```

The P10 recall column — the 10th percentile recall across your query set — is more informative than the mean. A mean recall of 0.95 can hide specific query categories that consistently miss. Those outliers often correspond to semantically sparse topics, short queries, or domain-specific vocabulary your embedding model handles poorly. Investigate them.

What to do when recall is too low:

- → Increase `oversampling` (Qdrant) or `nprobe` (Milvus/Faiss) — the fastest fix
- → Increase `ef` at query time on HNSW indexes — trades latency for recall
- → Switch from binary to int8 quantization — better recall at more memory cost
- → Use a higher-quality embedding model — more information preserved at same dimensions
- → Increase target dimensions if using MRL truncation — recall usually improves monotonically

Combining Both Techniques: Dimension Reduction + Quantization

The highest compression ratios come from stacking both techniques: reduce dimensions first, then quantize. The order matters — quantize the reduced vectors, not the original ones.

```
graph LR
    A["Full vector  
1536-dim float32  
6144 bytes"] --> B["MRL truncation  
512-dim float32  
2048 bytes"]
    B --> C["int8 quantization  
512-dim int8  
512 bytes (RAM)"]
    B -.kept on disk for rescoring.-> D["512-dim float32  
on disk"]
    C --> E["ANN candidate search"]
    D --> F["Rescore top candidates  
with full precision"]
    E --> F
    F --> G["Final top-k results"]
    style C fill:#c8e6c9
```

Dimension reduction happens first, then quantization compresses the already-reduced vector — reducing the original would discard information the truncation already preserved.

```

# Python – Combined pipeline: MRL truncation + int8 in Qdrant

from openai import OpenAI
from qdrant_client import QdrantClient
from qdrant_client.models import (
    VectorParams, Distance,
    ScalarQuantizationConfig, ScalarType,
    QuantizationSearchParams, PointStruct
)
import numpy as np
import uuid

openai_client = OpenAI()
qdrant = QdrantClient(url="http://localhost:6333")

TARGET_DIMS = 512    # MRL truncation: 1536 → 512 (3.3× smaller)
# Combined with int8: 512 × 1 byte = 512 bytes vs 1536 × 4 = 6144 bytes → 12×
# smaller

# Create the collection: 512-dim vectors with int8 quantization
qdrant.recreate_collection(
    collection_name="documents_optimized",
    vectors_config=VectorParams(
        size=TARGET_DIMS,
        distance=Distance.COSINE
    ),
    quantization_config=ScalarQuantizationConfig(
        type=ScalarType.INT8,
        quantile=0.99,
        always_ram=True
    )
)

def embed_documents(texts: list[str]) -> np.ndarray:
    """Embed at target dimensions using MRL truncation."""
    response = openai_client.embeddings.create(
        model="text-embedding-3-small",
        input=texts,
        dimensions=TARGET_DIMS
    )
    return np.array([item.embedding for item in response.data])

def ingest_batch(texts: list[str], ids: list[str] = None):
    """Embed and upsert a batch of documents."""
    embeddings = embed_documents(texts)

    if ids is None:
        ids = [str(uuid.uuid4()) for _ in texts]

    points = [
        PointStruct(
            id=id_,
            vector=embedding.tolist(),

```

```

        payload={"text": text}
    )
    for id_, text, embedding in zip(ids, texts, embeddings)
]

qdrant.upsert(collection_name="documents_optimized", points=points)
return ids

def search_optimized(query_text: str, top_k: int = 10) -> list[dict]:
    """
    Embed query at same truncated dimension, search with rescoring.
    """
    query_embedding = embed_documents([query_text])[0]

    results = qdrant.search(
        collection_name="documents_optimized",
        query_vector=query_embedding.tolist(),
        limit=top_k,
        search_params=QuantizationSearchParams(
            rescore=True,
            oversampling=2.0
        ),
        with_payload=True
    )

    return [
        {"id": r.id, "score": r.score, "text": r.payload.get("text", "")}
        for r in results
    ]

# Storage math for this configuration:
# Original: 1536 dims × 4 bytes = 6,144 bytes per vector
# Optimized: 512 dims × 1 byte = 512 bytes per vector (quantized in RAM)
# + 512 dims × 4 bytes = 2,048 bytes on disk for rescoring originals
# RAM savings: 12× for the hot index; disk holds originals at 3×

```

A practical comparison across compression strategies for 10 million vectors:

Strategy	RAM (index only)	Disk	Recall@10	Query Latency
float32, 1536 dims	~200 GB	58 GB	0.99	baseline
float32, 512 dims (MRL)	~67 GB	19 GB	0.97	faster
int8, 1536 dims + rescore	~50 GB	58 GB	0.97	similar
int8, 512 dims + rescore	~17 GB	19 GB	0.95	faster
binary, 1536 dims + rescore	~6 GB	58 GB	0.93	fastest search

| binary, 512 dims + rescore | ~2 GB | 19 GB | 0.90 | fastest overall |

The int8 + 512 dims row represents the practical optimum for most production RAG systems: it cuts RAM by more than 10× versus full float32 HNSW, preserves 95% recall with rescoring, and keeps query latency low. Binary quantization pushes further but requires more careful recall validation — acceptable for some use cases, not for others.

SQL Server Optimization Workflow

For teams running vector search inside SQL Server 2025 via native `VECTOR` and DiskANN, the optimization path uses the same principles as managed vector databases but expressed in T-SQL and DMVs.

```

-- SQL Server 2025 – full optimization workflow

-- Step 1: Baseline – full float32, DiskANN
CREATE TABLE dbo.Documents (
    DocId      INT IDENTITY(1,1) NOT NULL
        CONSTRAINT PK_Documents PRIMARY KEY,
    Content    NVARCHAR(MAX) NOT NULL,
    Source     NVARCHAR(200) NOT NULL,
    Embedding  VECTOR(1536) NOT NULL
);
CREATE VECTOR INDEX IX_Documents_Embedding
    ON dbo.Documents (Embedding)
    WITH (METRIC = 'cosine', TYPE = 'DiskANN', MAXDOP = 4);

-- Step 2: Add a truncated-dimension column for 3x storage savings
-- (populate from the application with MRL-truncated 512-dim embeddings)
ALTER TABLE dbo.Documents ADD Embedding512 VECTOR(512) NULL;
GO
-- Backfill in batches in production (UPDATE TOP ... in a loop);
-- single statement shown for clarity:
-- UPDATE dbo.Documents SET Embedding512 = ;
CREATE VECTOR INDEX IX_Documents_Embedding512
    ON dbo.Documents (Embedding512)
    WITH (METRIC = 'cosine', TYPE = 'DiskANN', MAXDOP = 4);
-- Storage: 512 x 4 bytes = 2 KB vs 1536 x 4 = 6 KB per row -> 3x reduction

-- Step 3: Validate recall parity between full and truncated columns
DECLARE @q VECTOR(1536) = '[0.1, ...]';
DECLARE @q512 VECTOR(512) = '[0.1, ...]';
WITH FullResults AS (
    SELECT TOP (20) DocId,
        VECTOR_DISTANCE('cosine', Embedding, @q) AS dist_full
    FROM dbo.Documents ORDER BY dist_full
),
HalfResults AS (
    SELECT TOP (20) DocId,
        VECTOR_DISTANCE('cosine', Embedding512, @q512) AS dist_half
    FROM dbo.Documents ORDER BY dist_half
)
SELECT COALESCE(f.DocId, h.DocId) AS DocId,
    f.dist_full, h.dist_half,
    CASE WHEN f.DocId IS NULL THEN 'only in truncated'
        WHEN h.DocId IS NULL THEN 'only in full'
        ELSE 'in both' END AS status
FROM FullResults AS f
FULL OUTER JOIN HalfResults AS h ON h.DocId = f.DocId
ORDER BY COALESCE(f.dist_full, h.dist_half);

-- Step 4: Measure index sizes via DMVs
SELECT i.name AS IndexName,
    SUM(s.used_page_count) * 8 / 1024 AS IndexMB
FROM sys.indexes AS i
JOIN sys.dm_db_partition_stats AS s

```

```

    ON s.object_id = i.object_id AND s.index_id = i.index_id
WHERE i.object_id = OBJECT_ID(N'dbo.Documents')
GROUP BY i.name
ORDER BY IndexMB DESC;

-- Step 5: Tune query-time recall via TOP_N over-fetch
-- DiskANN has no session-level ef_search equivalent; control the
-- recall/latency tradeoff by over-fetching candidates and rescoreing:
DECLARE @q2 VECTOR(512) = '[0.1, ...]';
SELECT TOP (10) DocId, exact_dist
FROM (
    SELECT TOP (100) DocId,                                -- over-fetch 10x
           VECTOR_DISTANCE('cosine', Embedding512, @q2) AS exact_dist
    FROM dbo.Documents
) AS candidates
ORDER BY exact_dist;
-- Increase the inner TOP_N for better recall, decrease for lower latency.

```

SQL Server 2025 includes a production-grade vector story natively: the `VECTOR` type, `VECTOR_DISTANCE`, and DiskANN indexes ship in the engine. For teams standardized on SQL Server, that means vector search lives in the same database as the operational data — one backup story, one security model, one monitoring stack — with Azure AI Search or a sidecar vector database (Qdrant, Weaviate) as the option when you need quantization or scale the in-engine index doesn't cover.

Operational Checklist: Before You Compress

Compression decisions are difficult to reverse once data is in production. Run through this checklist before changing your vector storage strategy:

1. Benchmark with your actual queries, not synthetic ones.

Random query vectors don't represent the distribution of queries your users send. Use a sample of real production queries or, at minimum, queries from your domain.

2. Establish a recall floor, not just a recall target.

Decide in advance: what is the minimum acceptable Recall@10 for your application? If your RAG system fails to retrieve the right context, downstream LLM quality degrades. Set the floor before seeing the numbers, or you'll be tempted to rationalize whatever the benchmark shows.

3. Test re-indexing time.

Changing dimensions or quantization type requires re-embedding and re-indexing. For 10 million vectors, this can take hours. Know your re-indexing window before committing to a strategy that might need to change.

4. Keep originals if you use binary quantization.

Binary quantization without rescoring loses too much recall for most applications. Rescoring requires access to original float32 vectors. If you compress and discard originals, you lose the ability to rescore.

5. Stage changes through a shadow index.

Build the new compressed index in parallel, run your recall benchmark against both indexes simultaneously, and cut over only after validating recall. Never optimize directly on the production index.

6. Monitor recall in production, not just at launch.

Query distributions shift over time. A compression strategy that meets your recall floor at launch may degrade as users ask questions about topics not well-represented in your corpus sample. Build a production recall monitor using human-labeled query-document pairs sampled weekly.

Key Takeaways

- → **Storage and memory costs scale with both vector count and vector size**: the two primary levers for controlling cost are dimensionality reduction (fewer dimensions) and quantization (fewer bits per dimension), and they can be stacked for compounding savings.
- → **MRL-trained models (OpenAI text-embedding-3-*, Cohere v3) enable native truncation**: you can reduce from 1536 to 512 dimensions by setting a single API parameter, with better quality than PCA projection because the model was explicitly trained for it.
- → **Scalar quantization (int8) with rescoring is the lowest-risk compression step**: it delivers 4× memory reduction with recall loss typically under 3%, using the original float32 vectors stored on disk for rescoring top candidates identified by the compressed index.
- → **The practical production optimum for most RAG systems is 512-dim MRL + int8 quantization**: this combination achieves roughly 12× memory reduction versus full float32 HNSW while preserving approximately 95% Recall@10, and it is straightforward to implement with SQL Server DiskANN (MRL-truncated columns) and Qdrant (native quantization).

- → **Recall benchmarking must use real queries and must track the P10 recall, not just the mean**: a high average recall can hide systematically poor retrieval on specific query categories, and those failures are the ones that degrade downstream LLM response quality in production.

Chapter 38: Edge Deployment of Vector Search

The push to run vector search closer to the data source—on mobile devices, embedded hardware, IoT gateways, retail point-of-sale systems, and CDN edge nodes—represents the next frontier for AI-enabled applications. Latency budgets that tolerate 200ms round trips to a central cloud region become unacceptable when you're building real-time product recommendations in a store with spotty connectivity, or semantic search on a drone with no uplink at all. This chapter examines the architectural patterns, tooling choices, and operational tradeoffs of deploying vector search at the edge: what "edge" actually means in this context, which vector engines survive the constraints of limited RAM and CPU, how to synchronize edge indexes with a central store, and how to build applications that degrade gracefully when the edge goes offline.

What "Edge" Means for Vector Search

Edge is not a single topology. The term covers at least four distinct deployment tiers, each with different resource profiles and connectivity expectations:

Tier 1: Browser and mobile client. WebAssembly runtimes, React Native apps, and native iOS/Android applications. RAM is 256MB to 2GB usable, storage is flash, and the device may have no network connection for extended periods. The vector index must be embedded in the application bundle or downloaded on first run.

Tier 2: Edge server / CDN PoP. Cloudflare Workers, AWS Lambda@Edge, Fastly Compute@Edge. Memory is severely constrained (128MB to 512MB per worker isolate), and you cannot run a persistent process. Vector search here means either a very small in-memory index loaded at cold start, or a sidecar service running alongside the worker runtime.

Tier 3: On-premise gateway / industrial edge. A Raspberry Pi 4, NVIDIA Jetson, or Intel NUC sitting in a retail store, factory floor, or hospital wing. These have 4–16GB RAM, local NVMe storage, and intermittent WAN connectivity. This is the most capable edge tier and can run a full vector database instance.

Tier 4: Near-edge / regional. A small VM or bare-metal server in a regional co-location facility, 10–30ms from end users rather than 80–120ms to a central cloud region. Full database capabilities, but still subject to the deployment patterns that make edge useful.

The database analogy is familiar: this mirrors the read-replica topology DBAs have used for decades. The central write master holds the source of truth; edge replicas serve reads locally. Vector databases add one wrinkle—the index is not just a data structure but an expensive artifact that takes time and compute to build. You cannot simply stream WAL records and reconstruct an HNSW graph incrementally the way you replay row changes into a B-tree.

| Tier | RAM | Storage | Connectivity | Can run |

|---|---|---|---|---|

| Browser/mobile | 256MB–2GB | Flash | Intermittent | SQLite-vec, Chroma (tiny) |

| CDN edge worker | 128–512MB | None (ephemeral) | Always-on | Static index shard |

| On-premise gateway | 4–16GB | NVMe | Intermittent WAN | Qdrant, SQLite-vec, SQL Server Express |

| Regional near-edge | 32–256GB | NVMe/SAN | Reliable | Full Qdrant, Weaviate, SQL Server Standard |

Understanding which tier you're targeting determines almost every subsequent decision: which engine to use, how to handle index updates, and how to write application code that degrades gracefully.

Choosing a Vector Engine for Constrained Environments

The dominant cloud vector databases—Pinecone, Weaviate Cloud, Qdrant Cloud—are designed for servers with abundant RAM and assume reliable network access to their control planes. Deploying them at Tier 1 or Tier 2 is not viable. Even Tier 3 requires careful selection.

SQLite-vec is the most portable option. It is a SQLite extension written in C that adds a `vec0` virtual table implementing flat KNN search and a quantized IVF-style index. The entire extension is a single shared library under 200KB. Because SQLite runs in-process, there is no network hop, no daemon to manage, and no port to expose. It is the right choice for mobile and browser-wasm deployments.

```

# Python
# SQLite-vec: embedded vector search for mobile/edge Python applications
# pip install sqlite-vec

import sqlite3
import sqlite_vec
import numpy as np
from sentence_transformers import SentenceTransformer

def build_local_index(documents: list[dict], db_path: str = "edge_search.db"):
    """
    Build a local vector index using SQLite-vec.
    Appropriate for Tier 3 edge gateways or bundled desktop applications.
    documents: list of {"id": str, "text": str, "metadata": dict}
    """
    model = SentenceTransformer("all-MiniLM-L6-v2") # 80MB, runs CPU-only
    embeddings = model.encode([d["text"] for d in documents],
normalize_embeddings=True)
    dim = embeddings.shape[1] # 384 for MiniLM

    conn = sqlite3.connect(db_path)
    sqlite_vec.load(conn) # register the extension

    conn.execute("DROP TABLE IF EXISTS vec_items")
    conn.execute("DROP TABLE IF EXISTS item_metadata")

    # Create the virtual vector table
    conn.execute(f"""
        CREATE VIRTUAL TABLE vec_items USING vec0(
            item_id INTEGER PRIMARY KEY,
            embedding FLOAT[{dim}]
        )
    """)

    # Store metadata in a regular SQLite table (join at query time)
    conn.execute("""
        CREATE TABLE item_metadata (
            item_id INTEGER PRIMARY KEY,
            doc_id TEXT,
            text TEXT,
            payload TEXT -- JSON blob for arbitrary metadata
        )
    """)

    import json
    for i, (doc, emb) in enumerate(zip(documents, embeddings)):
        conn.execute(
            "INSERT INTO vec_items(item_id, embedding) VALUES (?, ?)",
            [i, emb.astype(np.float32).tobytes()]
        )
        conn.execute(
            "INSERT INTO item_metadata(item_id, doc_id, text, payload) VALUES
(?, ?, ?, ?)",

```

```

        [i, doc["id"], doc["text"], json.dumps(doc.get("metadata", {}))]
    )

    conn.commit()
    print(f"Indexed {len(documents)} documents, dim={dim}, db_path={db_path}")
    print(f"DB size: {os.path.getsize(db_path) / 1024 / 1024:.1f} MB")
    conn.close()

def query_local_index(query_text: str, top_k: int = 5, db_path: str =
"edge_search.db"):
    """
    Query the local SQLite-vec index. No network required.
    Typical latency: 2-15ms for 50K vectors on Raspberry Pi 4.
    """
    model = SentenceTransformer("all-MiniLM-L6-v2")
    query_vec = model.encode([query_text], normalize_embeddings=True)[0]

    conn = sqlite3.connect(db_path)
    sqlite_vec.load(conn)

    results = conn.execute("""
        SELECT
            m.doc_id,
            m.text,
            m.payload,
            v.distance
        FROM vec_items v
        JOIN item_metadata m ON m.item_id = v.item_id
        WHERE v.embedding MATCH ?
            AND k = ?
        ORDER BY v.distance
    """, [query_vec.astype(np.float32).tobytes(), top_k]).fetchall()

    conn.close()
    return [
        {"doc_id": r[0], "text": r[1], "metadata": r[2], "distance": r[3]}
        for r in results
    ]

import os

# Example usage
if __name__ == "__main__":
    sample_docs = [
        {"id": "p001", "text": "Wireless noise-canceling headphones with 30-hour
battery", "metadata": {"category": "electronics", "price": 299}},
        {"id": "p002", "text": "Running shoes with carbon fiber plate for marathon
training", "metadata": {"category": "footwear", "price": 180}},
        {"id": "p003", "text": "Stainless steel French press coffee maker 1 liter",
"metadata": {"category": "kitchen", "price": 45}},
    ]
    build_local_index(sample_docs)
    hits = query_local_index("headphones for long flights")

```

```
for h in hits:
    print(f"{h['doc_id']} | score={h['distance']:.4f} | {h['text'][:60]}")
```

Qdrant ships as a single statically linked binary (~50MB) and is the best choice for Tier 3 on-premise gateways. It supports HNSW indexing, scalar quantization (SQ8), and binary quantization—all of which reduce the RAM footprint dramatically. A collection of 500K 1536-dimensional vectors (OpenAI ada-002 size) uses approximately 3GB uncompressed but drops to under 800MB with binary quantization, making it viable on a device with 4GB total RAM.

```

# shell
# Deploy Qdrant on a Raspberry Pi 4 / Jetson / x86 mini-PC
# Using the static binary release (no Docker required on constrained hardware)

# Download the ARM64 binary (replace with x86_64 for Intel NUC)
QDRANT_VERSION="v1.9.2"
ARCH="aarch64-unknown-linux-musl"
wget -q "https://github.com/qdrant/qdrant/releases/download/${QDRANT_VERSION}/
qdrant-${ARCH}.tar.gz"
tar -xzf "qdrant-${ARCH}.tar.gz"
chmod +x qdrant

# Minimal config for edge deployment – conservative memory usage
cat > config/config.yaml << 'EOF'
storage:
  storage_path: /data/qdrant/storage
  # Limit WAL to prevent filling flash storage
  wal:
    wal_capacity_mb: 32
    wal_segments_ahead: 0

service:
  host: 0.0.0.0
  http_port: 6333
  grpc_port: 6334
  # Disable telemetry on edge nodes
  enable_tls: false

# Critical for RAM-constrained devices
storage:
  mmap_threshold_kb: 1024      # Memory-map segments > 1MB (reads from flash, not
RAM)
  quantization:
    always_ram: false          # Allow quantized vectors to be mmap'd too

telemetry_disabled: true

# Reduce background work that spikes CPU/IO on edge
optimizer:
  default_segment_number: 2
  max_segment_size_kb: 102400 # 100MB max segment size
  memmap_threshold_kb: 20000
  indexing_threshold_kb: 10000
  flush_interval_sec: 30
  max_optimization_threads: 1 # Single core optimization on edge
EOF

# Run as a systemd service for automatic restart
cat > /etc/systemd/system/qdrant-edge.service << 'EOF'
[Unit]
Description=Qdrant Vector Search Edge Node
After=network.target

```



```

[Service]
ExecStart=/usr/local/bin/qdrant --config-path /etc/qdrant/config.yaml
Restart=on-failure
RestartSec=5
LimitNOFILE=65536
# Constrain RAM – OOM-kill before thrashing swap on flash storage
MemoryMax=2G
MemoryHigh=1800M

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
systemctl enable --now qdrant-edge
systemctl status qdrant-edge

# Create a collection with binary quantization for minimum RAM usage
curl -X PUT http://localhost:6333/collections/products \
  -H "Content-Type: application/json" \
  -d '{
    "vectors": {
      "size": 384,
      "distance": "Cosine"
    },
    "quantization_config": {
      "binary": {
        "always_ram": false
      }
    },
    "hnsw_config": {
      "m": 8,
      "ef_construct": 100,
      "full_scan_threshold": 5000,
      "on_disk": true
    },
    "optimizers_config": {
      "memmap_threshold": 10000
    }
  }'

echo "Qdrant edge node ready. Check RAM: $(free -h | awk '/^Mem/{print $3}')"

```

SQL Server Express on a gateway box is viable for Tier 3 when you already run SQL Server in your application stack (common in retail and healthcare edge deployments). SQL Server 2025 Express supports the native `VECTOR` type, `VECTOR_DISTANCE`, and `DiskANN` indexes — you get full T-SQL expressivity and can join vector search with transactional data in the same engine, within Express's resource limits. The tradeoff is a larger baseline footprint than Qdrant, but a single engine for everything.

For Tier 1 (browser/mobile), the realistic options narrow further: **Transformers.js** (WebAssembly port of ONNX Runtime) handles embedding generation in-browser, and **SQLite-vec compiled to WASM** handles the search. Both fit in a browser tab with no server round-trip.

Index Synchronization: Keeping Edge in Sync with Central

The hardest operational problem in edge vector deployment is not the initial data load—it's keeping edge indexes current as the central store evolves. This is the vector equivalent of replication lag, but with a fundamentally different character: you cannot apply vector index changes as a log of row mutations. An HNSW graph rebuild is not decomposable into atomic operations.

```
graph TD
    A[Central vector store  
canonical collection] -->|1: Snapshot-and-replace| B[Edge node:  
hot-swap full index]
    A -->|2: Delta sync via changelog| C[Edge node:  
incremental upserts]
    A -->|3: Versioned snapshot| D[Edge node:  
immutable, symlink swap]
    B --> E[Local queries served  
with no network hop]
    C --> E
    D --> E
```

Three synchronization patterns for keeping an edge index current with the central store — the right choice depends on update frequency and acceptable staleness.

The practical patterns are:

1. Snapshot-and-replace. Periodically export the full collection from the central store as a binary snapshot, ship it to edge nodes, and hot-swap the index. Qdrant supports snapshots natively via its REST API. This is appropriate when the dataset is stable (changes fewer than 5% of vectors per day) and the snapshot fits within your transfer window.

2. Delta sync with a changelog. Maintain a changelog table (or Kafka topic) of vectors inserted, updated, or deleted since the last sync. The edge node consumes the changelog and applies individual upserts. This works well with Qdrant's point upsert API and SQL Server's standard INSERT/UPDATE/DELETE against the vector table (rebuild or reorganize the DiskANN index after heavy delta application — quality remains acceptable until the index is 30–40% updated, at which point a rebuild improves recall by 3–8%).

3. Versioned read-only snapshots. Treat the edge index as immutable. Build a new snapshot for every major data version, push it to edge nodes, and switch a symlink atomically. No online mutation. This is operationally simple and eliminates index drift, at the cost of requiring full rebuilds for every update cycle.

```

# Python
# Edge synchronization manager: delta sync from central Qdrant to edge Qdrant
# Run this on the edge node as a periodic job (cron / systemd timer)

import time
import json
import hashlib
import sqlite3
import requests
from datetime import datetime, timezone
from qdrant_client import QdrantClient
from qdrant_client.models import PointStruct, UpdateStatus

CENTRAL_URL    = "https://central-qdrant.internal:6333"
CENTRAL_API_KEY = "central-api-key"
EDGE_URL       = "http://localhost:6333"
COLLECTION     = "products"
SYNC_STATE_DB  = "/data/sync_state.db"
BATCH_SIZE     = 256

def init_sync_state():
    """Track the last-synced offset per collection in local SQLite."""
    conn = sqlite3.connect(SYNC_STATE_DB)
    conn.execute("""
        CREATE TABLE IF NOT EXISTS sync_state (
            collection TEXT PRIMARY KEY,
            last_offset TEXT,    -- opaque cursor from central store
            last_sync TEXT,
            vectors_synced INTEGER DEFAULT 0
        )
    """)
    conn.commit()
    return conn

def get_last_offset(conn, collection: str) -> str | None:
    row = conn.execute(
        "SELECT last_offset FROM sync_state WHERE collection = ?",
        [collection]
    ).fetchone()
    return row[0] if row else None

def save_offset(conn, collection: str, offset: str, count: int):
    conn.execute("""
        INSERT INTO sync_state(collection, last_offset, last_sync, vectors_synced)
        VALUES (?, ?, ?, ?)
        ON CONFLICT(collection) DO UPDATE SET
            last_offset = excluded.last_offset,
            last_sync   = excluded.last_sync,
            vectors_synced = sync_state.vectors_synced + excluded.vectors_synced
    """, [collection, offset, datetime.now(timezone.utc).isoformat(), count])
    conn.commit()

def fetch_delta_from_central(last_offset: str | None, batch_size: int = BATCH_SIZE):

```

```

"""
Scroll through the central Qdrant collection from a given offset.
Returns (points, next_offset). next_offset is None when fully caught up.
"""
central = QdrantClient(url=CENTRAL_URL, api_key=CENTRAL_API_KEY)

scroll_result = central.scroll(
    collection_name=COLLECTION,
    offset=last_offset,          # None = start from beginning
    limit=batch_size,
    with_payload=True,
    with_vectors=True,
)
points, next_offset = scroll_result
return points, next_offset

def apply_delta_to_edge(points):
    """Upsert a batch of points into the local edge Qdrant instance."""
    if not points:
        return 0

    edge = QdrantClient(url=EDGE_URL)
    upsert_points = [
        PointStruct(
            id=p.id,
            vector=p.vector,
            payload=p.payload,
        )
        for p in points
    ]

    result = edge.upsert(
        collection_name=COLLECTION,
        points=upsert_points,
        wait=True,
    )

    if result.status != UpdateStatus.COMPLETED:
        raise RuntimeError(f"Edge upsert failed: {result.status}")

    return len(points)

def run_delta_sync():
    conn = init_sync_state()
    last_offset = get_last_offset(conn, COLLECTION)
    total_synced = 0
    start_time = time.monotonic()

    print(f"[{datetime.now().isoformat()}] Starting delta sync for '{COLLECTION}', offset={last_offset}")

    while True:
        try:

```

```

        points, next_offset = fetch_delta_from_central(last_offset)
    except Exception as e:
        print(f"ERROR fetching from central: {e}")
        break

    if not points:
        print(f"Sync complete. Synced {total_synced} vectors in
{time.monotonic()-start_time:.1f}s")
        break

    count = apply_delta_to_edge(points)
    total_synced += count
    last_offset = next_offset
    save_offset(conn, COLLECTION, str(next_offset) if next_offset else
"COMPLETE", count)

    print(f"  Batch: +{count} vectors | total={total_synced} |
next_offset={next_offset}")

    if next_offset is None:
        print("Reached end of collection.")
        break

    time.sleep(0.05)  # Brief pause – don't saturate edge IO

conn.close()
return total_synced

if __name__ == "__main__":
    synced = run_delta_sync()
    print(f"Delta sync finished. {synced} vectors applied to edge.")

```

A critical operational detail: on low-power flash storage (eMMC, SD card), the HNSW index build is IO-intensive enough to wear out the storage over months of frequent rebuilds. Profile the write amplification of your sync strategy. On a Raspberry Pi 4 with an SD card, a full rebuild of 100K vectors writes approximately 2–4GB to disk. At one rebuild per day, that is 60–120GB of writes per month—within the endurance budget of most industrial SD cards (10,000+ program/erase cycles at 8–32GB), but worth tracking.

Offline-First Application Architecture

An edge deployment that assumes continuous connectivity is not truly an edge deployment—it's a remote client. The application must be designed to function when the WAN link is down, when the central vector store is unreachable, and when the local index is days behind the central store. This is the same discipline as offline-first mobile databases, applied to vector search.

The pattern has three states:

- → **Online with sync.** The edge index is current. Queries run locally. Writes are forwarded to the central store and reflected in the next delta sync.
- → **Offline with stale index.** The edge index serves queries from its last-known-good state. Results may be incomplete (new vectors not yet synced) but are never wrong (no hallucinated data). The application should surface a freshness indicator to users: "Results reflect data as of 4 hours ago."
- → **Offline with missing index.** The edge node has just been provisioned and the initial sync has not completed, or the index has been deleted due to a storage failure. The application must degrade gracefully—either surfacing a text search fallback, a curated static list, or a "search unavailable" state.

```

# Python
# Offline-first vector search wrapper with graceful degradation
# Appropriate for a retail kiosk or industrial mobile app

import os
import time
import sqlite3
import sqlite_vec
import numpy as np
import json
import logging
from enum import Enum
from dataclasses import dataclass
from typing import Optional
from sentence_transformers import SentenceTransformer

logger = logging.getLogger("edge_search")

class SearchMode(Enum):
    VECTOR_LOCAL = "vector_local" # Local vector index, fully operational
    TEXT_FALLBACK = "text_fallback" # FTS5 fulltext search, no vectors
    STATIC_CATALOG = "static_catalog" # Pre-baked result list, last resort
    UNAVAILABLE = "unavailable" # All systems offline

@dataclass
class SearchResult:
    doc_id: str
    text: str
    score: float
    mode: SearchMode
    freshness_seconds: int # How old is the index data?

class OfflineFirstVectorSearch:
    """
    Edge vector search with graceful degradation through multiple fallback modes.
    Designed for environments where connectivity is intermittent.
    """

    def __init__(
        self,
        vector_db_path: str = "/data/edge_search.db",
        static_catalog_path: str = "/data/static_catalog.json",
        max_acceptable_staleness_hours: float = 48.0,
    ):
        self.vector_db_path = vector_db_path
        self.static_catalog_path = static_catalog_path
        self.max_staleness_seconds = max_acceptable_staleness_hours * 3600
        self._model: Optional[SentenceTransformer] = None

    def _get_model(self) -> SentenceTransformer:

```



```

if self._model is None:
    self._model = SentenceTransformer("all-MiniLM-L6-v2")
return self._model

def _check_index_health(self) -> tuple[bool, int]:
    """
    Returns (is_healthy, staleness_seconds).
    Reads a metadata row written by the sync process.
    """
    if not os.path.exists(self.vector_db_path):
        return False, -1

    try:
        conn = sqlite3.connect(self.vector_db_path)
        sqlite_vec.load(conn)

        # Check that the vector table exists and has rows
        row = conn.execute(
            "SELECT COUNT(*) FROM vec_items"
        ).fetchone()
        if row[0] == 0:
            conn.close()
            return False, -1

        # Check sync metadata written by the sync process
        meta = conn.execute("""
            SELECT last_sync_ts FROM sync_metadata
            ORDER BY last_sync_ts DESC LIMIT 1
        """).fetchone()
        conn.close()

        if meta is None:
            return True, 999999 # Index exists but no sync record

        last_sync_ts = meta[0] # Unix timestamp written by sync job
        staleness = int(time.time() - last_sync_ts)
        return True, staleness

    except Exception as e:
        logger.warning(f"Index health check failed: {e}")
        return False, -1

def _vector_search(self, query_text: str, top_k: int) -> list[SearchResult]:
    model = self._get_model()
    query_vec = model.encode([query_text], normalize_embeddings=True)[0]

    conn = sqlite3.connect(self.vector_db_path)
    sqlite_vec.load(conn)

    rows = conn.execute("""
        SELECT
            m.doc_id,
            m.text,
    """

```

```

        m.payload,
        v.distance,
        s.last_sync_ts
    FROM vec_items v
    JOIN item_metadata m ON m.item_id = v.item_id
    CROSS JOIN (SELECT MAX(last_sync_ts) AS last_sync_ts FROM sync_metadata)
S
    WHERE v.embedding MATCH ?
        AND k = ?
    ORDER BY v.distance
    """ , [query_vec.astype(np.float32).tobytes(), top_k]).fetchall()
conn.close()

last_sync_ts = rows[0][4] if rows else time.time()
staleness = int(time.time() - last_sync_ts)

return [
    SearchResult(
        doc_id=r[0],
        text=r[1],
        score=float(r[3]),
        mode=SearchMode.VECTOR_LOCAL,
        freshness_seconds=staleness,
    )
    for r in rows
]

def _fts_fallback(self, query_text: str, top_k: int) -> list[SearchResult]:
    """
    Full-text search using SQLite FTS5. Available even when vector index is
broken.
    Lower precision than vector search but always returns something for keyword
queries.
    """
    if not os.path.exists(self.vector_db_path):
        return []

    try:
        conn = sqlite3.connect(self.vector_db_path)
        rows = conn.execute("""
            SELECT
                m.doc_id,
                m.text,
                bm25(fts_items) AS score
            FROM fts_items
            JOIN item_metadata m ON m.doc_id = fts_items.doc_id
            WHERE fts_items MATCH ?
            ORDER BY score
            LIMIT ?
            """, [query_text, top_k]).fetchall()
        conn.close()

        return [

```

```

        SearchResult(
            doc_id=r[0],
            text=r[1],
            score=float(r[2]),
            mode=SearchMode.TEXT_FALLBACK,
            freshness_seconds=999999,
        )
        for r in rows
    ]
except Exception as e:
    logger.warning(f"FTS fallback failed: {e}")
    return []

def _static_catalog_fallback(self, top_k: int) -> list[SearchResult]:
    """
    Last-resort fallback: return top items from a pre-baked static catalog.
    The catalog is a JSON file bundled with the application, never stale.
    """
    if not os.path.exists(self.static_catalog_path):
        return []

    try:
        with open(self.static_catalog_path) as f:
            catalog = json.load(f)

        return [
            SearchResult(
                doc_id=item["id"],
                text=item["text"],
                score=0.0,
                mode=SearchMode.STATIC_CATALOG,
                freshness_seconds=0, # catalog is versioned, not time-based
            )
            for item in catalog[:top_k]
        ]
    except Exception as e:
        logger.warning(f"Static catalog fallback failed: {e}")
        return []

def search(self, query_text: str, top_k: int = 5) -> tuple[list[SearchResult],
SearchMode]:
    """
    Main search entry point. Returns results and the mode actually used.
    Callers should surface the mode to users when it is not VECTOR_LOCAL.
    """
    healthy, staleness = self._check_index_health()

    # Attempt vector search if index is healthy and not excessively stale
    if healthy and (staleness < self.max_staleness_seconds or staleness == -1):
        try:
            results = self._vector_search(query_text, top_k)
            if results:
                return results, SearchMode.VECTOR_LOCAL

```

```

        except Exception as e:
            logger.error(f"Vector search failed: {e}")

    # Fall back to FTS
    logger.info("Falling back to FTS search")
    fts_results = self._fts_fallback(query_text, top_k)
    if fts_results:
        return fts_results, SearchMode.TEXT_FALLBACK

    # Fall back to static catalog
    logger.info("Falling back to static catalog")
    static_results = self._static_catalog_fallback(top_k)
    if static_results:
        return static_results, SearchMode.STATIC_CATALOG

    return [], SearchMode.UNAVAILABLE

# Example: retail kiosk application loop
def kiosk_search_demo():
    searcher = OfflineFirstVectorSearch(
        max_acceptable_staleness_hours=24.0
    )

    queries = [
        "wireless headphones under $300",
        "running shoes for flat feet",
        "gifts for coffee lovers",
    ]

    for query in queries:
        results, mode = searcher.search(query, top_k=3)

        freshness_msg = ""
        if results and mode == SearchMode.VECTOR_LOCAL:
            staleness_h = results[0].freshness_seconds / 3600
            if staleness_h > 1:
                freshness_msg = f" (index is {staleness_h:.1f}h old)"

        mode_warning = {
            SearchMode.VECTOR_LOCAL: "",
            SearchMode.TEXT_FALLBACK: " [keyword search – vector index\nunavailable]",
            SearchMode.STATIC_CATALOG: " [showing popular items – search offline]",
            SearchMode.UNAVAILABLE: " [search unavailable]",
        }[mode]

        print(f"\nQuery: '{query}'{mode_warning}{freshness_msg}")
        for r in results:
            print(f" {r.doc_id} | {r.text[:60]}")

```

```
if __name__ == "__main__":  
    kiosk_search_demo()
```

The key insight in this pattern is that the application never throws an exception to the user. Every failure mode produces a degraded-but-functional result. This matches the principle that search on a retail floor must always show **something**—a blank screen is worse than a list of popular items that ignores the query.

WebAssembly: Bringing Vector Search to the Browser

Tier 1 edge—browser and mobile—requires a different execution model entirely. JavaScript runtimes cannot load native binaries; everything must run in WebAssembly. The two components you need are an embedding model and a vector search engine, both compiled to WASM.

Transformers.js provides ONNX-Runtime-compiled transformer models running in the browser via WebAssembly. Models are downloaded from Hugging Face and cached in the browser's Cache API. A quantized version of `all-MiniLM-L6-v2` is approximately 23MB after quantization—reasonable for a first-run download, cacheable thereafter.

SQLite-vec compiled to WASM is the search engine. The `sqlite-vec` project ships WASM builds. Combined with the `sql.js` or `wa-sqlite` SQLite WASM bindings, you get a fully in-browser vector index backed by the Origin Private File System (OPFS) for persistence.

```

// JavaScript / browser (ES modules)
// In-browser vector search using Transformers.js + SQLite-vec WASM
// No server round-trip for either embedding generation or search

import { pipeline } from "@xenova/transformers";
import initSqlJs from "sql.js";
// sqlite-vec WASM build; load as an ArrayBuffer from your CDN/assets
import sqliteVecWasm from "./sqlite_vec.wasm?url";

let db = null;
let embedder = null;
const DIM = 384; // all-MiniLM-L6-v2

/**
 * Initialize the embedding model and SQLite-vec database.
 * Call once at application startup.
 * Downloads ~23MB of model weights on first load (cached by browser thereafter).
 */
export async function initEdgeSearch() {
  // Load embedding model – runs entirely in WASM/WebWorker
  embedder = await pipeline(
    "feature-extraction",
    "Xenova/all-MiniLM-L6-v2",
    { quantized: true } // INT8 quantized: ~23MB vs ~90MB for fp32
  );

  // Load SQLite with sqlite-vec extension
  const SQL = await initSqlJs({
    locateFile: () => sqliteVecWasm,
  });

  // Attempt to restore from OPFS (Origin Private File System) for persistence
  let dbData = null;
  try {
    const root = await navigator.storage.getDirectory();
    const fileHandle = await root.getFileHandle("edge_search.db");
    const file = await fileHandle.getFile();
    dbData = new Uint8Array(await file.arrayBuffer());
    console.log(`Restored index from OPFS (${(dbData.length / 1024 /
1024).toFixed(1)} MB)`);
  } catch {
    console.log("No existing OPFS index; starting fresh.");
  }

  db = dbData ? new SQL.Database(dbData) : new SQL.Database();

  // Create tables if first run
  db.run(`
    CREATE VIRTUAL TABLE IF NOT EXISTS vec_items USING vec0(
      item_id INTEGER PRIMARY KEY,
      embedding FLOAT[${DIM}]
    )
  `);

```

```

db.run(`
  CREATE TABLE IF NOT EXISTS item_metadata (
    item_id INTEGER PRIMARY KEY,
    doc_id TEXT,
    text TEXT,
    payload TEXT
  )
`);

console.log("Edge search initialized.");
}

/**
 * Embed and index an array of documents into the in-browser SQLite-vec index.
 * @param {Array<{id: string, text: string, metadata?: object}>} documents
 */
export async function indexDocuments(documents) {
  if (!db || !embedder) throw new Error("Call initEdgeSearch() first");

  const texts = documents.map(d => d.text);

  // Generate embeddings in the browser via WASM
  const output = await embedder(texts, { pooling: "mean", normalize: true });
  const embeddings = output.tolist(); // float32[][]

  db.run("BEGIN TRANSACTION");
  try {
    const insertVec = db.prepare(
      "INSERT OR REPLACE INTO vec_items(item_id, embedding) VALUES (?, ?)"
    );
    const insertMeta = db.prepare(
      "INSERT OR REPLACE INTO item_metadata(item_id, doc_id, text, payload) VALUES"
    );

    for (let i = 0; i < documents.length; i++) {
      const emb = new Float32Array(embeddings[i]);
      insertVec.run([i, emb]);
      insertMeta.run([i, documents[i].id, documents[i].text,
        JSON.stringify(documents[i].metadata ?? {})]);
    }

    insertVec.free();
    insertMeta.free();
    db.run("COMMIT");
  } catch (e) {
    db.run("ROLLBACK");
    throw e;
  }

  // Persist to OPFS so the index survives page reload
  await persistToOPFS();
  console.log(`Indexed ${documents.length} documents in-browser.`);
}

```

```

}

/**
 * Search the in-browser vector index.
 * @param {string} queryText
 * @param {number} topK
 * @returns {Array<{docId, text, metadata, distance}>}
 */
export async function searchLocal(queryText, topK = 5) {
  if (!db || !embedder) throw new Error("Call initEdgeSearch() first");

  const output = await embedder([queryText], { pooling: "mean", normalize: true });
  const queryVec = new Float32Array(output.tolist()[0]);

  const stmt = db.prepare(`
    SELECT
      m.doc_id,
      m.text,
      m.payload,
      v.distance
    FROM vec_items v
    JOIN item_metadata m ON m.item_id = v.item_id
    WHERE v.embedding MATCH ?
      AND k = ?
    ORDER BY v.distance
  `);

  const rows = [];
  stmt.bind([queryVec, topK]);
  while (stmt.step()) {
    const [docId, text, payload, distance] =
      stmt.getAsObject(["doc_id", "text", "payload", "distance"]);
    rows.push({ docId, text, metadata: JSON.parse(payload), distance });
  }
  stmt.free();

  return rows;
}

async function persistToOPFS() {
  try {
    const data = db.export(); // Uint8Array of the SQLite file
    const root = await navigator.storage.getDirectory();
    const fileHandle = await root.getFileHandle("edge_search.db", { create: true });
    const writable = await fileHandle.createWritable();
    await writable.write(data);
    await writable.close();
  } catch (e) {
    console.warn("OPFS persistence failed (may not be supported in this browser):",
e);
  }
}

```


Browser-side vector search has concrete limits. A 50,000-vector index at 384 dimensions occupies roughly 75MB of memory as Float32 arrays, which is within the typical browser tab budget. At 768 dimensions (BERT-base), that doubles to 150MB and begins to compete with the rest of the page. For larger catalogs, apply scalar quantization to INT8 (4× reduction) before writing to the browser index, or shard the index across multiple OPFS files and use a routing layer to pick the relevant shard.

Security Considerations at the Edge

Edge deployments surface a threat model distinct from cloud deployments. A Qdrant instance running on a retail floor is physically accessible to an attacker in a way that a cloud database never is. The threat is not only network-based.

Physical access controls. The edge device should boot from an encrypted disk (LUKS on Linux, BitLocker on Windows IoT). The vector index itself may contain sensitive embeddings—for example, embeddings of customer images in a facial recognition system encode biometric information indirectly. If an attacker extracts the raw float vectors, inversion attacks (reconstructing approximate inputs from embeddings) are plausible for some model architectures.

Network segmentation. The Qdrant or SQLite-vec instance should not be reachable from the public network. Bind to `127.0.0.1` or a local VLAN interface. If the edge application is a web server that also serves customers, place the vector search on a separate internal interface and never proxy direct access to the vector DB port.

API key rotation. If the edge node connects to a central Qdrant or Pinecone instance to pull sync data, the API key on the edge device is a credential that could be extracted from the filesystem. Rotate these keys on a schedule and use short-lived tokens (OAuth 2.0 client credentials) where supported. Store keys in the OS keychain or TPM-backed secure enclave, not in plain configuration files.

Differential privacy for federated indexes. In cases where edge nodes generate embeddings from user behavior and those embeddings are sent back to the central store for model improvement, consider adding calibrated Laplace noise to embeddings before transmission. This prevents the central store from precisely reconstructing individual user interactions while preserving the aggregate signal for retraining.

Benchmarking Edge Hardware

Before committing to an edge hardware platform, profile the three operations that dominate vector search performance: embedding generation, index build, and query latency. The following shell-based benchmark harness uses Qdrant's built-in benchmark tool alongside a simple Python timing script.

```

# shell
# Edge hardware benchmarking: embedding + index build + query latency
# Run on the target edge device (Raspberry Pi 4 / Jetson Nano / NUC)

DEVICE_TAG=$(uname -m)-$(hostname)
RESULTS_DIR="/tmp/edge_bench_${DEVICE_TAG}_${date +%Y%m%d_%H%M%S}"
mkdir -p "$RESULTS_DIR"

echo "=== Edge Hardware Benchmark: $DEVICE_TAG ===" | tee "$RESULTS_DIR/report.txt"
echo "CPU: $(lscpu | grep 'Model name' | cut -d: -f2 | xargs)" | tee -a
"$RESULTS_DIR/report.txt"
echo "RAM: $(free -h | awk '/^Mem/{print $2}')" | tee -a "$RESULTS_DIR/report.txt"
echo "Storage: $(df -h /data | awk 'NR==2{print $2, $4, "free"}')" | tee -a
"$RESULTS_DIR/report.txt"
echo "" | tee -a "$RESULTS_DIR/report.txt"

# 1. Embedding generation throughput (sentences/sec)
echo "--- Embedding Generation Throughput ---" | tee -a "$RESULTS_DIR/report.txt"
python3 - << 'PYEOF' | tee -a "$RESULTS_DIR/report.txt"
import time
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")
# Warm-up
model.encode(["warmup sentence"])

sentences = [f"This is test sentence number {i} for benchmarking purposes." for i in
range(500)]

t0 = time.perf_counter()
_ = model.encode(sentences, batch_size=32, show_progress_bar=False)
elapsed = time.perf_counter() - t0

print(f"Embedded 500 sentences in {elapsed:.2f}s → {500/elapsed:.1f} sentences/sec")
print(f"Estimated time to embed 100K docs: {100000 / (500/elapsed) / 60:.1f}
minutes")
PYEOF

# 2. SQLite-vec index build time
echo "" | tee -a "$RESULTS_DIR/report.txt"
echo "--- SQLite-vec Index Build (10K vectors, dim=384) ---" | tee -a "$RESULTS_DIR/
report.txt"
python3 - << 'PYEOF' | tee -a "$RESULTS_DIR/report.txt"
import time, os, sqlite3, numpy as np
import sqlite_vec

BENCH_DB = "/tmp/bench_vec.db"
DIM = 384
N = 10000

conn = sqlite3.connect(BENCH_DB)
sqlite_vec.load(conn)
conn.execute(f"CREATE VIRTUAL TABLE bench_vecs USING vec0(id INTEGER PRIMARY KEY, v

```

```

FLOAT[{DIM}]))")

vecs = np.random.randn(N, DIM).astype(np.float32)
vecs /= np.linalg.norm(vecs, axis=1, keepdims=True)

t0 = time.perf_counter()
conn.execute("BEGIN")
for i, v in enumerate(vecs):
    conn.execute("INSERT INTO bench_vecs(id, v) VALUES (?, ?)", [i, v.tobytes()])
conn.execute("COMMIT")
elapsed = time.perf_counter() - t0

db_size_mb = os.path.getsize(BENCH_DB) / 1024 / 1024
print(f"Inserted {N} vectors in {elapsed:.2f}s → {N/elapsed:.0f} vecs/sec")
print(f"DB size: {db_size_mb:.1f} MB for {N} × dim{DIM}")

# Query latency
query = np.random.randn(DIM).astype(np.float32)
query /= np.linalg.norm(query)

latencies = []
for _ in range(100):
    t0 = time.perf_counter()
    conn.execute(
        "SELECT id, distance FROM bench_vecs WHERE v MATCH ? AND k = 10 ORDER BY distance",
        [query.tobytes()]
    ).fetchall()
    latencies.append((time.perf_counter() - t0) * 1000)

import statistics
print(f"Query latency (100 runs, k=10): "
      f"p50={statistics.median(latencies):.1f}ms "
      f"p95={sorted(latencies)[95]:.1f}ms "
      f"p99={sorted(latencies)[99]:.1f}ms")

conn.close()
os.unlink(BENCH_DB)
PYEOF

# 3. Qdrant query latency (if running)
echo "" | tee -a "$RESULTS_DIR/report.txt"
echo "--- Qdrant Edge Query Latency ---" | tee -a "$RESULTS_DIR/report.txt"
if curl -sf http://localhost:6333/healthz > /dev/null 2>&1; then
    python3 - << 'PYEOF' | tee -a "$RESULTS_DIR/report.txt"
import time, statistics, numpy as np
from qdrant_client import QdrantClient

client = QdrantClient(url="http://localhost:6333")
collections = client.get_collections().collections
if not collections:
    print("No collections found – skipping Qdrant benchmark")
else:

```

```

col = collections[0].name
info = client.get_collection(col)
dim = info.config.params.vectors.size
print(f"Benchmarking collection '{col}' (dim={dim})")

latencies = []
for _ in range(200):
    q = np.random.randn(dim).astype(np.float32)
    q /= np.linalg.norm(q)
    t0 = time.perf_counter()
    client.search(collection_name=col, query_vector=q.tolist(), limit=10)
    latencies.append((time.perf_counter() - t0) * 1000)

print(f"Query latency (200 runs, k=10): "
      f"p50={statistics.median(latencies):.1f}ms "
      f"p95={sorted(latencies)[int(len(latencies)*0.95)]:.1f}ms "
      f"p99={sorted(latencies)[int(len(latencies)*0.99)]:.1f}ms")
PYEOF
else
    echo "Qdrant not running – skipping"
fi

echo "" | tee -a "$RESULTS_DIR/report.txt"
echo "Benchmark complete. Results in $RESULTS_DIR/report.txt"
cat "$RESULTS_DIR/report.txt"

```

When evaluating edge hardware, the metric that most surprises engineers is not raw query latency—HNSW on modern ARM chips is fast—but **embedding generation throughput**. A MiniLM model running on a Raspberry Pi 4 CPU produces roughly 40–80 sentences per second. If your application must embed user queries at search time, that is imperceptible (one query = one embedding $\approx$ 12–25ms). If it must re-embed a 100K document catalog on every full sync, that is 20–40 minutes of CPU time. The right mitigation is to embed at the central store and ship float vectors in the sync payload rather than re-embedding on edge.

Putting It Together: A Retail Edge Deployment

To make the patterns concrete, consider a mid-size retailer deploying semantic product search on 800 point-of-sale kiosks in stores across a country. Each kiosk runs on an Intel NUC (16GB RAM, 512GB NVMe, intermittent WAN via store Wi-Fi). The product catalog has 250,000 SKUs, each embedded at 384 dimensions using `all-MiniLM-L6-v2`.

Central infrastructure. A Qdrant Cloud instance holds the canonical collection. Product catalog updates—new SKUs, price changes, discontinuations—are pushed by the central ERP system via a Kafka topic consumed by an ingestion service that upserts into Qdrant.

Edge provisioning. New kiosks run a provisioning script on first boot that pulls a full Qdrant snapshot from an S3-compatible store (pre-built nightly from the central collection), hydrates the local Qdrant instance, and writes a sync state record. Snapshot size for 250K × 384-dim vectors with binary quantization is approximately 125MB—a reasonable first-boot download even on slow store Wi-Fi.

Ongoing sync. A systemd timer runs the delta sync script every 15 minutes. If the WAN is down, the job exits cleanly and retries on the next cycle. The local index stays at whatever state it last successfully synced to.

Application layer. The kiosk application uses the `OfflineFirstVectorSearch` wrapper. If the local Qdrant instance is reachable and the index is less than 24 hours stale, queries go local (sub-10ms). If Qdrant is unhealthy (OOM-killed, disk full), the app falls back to an FTS5 index over product names maintained in SQLite. If the SQLite database is also missing, the app shows the store's top-50 bestsellers from a JSON file bundled with the application binary.

Observability. Each kiosk pushes a lightweight heartbeat every 5 minutes to a central monitoring endpoint when WAN is available: sync lag, index size, p95 query latency, and current search mode. The ops team can see at a glance how many kiosks are in `VECTOR_LOCAL` vs `TEXT_FALLBACK` vs `STATIC_CATALOG` mode and dispatch store IT when a device drifts into persistent fallback.

This architecture means a network outage or a Qdrant crash on a kiosk produces a degraded-but-functional experience—keyword search still works—rather than a broken one. A full store network outage still allows customers to find products. The catalog data, being relatively stable, makes stale vector results a minor quality concern rather than a correctness issue.

Key Takeaways

- → **Edge vector search spans four distinct tiers**—browser/mobile, CDN worker, on-premise gateway, and near-edge regional—each with different RAM budgets, storage media, and connectivity assumptions. Match the engine to the tier: SQLite-vec for embedded/browser use, Qdrant binary with quantization for on-premise gateways, full-featured engines for near-edge.
- → **Index synchronization is the defining operational challenge.** Unlike row-based replication, HNSW graphs cannot be updated as a log of atomic mutations indefinitely without quality degradation. Design a sync strategy—snapshot-and-replace, delta upsert, or versioned immutable snapshots—that matches your update frequency and acceptable recall budget.

- → **Offline-first architecture requires multiple fallback layers.** A production edge search system should gracefully degrade from local vector search → full-text search → static curated results rather than failing hard. Surface index freshness to users when results may be stale; never present stale data as current without disclosure.
- → **Embedding generation throughput, not query latency, is the edge bottleneck.** Generating embeddings on low-power ARM hardware is 10–20× slower than querying an HNSW index. Push embedding computation to the central store and ship float vectors in sync payloads to avoid re-embedding large catalogs on constrained hardware.
- → **Edge deployments introduce physical security concerns.** Vector indexes may encode sensitive biometric or behavioral signals indirectly. Encrypt edge storage at rest, bind database ports to localhost or internal interfaces only, use short-lived credentials for central sync, and add differential privacy noise to any embeddings transmitted back to central stores for model improvement.

Chapter 39: The Future: Hybrid Relational-Vector Databases

The Convergence Is Already Happening

The database industry has seen consolidation cycles before — the document store era eventually gave way to multi-model databases, time-series engines absorbed into relational extensions, and graph capabilities folded into relational systems. Vector search is following the same trajectory, but faster and with higher stakes. This chapter examines where the hybrid relational-vector database is going: what architectural patterns are winning, which vendors are placing serious bets, and how DBAs should position their systems and skills for the next five years. The question is no longer whether your relational database will support vector search — SQL Server 2025 already does. The question is whether that integration will be deep enough to matter, and what trade-offs you accept in exchange for keeping everything in one system.

- → **SQL Server 2025**: Native `VECTOR` type, `VECTOR_DISTANCE`, and DiskANN indexing in the engine — exact and approximate nearest neighbor search in T-SQL
- → **Azure SQL Database**: Native vector type and functions generally available; DiskANN approximate search in preview — the managed path for SQL Server vector workloads
- → **Oracle 23ai**: `VECTOR` type with HNSW and IVF indexes — function-based syntax close to SQL Server's
- → **SingleStore / AlloyDB**: Distributed SQL with vector indexes for scale-out hybrid workloads

This is not yet feature parity with dedicated vector databases. Pinecone still outperforms in-engine indexes on billion-scale approximate nearest neighbor queries. Weaviate's hybrid BM25+vector search is more sophisticated than what most relational extensions offer today. Qdrant's payload-indexed filtering during graph traversal prevents index degradation in ways current relational implementations do not. But these gaps are engineering problems, not fundamental limitations, and engineering problems get solved.

Hybrid Query Architectures: SQL Meets Semantic Search

The defining query pattern of the hybrid era combines semantic similarity with structured predicates in a single statement — "find documents semantically similar to X, published after date Y, owned by tenant Z, excluding restricted classification." In SQL Server 2025 this is one T-SQL query, not an application-side join of two systems:


```
-- SQL Server 2025: hybrid semantic + structured query in one statement
DECLARE @q VECTOR(1536) = '[0.1, ...]';
DECLARE @tenant INT = 42;

SELECT TOP (20) c.ChunkId, c.ChunkText,
               VECTOR_DISTANCE('cosine', c.Embedding, @q) AS SemDistance,
               fts.[RANK] AS FtsRank
FROM dbo.DocumentChunks AS c
LEFT JOIN CONTAINSTABLE(dbo.DocumentChunks, ChunkText,
                       'refund policy') AS fts
      ON fts.[KEY] = c.ChunkId
WHERE c.TenantId = @tenant
      AND c.PublishedAt >= '2025-01-01'
      AND c.Classification <> 'Restricted'
ORDER BY (0.7 * (1 - VECTOR_DISTANCE('cosine', c.Embedding, @q)))
        + (0.3 * (fts.[RANK] / 1000.0)) DESC;
-- Semantic similarity + full-text rank + relational predicates,
-- one engine, one transaction, one security model.
```

This pattern — over-fetching from the ANN index, joining with full-text results, and fusing scores — is the workhorse of production hybrid search. The `0.7 / 0.3` weighting is a starting point; in practice you tune it against offline evaluation datasets (Chapter 27).

Weaviate and Elasticsearch support reciprocal rank fusion (RRF) natively. In SQL Server it is a straightforward table-valued pattern — no extension required:

```

-- SQL Server 2025: reciprocal rank fusion in T-SQL
-- Two ranked lists (semantic top-50, full-text top-50) fused by RRF.
DECLARE @q VECTOR(1536) = '[0.1, ...]';
DECLARE @k FLOAT = 60.0;    -- RRF constant

WITH Semantic AS (
    SELECT TOP (50) ChunkId,
           ROW_NUMBER() OVER (
               ORDER BY VECTOR_DISTANCE('cosine', Embedding, @q)) AS rnk
    FROM dbo.DocumentChunks
    WHERE TenantId = 42
),
FullText AS (
    SELECT TOP (50) [KEY] AS ChunkId,
           ROW_NUMBER() OVER (ORDER BY [RANK] DESC) AS rnk
    FROM CONTAINSTABLE(dbo.DocumentChunks, ChunkText, 'refund policy')
)
SELECT TOP (10) COALESCE(s.ChunkId, f.ChunkId) AS ChunkId,
               ISNULL(1.0 / (@k + s.rnk), 0)
               + ISNULL(1.0 / (@k + f.rnk), 0) AS RrfScore
FROM Semantic AS s
FULL OUTER JOIN FullText AS f ON f.ChunkId = s.ChunkId
ORDER BY RrfScore DESC;
-- RRF needs no score normalization – it fuses ranks, not distances –
-- which makes it robust when the two lists use incomparable scales.

```

Vector-Native Extensions to the SQL Standard

Compare the syntax surface across the two major relational vendors shipping native vector support today:

| Operation | SQL Server 2025 | Oracle 23ai |

| Distance | `VECTOR_DISTANCE('cosine', a, b)` | `VECTOR_DISTANCE(a, b, COSINE)` |

| ANN index | `CREATE VECTOR INDEX ... TYPE='DiskANN'` | `CREATE VECTOR INDEX ... HNSW` |

| ANN query | `VECTOR_SEARCH(...)` | `VECTOR_SEARCH(...)` |

The function-based syntax that SQL Server and Oracle chose is closer to what a future SQL standard will look like — it is easier to implement across different vendor parsers than operator overloading. Expect convergence on `VECTOR_DISTANCE`-style functions; the index DDL will stay vendor-specific for years, exactly as B-tree options did.

Dedicated Vector Databases: Where They Remain Superior

Intellectual honesty requires naming where dedicated systems still win:

| Workload | Why dedicated wins |

| >500M vectors, single namespace | Native graph sharding; SQL Server's DiskANN index lives on one instance |

| Multi-tenancy via namespaces (thousands of isolated indexes) | Pinecone namespaces have no in-engine equivalent; SQL Server needs RLS + tenant predicates |

| Sparse+dense hybrid search (SPLADE + dense vectors) | Weaviate and Qdrant support natively; SQL Server needs application-side fusion |

| Sub-10ms p99 at massive scale | Purpose-built storage engines beat a general-purpose rowstore |

Pinecone's serverless architecture, for example, decouples vector storage from similarity computation, serving from object storage with millisecond latency via proprietary compression. SQL Server's DiskANN index lives in the database's storage engine — the I/O model is fundamentally a rowstore's, which is the price of transactional consistency and operational unity.

Here is the namespace-based multi-tenancy pattern that has no direct SQL Server equivalent — one Pinecone index, thousands of isolated tenants:

```
# Python – Pinecone namespace isolation (no SQL Server equivalent)
from pinecone import Pinecone
pc = Pinecone(api_key="...")
index = pc.Index("multitenant-rag")

# Each tenant is a namespace: storage-level isolation, zero filter cost
index.upsert(vectors=[("doc-1", emb, {"text": "..."})],
             namespace="tenant-42")
results = index.query(vector=q_emb, top_k=10, namespace="tenant-42")
# The SQL Server equivalent is RLS + TenantId predicates (Chapter 34):
# same logical isolation, enforced per-query rather than per-namespace.
# At thousands of tenants the predicate approach costs planner and
# execution overhead that namespaces avoid entirely.
```

The Next Architectural Patterns

Storage-compute separation for vectors. The direction Pinecone's serverless points to — vectors in cheap object storage, compute scaled independently — will reach relational engines as DiskANN matures. Expect SQL Server's vector story to gain tiered storage options the way columnstore gained archival compression.

Quantization inside the engine. SQL Server's `VECTOR` is float32 today. Scalar and product quantization inside the storage engine (smaller types, compressed graph layouts) is the obvious next step — the same path columnstore took from uncompressed to `COLUMNSTORE_ARCHIVE`.

Multi-vector and late interaction. ColBERT-style multi-vector retrieval (one vector per token, MaxSim scoring) is handled natively by dedicated systems first. In SQL Server today the pattern is a child table of token vectors joined to the parent — workable at moderate scale, and a candidate for native support as the pattern proves out:

```
-- SQL Server: multi-vector (ColBERT-style) retrieval with a child table
CREATE TABLE dbo.ChunkTokenVectors (
    TokenVectorId BIGINT IDENTITY(1,1) NOT NULL
        CONSTRAINT PK_ChunkTokenVectors PRIMARY KEY,
    ChunkId        BIGINT NOT NULL
        CONSTRAINT FK_TokVec_Chunk REFERENCES dbo.DocumentChunks (ChunkId),
    TokenVector    VECTOR(128) NOT NULL
);
CREATE VECTOR INDEX IX_TokVec_TokenVector
    ON dbo.ChunkTokenVectors (TokenVector)
    WITH (METRIC = 'cosine', TYPE = 'DiskANN', MAXDOP = 4);
GO
-- MaxSim approximation: best token match per chunk, then aggregate
DECLARE @qt VECTOR(128) = '[0.1, ...]'; -- one query token vector
SELECT TOP (10) ChunkId, MIN(
    VECTOR_DISTANCE('cosine', TokenVector, @qt)) AS BestTokenDistance
FROM dbo.ChunkTokenVectors
GROUP BY ChunkId
ORDER BY BestTokenDistance;
-- Full MaxSim scores every query token against every doc token;
-- this is the single-token building block, aggregated per query token
-- in the application layer.
```

The Operational Convergence: One Platform, Many Workloads

Beyond query capabilities, the strategic advantage of hybrid databases is operational: one backup strategy, one monitoring stack, one set of RBAC policies, one disaster recovery plan. For organizations already running SQL Server in production, native vectors mean:

- → Backups cover vectors automatically — no separate vector-store export schedule (Chapter 36)
- → Row-Level Security applies to vector tables unchanged (Chapter 34)
- → Availability Groups replicate vectors with the log stream — warm index on failover (Chapter 35)
- → Query Store, DMVs, and Extended Events observe vector queries like any workload (Chapter 14)

Here is a practical monitoring query that demonstrates the operational integration — standard SQL Server administration, working on vector indexes without modification:

```
-- SQL Server: vector index health from standard DMVs
-- Index space usage (works for DiskANN indexes transparently)
SELECT i.name AS IndexName, i.type_desc,
       SUM(s.used_page_count) * 8 / 1024 AS UsedMB,
       SUM(s.reserved_page_count) * 8 / 1024 AS ReservedMB
FROM sys.indexes AS i
JOIN sys.dm_db_partition_stats AS s
  ON s.object_id = i.object_id AND s.index_id = i.index_id
WHERE i.object_id = OBJECT_ID(N'dbo.DocumentChunks')
GROUP BY i.name, i.type_desc
ORDER BY UsedMB DESC;

-- Query Store: find the most expensive vector queries by CPU
-- (filter qsqt.query_sql_text for VECTOR_DISTANCE / VECTOR_SEARCH)
SELECT TOP (10) qsqt.query_sql_text,
       SUM(qrs.avg_cpu_time * qrs.count_executions) / 1000.0 AS TotalCpuMs
FROM sys.query_store_query_text AS qsqt
JOIN sys.query_store_query AS qsq ON qsq.query_text_id = qsqt.query_text_id
JOIN sys.query_store_plan AS qsp ON qsp.query_id = qsq.query_id
JOIN sys.query_store_runtime_stats AS qrs ON qrs.plan_id = qsp.plan_id
WHERE qsqt.query_sql_text LIKE '%VECTOR\_DISTANCE%' ESCAPE '\'
GROUP BY qsqt.query_sql_text
ORDER BY TotalCpuMs DESC;
```

The Five-Year Horizon: Predictions and Preparation

- → **Every major relational engine will ship ANN indexes with quantization enabled by default.** Renting vector search as a separate service for workloads under 100M vectors will be economically unjustifiable for most organizations.
- → **The SQL standard will absorb vector functions.** `VECTOR_DISTANCE`-style functions will standardize; index DDL stays vendor-specific.
- → **Hybrid search becomes the default query shape.** Semantic + full-text + relational predicates in one statement, with RRF as the standard fusion.
- → **Dedicated systems consolidate upward.** Pinecone, Weaviate, and Qdrant compete on billion-scale, multi-modal, and agentic workloads — the long tail moves into the database.
- → **DBA skills transfer directly.** Index maintenance, query tuning, backup strategy, security policy — the vector chapter of each discipline, not a new discipline.

Choosing Your Architecture Today

```
graph TD
    A[Vector dataset size?] -->| "< 10M" | B[SQL Server 2025  
native VECTOR + DiskANN]
    A -->| "10M - 500M" | C[Latency SLA?]
    C -->| Yes: sub-20ms p99 | D[Dedicated: Qdrant / Pinecone  
+ SQL Server for metadata]
    C -->| No | E[SQL Server tuned DiskANN  
over-fetch + rescore]
    E -->| "> 500M" | F[Sharded dedicated system  
SQL Server for source of truth]
    F --> G[Multi-tenant?]
    G -->| Thousands of tenants | H[Pinecone namespaces  
or SQL Server RLS]
    G -->| Dozens of tenants | I[SQL Server RLS  
+ TenantId predicates]
```

The decision tree is deliberately boring: start in the database you already operate. Move vector search out only when a measured limit — scale, latency, tenancy model — forces the move. Every dedicated vector database you don't deploy is a backup schedule you don't maintain, a security model you don't audit, and an on-call rotation you don't staff.

Key Takeaways

- → **Convergence is the trajectory:** SQL Server 2025 native vectors put semantic search in the engine you already operate — backup, RLS, AGs, and DMVs apply unchanged.

- → **Hybrid queries are the native shape:** semantic similarity + full-text + relational predicates in one T-SQL statement, fused with weighted scores or RRF.
- → **Function-based vector syntax will standardize;** index DDL stays vendor-specific — write portable distance logic, isolate index DDL.
- → **Dedicated systems keep the extremes:** billion-scale, namespace multi-tenancy, sparse+dense hybrid, sub-10ms p99 — move out only on measured limits.
- → **DBA skills transfer:** the next five years reward index tuning, query plans, and backup strategy applied to vectors — not a new discipline.

Chapter 40: Vector Database Monitoring Dashboard

The Three-Layer Monitoring Model

Monitoring a vector database in production is not the same as monitoring a relational database. The familiar metrics — query throughput, lock waits, buffer hit ratio, Always On sync health — still matter when the underlying engine is SQL Server, but they tell you nothing about the health of your embedding pipeline, the quality of your search results, or whether your DiskANN indexes are serving queries efficiently under load. This chapter brings together everything from earlier in the book and shows you how to build a dashboard that covers all three layers: infrastructure metrics, vector-specific operational metrics, and application-layer quality signals. By the end, you will have runnable T-SQL for SQL Server DMV and Query Store monitoring, a Python quality probe that runs continuously against your RAG pipeline, and a pattern for surfacing drift and recall degradation before your users notice.

```
graph TD
    L1[Layer 1: Infrastructure  
CPU, memory, I/O, AG health] --> D[Dashboard]
    L2[Layer 2: Vector operations  
DiskANN health, query latency,  
index size, pipeline lag] --> D
    L3[Layer 3: Quality signals  
recall@k probes, drift,  
user feedback] --> D
```

Layer 1 is your existing SQL Server monitoring — keep it. Layer 2 is new: DiskANN index health, vector query latency from Query Store, embedding pipeline lag. Layer 3 is the one teams skip and regret: continuous quality probes that prove the system still retrieves the right documents.

Instrumenting SQL Server Vectors with DMVs and Query Store

SQL Server exposes vector workload telemetry through the same DMVs and Query Store you already use — no exporter required. The queries below form the Layer 2 core.


```

-- SQL Server 2025: Layer 2 vector operations monitoring
-- 1. DiskANN index space and growth (run hourly, trend in Grafana)
SELECT DB_NAME() AS DbName, i.name AS IndexName,
       SUM(s.used_page_count) * 8 / 1024 AS UsedMB,
       SUM(s.row_count) AS VectorRows
FROM sys.indexes AS i
JOIN sys.dm_db_partition_stats AS s
  ON s.object_id = i.object_id AND s.index_id = i.index_id
WHERE i.object_id = OBJECT_ID(N'dbo.DocumentChunks')
GROUP BY i.name;
-- 2. Vector query latency from Query Store (top offenders, last hour)
SELECT TOP (10) qsqt.query_sql_text,
       SUM(qrs.count_executions) AS Execs,
       SUM(qrs.avg_duration * qrs.count_executions) / 1000.0 AS TotalMs,
       MAX(qrs.max_duration) / 1000.0 AS MaxMs
FROM sys.query_store_query_text AS qsqt
JOIN sys.query_store_query AS qsq ON qsq.query_text_id = qsqt.query_text_id
JOIN sys.query_store_plan AS qsp ON qsp.query_id = qsq.query_id
JOIN sys.query_store_runtime_stats AS qrs ON qrs.plan_id = qsp.plan_id
WHERE qsqt.query_sql_text LIKE '%VECTOR\_DISTANCE%' ESCAPE '\'
      OR qsqt.query_sql_text LIKE '%VECTOR\_SEARCH%' ESCAPE '\'
GROUP BY qsqt.query_sql_text
ORDER BY TotalMs DESC;
-- 3. Embedding pipeline lag: unprocessed queue depth + oldest item age
SELECT COUNT(*) AS PendingJobs,
       DATEDIFF(minute, MIN(EnqueuedAt), SYSUTCDATETIME()) AS OldestAgeMin
FROM dbo.EmbeddingJobs
WHERE Status = 'pending';
-- 4. AG replica health for the vector database (Chapter 35)
SELECT ars.role_desc AS Role, ars.synchronization_health_desc AS Health,
       drs.log_send_queue_size AS SendQueueKB,
       drs.redo_queue_size AS RedoQueueKB
FROM sys.dm_hadr_availability_replica_states AS ars
JOIN sys.dm_hadr_database_replica_states AS drs
  ON drs.replica_id = ars.replica_id AND drs.group_id = ars.group_id
WHERE drs.database_id = DB_ID(N'VectorRagDb');

```

This uses Query Store rather than wrapping every query with timing code — it captures the full query load from all connections, not just the ones your application explicitly instruments. Keep Query Store in read-write mode with a sensible retention (30 days is a good default for vector workloads) so latency trends survive long enough to correlate with recall-probe history.

For Grafana, expose these as a Prometheus pushgateway feed or a SQL data source: a collector script runs the four queries every 60 seconds via `pyodbc` and pushes gauges (`sqlserver_vector_index_mb`, `sqlserver_vector_query_ms`, `sqlserver_embedding_queue_depth`). The PromQL for the dashboard's slowest-query panel is then trivial:

```
# Prometheus alerting rules (vector_alerts.yaml)
# Warn when p99-ish vector query latency exceeds 200ms for 10 minutes:
# expr: max_over_time(sqlserver_vector_query_max_ms[10m]) > 200
# Crit when the embedding queue stops draining (depth growing 30 min):
# expr: deriv(sqlserver_embedding_queue_depth[30m]) > 0
# Crit when DiskANN index size grows >20%/day unexpectedly (runaway ingest):
# expr: deriv(sqlserver_vector_index_mb[1d]) > 0.2 * sqlserver_vector_index_mb
```

Monitoring Managed Services via Their Native APIs

Where part of the estate lives in Pinecone, Weaviate, or Qdrant, fold their native metrics into the same dashboard rather than maintaining a second one. Pinecone exposes index stats (vector count, dimension, index fullness) via its API; Weaviate exposes a Prometheus endpoint natively; Qdrant exposes cluster telemetry at `/metrics`.

```
# Python – managed-service metrics into the same pushgateway
# Requires: pinecone-client>=3.0, requests, prometheus_client, pyodbc
import os
from pinecone import Pinecone
from prometheus_client import CollectorRegistry, Gauge, push_to_gateway

registry = CollectorRegistry()
pc_fullness = Gauge("pinecone_index_fullness_ratio",
                    "Index fullness 0-1", registry=registry)
pc_vectors = Gauge("pinecone_vector_count",
                  "Total vectors", ["index"], registry=registry)

pc = Pinecone(api_key=os.environ["PINECONE_API_KEY"])
for desc in pc.list_indexes():
    stats = pc.Index(desc.name).describe_index_stats()
    pc_vectors.labels(index=desc.name).set(stats["total_vector_count"])
    pc_fullness.set(stats.get("index_fullness", 0.0))
    # Fullness > 0.8 on a pod-based index means scale the pods
    # BEFORE latency degrades – this is the managed equivalent of
    # watching DiskANN index growth in the previous section.

push_to_gateway("localhost:9091", job="managed_vector_metrics",
               registry=registry)

# Weaviate needs no collector: point Prometheus at its native endpoint
# (PROMETHEUS_MONITORING_ENABLED=true) and alert on
# weaviate_async_replication_queue_length and query latency histograms.
```

Embedding Quality and Recall Probes

Infrastructure metrics tell you the system is *running*. Quality probes tell you it is *working*. A probe runs a fixed set of queries with known-good answers against the live retrieval path every five minutes and records recall@k — the earliest warning of index corruption, bad deployments, or embedding drift.

```

# Python – continuous recall probe against SQL Server (pyodbc)
# Requires: pyodbc, openai, prometheus_client, numpy
import json, os, time
import pyodbc, openai
from prometheus_client import CollectorRegistry, Gauge, push_to_gateway

GOLDEN = json.load(open("golden_dataset.json"))
# [{"query": "...", "expected_chunk_ids": [101, 102], "k": 10}, ...]
client = openai.OpenAI()
conn = pyodbc.connect(os.environ["SQLSERVER_DSN"])

def recall_at_k(retrieved, expected, k):
    return len(set(retrieved[:k]) & set(expected)) / max(len(expected), 1)

def probe():
    registry = CollectorRegistry()
    g_recall = Gauge("rag_recall_at_k", "Probe recall@k",
                    ["query_id"], registry=registry)
    g_dist = Gauge("rag_top1_distance", "Top-1 cosine distance",
                  ["query_id"], registry=registry)
    cur = conn.cursor()
    for i, q in enumerate(GOLDEN):
        emb = client.embeddings.create(
            input=[q["query"]],
            model="text-embedding-3-small").data[0].embedding
        vec = "[" + ", ".join(str(v) for v in emb) + "]"
        rows = cur.execute("""SELECT TOP (?) ChunkId,
                               VECTOR_DISTANCE('cosine', Embedding,
                               CAST(? AS VECTOR(1536))) AS d
                               FROM dbo.DocumentChunks
                               WHERE TenantId = ?
                               ORDER BY d""",
                            (q["k"], vec, q.get("tenant_id", 1))).fetchall()
        retrieved = [r[0] for r in rows]
        g_recall.labels(query_id=str(i)).set(
            recall_at_k(retrieved, q["expected_chunk_ids"], q["k"]))
        g_dist.labels(query_id=str(i)).set(rows[0][1] if rows else 1.0)
    push_to_gateway("localhost:9091", job="rag_quality_probe",
                    registry=registry)
    print(f"[{time.strftime('%H:%M:%S')}] Probe complete: "
          f"{len(GOLDEN)} queries")

if __name__ == "__main__":
    while True:
        probe()
        time.sleep(300) # every 5 minutes

```

Alert on the probe metrics, not just the infrastructure ones: a 5-point drop in mean recall@k sustained over 30 minutes pages the on-call engineer; a single query's top-1 distance drifting upward week over week is the early signal of embedding drift (Chapter 30) or a degraded DiskANN index in need of rebuild.

Building the Golden Dataset

The probe is only as good as its golden dataset. Build it deliberately:

- → **Start with 50–100 queries** drawn from real production query logs — the questions users actually ask, not the ones engineers imagine.
- → **Label expected chunks by hand** for the first version. A domain expert marking "these 2–3 chunks answer this question" beats any automated labeling for probe trustworthiness.
- → **Cover the failure modes:** paraphrased duplicates (tests threshold tuning), multi-hop questions (tests top-k depth), tenant-boundary queries (tests RLS — the probe must *not* retrieve cross-tenant chunks).
- → **Version the dataset** alongside the embedding model version. A model rotation (Chapter 30) invalidates golden expectations — re-validate, don't just re-run.
- → **Store it in the database** (`dbo.GoldenQueries`) rather than a JSON file once it stabilizes — versioned, backed up, auditable.

Alerting Rules and Escalation Thresholds

- → **Critical — page immediately:** recall@k probe drops >5 points for 30 minutes; AG replica health = NOT_HEALTHY; embedding queue depth growing for 30 minutes (ingest stalled).
- → **Warning — ticket for business hours:** vector query p99 latency >200ms for 10 minutes; DiskANN index growth >20%/day; Pinecone index fullness >0.8; probe top-1 distance trending up over 7 days.
- → **Info — dashboard only:** backup completion, index rebuild completion, model rotation events, golden dataset version changes.

Two-tier thresholds (warning then critical) on every metric that predicts user-facing degradation. The probe alerts are the most important in the whole stack — they are the only ones that fire when the system is running perfectly and returning the wrong answers.

Grafana Dashboard Layout

One dashboard, three rows mirroring the three layers:

- → **Row 1 — Quality (the top row, the one executives see).** Mean recall@k probe gauge, probe pass/fail timeline, top-1 distance trend, user feedback rate (thumbs-down per 1K queries).

- → **Row 2 — Vector operations.** Query latency p50/p99 from Query Store, DiskANN index size growth, embedding queue depth and oldest-item age, per-namespace hit rates.
- → **Row 3 — Infrastructure.** CPU, memory, buffer pool, AG sync health, backup age, managed-service fullness/queue metrics.

Quality on top is deliberate: when the 2 AM page fires for a latency spike, the first question is "are users getting wrong answers?" — and the answer is one glance up, not three dashboards over.

Handling Embedding Model Rotation

Model rotation is a monitoring event, not just a deployment event (Chapter 30). When the embedding model changes:

- → **Freeze the golden dataset version** and re-validate expectations against the new model's geometry before trusting probe results again.
- → **Run old and new probes in parallel** during the side-by-side backfill — two probe jobs, two metric series, one dashboard comparing them.
- → **Alert on backfill lag** as a first-class metric: `sqlserver_backfill_remaining` trending to zero is the cutover gate; a stalled backfill pages like any stalled pipeline.
- → **Annotate the dashboard** at cutover time — six months later, the `recall@k` step-change needs its explanation attached.

Putting It All Together: The Monitoring Stack Deployment

A minimal `docker-compose.yml` wires together Prometheus, the pushgateway, Grafana, the SQL Server collector, and the recall probe:

```
# docker-compose.yml – vector monitoring stack
services:
  prometheus:
    image: prom/prometheus:v2.51.0
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - ./vector_alerts.yml:/etc/prometheus/rules/vector_alerts.yml:ro
      - prometheus_data:/prometheus
    command:
      - "--config.file=/etc/prometheus/prometheus.yml"
    ports: ["9090:9090"]
  pushgateway:
    image: prom/pushgateway:v1.7.0
    ports: ["9091:9091"]
  grafana:
    image: grafana/grafana:10.4.0
    environment:
      GF_SECURITY_ADMIN_PASSWORD: "${GRAFANA_ADMIN_PASSWORD}"
    volumes: ["grafana_data:/var/lib/grafana"]
    ports: ["3000:3000"]
    depends_on: [prometheus]
  sqlserver_collector:
    # runs the DMV/Query Store queries hourly
    build: ./collectors
    environment:
      SQLSERVER_DSN: "${SQLSERVER_DSN}"
      PUSHGATEWAY: "pushgateway:9091"
    command: python sqlserver_vector_collector.py
  rag_quality_probe:
    # recall probe every 5 minutes
    build: ./collectors
    environment:
      SQLSERVER_DSN: "${SQLSERVER_DSN}"
      OPENAI_API_KEY: "${OPENAI_API_KEY}"
      PUSHGATEWAY: "pushgateway:9091"
    command: python rag_quality_probe.py
volumes:
  prometheus_data:
  grafana_data:
```

The `prometheus.yml` scrapes the pushgateway (for pushed metrics from all collectors) and Weaviate's native endpoint directly:

```
# prometheus.yml
global:
  scrape_interval: 60s
rule_files:
  - /etc/prometheus/rules/*.yaml
scrape_configs:
  - job_name: pushgateway          # SQL Server collector + probes + Pinecone
    static_configs:
      - targets: ["pushgateway:9091"]
  - job_name: weaviate             # native endpoint, no collector needed
    static_configs:
      - targets: ["weaviate:2112"]  # PROMETHEUS_MONITORING_ENABLED=true
  - job_name: prometheus_self
    static_configs:
      - targets: ["localhost:9090"]
```

Key Takeaways

- → **Monitor three layers:** SQL Server infrastructure (DMVs, AG health), vector operations (Query Store latency, DiskANN growth, pipeline lag), and quality signals (recall probes, drift, feedback).
- → **Query Store is your vector APM** — it captures every vector query's latency from all connections with no code changes; trend it, alert on it.
- → **DiskANN index growth and embedding queue depth are the two backend metrics that most directly predict user-facing degradation** — alert with two-tier thresholds.
- → **Continuous quality probes make the invisible visible.** Recall@k every five minutes against a golden dataset is the only metric that fires when the system runs perfectly and returns wrong answers.
- → **Treat model rotation as a monitoring event:** parallel probes, backfill-lag alerts, frozen golden versions, annotated dashboards.
- → **Quality row on top.** When the page fires, "are users getting wrong answers?" must be answerable in one glance.